

PsiloPrep: Preprocesamiento fMRI de ds006072 (psilocibina)

Preprocesamiento NORDIC + fMRIPrep + TEDANA + extracción de conectividad para ds006072

Juan

2026-06-03

Table of contents

1	1 NORDIC: reducción de ruido térmico previa a fMRIPrep	2
1.1	1.1 Por qué NORDIC debe aplicarse <i>antes</i> de fMRIPrep (y no después)	3
1.2	1.2 Por qué aplicar NORDIC en este pipeline	3
1.3	1.3 Algoritmo	4
1.4	1.4 Modos NORDIC por adquisición: dos ejes de decisión (fase × ruido)	6
1.5	1.5 Normalización del dtype	9
1.6	1.6 Implementación: <code>oct2py</code> + Octave + <code>NIFTI_NORDIC.m</code>	11
1.7	1.7 Estrategia de idempotencia y de seguridad en sobrescritura	13
1.8	1.8 Chunk de aplicación de NORDIC	14
1.9	1.9 QC cuantitativo del bulk run NORDIC	30
1.9.1	1.9.1 Métricas de QC	30
2	2 Preprocesamiento fMRI (fMRIPrep)	42
2.1	2.1 Requisitos previos	43
2.1.1	2.1.1 Instalación de Docker y NiPreps	43
2.1.2	2.1.2 Docker Desktop	43
2.1.3	2.1.3 Configuración de recursos WSL 2	43
2.1.4	2.1.4 Imágenes Docker de NiPreps	44
2.1.5	2.1.5 Licencia de FreeSurfer	44
2.2	2.2 Ejecución de fMRIPrep para ds006072 (multi-echo)	44
2.2.1	2.2.1 Explicación de los argumentos	46
2.2.2	2.2.2 Reanudación tras fallo	49
2.2.3	2.2.3 Outputs esperados	49
2.2.4	2.2.4 Interpretación de los resultados de fMRIPrep	57

2.3	2.3 QC cuantitativo del bulk run fMRIPrep	57
2.3.1	2.3.1 Métricas de QC	58
2.3.2	2.3.2 Comprobaciones adicionales recomendadas (no automatizadas) . .	61
3	3 Denoising multi-echo con TEDANA	72
3.1	3.1 Fundamento: separación BOLD vs. non-BOLD basada en TE-dependence .	72
3.2	3.2 Ventaja sobre métodos single-echo	73
3.3	3.3 Justificación de la estrategia de denoising multi-echo	73
3.4	3.4 Impacto de TEDANA sobre las correlaciones de conectividad funcional . .	75
3.5	3.5 Posición en el pipeline	75
3.6	3.6 Consideración sobre GSR post-TEDANA	76
3.7	3.7 Ejecución de TEDANA	76
3.8	3.8 Heterogeneidad de phase encoding y número de runs en ds006072	76
3.9	3.9 TEDANA en espacio nativo	77
3.10	3.10 Justificación de las opciones de TEDANA	81
3.11	3.11 Resampleo post-TEDANA a MNI152Nlin2009cAsym	83
3.12	3.12 Outputs de TEDANA	86
3.13	3.13 QC cuantitativo del bulk run TEDANA	86
3.13.1	3.13.1 Métricas de QC	87
3.13.2	3.13.2 Comprobaciones adicionales recomendadas (no automatizadas) .	91
4	4 Extracción de conectividad funcional para ds006072 (Python)	106
4.1	4.1 Por qué NO aplicar GSR después de TEDANA (en el pipeline primario) . .	106
4.2	4.2 Atlas de parcelación: corteza + subcorteza (Schaefer 200 + Tian S2)	107
4.3	4.3 Diseño de triangulación para ds006072	112
4.4	4.4 Filtrado temporal y censurado: decisiones revisadas	120
4.5	4.5 Series temporales y matrices de conectividad	121
4.6	4.6 Promedios de conectividad por sujeto y grupo	137
5	5 Control de calidad	142
5.1	5.1 Framewise Displacement, DVARs y QC-FC	145
5.2	5.2 Triangulación inter-pipeline + tDOF + convergencia de efectos	160

1 1 NORDIC: reducción de ruido térmico previa a fMRIPrep

NORDIC (*NOise Reduction with DIstribution Corrected PCA*; Moeller et al., 2021; Vizioli et al., 2021) es un algoritmo de *low-rank denoising* basado en SVD local por parches que opera sobre datos complejos (magnitud + fase) reconstruidos, antes de cualquier interpolación espacial o temporal. Su objetivo es suprimir el **ruido térmico** —ruido gaussiano, espacialmente i.i.d. tras corrección por *g-factor*, generado en los circuitos del sistema de RM y amplificado por las aceleraciones SMS/MB— sin atenuar la señal BOLD ni alterar la *point-spread function* funcional (Vizioli et al., 2021, Figs. 5–6). Vizioli et al. (2021) demostraron mejoras sustanciales

en tSNR y en la detección/reproducibilidad de mapas funcionales a 7 T en 10 datasets (3 T y 7 T, distintas resoluciones y paradigmas) sin alterar la amplitud de la respuesta ni la precisión espacial. Dowdle et al. (2023) extendieron la evaluación a 3 T multi-echo (el escenario de ds006072) y reportaron que los incrementos en *t-values* tras NORDIC son comparables a los obtenidos suavizando con un kernel gaussiano de 1.5 vóxeles, pero con un aumento de la anchura espacial efectiva inferior al 4% (vs. 51–142% con suavizado explícito). Siegel et al. (2024) aplicaron NORDIC como **primer paso** del pipeline ds006072.

1.1 1.1 Por qué NORDIC debe aplicarse *antes* de fMRIPrep (y no después)

El orden **NORDIC** $\rightarrow$ **fMRIPrep** es mandatorio por construcción del algoritmo, no por convención. Sus tres canónicos lo reflejan explícitamente:

- Vizioli et al. (2021, §Results y Fig. 2): comparan NORDIC vs. *Standard* “before any preprocessing for fMRI analysis” sobre imágenes reconstruidas de k-space, y realizan después *slice-timing correction*, *3D rigid-body motion correction* y *anatomical alignment* en BrainVoyager.
- Dowdle et al. (2023, §2.3): “Prior to any additional processing, we examined the noise removed by NORDIC”; las etapas posteriores (*slice timing*, *motion correction* con *Fourier interpolation*, escalado temporal, GLM) se ejecutan sobre la salida de NORDIC.
- Moeller et al. (2021, dMRI): NORDIC se aplica sobre la reconstrucción compleja *offline* del k-space crudo, antes de cualquier corrección (incluyendo *eddy-currents*).

El motivo técnico es que NORDIC requiere **ruido gaussiano, zero-mean, i.i.d.** tras normalización por *g-factor* (Moeller et al., 2021, Eq. 1; Vizioli et al., 2021, Fig. 7): la distribución Marchenko-Pastur utilizada para umbralizar los valores singulares sólo es válida bajo esa asunción. Cualquier operación que introduzca una interpolación espacial o temporal —*slice-timing correction*, *head-motion correction*, *susceptibility distortion correction*, *co-registration* EPI $\rightarrow$ T1w, *spatial normalization*— correlaciona el ruido entre vóxeles vecinos y lo convierte en espacialmente coloreado, invalidando el umbral analítico y reduciendo la eficacia del denoising (o, peor, atenuando señal real). Por la misma razón, NORDIC no puede re-aplicarse sobre los outputs de fMRIPrep en espacio MNI.

Aplicar NORDIC *después* de fMRIPrep también desperdiciaría los 3 volúmenes de ruido terminales adquiridos sin pulso de excitación, ya que fMRIPrep los procesaría como si fueran volúmenes BOLD legítimos (corregiría por movimiento, interpolaría slice-timing asumiendo excitación, etc.), corrompiendo la estimación empírica de σ .

1.2 1.2 Por qué aplicar NORDIC en este pipeline

1. **Complementariedad con TEDANA (multi-echo ICA denoising).** TEDANA separa componentes por TE-dependencia: κ alto (BOLD-like, monotónico con TE)

vs. ρ alto (non-BOLD, independiente de TE; Kundu et al., 2012). El ruido térmico es independiente de TE y, en ausencia de reducción previa, se reparte arbitrariamente entre componentes, contaminando ambas categorías y reduciendo la separabilidad κ/ρ . Dowdle et al. (2023, §2.3) afirman explícitamente que “NORDIC is expected to complement denoising strategies that remove structured noise, such as ICA-AROMA, multi-echo ICA denoising with tedana”. Ese es el orden del pipeline de Siegel et al. (2024).

2. **Replicación fiel del pipeline de Siegel.** El protocolo de adquisición de ds006072 (BOLDREST{1,2}NORDICME52mm) fue diseñado **explícitamente** para NORDIC: incluye 3 volúmenes de ruido al final de cada run (sin pulso de excitación) que permiten estimar la desviación estándar empírica del ruido térmico en lugar de inferirla del propio signal (Vizioli et al., 2021, Methods §“The degree of noise removal”). Omitir NORDIC desperdicia el diseño de adquisición y se desvía del pipeline original.
3. **Necesidad metodológica para multi-echo 2 mm $\times$ MB6.** El contraste BOLD en ds006072 opera cerca del límite de SNR útil por vóxel individual. Sin reducción de ruido térmico, la varianza no estructurada compite con la señal BOLD durante la combinación óptima por TE y durante la descomposición ICA, atenuando el contraste efectivo. Dado el bajo N ($N = 7$), maximizar la calidad por observación es crítico.

Caveat sobre la asunción i.i.d. ds006072 se adquirió con *partial Fourier*, lo que introduce autocorrelación espacial residual en el ruido tras reconstrucción (Dowdle et al., 2023, §Limitations). Esto hace que el umbral estimado sea ligeramente conservador —permitiendo que sobreviva algo más de ruido gaussiano residual—, pero **no invalida el denoising**: simplemente reduce la ganancia máxima de tSNR frente a adquisiciones *full-Fourier*. No existe una manera tratable de corregirlo ex post sin re-adquirir datos.

1.3 1.3 Algoritmo

NORDIC opera en cinco pasos sobre los datos complejos reconstruidos (Moeller et al., 2021; Vizioli et al., 2021, Methods y Fig. 7):

1. **Reconstrucción compleja.** Combina magnitud y fase en un volumen complejo $C = M \cdot e^{i\phi}$. Operar en dominio complejo evita el sesgo no-gaussiano de la distribución Rician de la magnitud (Gudbjartsson & Patz, 1995) y preserva la asunción de ruido zero-mean gaussiano requerida por la distribución Marchenko-Pastur. NIFTI_NORDIC.m aplica además filtrado temporal y por slice de la fase (parámetros `temporal_phase` y `phase_filter_width`) para suprimir errores de fase $B0$ -dependientes que, si no se corrigen, serían interpretados por la SVD como estructura de bajo rango.
2. **Normalización por g -factor y estimación del nivel de ruido σ .** El ruido de imagen tras reconstrucción paralela es espacialmente variable (Breuer et al., 2009; Pruessmann et al., 1999): NORDIC mapea este ruido a distribución espacialmente homogénea dividiendo cada vóxel por su g -factor (estimado internamente mediante MP-PCA sobre la propia

serie temporal en NIFTI_NORDIC.m; en la versión *raw-k-space* de Moeller 2021 se deriva directamente del kernel SMS/GRAPPA). Los volúmenes de ruido finales adquiridos sin excitación RF se utilizan para estimar σ tras *g-factor*-normalización. El parámetro `ARG.noise_volume_last = 3` **identifica** el bloque de 3 frames de ruido al final de la serie, pero NIFTI_NORDIC.m no los promedia: la línea 488 del código asigna `KSP2_NOISE = KSP2(:, :, :, end+1-ARG.noise_volume_last)`, es decir, **selecciona un único volumen** —el primero del bloque de ruido, posición $Q-2$ para $Q = 513$ y $N=3$ — y calcula σ como `std` de sus vóxeles no nulos (línea 525). Los tres frames del bloque se excluyen además del cómputo de `meanphase` (línea 282) —el promedio de fase por slice para el centrado de *k-space*—, de modo que el ruido sin excitación RF no contamina esa estimación. En modo complex, σ se divide adicionalmente por $\sqrt{2}$ (líneas 531–533) para compensar que el ruido se distribuye por igual en partes real e imaginaria; en modo magnitude-only no se aplica este factor.

3. **Descomposición local por parches (*locally low-rank*, LLR).** Se forman matrices de Casorati $Y_{\text{patch}} \in \mathbb{C}^{M \times Q}$ con M = vóxeles del parche, Q = frames temporales. Por defecto NIFTI_NORDIC.m (línea 562) elige el lado del parche como `round((11 * Q)^(1/3))`, manteniendo ratio $M:Q \approx 11:1$ (Moeller et al., 2021). Para $Q = 513$ esto da `round(5643^(1/3)) = round(17.81) = 18`, por lo que los parches son cubos de $18^3 = 5832$ **vóxeles** con ratio efectivo $5832/513 \approx 11.37$. Cada parche se descompone vía SVD.
4. **Umbralización por distribución Marchenko-Pastur con corrección de distribución.** NORDIC no usa el umbral analítico asintótico de Marchenko-Pastur, sino un umbral numérico calibrado por simulación Monte-Carlo sobre matrices $M \times Q$ con σ del paso 2 (Vizioli et al., 2021, Methods §“Numerical evaluation of threshold choice”; Veraart et al., 2016). La corrección de distribución (la “DC” de NORDIC) se refiere a esta calibración no-asintótica, más estricta que el umbral MP asintótico que usa MP-PCA/dwidenoise. Los valores singulares por debajo del umbral se anulan.
5. **Reconstrucción y output.** Los parches denoised se re-insertan con promedio de parches solapados (Vizioli et al., 2021, Methods §“Patch averaging”); se deshace la normalización por *g-factor* y los filtros de fase; se escribe la magnitud (o, si se solicita explícitamente, también la fase). **NIFTI_NORDIC.m conserva los volúmenes de ruido en la salida:** el archivo denoised mantiene $Q = 513$ frames, y el pipeline debe recortar los 3 últimos explícitamente antes de fMRIPrep (la trimación no la realiza la rutina MATLAB).

A diferencia de MP-PCA “magnitude-only” (Veraart et al., 2016; implementado como `dwidenoise` en MRtrix3), NORDIC opera sobre datos complejos y aplica una calibración de umbral adaptada a tamaño finito de matriz, lo que produce mayor reducción de ruido para el mismo nivel de preservación de señal (Dowdle et al., 2023, Fig. 5).

1.4 1.4 Modos NORDIC por adquisición: dos ejes de decisión (fase × ruido)

Un análisis sistemático del dataset reveló que ds006072 no es homogéneo en **dos** ejes ortogonales relevantes para NORDIC, no en uno. El eje de fase (presencia o ausencia del NIfTI `_part-phase_bold.nii.gz`) es visible inspeccionando nombres de archivo y está documentado en [OpenNeuro_directory.txt](#); el eje de ruido (presencia o ausencia del bloque terminal de 3 frames sin excitación RF del protocolo BOLDREST*NORDICME52mm) **sólo se puede verificar abriendo los datos** y midiendo la intensidad cerebral de los últimos frames. Ambos ejes determinan ramas distintas dentro de NIFTI_NORDIC.m y deben seleccionarse por separado:

1. **Eje fase** — determina `ARG.magnitude_only` (0 en complex, 1 en magnitude-only). Decide si la SVD opera sobre una matriz compleja (real + imaginaria, con σ dividida por $\sqrt{2}$) o sobre una matriz real positiva (aproximación Gaussiana a la distribución Riciana).
2. **Eje ruido** — determina `ARG.noise_volume_last` (3 cuando hay noise volumes terminales, 0 cuando no los hay). Decide si σ se mide empíricamente del primer frame del bloque de ruido (L519–525 del código) o si se delega al estimador interno MP-PCA vía normalización por g-factor (L527).

Ambos ejes se verificaron empíricamente con [src/python/survey_noise_volumes_all.py](#), que recorre todos los runs BOLDREST* (solo echo-1 por eficiencia) y calcula por run el ratio entre la intensidad cerebral media de los 3 últimos frames y la del frame central. Un ratio cercano a 0.03–0.05 indica noise volumes auténticos (adquiridos sin pulso RF → caída del ~95–97 % respecto a la señal); un ratio ~1.0 indica que los últimos 3 frames son señal normal, es decir, que **no existen noise volumes** en ese run. El resultado fue categórico:

Sujetos	Fase	Ruido terminal (ratio < 0.2)	Dirección
P1–P3		— ratio 0.95–1.00	AP
P4, P5, P7		— ratio 0.03–0.04	PA
P6		(5/8 runs) / (3/8 truncados; Section 1.4)	PA

Hallazgo metodológico: P1–P3 no incluyen volúmenes de ruido. La hipótesis inicial del pipeline —que el protocolo BOLDREST*NORDICME52mm grababa noise volumes desde el inicio del reclutamiento (Siegel et al., 2024, marzo 2021 – mayo 2023), independientemente de la activación de la reconstrucción de fase— resultó falsa al contrastarse con los datos. Una versión preliminar del chunk, ejecutada con `noise_volume_last = 3` también en P1–P3, produjo **sobre-denoising catastrófico** en el canary sub-P2 ses-16 (tSNR gain 6–13×, `sd_fg_ratio` 0.08–0.17 a través de los 5 ecos): NIFTI_NORDIC.m estimó σ a partir de un frame de señal (std 3217, vs 100–200 en noise frames auténticos de P4), inflando el umbral Marchenko-Pastur ~20× y colapsando la serie temporal al *mean image* (todos los valores singulares salvo el primer modo espacial cayeron bajo el umbral). El diagnóstico completo por frame y la comparación

pre/post se preservan en [src/python/inspect_noise_volume_stats.py](#); los backups `.pre-nordic` de los canarys permitieron revertir el procesado. Este episodio motiva que el pipeline definitivo **detecte la presencia de ruido por-run** en lugar de asumir el protocolo.

Dos ramas de estimación de σ en NIFTI_NORDIC.m (L519–528 del commit pinneado):

```
if ARG.noise_volume_last>0
    %tmp_noise=KSP2(:,:,end+1-ARG.noise_volume_last);
    tmp_noise=KSP2_NOISE;

    tmp_noise(isnan(tmp_noise))=0;
    tmp_noise(isinf(tmp_noise))=0;
    ARG.measured_noise=std(tmp_noise(tmp_noise~=0)); % sqrt(2) for real and complex
else
    ARG.measured_noise=1; % IF COMPLEX DATA
end
```

- **Con `noise_volume_last > 0`** (P4–P7 sanos): σ se estima directamente del primer frame del bloque de ruido tras normalización por g-factor (ver Section 1, paso 2). En modo complex se divide adicionalmente por $\sqrt{2}$ (L531–533) para reflejar la distribución del ruido en partes real/imaginaria. Es el modo de máxima precisión (Vizioli et al., 2021, Methods §“The degree of noise removal”).
- **Con `noise_volume_last = 0`** (P1–P3): el estimador explícito se omite y `measured_noise` se fija a 1. La normalización por g-factor —cuyo mapa se estima internamente por MP-PCA sobre la propia serie 4D (Veraart et al., 2016; Moeller et al., 2021, §2.2)— absorbe el nivel de ruido, dejando el espectro de valores singulares en escala unitaria. El umbral Marchenko-Pastur calibrado numéricamente por NORDIC actúa sobre esa escala sin necesidad de σ medida. Es el modo por defecto de `dwidenoise` (MRtrix3) y el fallback canónico de NIFTI_NORDIC cuando no hay noise scan disponible. Dowdle et al. (2023) lo evalúan explícitamente a 3 T multi-echo y confirman denoising eficaz, aunque con ganancia parcial respecto al modo con noise scan.

Clasificación definitiva ds006072 y parámetros NORDIC por clase:

Clase	Sujetos	Fase	Ruido	<code>magnitude_only</code>	<code>noise_volume_last</code>	Trim tras NORDIC
A	P4–P7 (runs sanos)			0	3	3 frames
B	P1–P3			1	0	0 frames
C	P6 ses-9			—	—	excluidos
	BOLDREST1/BOL- DREST2, ses-5		(abor- ta-			(Section 1.4)
	BOLDREST3		dos/corto)			

Las clases D (fase $\times$ ruido no abortado) y E (fase $\times$ ruido) son teóricamente posibles pero no observadas en ds006072; el chunk las trataría de forma consistente con los ejes por separado.

Expectativa de ganancia tSNR por clase (Dowdle et al., 2023, §3.4; Vizioli et al., 2021, Fig. 5; Chan et al., 2024):

- **Clase A — Complex + noise scan:** $\sim 1.5\text{--}3\times$ en gris cortical, $\sim 1.3\text{--}1.8\times$ en estructuras profundas; inflado de PSF $< 4\%$.
- **Clase B — Magnitude-only + MP-PCA interno:** $\sim 1.2\text{--}1.6\times$ tSNR; inflado de PSF 4–8 %. La magnitud en régimen de bajo SNR sigue Rayleigh (no Gaussiana), introduciendo un sesgo que MP-PCA estima con buena aproximación pero no elimina por completo; el umbral resulta ligeramente conservador y parte de la ganancia se pierde. La pérdida es por construcción, no un fallo; la comparación cuantitativa clase A vs B se reporta como control en Section 1.9.

Consistencia con Siegel et al. (2024). El paper describe la aplicación de NORDIC a todos los sujetos del estudio sin distinguir protocolos. Dado que P1–P3 no tienen noise volumes —lo que imposibilita técnicamente `noise_volume_last > 0`—, la única ruta compatible con esa descripción es `noise_volume_last = 0 + magnitude_only = 1`, que es el camino canónico de NIFTI_NORDIC.m para ese caso. Este pipeline replica esa decisión implícita de Siegel de forma explícita, verificable y trazable run-a-run vía sidecar BIDS.

Nota sobre la VST (Foi 2011): algunas descripciones de NORDIC en la bibliografía secundaria mencionan una *variance-stabilizing transformation* previa al PCA para compensar el sesgo Riciano en magnitud. **La versión actual de NIFTI_NORDIC.m (commit 0861968, mayo 2025) no implementa una VST explícita:** el modo magnitude-only se limita al ajuste de umbral descrito arriba. Registramos este hecho para ser consistentes con el código real que ejecutamos.

Automatización por-adquisición. El chunk detecta ambos ejes por run sin intervención manual:

- **Eje fase** — existencia del archivo `_part-phase_bold.nii.gz` emparejado con el `_part-mag_bold.nii.gz` a procesar (ruta BIDS canónica, O(1) por archivo).
- **Eje ruido** — ratio entre la intensidad cerebral media de los últimos 3 frames y la del frame central del run, calculado vía `img.dataobj` (4 frames en memoria, O(ms) por archivo). Un ratio < 0.2 indica noise volumes presentes; > 0.5 indica ausencia. El rango intermedio 0.2–0.5 (no observado en ds006072) se considera ambiguo: `detect_noise_volumes` detiene el run con un `ValueError` que obliga a inspeccionarlo manualmente antes de decidir la estrategia NORDIC, en lugar de adivinar la rama y arriesgar un sobre-denoising.

El JSON sidecar registra por run ambas decisiones y su trazabilidad:

- `NORDICMode` `{"complex", "magnitude-only"}`

- `NoiseVolumesDetected` {true, false}
- `NoiseLast3RatioToCentral` (float) — diagnóstico reproducible del eje ruido
- `NORDICArgs.noise_volume_last` {0, 3} — parámetro efectivo pasado a `NIFTI_NORDIC`
- `NoiseVolumesTrimmed` {0, 3} — frames eliminados tras el retorno de `NIFTI_NORDIC`
- `NumVolumesBeforeNORDIC`, `NumVolumesAfterNORDIC`

Caveat metodológico. La heterogeneidad de denoising entre clase A (P4–P7: complex + σ explícita) y clase B (P1–P3: mag-only + σ interna MP-PCA) es una fuente de varianza inter-sujeto adicional al diseño. En un estudio *crossover* dentro-sujeto como ds006072 (PSIL vs MTP en cada participante), esta heterogeneidad no contamina el contraste de interés (PSIL – MTP es intra-sujeto y por tanto comparte la clase NORDIC dentro de cada par). Sí podría afectar comparaciones inter-sujeto (por ejemplo magnitud absoluta de FC); por ello §4 reporta la clase NORDIC como covariable de control en los QC plots y PsyConn §2 verifica que los efectos PSIL – MTP no se concentran sistemáticamente en una clase.

1.5 1.5 Normalización del dtype

NORDIC altera un aspecto del header NIFTI que la aserción de Section 1 —“*NORDIC no altera dimensiones, geometría ni metadata*”— no cubre: el `datatype`. El desvío depende de la clase de Section 1.4:

Clase	Sujetos	NORDIC args	Dtype post-NORDIC
A	P4, P5, P7	<code>magnitude_only=0,</code> <code>noise_last=3</code>	<code>uint16</code> (preservado)
A	P6	<code>magnitude_only=0,</code> <code>noise_last=3</code>	<code>int16</code> (preservado)
B	P1, P2, P3	<code>magnitude_only=1,</code> <code>noise_last=0</code>	<code>float32</code> (introducido)

Mecanística. El desvío nace dentro de `NIFTI_NORDIC.m`, no en el wrapper Python: el chunk invoca la rutina con `nout=0` —NORDIC escribe el `.nii.gz` y el chunk se limita a releerlo—, así que no hay ningún round-trip de la matriz por `oct2py` ni por `nibabel` al guardar. En modo complex, la rutina conserva en `info.Datatype` el `uint16/int16` que `niftiinfo` leyó del header de entrada, y el bloque de escritura (L740–748) despacha sobre esa cadena, emitiendo un volumen entero. En modo magnitude-only, la rutina ejecuta `info.Datatype = class(I_M)` (L242) e `I_M` es `single`; con `info.Datatype` así sobrescrito, el despacho de escritura no encuentra `uint16` ni `int16` ni `double` y cae a la rama por defecto `single`, escribiendo `float32`. En ambos modos el volcado pasa por el shim `niftiwrite` → `save_untouch_nii`; lo que cambia entre clases es la cadena `info.Datatype`, no la función de escritura.

Impacto operativo. El cambio de `int16/uint16` (2 B/vóxel) a `float32` (4 B/vóxel) duplica el footprint de RAM de la serie BOLD sin beneficio informacional, ya que el ruido Riciano

residual tras NORDIC tiene $\sim 50\text{--}100$ unidades de intensidad y la precisión sub-entera no aporta señal relevante. En fMRIPrep 25.2.5 multi-echo el nodo `t2smap_node` (wrapper de `tedana.workflows.t2smap_workflow`) carga los 5 ecos simultáneamente para el fit log-lineal de T_2^* y aplica internamente un cast a `float64`: con las dimensiones de ds006072 ($110 \times 110 \times 72 \times 513$ vóxeles) el pico de RAM por nodo escala desde ~ 10 GB (entrada entera $\rightarrow$ `float64` interno) a ~ 20 GB (entrada `float32` $\rightarrow$ `float64` interno), excediendo los 24 GB asignados a WSL 2 (§1, `.wslconfig`) y activando el OOM-killer del kernel Linux (Killed silencioso, sin traceback Python; observado en la ejecución del 24/abr/2026 sobre sub-P1 con `--nprocs 1` y `--me-t2s-fit-method loglin`, confirmando que la causa no era concurrencia sino footprint absoluto del nodo).

Solución in situ. La normalización **no es un paso ni un chunk aparte**: se aplica dentro del propio chunk de NORDIC (§1.8). Como paso final de `apply_nordic()` —tras la ejecución de `NIFTI_NORDIC.m` y antes del trim de los volúmenes de ruido— se invoca `cast_inplace_to_dtype(out_nii, dtype_orig)`, donde `dtype_orig` es el `datatype` del `part-mag` de entrada, capturado en la validación de geometría previa. En clase A el cast es un no-op (el dtype ya se preservó); en clase B revierte el `float32` al entero original. El árbol BIDS post-NORDIC queda así homogéneo en dtype sin un chunk ni una pasada de disco adicionales.

`cast_inplace_to_dtype()` es defensivo e idempotente:

1. **Detección por header, no por clase** — compara `get_data_dtype()` del output con el dtype objetivo y actúa sólo si difieren; un fichero ya en el dtype correcto se salta (re-ejecutar es un no-op).
2. **Cast a entero seguro** — `np.nan_to_num + np.clip` al rango `[info.min, info.max]` + redondeo al entero más cercano. La SVD truncada de NORDIC no introduce negatividad en datos magnitud reales (verificado en P1 ses-11 echo-1: `min = 0.000`) y los máximos observados (`max = 16 631` en P1) quedan holgadamente dentro del rango entero, sin saturación.
3. **Pérdida de precisión nula en términos efectivos** — el redondeo introduce un error máximo de ± 0.5 unidades, ~ 3 órdenes de magnitud inferior a residual NORDIC ($\sim 50\text{--}100$) y 2 órdenes inferior a la fluctuación BOLD típica en gris cortical a 3 T ($\Delta S/S$ 1 % $\rightarrow \sim 20\text{--}30$ unidades sobre una media de $\sim 2\,000$). Es irrelevante para cualquier análisis posterior (T2*-fitting, regresión 24P, correlaciones Schaefer).
4. **Invariancia del resto del header** — el cast no modifica `affine`, `zooms`, `slice timing` ni el número de frames; fija `scl_slope = 1.0` y `scl_inter = 0.0` para que los valores raw coincidan con los físicos, y verifica la escritura reabriendo el fichero (compara dtype, shape y affine). Los sidecars JSON no requieren actualización: el `datatype` no es metadata BIDS-canónica y los readers (`nibabel`, `nilearn`, `fMRIPrep`, `TEDANA`) lo obtienen directamente del header.

1.6 1.6 Implementación: oct2py + Octave + NIFTI_NORDIC.m

La implementación canónica de NORDIC es la rutina MATLAB `NIFTI_NORDIC.m` distribuida por el Center for Magnetic Resonance Research (CMRR, Universidad de Minnesota; Moeller et al., 2021): https://github.com/SteenMoeller/NORDIC_Raw. Es la versión empleada en Vizioli et al. (2021), Dowdle et al. (2023) y Siegel et al. (2024), y la única validada en la bibliografía clínica/metodológica. No existe a fecha de este TFM una versión Python con la misma cobertura de validación; las portaciones comunitarias divergen en el cálculo del g-factor y la selección de tamaño de parche.

Para evitar la dependencia de una licencia MATLAB, se ejecuta `NIFTI_NORDIC.m` desde Python a través de **GNU Octave** (intérprete libre del lenguaje MATLAB) usando el puente **oct2py** (Blink, 2014). `oct2py` serializa arrays NumPy a Octave de forma transparente, permitiendo orquestar todo el flujo (descubrimiento de archivos, idempotencia, escritura de metadatos JSON) en Python. El coste computacional es comparable al de MATLAB nativo: `NIFTI_NORDIC` sobre $5 \text{ ecos} \times 513 \text{ frames} \times 2 \text{ mm}^3$ tarda $\sim 3\text{--}5$ minutos por run en CPU multi-core (los pasos PCA están vectorizados).

Nota sobre la instalación en Windows. `conda-forge` no empaqueta `octave` para `win-64`, por lo que utilizamos el **instalador oficial de Octave para Windows** (<https://octave.org/download>), que entrega exactamente el mismo intérprete GNU Octave que `oct2py` requiere (Blink, 2014). Esta elección preserva la atomicidad del swap NTFS en la misma partición donde reside el dataset ($\sim 100 \text{ GB}$ en `D:/`), que se perdería al cruzar el límite de filesystem en una solución WSL2 o Docker.

Interferencia de antivirus durante la instalación. Durante la preparación de este entorno observamos un modo de fallo no trivial en Windows: Windows Defender, al escanear en tiempo real mientras el instalador de Octave escribe los $>20\,000$ archivos `.m` de su librería estándar, puede **truncar a 0 bytes** algunos de ellos (p. ej. `miscellaneous/ispc.m`, `miscellaneous/version.m`). El síntoma es que `octave-cli --eval "disp(version)"` falla con `error: invalid call to script ... ispc.m` porque el archivo ha perdido su cabecera `function`. El mismo mecanismo corrompió previamente metadatos de paquetes `pip` en el environment (RECORD files leídos como versión `None`). Para evitarlo, antes de instalar Octave se desactivó temporalmente la protección en tiempo real de Windows Defender y se añadieron exclusiones permanentes para `C:\Program Files\GNU Octave\` y la raíz del environment `conda`.

Shims de I/O para Octave. `{#sec-nordic-octave-shims}` `NIFTI_NORDIC.m` invoca las funciones `niftiread`, `niftiinfo` y `niftiwrite` del *Image Processing Toolbox* de MATLAB (L171–178, L229–234, L458, L695, L729, L750), que **no existen en Octave** — es el primer obstáculo al portar la rutina. Para ejecutarla **sin modificar una sola línea** del código CMRR — y así preservar la equivalencia bit a bit con la implementación de referencia citada en Vizioli et al. (2021), Dowdle et al. (2023) y Siegel et al. (2024) — proveemos tres *shims* en `src/octave_shims/` (`niftiread.m`, `niftiinfo.m`, `niftiwrite.m`) que reenvían esas llamadas

a la toolbox *Tools for NIfTI and ANALYZE image* de Jimmy Shen (MATLAB FileExchange #8797, compatible con Octave sin cambios). Los shims conservan el header original (sform, qform, pixdim, intent codes) a través de un campo `.raw` en la struct `info`, y actualizan únicamente la dimensión temporal y el `datatype` del array de salida para reflejar el trimado de los 3 noise volumes. El diseño aprovecha que `NIFTI_NORDIC.m` sólo consume `info.Datatype` como cadena (vía `strmatch`); el resto del struct se pasa opaco a `niftiwrite`, por lo que el round-trip a través del shim es lossless para los campos geométricos.

Compresión gzip. Un detalle adicional: la toolbox de Jimmy Shen en su variante para Octave **no descomprime .nii.gz de forma nativa** — `load_untouch_nii` espera un `gunzip` backed por la JVM de MATLAB que no está disponible en Octave, y falla con `error: Please use MATLAB 7.1 (with java) and above, or run gunzip outside MATLAB`. Los shims resuelven esto transparentemente: **en lectura**, `niftiread/niftiinfo` descomprimen el `.nii.gz` a un `.nii` temporal con la función `gunzip` nativa de Octave (disponible en core desde 3.8), cargan el `.nii` plano y limpian el temporal vía `onCleanup`; **en escritura**, `niftiwrite` guarda un `.nii` plano con `save_untouch_nii`, lo comprime con `gzip` de Octave y elimina el `.nii`. El resultado es equivalente al flujo MATLAB nativo (`niftiwrite(..., 'Compressed', true)`) pero usando únicamente herramientas del ecosistema Octave.

Dependencias (instalar una sola vez):

```
# 0) (Windows) Añadir C:\Program Files\GNU Octave\ y la raíz del env conda
#     como exclusiones en Windows Defender ANTES de instalar Octave.
#     Motivo: el AV puede truncar ficheros durante la escritura del instalador,
#     dejando parte de la librería estándar de Octave corrupta (ispc.m,
#     version.m, ...) y rompiendo el arranque. Ver nota metodológica @sec-nordic-octave-shims

# 1) Instalador oficial de Octave para Windows (~540 MB).
#     Descargar "octave-11.1.0-w64-installer.exe" desde https://octave.org/download
#     (variante "Windows-64 (recommended)"; NO la build "64-bit linear algebra",
#     que sólo aporta ventaja con arrays > 2e9 elementos) y marcar la opción
#     "Add Octave to PATH" durante la instalación.
#     Alternativamente, definir OCTAVE_EXECUTABLE apuntando a octave-cli.exe.

# 2) Puente Python  Octave, instalado en el environment activo.
pip install oct2py

# 3a) Verificación de Octave standalone (debe imprimir la versión).
octave-cli --eval "disp(version)"

# 3b) Verificación del puente oct2py  Octave vía script (evita problemas
#     de codificación UTF-16 de PowerShell con `python -c`).
python -c "from oct2py import Oct2Py; print(Oct2Py().eval('version'))"
```

```
# 4) Clonar la rutina canónica NORDIC_Raw desde el repositorio del CMRR.
git clone https://github.com/SteenMoeller/NORDIC_Raw.git D:/ProfessionalProjects/PsyConn/src/

# 5) Toolbox NIfTI de Jimmy Shen (requerida por los shims de Octave).
#   Descargar "Tools for NIfTI and ANALYZE image" (MATLAB FileExchange #8797):
#   https://www.mathworks.com/matlabcentral/fileexchange/8797
#   Requiere cuenta gratuita de MathWorks. Extraer el ZIP en:
#   D:/ProfessionalProjects/PsyConn/src/external/nifti_tools/
#   de modo que load_untouch_nii.m, save_untouch_nii.m y make_nii.m queden
#   accesibles directamente en esa carpeta (sin subdirectorio intermedio).
#   Verificación: `ls D:/ProfessionalProjects/PsyConn/src/external/nifti_tools/load_untouch_

# 6) Shims propios (niftiread.m, niftiinfo.m, niftiwrite.m).
#   Ya están versionados en D:/ProfessionalProjects/PsyConn/src/octave_shims/.
#   No requieren instalación; el chunk de NORDIC los añade al path de Octave
#   junto a nifti_tools y NORDIC_Raw antes de invocar NIFTI_NORDIC.
```

Al cierre del intérprete, `oct2py` puede emitir un `PermissionError` no fatal al intentar borrar su directorio temporal (`writer.mat` todavía bloqueado por el proceso Octave en terminación). Es cosmético y no afecta al resultado del denoising; se suprime en el chunk principal llamando explícitamente `oc.exit()` antes de que Python termine.

1.7 Estrategia de idempotencia y de seguridad en sobrescritura

NORDIC sobrescribe los archivos `part-mag_bold.nii.gz in-place` (el output es magnitud, como el input), por lo que la ejecución es destructiva. Para minimizar riesgo:

1. **Escritura atómica mediante fichero temporal.** `NIFTI_NORDIC.m` escribe su output a un archivo con sufijo `_NORDIC_tmp.nii.gz` en el mismo directorio. El chunk verifica que ese archivo existe, abre la cabecera, valida que las dimensiones espaciales coinciden con el input y que el número de frames es exactamente el esperado (Q entrada = Q salida, porque `NIFTI_NORDIC.m` no trimma); si el run trae bloque terminal de ruido (detección empírica por ratio, Section 1.4), recorta los 3 últimos frames a un segundo temporal; en caso contrario el archivo temporal se conserva con sus Q frames íntegros; y sólo entonces realiza un `os.replace()` atómico sobre el archivo original.
2. **Backup previo opcional** (`KEEP_BACKUP = True`): antes del `replace`, el archivo original se renombra a `<base>.nii.gz.pre-nordic` para facilitar rollback si se detecta un problema en QC. Por defecto se desactiva (el usuario ya mantiene una copia de seguridad externa del dataset crudo).
3. **Trazabilidad por JSON.** El sidecar `BIDS_part-mag_bold.json` se actualiza con (ejemplo de run P4 complex con bloque de ruido):

```

"DenoisingMethod": "NORDIC",
"NORDICMode": "complex",
"DenoisingSoftware": "NIFTI_NORDIC.m (CMRR, commit 0861968) via oct2py + Octave",
"NORDICArgs": {"noise_volume_last": 3, "temporal_phase": 1, "phase_filter_width": 10, "magnit
"NoiseVolumesDetected": true,
"NoiseLast3RatioToCentral": 0.0343,
"NoiseVolumesUsed": 3,
"NoiseVolumesTrimmed": 3,
"NumVolumesBeforeNORDIC": 513,
"NumVolumesAfterNORDIC": 510

```

Para un run P1–P3 sin bloque de ruido, NORDICMode sería "magnitude-only", NORDICArgs.noise_volume_last y NoiseVolumesUsed serían 0, NoiseVolumesDetected sería false, NoiseLast3RatioToCentral = 0.99, NoiseVolumesTrimmed sería 0 y NumVolumesAfterNORDIC = NumVolumesBeforeNORDIC = 513. Antes de procesar un run el chunk consulta DenoisingMethod == "NORDIC" y, si existe, omite el run. Tras procesar, las imágenes de fase (part-phase_bold.nii.gz y sus JSON) pueden eliminarse para liberar ~100 GB, ya que NORDIC consume la fase pero escribe únicamente magnitud.

1.8 1.8 Chunk de aplicación de NORDIC

```

#=====
# Aplicación de NORDIC PCA a todos los runs BOLD multi-echo de ds006072
# Moeller et al. (2021); Vizioli et al. (2021); Dowdle et al. (2023);
# pipeline original de Siegel et al. (2024).
#
# Contrato con NIFTI_NORDIC.m (commit 0861968):
# - Input: un par (magn, phase) 4D por eco y por run (o solo magn
#           si magnitude_only=1). Si el run trae los 3 volúmenes
#           terminales de ruido (protocolo NORDICME5), se detectan
#           empíricamente y se declaran vía ARG.noise_volume_last=3;
#           si no los trae (P1-P3, o sub-P6 ses-9 abortados), se
#           pasa noise_volume_last=0 y NIFTI_NORDIC estima vía
#           MPPCA interno sobre el propio g-factor (@sec-nordic-magonly).
# - Output: magnitud 4D con el MISMO número de frames que el input
#           (NIFTI_NORDIC NO trimma los volúmenes de ruido).
# - Nombre de output: ARG.DIROUT + fn_out + '.nii' (o '.nii.gz' si
#           ARG.write_gzipped_niftis = 1, default 0).
# - Parámetros fMRI recomendados (docstring de NIFTI_NORDIC.m, L2-8):
#           ARG.temporal_phase = 1

```

```

#     ARG.phase_filter_width = 10
#     (los defaults internos 1/3 son para dMRI y degradarían la
#     reconstrucción de fase en fMRI).
# - ARG.make_complex_nii se evalúa con isfield(), no con su valor →
#   NO añadir la key si queremos output solo magnitud.
# - Dtype del output (@sec-nordic-dtype-cast): en modo complex
#   NIFTI_NORDIC.m conserva el datatype del header original
#   (int16/uint16) y emite un volumen entero. En modo
#   magnitude-only sobrescribe info.Datatype a 'single' (L242)
#   y el despacho de escritura cae a la rama por defecto,
#   emitiendo float32. apply_nordic() llama
#   cast_inplace_to_dtype() tras la ejecución para normalizar
#   siempre al dtype del mag de entrada y así evitar OOM en
#   fMRIPrep t2smap_node.
#
# Estrategia:
# - Un eco por llamada (NIFTI_NORDIC es 4D → mag/fase emparejados
#   por eco); los 5 ecos de un run se procesan secuencialmente.
# - Escritura atómica: NIFTI_NORDIC escribe a un nombre temporal;
#   luego trimmamos los noise volumes (sólo si los hay) y
#   reemplazamos el part-mag original con os.replace (atómico
#   en misma partición).
# - Backup opcional con extensión .pre-nordic para rollback manual.
# - JSON sidecar marcado con DenoisingMethod=NORDIC para idempotencia.
# - Detección automática en dos ejes (@sec-nordic-magonly):
#   * fase : complex existe part-phase emparejado (P4-P7);
#             magnitude-only si no (P1-P3).
#   * ruido : ratio_last3/central < 0.2 → presentes
#             (noise_volume_last=3, trim 3 al final);
#             > 0.5 → ausentes (noise_volume_last=0, sin trim,
#             vía MPPCA interno de NIFTI_NORDIC).
#
# Dependencias: octave, oct2py, nibabel, NORDIC_Raw clonado, toolbox NifTI
# de Jimmy Shen + shims Octave de niftiread/niftiinfo/niftiwrite (ver @sec-nordic-octave-shi
# =====

import os
import re
import json
import glob
import shutil
import subprocess

```

```

import datetime as dt
import functools
import traceback

import numpy as np
import nibabel as nib
from oct2py import Oct2Py

print = functools.partial(print, flush=True)

# -----
# 1. Rutas y parámetros
# -----
# BIDS filtrado por tier (ver PsiloData §6). El chunk `build_tier_dataset`
# de PsiloData copia los runs del tier activo a
# derivatives/ds006072/derivatives/nordic/data/<tier>; NORDIC procesa in-place lo
# que exista ahí, dejando intacto el dataset fuente pristino en F:\.
# Dataset fuente pristino (backup, sin tocar): F:/JIVillL/openneuro/ds006072
TIER = "t0" # ← cambiar a "t1"/"texc" para procesar otro tier
NORDIC_DATA_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/derivatives/nordic/data"
BIDS_DIR = f"{NORDIC_DATA_DIR}/{TIER}"
NORDIC_PATH = "D:/ProfessionalProyectos/PsyConn/src/external/NORDIC_Raw"
NIFTI_TOOLS_PATH = "D:/ProfessionalProyectos/PsyConn/src/external/nifti_tools"
SHIMS_PATH = "D:/ProfessionalProyectos/PsyConn/src/octave_shims"
SUBJECTS = ["sub-P1", "sub-P2", "sub-P3", "sub-P4",
            "sub-P5", "sub-P6", "sub-P7"]
N_NOISE_VOL = 3 # volúmenes de ruido al final (protocolo NORDICME5)
MIN_FRAMES = 50 # sanity: rechazar runs con muy pocos frames
DROP_PHASE = False # eliminar part-phase tras NORDIC para liberar disco
KEEP_BACKUP = False # True → mover el original a <base>.nii.gz.pre-nordic
DRY_RUN = True # True → descubre runs y simula, sin llamar NORDIC
DERIV_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/derivatives/nordic/dry_run"

# Detección empírica de noise volumes por-run (@sec-nordic-magonly):
# ratio = mean(intensidad cerebral, últimos 3 frames) / mean(frame central).
# Validado sobre los 55 BOLDREST* de ds006072 (survey_noise_volumes_all.py):
# runs con ruido caen en 0.03-0.05; runs sin ruido caen en 0.95-1.01; no hay
# distribución intermedia. Los thresholds dejan un margen amplio a ambos lados.
NOISE_RATIO_PRESENT = 0.20 # ratio < thr → noise volumes presentes
NOISE_RATIO_ABSENT = 0.50 # ratio > thr → noise volumes ausentes

# Runs excluidos de NORDIC tras diagnóstico empírico (ver @sec-nordic-magonly).

```



```

# Clave: (subj, ses, task). Valor: motivo (también escrito al log por run saltado).
# Granularidad a nivel de tarea porque dentro de ses-9 coexisten runs íntegros
# (BOLDREST3, BOLDREST4) con runs abortados (BOLDREST1, BOLDREST2).
EXCLUDED_RUNS = {
    ("sub-P6", "ses-9", "BOLDREST1"): (
        "Run abortado prematuramente (208 frames vs 513 nominal). "
        "El esquema de dos ejes (@sec-nordic-magonly) sí permite NORDIC "
        "sin noise block (detect_noise_volumes → noise_volume_last=0, "
        "vía MPPCA interno), así que NORDIC es técnicamente aplicable. "
        "Pero tras censoring por movimiento el run efectivo caerá por "
        "debajo del mínimo aceptable para FC estable (PsiloData §4): "
        "excluido por insuficiencia de datos, no por "
        "imposibilidad técnica."
    ),
    ("sub-P6", "ses-9", "BOLDREST2"): (
        "Run abortado prematuramente (135 frames vs 513 nominal). "
        "Mismo criterio que BOLDREST1: NORDIC aplicable con "
        "noise_volume_last=0, pero el run es demasiado corto para "
        "sustentar análisis de conectividad tras censoring."
    ),
    ("sub-P6", "ses-5", "BOLDREST3"): (
        "Run corto (245 frames vs 513 nominal, ~50 % de longitud). "
        "NORDIC técnicamente aplicable, pero los minutos BOLD efectivos "
        "tras censoring quedan muy por debajo del umbral de estabilidad "
        "individual (PsiloData §4): excluido por "
        "insuficiencia de datos para FC estable."
    ),
}

# Parámetros de NORDIC (docstring NIFTI_NORDIC.m, ejemplo fMRI)
NORDIC_BASE_ARGS = {
    "temporal_phase": 1,
    "phase_filter_width": 10,
    "NORDIC": 1, # threshold basado en empírico (default)
    "MP": 0, # no MPPCA
    "write_gzipped_niftis": 1, # escribir directamente .nii.gz
}

# -----
# 2. Pin de versión: commit hash del clon local de NORDIC_Raw
# -----
try:

```

```

    NORDIC_COMMIT = subprocess.check_output(
        ["git", "-C", NORDIC_PATH, "rev-parse", "--short=7", "HEAD"],
        text=True,
    ).strip()
except (subprocess.CalledProcessError, FileNotFoundError):
    NORDIC_COMMIT = "unknown"
SOFTWARE_TAG = (f"NIFTI_NORDIC.m (CMRR, commit {NORDIC_COMMIT}) "
                f"via oct2py + Octave")
print(f"NORDIC_Raw commit: {NORDIC_COMMIT}")

# -----
# 3. Inicializar Octave + cargar rutina NORDIC
# -----
for _p, _probe in [
    (NIFTI_TOOLS_PATH, "load_untouch_nii.m"),
    (SHIMS_PATH, "niftiread.m"),
    (NORDIC_PATH, "NIFTI_NORDIC.m"),
]:
    if not os.path.exists(os.path.join(_p, _probe)):
        raise FileNotFoundError(
            f"Falta {_probe} en {_p}. Ver @sec-nordic-octave-shims "
            "(pasos 4-6 de la instalación)."
        )

oc = Oct2Py()
# Orden: nifti_tools (backend I/O) → octave_shims (adapta niftiread/
# niftiinfo/niftiwrite a Jimmy Shen) → NORDIC_Raw. addpath añade al
# INICIO del path, por lo que NORDIC_Raw queda resuelto primero y, al
# invocar niftiread/etc, Octave los encuentra en octave_shims antes que
# en cualquier otro sitio.
oc.addpath(NIFTI_TOOLS_PATH)
oc.addpath(SHIMS_PATH)
oc.addpath(NORDIC_PATH)
# NIFTI_NORDIC usa tukeywin (filtro temporal de fase) del paquete signal.
oc.eval("pkg load signal;")
print("Octave inicializado:")
print(f"  NORDIC_Raw:  {NORDIC_PATH}")
print(f"  octave_shims: {SHIMS_PATH}")
print(f"  nifti_tools:  {NIFTI_TOOLS_PATH}")

# -----
# 4. Helpers JSON (actualización in-place preservando formato)

```

```

# -----
def set_json_field(json_path, field_name, field_value):
    """Añade o actualiza un campo en un JSON preservando formato.

    Sustitución regex sobre el texto crudo (mismo patrón que las correcciones
    BIDS de PsiloData §5) para mantener indentación, line endings y precisión.
    """
    with open(json_path, "rb") as f:
        text = f.read().decode("utf-8")

    indent_match = re.search(r'\n([\t]+)', text)
    indent = indent_match.group(1) if indent_match else "\t"
    value_str = json.dumps(field_value)

    if f'"{field_name}"' in text:
        existing = json.loads(text)
        if existing.get(field_name) == field_value:
            return
        pattern = (rf'("{field_name}"\s*:\s*)'
                    rf'(\{[^\{]*\}|\[.*?\]|"[\^"]*"|-\?\d+(?:\.\d+)?|true|false|null)')
        new_text = re.sub(pattern, rf'\g<1>{value_str}', text, count=1, flags=re.DOTALL)
    else:
        new_field = f',\n{indent}"{field_name}": {value_str}'
        new_text = re.sub(r'\s*\}\s*$', f'{new_field}\n}\n', text)

    parsed = json.loads(new_text)
    assert parsed.get(field_name) == field_value

    with open(json_path, "wb") as f:
        f.write(new_text.encode("utf-8"))

def mark_json_nordic(json_path, mode, n_in, n_out, args_used,
                     has_noise, noise_ratio):
    """Escribe los campos NORDIC al sidecar BIDS.

    Los campos `NoiseVolumesDetected` y `NoiseLast3RatioToCentral` registran
    la clasificación empírica (@sec-nordic-magonly); `NoiseVolumesUsed`
    refleja el valor efectivo de `ARG.noise_volume_last` (0 o 3) y
    `NoiseVolumesTrimmed` el número de frames realmente descartados.
    """
    set_json_field(json_path, "DenoisingMethod", "NORDIC")

```

```

set_json_field(json_path, "NORDICMode", mode)
set_json_field(json_path, "DenoisingSoftware", SOFTWARE_TAG)
set_json_field(json_path, "NORDICArgs", args_used)
set_json_field(json_path, "NoiseVolumesDetected", bool(has_noise))
set_json_field(json_path, "NoiseLast3RatioToCentral",
                round(float(noise_ratio), 4))
set_json_field(json_path, "NoiseVolumesUsed",
                int(args_used.get("noise_volume_last", 0)))
set_json_field(json_path, "NoiseVolumesTrimmed",
                int(N_NOISE_VOL if has_noise else 0))
set_json_field(json_path, "NumVolumesBeforeNORDIC", int(n_in))
set_json_field(json_path, "NumVolumesAfterNORDIC", int(n_out))
set_json_field(json_path, "DenoisingDate",
                dt.datetime.now().isoformat(timespec="seconds"))

# -----
# 5. Helpers NIfTI (validación pre/post y trim de volúmenes de ruido)
# -----
def load_shape(nii_path):
    """Lee la cabecera sin cargar los datos."""
    img = nib.load(nii_path)
    return tuple(img.shape), img.header.get_zooms(), img.header.get_data_dtype()

def validate_pair(mag_path, phase_path):
    """Comprueba que mag y phase tienen la misma geometría 4D. Devuelve
    la shape y el dtype del mag (necesario para la normalización de
    dtype post-NORDIC, @sec-nordic-dtype-cast)."""
    ms, mz, mdt = load_shape(mag_path)
    ps, pz, _ = load_shape(phase_path)
    if ms != ps:
        raise ValueError(f"Dims mag phase: {ms} vs {ps}")
    if not np.allclose(mz[:3], pz[:3], rtol=1e-4):
        raise ValueError(f"Voxel size mag phase: {mz} vs {pz}")
    return ms, mdt

def trim_trailing_volumes(nii_path, n_trim):
    """Descarta los últimos n_trim volúmenes de un NIfTI 4D in-place."""
    img = nib.load(nii_path)
    data = img.get_fdata(dtype=np.float32, caching="unchanged")
    if data.ndim != 4 or data.shape[3] <= n_trim:
        raise ValueError(f"4D trim imposible: shape={data.shape}, n_trim={n_trim}")
    trimmed = data[..., :-n_trim]

```

```

new_img = nib.Nifti1Image(trimmed, img.affine, img.header)
new_img.header.set_data_dtype(img.header.get_data_dtype())
# Escribir a un .tmp y renombrar (atómico en la misma partición)
tmp_path = nii_path + ".trim.nii.gz"
nib.save(new_img, tmp_path)
os.replace(tmp_path, nii_path)
return trimmed.shape[3]

```

```
def cast_inplace_to_dtype(nii_path, target_dtype):
```

```

    """Recasta un NIfTI 4D in-place al dtype objetivo preservando header,
    affine y valores físicos. No-op si el fichero ya está en target_dtype.

```

Motivación (@sec-nordic-dtype-cast): NIFTI_NORDIC.m en modo magnitude-only escribe el output en float32 independientemente del dtype de entrada (el round-trip SVD → double → save_nii pierde el datatype original del header). En modo complex (save_untouch_nii) el dtype se preserva. Para homogeneizar el árbol BIDS post-NORDIC al dtype original de Siemens (int16/uint16) y evitar la duplicación del footprint de RAM en fMRIPrep t2smmap_node, se normaliza aquí como paso final del apply_nordic.

Para cast a dtype entero: clip defensivo a [info.min, info.max], nan_to_num, round al entero más cercano. Fija scl_slope=1.0 y scl_inter=0.0 para que los valores raw coincidan con los físicos tras el cast.

```

    """
    img = nib.load(nii_path)
    if img.get_data_dtype() == target_dtype:
        return # idempotente
    data = np.asanyarray(img.dataobj)
    data = np.nan_to_num(data, nan=0.0, posinf=0.0, neginf=0.0)
    if np.issubdtype(target_dtype, np.integer):
        info = np.iinfo(target_dtype)
        vmin, vmax = float(data.min()), float(data.max())
        if vmin < info.min or vmax > info.max:
            raise ValueError(
                f"Valores [{vmin}, {vmax}] fuera de rango {target_dtype} "
                f"({info.min}, {info.max}) en {os.path.basename(nii_path)}"
            )
        data = np.round(np.clip(data, info.min, info.max)).astype(target_dtype)
    else:
        data = data.astype(target_dtype)
    new_hdr = img.header.copy()
    new_hdr.set_data_dtype(target_dtype)

```

```

new_hdr["scl_slope"] = 1.0
new_hdr["scl_inter"] = 0.0
new_img = nib.Nifti1Image(data, img.affine, new_hdr)
tmp_path = nii_path + ".dtype.nii.gz"
nib.save(new_img, tmp_path)
os.replace(tmp_path, nii_path)
# Verificación post-escritura (levanta si algo se desvió)
check = nib.load(nii_path)
assert check.get_data_dtype() == target_dtype
assert check.shape == img.shape
assert np.allclose(check.affine, img.affine)

def detect_noise_volumes(nii_path, n_noise=N_NOISE_VOL,
                        thr_present=NOISE_RATIO_PRESENT,
                        thr_absent=NOISE_RATIO_ABSENT):
    """Detecta empíricamente si los últimos n_noise frames son noise volumes.

    Los volúmenes de ruido NORDIC se adquieren sin pulso RF, por lo que su
    intensidad cerebral media cae 1-2 órdenes de magnitud respecto a la de
    un frame analítico. Comparando el promedio cerebral de los últimos
    n_noise frames con el del frame central obtenemos un ratio bimodal
    (~0.03-0.05 si hay ruido, ~0.95-1.01 si no; ver @sec-nordic-magonly).

    Lee solo 4 frames vía img.dataobj para memoria constante.
    Devuelve (has_noise: bool, ratio: float). Lanza ValueError si el ratio
    cae en el rango ambiguo [thr_present, thr_absent] - un run así debe
    inspeccionarse manualmente antes de decidir la estrategia NORDIC.
    """
    img = nib.load(nii_path)
    if len(img.shape) != 4:
        raise ValueError(f"No 4D: {img.shape}")
    nt = img.shape[3]
    if nt <= n_noise:
        raise ValueError(f"Solo {nt} frames, no alcanza para {n_noise} noise")
    dataobj = img.dataobj
    mid = np.asarray(dataobj[..., nt // 2], dtype=np.float32)
    mid_max = float(mid.max())
    if mid_max <= 0:
        raise ValueError("Frame central vacío (max 0)")
    brain = mid > 0.20 * mid_max
    n_brain = int(brain.sum())
    if n_brain < 1000:

```

```

        raise ValueError(f"Máscara cerebral demasiado pequeña: {n_brain}")
analytic = float(mid[brain].mean())
last_means = [
    float(np.asarray(dataobj[..., nt - k], dtype=np.float32)[brain].mean())
    for k in range(n_noise, 0, -1)
]
ratio = float(np.mean(last_means) / analytic)
if ratio < thr_present:
    return True, ratio
if ratio > thr_absent:
    return False, ratio
raise ValueError(
    f"Ratio ambiguo {ratio:.4f} (ni <{thr_present} ni >{thr_absent}); "
    f"inspeccionar {os.path.basename(nii_path)} manualmente."
)

# -----
# 6. Núcleo: aplicar NORDIC a un par (mag, phase) → mag denoised + trimmed
# -----
def apply_nordic(mag_path, phase_path):
    """Ejecuta NIFTI_NORDIC, valida el output, trimma (si procede) los
    volúmenes de ruido y reemplaza atómicamente el part-mag_bold.nii.gz
    original.

    Decide en dos ejes (@sec-nordic-magonly):
    * fase : complex si existe part-phase; magnitude-only si no.
    * ruido: detect_noise_volumes() sobre el mag de entrada. Si los
        detecta, ARG.noise_volume_last=3 y trim 3 al final;
        si no, ARG.noise_volume_last=0 ( vía MPPCA interno)
        y no se trimma nada.

    Devuelve (mode, n_in, n_out, args_used, has_noise, noise_ratio).
    """
    work_dir = os.path.dirname(mag_path)
    base     = os.path.basename(mag_path)
    assert base.endswith(".nii.gz"), f"Input debe ser .nii.gz: {base}"
    stem     = base[:-len(".nii.gz")]

    # ---- Validación de geometría 4D (pre-ejecución) ----
    use_phase = phase_path is not None and os.path.exists(phase_path)
    mode      = "complex" if use_phase else "magnitude-only"

```

```

if use_phase:
    shape, dtype_orig = validate_pair(mag_path, phase_path)
else:
    shape, _, dtype_orig = load_shape(mag_path)
if len(shape) != 4 or shape[3] < MIN_FRAMES:
    raise ValueError(f"Geometría 4D inválida: {shape}")
n_in = int(shape[3])
if n_in <= N_NOISE_VOL:
    raise ValueError(f"No hay frames analíticos: n_in={n_in} {N_NOISE_VOL}")

# ---- Detección empírica de noise volumes ----
has_noise, noise_ratio = detect_noise_volumes(mag_path)
n_noise_used = N_NOISE_VOL if has_noise else 0
n_expected_out = n_in - n_noise_used

# ---- Struct ARG para NIFTI_NORDIC.m ----
arg = dict(NORDIC_BASE_ARGS)
arg["DIROUT"] = work_dir.replace("\\", "/") + "/"
arg["noise_volume_last"] = n_noise_used # 0 → MPPCA interno; 3 → empírico
arg["magnitude_only"] = 0 if use_phase else 1
# IMPORTANTE: no incluir 'make_complex_nii' - NIFTI_NORDIC comprueba
# isfield() y generaría un NifTI de fase adicional no deseado.

fn_out = f"{stem}_NORDIC_tmp"
out_nii = os.path.join(work_dir, f"{fn_out}.nii.gz")
if os.path.exists(out_nii):
    os.remove(out_nii) # limpiar restos de una ejecución previa fallida

if DRY_RUN:
    print(f"    [dry-run] skipping NIFTI_NORDIC call; "
          f"n_in={n_in}, mode={mode}, "
          f"has_noise={has_noise}, ratio={noise_ratio:.4f}, "
          f"noise_volume_last={n_noise_used}")
    return mode, n_in, n_expected_out, arg, has_noise, noise_ratio

# ---- Llamada a MATLAB ----
if use_phase:
    oc.NIFTI_NORDIC(mag_path, phase_path, fn_out, arg, nout=0)
else:
    # Modo magnitude-only: la firma sigue requiriendo 4 args; pasamos
    # cadena vacía como fn_phase y NIFTI_NORDIC la ignora al detectar
    # magnitude_only=1 (L249-253 de NIFTI_NORDIC.m).

```



```

        oc.NIFTI_NORDIC(mag_path, "", fn_out, arg, nout=0)

    if not os.path.exists(out_nii):
        raise RuntimeError(f"NORDIC ({mode}) no produjo output: {out_nii}")

    # ---- Validación post-ejecución ----
    out_shape, _, out_dtype = load_shape(out_nii)
    if out_shape[:3] != shape[:3] or out_shape[3] != n_in:
        raise RuntimeError(
            f"Shape inesperado tras NORDIC: in={shape} → out={out_shape}"
        )

    # ---- Normalización de dtype (@sec-nordic-dtype-cast) ----
    # NIFTI_NORDIC.m en modo magnitude-only emite float32 por el round-trip
    # SVD → double → save_nii que ignora el datatype del header original.
    # En modo complex (save_untouch_nii) el dtype se preserva y el cast es
    # un no-op. El fix se aplica ANTES del trim para que éste herede el
    # dtype ya normalizado vía set_data_dtype(img.header.get_data_dtype()).
    if out_dtype != dtype_orig:
        cast_inplace_to_dtype(out_nii, dtype_orig)

    # ---- Trim de los volúmenes de ruido (sólo si los hubiera) ----
    if has_noise:
        n_after_trim = trim_trailing_volumes(out_nii, N_NOISE_VOL)
        if n_after_trim != n_expected_out:
            raise RuntimeError(
                f"Trim incorrecto: esperado {n_expected_out}, obtenido {n_after_trim}"
            )

    # ---- Swap atómico: backup opcional + replace ----
    if KEEP_BACKUP:
        backup = mag_path + ".pre-nordic"
        if os.path.exists(backup):
            os.remove(backup)
        os.rename(mag_path, backup)
    os.replace(out_nii, mag_path)

    return mode, n_in, n_expected_out, arg, has_noise, noise_ratio

# -----
# 7. Loop principal: descubrir y procesar todos los runs
# -----

```

```

n_complex      = 0
n_magonly      = 0
skipped        = 0
excluded       = 0
errors        = []
dryrun_records = [] # poblado en el loop, guardado al final si DRY_RUN=True

def session_of(mag_nii: str) -> str:
    """Extrae 'ses-XX' del path BIDS (.../sub-PX/ses-XX/func/...)."""
    parts = mag_nii.replace("\\", "/").split("/")
    for p in parts:
        if p.startswith("ses-"):
            return p

def task_of(mag_nii: str) -> str:
    """Extrae la etiqueta task-<label> del filename BIDS (sin el prefijo task-)."""
    m = re.search(r"task-([A-Za-z0-9]+)", os.path.basename(mag_nii))
    return m.group(1) if m else ""

for subj in SUBJECTS:
    mag_jsons = sorted(glob.glob(os.path.join(
        BIDS_DIR, subj, "ses-*/func",
        "*task-BOLDREST*_part-mag_bold.json"
    )))
    n_phase_subj = len(glob.glob(os.path.join(
        BIDS_DIR, subj, "ses-*/func",
        "*task-BOLDREST*_part-phase_bold.nii.gz"
    )))
    expected_mode = "complex" if n_phase_subj > 0 else "magnitude-only"
    print(f"\n=== {subj} ({len(mag_jsons)} ecos mag, "
          f"{n_phase_subj} fase → modo esperado: {expected_mode}) ===")

    for mag_json in mag_jsons:
        mag_nii = mag_json.replace(".json", ".nii.gz")
        if not os.path.exists(mag_nii):
            errors.append(f" Falta NIfTI: {os.path.basename(mag_nii)}")
            continue

    # ---- Exclusiones por run (scans abortados, sin noise volumes) ----

```

```

ses = session_of(mag_nii)
task = task_of(mag_nii)
if (subj, ses, task) in EXCLUDED_RUNS:
    excluded += 1
    print(f" EXCLUIDO [{subj} {ses} {task}]: {os.path.basename(mag_nii)}")
    continue

# ---- Idempotencia: ¿ya procesado? ----
try:
    with open(mag_json) as f:
        meta = json.load(f)
        if meta.get("DenoisingMethod") == "NORDIC":
            skipped += 1
            continue
except (json.JSONDecodeError, OSError) as e:
    errors.append(f" JSON ilegible: {os.path.basename(mag_json)} ({e})")
    continue

phase_nii = mag_nii.replace("_part-mag_", "_part-phase_")
has_phase = os.path.exists(phase_nii)

try:
    mode_tag = "complex" if has_phase else "magnitude-only"
    print(f" NORDIC [{mode_tag}]: {os.path.basename(mag_nii)}")
    (mode, n_in, n_out, args_used,
     has_noise, noise_ratio) = apply_nordic(
        mag_nii, phase_nii if has_phase else None
    )
    n_trimmed = (n_in - n_out)
    noise_tag = (f"noise-present ratio={noise_ratio:.4f}"
                 if has_noise
                 else f"noise-absent ratio={noise_ratio:.4f} ( vía MPPCA)")
    print(f" {n_in} → {n_out} frames (trimmed {n_trimmed}); {noise_tag}")

    dryrun_records.append({
        "subj": subj, "ses": ses, "task": task,
        "run": re.search(r"(run-\d+)", os.path.basename(mag_nii)).group(1),
        "echo": re.search(r"echo-(\d+)", os.path.basename(mag_nii)).group(1),
        "mode": mode,
        "has_noise": has_noise,
        "noise_ratio": round(noise_ratio, 4),
        "n_in": n_in,
    })

```

```

        "n_out": n_out,
        "n_trimmed": n_trimmed,
        "noise_volume_last": args_used["noise_volume_last"],
        "magnitude_only": args_used["magnitude_only"],
        "mag_path": mag_nii,
    })

    if not DRY_RUN:
        mark_json_nordic(mag_json, mode, n_in, n_out, args_used,
                          has_noise, noise_ratio)
        if DROP_PHASE and has_phase:
            os.remove(phase_nii)
            phase_json = phase_nii.replace(".nii.gz", ".json")
            if os.path.exists(phase_json):
                os.remove(phase_json)

        if mode == "complex":
            n_complex += 1
        else:
            n_magonly += 1
    except Exception:
        errors.append(f" NORDIC falló: {os.path.basename(mag_nii)}")
        traceback.print_exc()

oc.exit()
print(f"\n=== Resumen NORDIC ===")
print(f" Procesados (complex):          {n_complex}")
print(f" Procesados (magnitude-only):      {n_magonly}")
print(f" Ya denoised (skip):                 {skipped}")
print(f" Excluidos (ses. invalidadas):       {excluded}")
print(f" Errores:                           {len(errors)}")
for e in errors:
    print(f"      {e}")
for (subj_x, ses_x, task_x), reason in EXCLUDED_RUNS.items():
    print(f" [excluido] {subj_x}/{ses_x}/{task_x}: {reason}")

if DRY_RUN and dryrun_records:
    import pandas as _pd
    os.makedirs(DERIV_DIR, exist_ok=True)
    dryrun_csv = os.path.join(DERIV_DIR, "dryrun.csv")
    _pd.DataFrame(dryrun_records).to_csv(dryrun_csv, index=False)
    print(f"\n[dry-run] plan guardado en {dryrun_csv} ")

```

```
f"({len(dryrun_records)} filas = runs × ecos)")
```

i Cuándo ejecutar y protocolo de seguridad

NORDIC se ejecuta **después** de la corrección de `B0FieldIdentifier` (PsiloData §5) y **antes** de `fMRIPrep` (§2), por los motivos expuestos en Section 1.1. El chunk sobrescribe los archivos `part-mag_bold.nii.gz` in-place con la magnitud denoised. El número de frames resultante depende del eje de ruido (Section 1.4): *runs* con bloque de ruido detectado (P4–P7 BOLDREST) *quedan con $Q - 3$ frames (el trimado lo realiza el propio chunk, no NIFTI_NORDIC.m)*; *runs sin bloque (todos los BOLDREST de P1–P3 y los abortados/cortos de P6) conservan los Q frames originales.*

Protocolo de seguridad recomendado (dada la irreversibilidad local del paso):

1. **Primera ejecución en modo `DRY_RUN = True`:** valida descubrimiento de archivos, emparejamiento mag/fase, dimensiones 4D y número de frames sin invocar MATLAB ni modificar archivos.
2. **Bulk run sobre todos los sujetos.** `ARG.magnitude_only = 0` y `ARG.magnitude_only = 1` son ramas distintas dentro de `NIFTI_NORDIC.m` (scaling del umbral por `sqrt(2)` y manejo real/imaginaria solo en complex; Section 1.4), por lo que ambas se ejercitan en la misma corrida (P4–P7 complex, P1–P3 magnitude-only). Con `KEEP_BACKUP = True` cada original se renombra a `<base>.nii.gz.pre-nordic` antes del swap, para rollback. La idempotencia por JSON sidecar hace que los scans ya denoised se salten automáticamente en re-ejecuciones.
3. Inspeccionar al menos un run de cada modo con `fsleyes` y QC cuantitativo (Section 1.9): verificar geometría preservada, `NumVolumesAfterNORDIC = NumVolumesBeforeNORDIC - NoiseVolumesTrimmed` en el JSON (p. ej. $513 - 3 = 510$ en runs complex con bloque de ruido; $513 - 0 = 513$ en runs magnitude-only sin bloque), y ganancia de tSNR positiva. **Expectativa diferenciada:** tSNR complex típicamente $\sim 1.5 \times - 3 \times$ en gris cortical (Class A: complex + noise); tSNR magnitude-only $\sim 1.2 \times - 1.8 \times$ (Class B: magnitude-only + no-noise, vía MPPCA interno; ganancia menor por construcción, no es un fallo).
4. Con QC validado, eliminar backups (si se generaron): `find $BIDS_DIR -name "*.pre-nordic" -delete`.

El modo NORDIC se decide **automáticamente por adquisición** según la presencia del `_part-phase_bold.nii.gz` emparejado: P4–P7 reciben NORDIC complex-valued, P1–P3 reciben NORDIC magnitude-only (Section 1.4). La heterogeneidad resultante está tratada como covariable en §4 y PsyConn §2.

Si `DROP_PHASE = True`, las imágenes de fase y sus JSONs se eliminan tras consumirse (libera ~ 100 GB para P4–P7). Dejar `DROP_PHASE = False` si se desea poder re-ejecutar NORDIC con parámetros distintos; en ese caso desmarcar primero el JSON (`DenoisingMethod`) para romper la idempotencia y restaurar el mag desde el backup.

1.9 1.9 QC cuantitativo del bulk run NORDIC

Lanzado NORDIC sobre todo el Tier 0 en `prep/nordic/data/`, el siguiente chunk reproduce las métricas de QC sobre *cada* run denoised del bulk. Dos rasgos de diseño:

1. **Origen del pre-NORDIC:** el bulk NORDIC opera in-place sobre las copias por tier en `derivatives/ds006072/prep/nordic/data/{t0,t1,texc}`, de modo que los NIFTI originales en `F:\JIVilll\openneuro\ds006072` siguen siendo **pristinos** (son directamente el estado pre). El par `pre/post` se resuelve por tanto entre discos: pre desde `F:\`, post desde `D:\`.
2. **Idempotencia por filesystem:** al terminar cada run se escriben dos PNG y una fila en `metrics.csv`. En re-ejecuciones, si las dos PNG ya existen en `bulck-run/sub-X/ses-Y/` el run se salta (sin leer NIFTI). Esto permite añadir tiers posteriores a `prep/nordic/data/` y relanzar el chunk sin recomputar el Tier 0. **No se modifican los JSON sidecar** (a diferencia del marcado idempotente del chunk NORDIC); el registro queda en el propio sistema de archivos de derivatives.

1.9.1 1.9.1 Métricas de QC

1. **Integridad estructural:** misma shape 3D, mismo affine, mismo dtype; recuento de frames `post = pre - NoiseVolumesTrimmed` (donde `NoiseVolumesTrimmed` vale 3 si el run traía bloque de ruido terminal y 0 si no, según Section 1.4).
2. **Integridad del sidecar:** `DenoisingMethod == "NORDIC"`, `NORDICMode` coincide con la rama esperada según presencia de fase, `NoiseVolumesDetected` coincide con la presencia empírica de bloque de ruido, `NumVolumesAfterNORDIC = NumVolumesBeforeNORDIC - NoiseVolumesTrimmed`.
3. **Conservación de la señal:** la media temporal dentro de la máscara cerebral debe variar menos de un 1 % (NORDIC es un filtro de ruido; no debe alterar la señal). Si la media cambia más → bug en swap/trim.
4. **Ganancia de tSNR (mean/std temporal voxel-a-voxel):** mediana y percentil 90 dentro de una máscara foreground derivada por umbral percentil-75 sobre el volumen medio pre (no requiere segmentación). **Expectativa:** $\sim 1.5\times-3\times$ en complex, $\sim 1.2\times-1.8\times$ en magnitude-only.
5. **Reducción de temporal:** mediana de `std(t)` post debe ser claramente $<$ pre en foreground, y comparable en background (NORDIC no debe alterar el fondo).
6. **Residual (pre — post):** desviación típica del residual debe ser aproximadamente uniforme espacialmente. Bandas o parches de 15–20 vóxeles son la firma del tamaño de parche LLR y son esperables; estructura anatómica nítida en el residual (córtex visible) indicaría *over-denoising* (NORDIC se comió señal BOLD).
7. **Espectro de potencia global:** el espectro post debe ser $<$ pre uniformemente en frecuencias altas (ruido térmico blanco), sin nuevos picos ni bandas.

8. **Autocorrelación espacial (FWHM aproximado):** NORDIC no debería ensanchar la FWHM espacial más de un 5–10 %. Si lo hace, sospechar un kernel LLR demasiado grande (no aplicable con los defaults, pero útil como control).

Métrica	Rango esperado (complex)	Rango esperado (mag-only)	Si sale fuera
same_shape3d, same_affine, same_dtype, frames_ok mean_drift	todas True < 0.02 (2 %)	todas True < 0.02 (2 %)	bug grave: revertir el swap desde .pre-nordic y rehacer NORDIC está alterando la señal media → revisar trim y escritura < fg_lo over-denoising (mata BOLD); > 0.95 sin denoising 1 no se removió térmico; <0.10 fondo machacado (artefacto) > hi over-denoising (gain irreal); < lo sin denoising
sd_fg_ratio	0.30–0.95	0.40–0.95	la cola alta de tSNR (GM bien señalado) debe ganar más > 1.15 suavizado espacial (parche LLR demasiado grande)
sd_bg_ratio	0.10–0.95	0.10–0.95	estructura anatómica visible <i>over-denoising</i>
tsnr_gain_med	1.15–4.0	1.05–2.5	nuevos picos artefacto de la implementación
tsnr_gain_p90	tsnr_gain_med	tsnr_gain_med	
fwhm_ratio	0.90–1.15	0.90–1.15	
Mapa (residual)	uniforme, aspecto de ruido	uniforme	
PSD pre vs post	post pre en todas las f; reducción del suelo plano	igual	

Referencias: Vizioli 2021, Moeller 2021 (ganancia tSNR complex ~1.5–3×, mag-only ~1.2–1.8×).

```

# =====
# QC del bulk run NORDIC - panel diagnóstico recorrido sobre todos los runs
# denoised en prep/nordic/data/{t0,t1,texc}.
#
# Entrada:
#   - pre : F:/JIVillL/openneuro/ds006072/sub-X/ses-Y/func/*_part-mag_bold.nii.gz (pristino)
#   - post: prep/nordic/data/{t0,t1,texc}/sub-X/ses-Y/func/*_part-mag_bold.nii.gz (denoised)
#
# Salida:
#   - D:/.../nordic/bulck-run/sub-X/ses-Y/<run>_summary.png
#   - D:/.../nordic/bulck-run/sub-X/ses-Y/<run>_psd.png
#   - D:/.../nordic/bulck-run/metrics.csv
#
# Skip logic:
#   - Si los dos PNG del run ya están en bulck-run/sub-X/ses-Y → SKIP idempotente.
# =====

import os, glob, json, re, gzip, io
from collections import defaultdict
import numpy as np
import pandas as pd
import nibabel as nib
import matplotlib.pyplot as plt
from scipy import ndimage, signal

SRC_BIDS_PRE = "F:/JIVillL/openneuro/ds006072"
# NORDIC denoised data - organizado en tres tandas bajo prep/nordic/data/.
# El QC recorre las tres; cada run resuelve su pre pristino por ruta relativa.
NORDIC_DATA_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/nordic/data"
POST_BIDS_DIRS = [os.path.join(NORDIC_DATA_DIR, b) for b in ("t0", "t1", "texc")]
BULK_QC_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/nordic/bulck-run"
MASK_PCTL = 75
BG_MARGIN = 4
os.makedirs(BULK_QC_DIR, exist_ok=True)

# -----
# Helpers de QC (pre/post: tSNR, residual, PSD, FWHM)
# -----

def load_nifti_any(path):
    if path.endswith(".nii") or path.endswith(".nii.gz"):
        return nib.load(path)
    if ".nii.gz." in path:
        raw = gzip.open(path, "rb").read()

```



```

elif ".nii." in path:
    raw = open(path, "rb").read()
else:
    return nib.load(path)
fh = nib.FileHolder(fileobj=io.BytesIO(raw))
return nib.Nifti1Image.from_file_map({"header": fh, "image": fh})

def foreground_mask(mean3d, pct=MASK_PCTL):
    t = np.percentile(mean3d[mean3d > 0], pct)
    m = mean3d > t
    m = ndimage.binary_opening(m, iterations=2)
    lbl, nl = ndimage.label(m)
    if nl > 1:
        sizes = ndimage.sum(m, lbl, index=np.arange(1, nl + 1))
        m = (lbl == int(np.argmax(sizes)) + 1)
    m = ndimage.binary_fill_holes(m)
    m = ndimage.binary_erosion(m, iterations=2)
    return m

def background_mask(shape3d, margin=BG_MARGIN):
    bg = np.zeros(shape3d, dtype=bool)
    bg[:margin, :, :] = True; bg[-margin:, :, :] = True
    bg[:, :margin, :] = True; bg[:, -margin:, :] = True
    bg[:, :, :margin] = True; bg[:, :, -margin:] = True
    return bg

def fwhm_voxels(vol3d, mask):
    out = []
    for ax in range(3):
        v1 = np.take(vol3d, range(0, vol3d.shape[ax]-1), axis=ax)
        v2 = np.take(vol3d, range(1, vol3d.shape[ax] ), axis=ax)
        m1 = np.take(mask, range(0, mask.shape[ax]-1), axis=ax)
        m2 = np.take(mask, range(1, mask.shape[ax] ), axis=ax)
        m = m1 & m2
        if m.sum() < 100:
            out.append(np.nan); continue
        x, y = v1[m], v2[m]
        x = x - x.mean(); y = y - y.mean()
        rho = (x*y).sum() / np.sqrt((x*x).sum() * (y*y).sum() + 1e-12)
        rho = max(min(rho, 0.9999), 1e-6)
        sigma = 1.0 / np.sqrt(-2.0 * np.log(rho))
        out.append(2.0 * np.sqrt(2.0 * np.log(2.0)) * sigma)

```

```

return out

def qc_pair(pre_path, post_path, tr=1.761):
    """QC pre/post de un eco, parametrizado por paths arbitrarios."""
    pre_img = load_nifti_any(pre_path)
    post_img = load_nifti_any(post_path)
    n_trim = int(pre_img.shape[3] - post_img.shape[3])
    same_shape3d = pre_img.shape[:3] == post_img.shape[:3]
    same_affine = np.allclose(pre_img.affine, post_img.affine, atol=1e-5)
    same_dtype = pre_img.get_data_dtype() == post_img.get_data_dtype()

    # Integridad de frames (§1.9.1): el recuento debe cuadrar con el sidecar
    # NORDIC - post == NumVolumesAfterNORDIC, n_trim == NoiseVolumesTrimmed, y
    # NumVolumesBeforeNORDIC == NumVolumesAfterNORDIC + NoiseVolumesTrimmed.
    js_post = post_path.replace(".nii.gz", ".json")
    try:
        with open(js_post) as _f:
            _meta = json.load(_f)
            _n_before = _meta["NumVolumesBeforeNORDIC"]
            _n_after = _meta["NumVolumesAfterNORDIC"]
            _n_trimmed = _meta["NoiseVolumesTrimmed"]
            frames_ok = (
                _n_before == int(pre_img.shape[3])
                and _n_after == int(post_img.shape[3])
                and _n_trimmed == n_trim
                and _n_before == _n_after + _n_trimmed
            )
    except (OSError, KeyError, json.JSONDecodeError, ValueError, TypeError):
        frames_ok = False

    pre = pre_img.get_fdata(dtype=np.float32)
    post = post_img.get_fdata(dtype=np.float32)
    pre_analytic = pre[..., :pre.shape[-1] - n_trim] if n_trim > 0 else pre
    T = post.shape[-1]
    shape3d = post.shape[:3]
    z_ctr = shape3d[2] // 2

    mu_pre = pre_analytic.mean(-1)
    fg = foreground_mask(mu_pre)
    bg = background_mask(shape3d)

    sd_pre = pre_analytic.std(-1, dtype=np.float32)

```

```

mu_post = post.mean(-1, dtype=np.float32)
sd_post = post.std(-1, dtype=np.float32)
tsnr_pre = np.zeros(shape3d, dtype=np.float32)
tsnr_post = np.zeros(shape3d, dtype=np.float32)
np.divide(mu_pre, sd_pre, out=tsnr_pre, where=(sd_pre > 0))
np.divide(mu_post, sd_post, out=tsnr_post, where=(sd_post > 0))

mean_drift = abs(mu_post[fg].mean() - mu_pre[fg].mean()) / (mu_pre[fg].mean() + 1e-9)
sd_fg_ratio = np.median(sd_post[fg]) / (np.median(sd_pre[fg]) + 1e-9)
sd_bg_ratio = np.median(sd_post[bg]) / (np.median(sd_pre[bg]) + 1e-9)
tsnr_gain_med = np.median(tsnr_post[fg]) / (np.median(tsnr_pre[fg]) + 1e-9)
tsnr_gain_p90 = np.percentile(tsnr_post[fg], 90) / (np.percentile(tsnr_pre[fg], 90) + 1e-9)

fwhm_pre = fwhm_voxels(mu_pre, fg)
fwhm_post = fwhm_voxels(mu_post, fg)
fwhm_ratio = np.nanmean([a/b for a, b in zip(fwhm_post, fwhm_pre) if b > 0])

resid_sd_map = np.zeros(shape3d, dtype=np.float32)
sum_sq_fg = 0.0; n_fg_t = 0
sum_sq_bg = 0.0; n_bg_t = 0
ts_pre_sum = np.zeros(T, dtype=np.float64)
ts_post_sum = np.zeros(T, dtype=np.float64)
n_fg_vox = 0
for z in range(shape3d[2]):
    pr = pre_analytic[:, :, z, :]
    po = post[:, :, z, :]
    rz = pr - po
    resid_sd_map[:, :, z] = rz.std(-1)
    fg_z, bg_z = fg[:, :, z], bg[:, :, z]
    if fg_z.any():
        sum_sq_fg += float((rz[fg_z] ** 2).sum())
        n_fg_t += int(fg_z.sum()) * T
        ts_pre_sum += pr[fg_z].sum(0)
        ts_post_sum += po[fg_z].sum(0)
        n_fg_vox += int(fg_z.sum())
    if bg_z.any():
        sum_sq_bg += float((rz[bg_z] ** 2).sum())
        n_bg_t += int(bg_z.sum()) * T
resid_sd_fg = float(np.sqrt(sum_sq_fg / max(n_fg_t, 1)))
resid_sd_bg = float(np.sqrt(sum_sq_bg / max(n_bg_t, 1)))
ts_pre_mean = ts_pre_sum / max(n_fg_vox, 1)
ts_post_mean = ts_post_sum / max(n_fg_vox, 1)

```

```

nperseg = min(256, T)
f_psd, P_pre = signal.welch(ts_pre_mean, fs=1/tr, nperseg=nperseg)
_, P_post = signal.welch(ts_post_mean, fs=1/tr, nperseg=nperseg)

del pre, post, pre_analytic, sd_pre, sd_post, mu_post

return dict(
    same_shape3d = same_shape3d,
    same_affine = same_affine,
    same_dtype = same_dtype,
    frames_ok = frames_ok,
    n_pre = pre_img.shape[3],
    n_post = post_img.shape[3],
    n_trim = n_trim,
    mean_drift = float(mean_drift),
    sd_fg_ratio = float(sd_fg_ratio),
    sd_bg_ratio = float(sd_bg_ratio),
    tsnr_pre_med = float(np.median(tsnr_pre[fg])),
    tsnr_post_med = float(np.median(tsnr_post[fg])),
    tsnr_gain_med = float(tsnr_gain_med),
    tsnr_gain_p90 = float(tsnr_gain_p90),
    fwhm_ratio = float(fwhm_ratio),
    resid_sd_fg = resid_sd_fg,
    resid_sd_bg = resid_sd_bg,
    _fg_slice = fg[:, :, z_ctr].copy(),
    _tsnr_pre_slice = tsnr_pre[:, :, z_ctr].copy(),
    _tsnr_post_slice = tsnr_post[:, :, z_ctr].copy(),
    _resid_sd_slice = resid_sd_map[:, :, z_ctr].copy(),
    _tsnr_pre_fg = tsnr_pre[fg].astype(np.float32),
    _tsnr_post_fg = tsnr_post[fg].astype(np.float32),
    _psd_f = f_psd,
    _psd_pre = P_pre,
    _psd_post = P_post,
)

def plot_run_summary(subj, ses, task, run, rows, out_dir):
    """Una figura por (subj, ses, task, run) con 1 fila por eco."""
    from copy import copy
    cmap_tsnr = copy(plt.get_cmap("magma")); cmap_tsnr.set_bad("#111111")
    cmap_resid = copy(plt.get_cmap("viridis")); cmap_resid.set_bad("#111111")
    n = len(rows)
    fig, axes = plt.subplots(n, 4, figsize=(16, 3.2 * n),

```

```

        gridspec_kw=dict(width_ratios=[1.1, 1.1, 1.1, 1.1]))
if n == 1: axes = axes[None, :]
for i, (it, q) in enumerate(rows):
    fg_slc = q["_fg_slc"]
    tsnr_pre_slc = np.where(fg_slc, q["_tsnr_pre_slice"], np.nan)
    tsnr_post_slc = np.where(fg_slc, q["_tsnr_post_slice"], np.nan)
    resid_slc = np.where(fg_slc, q["_resid_sd_slice"], np.nan)
    tsnr_pre_fg = q["_tsnr_pre_fg"]
    tsnr_post_fg = q["_tsnr_post_fg"]
    ax = axes[i, 0]
    vmax_pre = np.percentile(tsnr_pre_fg[np.isfinite(tsnr_pre_fg)], 99) if tsnr_pre_fg.size else 1.0
    ax.imshow(tsnr_pre_slc.T, origin="lower", cmap=cmap_tsnr, vmin=0, vmax=vmax_pre)
    ax.set_title(f"echo-{it['echo']} tSNR pre (mediana={q['_tsnr_pre_med']:.1f})")
    ax.set_xticks([]); ax.set_yticks([])
    ax = axes[i, 1]
    vmax_post = np.percentile(tsnr_post_fg[np.isfinite(tsnr_post_fg)], 99) if tsnr_post_fg.size else 1.0
    ax.imshow(tsnr_post_slc.T, origin="lower", cmap=cmap_tsnr, vmin=0, vmax=vmax_post)
    ax.set_title(f"tSNR post (mediana={q['_tsnr_post_med']:.1f}, ganancia×{q['_tsnr_gain_med']:.1f})")
    ax.set_xticks([]); ax.set_yticks([])
    ax = axes[i, 2]
    hp = tsnr_pre_fg[np.isfinite(tsnr_pre_fg)]
    hq = tsnr_post_fg[np.isfinite(tsnr_post_fg)]
    bins = np.linspace(0, np.percentile(np.r_[hp, hq], 99), 60)
    ax.hist(hp, bins=bins, alpha=0.55, label="pre", color="#888")
    ax.hist(hq, bins=bins, alpha=0.55, label="post", color="#c2185b")
    ax.axvline(q["_tsnr_pre_med"], color="#444", linestyle="--", linewidth=1)
    ax.axvline(q["_tsnr_post_med"], color="#c2185b", linestyle="--", linewidth=1)
    ax.set_title("tSNR foreground")
    ax.set_xlabel("tSNR"); ax.legend(loc="upper right", fontsize=8)
    ax = axes[i, 3]
    resid_vals = resid_slc[np.isfinite(resid_slc)]
    vmax_r = np.percentile(resid_vals, 99) if resid_vals.size else 1.0
    ax.imshow(resid_slc.T, origin="lower", cmap=cmap_resid, vmin=0, vmax=vmax_r)
    ax.set_title(f"(residual) fg={q['_resid_sd_fg']:.1f} bg={q['_resid_sd_bg']:.1f}")
    ax.set_xticks([]); ax.set_yticks([])
fig.suptitle(f"QC NORDIC - {subj} {ses} {task} {run} ({rows[0][0]['mode']})",
            y=1.001, fontsize=12)
fig.tight_layout()
out_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_summary.png")
fig.savefig(out_png, dpi=120, bbox_inches="tight")
plt.close(fig)
return out_png

```

```

def plot_run_psd(subj, ses, task, run, rows, out_dir, tr=1.761):
    fig, ax = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
    cmap = plt.cm.viridis(np.linspace(0.1, 0.9, len(rows)))
    for (it, q), c in zip(rows, cmap):
        ax[0].semilogy(q["_psd_f"], q["_psd_pre"], color=c, label=f"echo-{it['echo']}")
        ax[1].semilogy(q["_psd_f"], q["_psd_post"], color=c, label=f"echo-{it['echo']}")
    ax[0].set_title(f"{subj} {ses} {run} - PSD pre"); ax[0].set_xlabel("Hz")
    ax[1].set_title(f"{subj} {ses} {run} - PSD post"); ax[1].set_xlabel("Hz")
    ax[0].legend(fontsize=8, ncol=2); ax[0].set_ylabel("power (log)")
    fig.tight_layout()
    out_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_psd.png")
    fig.savefig(out_png, dpi=120, bbox_inches="tight")
    plt.close(fig)
    return out_png

def verdict(r):
    is_complex = r["mode"] == "complex"
    gain_lo = 1.15 if is_complex else 1.05
    gain_hi = 4.0 if is_complex else 2.5
    fg_lo = 0.30 if is_complex else 0.40
    fg_hi = 0.95
    bg_lo = 0.08 if is_complex else 0.03
    bg_hi = 0.95
    structural_ok = all([r["same_shape3d"], r["same_affine"], r["frames_ok"]])
    if (r["sd_fg_ratio"] < fg_lo) or (r["tsnr_gain_med"] > gain_hi):
        return " OVER-DENOISING"
    if (r["sd_fg_ratio"] > fg_hi) or (r["tsnr_gain_med"] < gain_lo):
        return " SIN DENOISING"
    checks = [
        structural_ok,
        r["mean_drift"] < 0.02,
        fg_lo <= r["sd_fg_ratio"] <= fg_hi,
        bg_lo <= r["sd_bg_ratio"] <= bg_hi,
        gain_lo <= r["tsnr_gain_med"] <= gain_hi,
        0.90 <= r["fwhm_ratio"] <= 1.15,
    ]
    out = " PASS" if all(checks) else " REVISAR"
    if not r["same_dtype"]:
        out += " (dtype inflated)"
    return out

# -----

```

```

# Descubrimiento: agrupa cada _part-mag_bold.nii.gz de prep/nordic/data/ por
# run y localiza su pre pristino en F:/.
# -----
ECHO_RE = re.compile(
    r"(sub-P\d+)_ (ses-[^_]+)_task-([^_]+)_dir-([^_]+)_ (run-\d+)_echo-(\d+)_part-mag_bold\.nii\.gz"
)

def discover_bulk_runs():
    """Devuelve dict key=(subj,ses,task,run) → list[items ordenados por eco].
    Recorre las tres tandas NORDIC (t0, t1, texc): cada run se globbea en su
    tanda y su pre pristino se resuelve por ruta relativa a esa tanda.

    Si un run aparece en más de una tanda, se conserva la primera ocurrencia
    (orden t0 → t1 → texc) y se descarta el duplicado de las tandas
    posteriores -en la práctica, el de texc-. Sin esto los ecos del run se
    acumulaban en la misma clave y duplicaban filas en metrics.csv."""
    runs = defaultdict(list)
    claimed_by = {} # (subj,ses,task,run) → post_dir que lo reclamó primero
    for post_dir in POST_BIDS_DIRS:
        pattern = os.path.join(post_dir, "sub-*", "ses-*", "func",
                                "*_part-mag_bold.nii.gz")
        for post in sorted(glob.glob(pattern)):
            if post.endswith(".pre-nordic"):
                continue
            m = ECHO_RE.search(os.path.basename(post))
            if not m:
                continue
            subj, ses, task, direction, run, echo = m.groups()
            run_key = (subj, ses, task, run)
            # dedup entre tandas: si el run ya lo reclamó una tanda previa
            # (orden t0→t1→texc), este es un duplicado y se descarta.
            if claimed_by.setdefault(run_key, post_dir) != post_dir:
                continue
            rel = os.path.relpath(post, post_dir)
            pre = os.path.join(SRC_BIDS_PRE, rel).replace("\\", "/")
            post = post.replace("\\", "/")
            js_post = post.replace(".nii.gz", ".json")
            if not (os.path.exists(pre) and os.path.exists(js_post)):
                continue
            meta = json.load(open(js_post))
            if meta.get("DenoisingMethod") != "NORDIC":
                continue # run sin NORDIC aplicado → fuera del QC

```

```

        it = dict(
            subj=subj, ses=ses, task=task, run=run, echo=int(echo),
            direction=direction,
            mode=meta.get("NORDICMode", "?"),
            n_pre=meta.get("NumVolumesBeforeNORDIC"),
            n_post=meta.get("NumVolumesAfterNORDIC"),
            pre=pre, post=post, json=js_post,
        )
        runs[run_key].append(it)
# ordena ecos
for k in runs:
    runs[k].sort(key=lambda x: x["echo"])
return runs

# -----
# Loop principal
# -----

runs = discover_bulk_runs()
print(f"[discover] {len(runs)} runs (subj×ses×task×run) en t0+t1+texc")

bulk_rows = []
skipped_idem = []
processed = []

for key, items in sorted(runs.items()):
    subj, ses, task, run = key
    out_dir = os.path.join(BULK_QC_DIR, subj, ses)
    os.makedirs(out_dir, exist_ok=True)
    summary_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_summary.png")
    psd_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_psd.png")

    # (1) Idempotencia por filesystem
    if os.path.exists(summary_png) and os.path.exists(psd_png):
        skipped_idem.append(key)
        continue

    # (2) Procesado real - qc por eco
    print(f"[qc] {subj} {ses} {task} {run} ({len(items)} ecos, "
          f"mode={items[0]['mode']})")
    rows = []
    for it in items:
        try:

```



```

        q = qc_pair(it["pre"], it["post"])
    except Exception as e:
        print(f"      [!] error echo-{it['echo']}: {type(e).__name__}: {e}")
        continue
    rows.append((it, q))
    bulk_rows.append({**{k: v for k, v in it.items() if not k.startswith("_")},
                       **{k: v for k, v in q.items() if not k.startswith("_")}})

if not rows:
    print(f"      [!] sin ecos QCed para {key}, se omite plot")
    continue
plot_run_summary(subj, ses, task, run, rows, out_dir)
plot_run_psd(subj, ses, task, run, rows, out_dir)
processed.append(key)

print(f"\n[summary] procesados={len(processed)} "
      f"idempotent_skipped={len(skipped_idem)}")

# -----
# metrics.csv del bulk run
# -----
df_bulk = pd.DataFrame(bulk_rows)

if not df_bulk.empty:
    df_bulk["verdict"] = df_bulk.apply(verdict, axis=1)
    show_cols = [c for c in ["subj", "ses", "task", "run", "echo", "mode",
                             "n_pre", "n_post", "mean_drift",
                             "sd_fg_ratio", "sd_bg_ratio",
                             "tsnr_pre_med", "tsnr_post_med",
                             "tsnr_gain_med", "tsnr_gain_p90",
                             "fwhm_ratio", "verdict"]
                 if c in df_bulk.columns]
    with pd.option_context("display.float_format", "{:.3f}".format,
                           "display.max_columns", None,
                           "display.width", 220):
        print("\n" + df_bulk[show_cols].to_string(index=False))
    out_csv = os.path.join(BULK_QC_DIR, "metrics.csv")
    df_bulk[[c for c in df_bulk.columns if not c.startswith("_)]]].to_csv(
        out_csv, index=False)
    print(f"\nMétricas guardadas: {out_csv}")
else:
    print("[!] el bulk no produjo filas; no se escribe metrics.csv")

```

2 2 Preprocesamiento fMRI (fMRIPrep)

fMRIPrep es una aplicación de [NiPreps](#) (NeuroImaging PREProcessing toolS) diseñada para el preprocesamiento de datos fMRI tanto de tarea como de estado de reposo, incluyendo adquisiciones multi-eco (Esteban et al., 2019, 2020). El pipeline realiza un *preprocesamiento mínimo*: corrección de movimiento, corrección de distorsión de campo (*field unwarping*), normalización espacial, corrección de campo de sesgo (*bias field correction*) y extracción cerebral (*skull-stripping*). Internamente combina herramientas de FSL, ANTs, FreeSurfer y AFNI, seleccionando automáticamente la configuración óptima según los metadatos BIDS del dataset de entrada.

El flujo de trabajo comprende dos bloques principales (Esteban et al., 2019, 2020):

1. **Preprocesamiento anatómico (T1w)**: conformado de imágenes T1w, extracción cerebral (`antsBrainExtraction.sh` de ANTs), segmentación de tejido (FSL `fast`), reconstrucción de superficies corticales (FreeSurfer `recon-all`) y normalización espacial no lineal al template estándar (`antsRegistration`).
2. **Preprocesamiento funcional (BOLD)**: estimación de imagen de referencia BOLD, corrección de movimiento de cabeza (FSL `mcflirt`), corrección de *slice-timing* (AFNI `3dTshift`, si `SliceTiming` está disponible en los metadatos), corrección de distorsión por susceptibilidad (SDCFlows), corregistro EPI→T1w (*boundary-based registration* vía FreeSurfer `bbregister`), remuestreo al espacio estándar mediante concatenación de todas las transformaciones en una sola interpolación (kernel Lanczos), y estimación de confounds (señal global media, señales de tejido, `tCompCor`, `aCompCor`, *framewise displacement*, 6 parámetros de movimiento, DVARS y *spike regressors*). Para datos **multi-eco**, fMRIPrep estima los parámetros de movimiento a partir de un único eco y aplica las mismas transformaciones a todos los ecos, preservando la relación de amplitudes entre TEs (Mao et al., 2024). Adicionalmente, el workflow `init_bold_t2s_wf()` genera un mapa T2* adaptativo y una combinación óptima ponderada por T2* de todos los ecos, utilizando internamente la rutina `t2smap_workflow` de tedana (Kundu et al., 2012). Esta serie combinada óptimamente es la que continúa a través de los pasos de corregistro y remuestreo subsiguientes.

fMRIPrep **no** realiza suavizado espacial ni denoising ICA, para mantenerse agnóstico respecto al análisis posterior (Esteban et al., 2019). Estos pasos quedan a cargo del investigador y deben aplicarse **después** de fMRIPrep, sobre las imágenes preprocesadas en espacio estándar. En el caso multi-eco, fMRIPrep realiza la combinación óptima T2* pero **no** ejecuta la descomposición ICA ni la clasificación TE-dependiente de componentes; el denoising multi-eco completo requiere una herramienta externa como TEDANA (§3).

2.1 2.1 Requisitos previos

2.1.1 2.1.1 Instalación de Docker y NiPreps

El pipeline de neuroimagen se ejecuta dentro de contenedores Docker, lo que garantiza la reproducibilidad al encapsular todas las dependencias (FSL, ANTs, FreeSurfer, AFNI) en imágenes preconfiguradas. Cada herramienta de la familia [NiPreps](#) (NeuroImaging PREProcessing toolS) se distribuye como una imagen Docker independiente; es decir, instalar fMRIPrep **no** instala automáticamente fMRIPost-AROMA ni otras herramientas NiPreps.

2.1.2 2.1.2 Docker Desktop

Instalar [Docker Desktop](#) para Windows y verificar que el servicio esté activo:

```
docker info
```

En Windows, es necesario habilitar las *Shared Drives* desde la configuración de Docker Desktop (Settings → Resources → File Sharing) para que los contenedores puedan acceder a los directorios de datos del proyecto mediante montajes `-v`.

2.1.3 2.1.3 Configuración de recursos WSL 2

Docker Desktop con backend WSL 2 hereda los límites de memoria y CPU de la máquina virtual WSL. Por defecto, WSL asigna el 50% de la RAM del sistema, lo que puede ser insuficiente para fMRIPrep (especialmente con FreeSurfer `recon-all`, normalización ANTs y la estimación T2* multi-echo). Para ajustar estos límites, crear o editar el archivo `%UserProfile%\wslconfig`:

```
[wsl2]
memory=24GB
swap=16GB
swapFile=E:\\wsl_swap.vhdx
processors=10
```

- `memory=24GB`: asigna 24 GB de los 32 GB físicos a WSL, reservando ~8 GB para Windows.
- `swap=16GB`: añade 16 GB de *swap* como colchón para picos transitorios (nodos T2SMap con `curvefit`, `antsRegistration`). Evita que el *OOM killer* del kernel Linux mate procesos cuando la memoria residente sobrepasa momentáneamente el límite físico asignado.
- `swapFile=E:\\wsl_swap.vhdx`: ubica el archivo de swap en la unidad E: para no consumir espacio en C:. Por defecto WSL lo coloca en `%USERPROFILE%\\AppData\\Local\\Temp`, que vive en C:.

- `processors=10`: limita WSL a 10 de los 12 *threads* del i5-10400F, reservando 2 para Windows e impidiendo saturación del sistema operativo durante ejecuciones intensivas.

Tras modificar `.wslconfig`, reiniciar WSL desde PowerShell para aplicar los cambios:

```
wsl --shutdown
```

Verificar la asignación dentro de WSL con `wsl free -h`.

2.1.4 2.1.4 Imágenes Docker de NiPreps

Descargamos las imágenes de las herramientas utilizadas en este pipeline:

```
# fMRIPrep - preprocesamiento mínimo de fMRI
docker pull nipreps/fmriprep:25.2.5
```

Verificamos que la imagen esté disponible localmente:

```
docker images
```

2.1.5 2.1.5 Licencia de FreeSurfer

fMRIPrep requiere una licencia gratuita de FreeSurfer para la reconstrucción de superficies corticales. Obtenerla en <https://surfer.nmr.mgh.harvard.edu/registration.html> y guardarla como `license.txt` en la carpeta raíz del proyecto (D:\ProfessionalProyects\PsyConn\license.txt).

2.2 2.2 Ejecución de fMRIPrep para ds006072 (multi-echo)

i Inputs ya denoised por NORDIC

A partir de aquí, los archivos `part-mag_bold.nii.gz` que fMRIPrep encuentra en el árbol BIDS han sido **sobreescritos in-place por NORDIC** (Section 1) y su `datatype` ha sido posteriormente **normalizado a uint16** en los sujetos de clase B (P1–P3) por Section 1.5. NORDIC opera en el dominio espacio-temporal sin alterar dimensiones, geometría ni sidecars BIDS; el desvío de `datatype` que NIFTI_NORDIC.m introduce en la rama `magnitude-only` se revierte explícitamente en Section 1.5 como paso final del preprocesamiento NORDIC. Con esa normalización aplicada, **el comando de fMRIPrep no requiere ninguna modificación**: el árbol BIDS sigue siendo válido y `SDCFlows / init_bold_t2s_wf()` operan exactamente igual, con un footprint de RAM en

`t2smap_node` equivalente al del dataset raw. Omitir Section 1.5 activa el OOM-killer del kernel sobre P1–P3 (Killed silencioso tras cargar los 5 ecos en `t2smap_node`) — incidente reproducido empíricamente el 24/abr/2026. El número de frames por serie depende del eje de ruido del run (Section 1.4): $Q - 3$ (típicamente 510) para runs con bloque de ruido terminal detectado (P4–P7 BOLDREST); Q íntegro (típicamente 513) para runs sin bloque (P1–P3 BOLDREST y runs abortados/cortos de P6), donde NIFTI_NORDIC estimó por MPPCA interno sin consumir los 3 últimos frames. El JSON sidecar de cada eco lleva ahora los campos `DenoisingMethod`: "NORDIC", `NoiseVolumesDetected` y `NoiseVolumesTrimmed` como trazabilidad; fMRIPrep ignora estos campos (no son BIDS-canónicos) sin error. Si NORDIC no se ejecutó, fMRIPrep procesa los Q frames raw sin atenuación de ruido térmico — pipeline subóptimo pero técnicamente válido.

El dataset ds006072 contiene datos BOLD multi-echo (5 ecos), lo que requiere una configuración específica de fMRIPrep. Cuando fMRIPrep detecta datos multi-echo a través de los metadatos BIDS, automáticamente activa el workflow `init_bold_t2s_wf()` que genera un mapa $T2^*$ adaptativo y la combinación óptima ponderada por $T2^*$ de todos los ecos (Kundu et al., 2012). La estimación de $T2^*$ y S se realiza por defecto mediante regresión no lineal (`--me-t2s-fit-method curvefit`). La opción `--me-output-echos` instruye adicionalmente a fMRIPrep para que guarde los ecos individuales preprocesados — corregidos por *slice-timing*, movimiento y distorsión de susceptibilidad, y remuestreados al espacio estándar — además de la serie *optimally combined* (DuPre et al., 2021). Estos ecos individuales preprocesados son el insumo necesario para el denoising posterior con TEDANA (§3).

El *workdir* temporal de fMRIPrep (intermedios de *nipype*, resampling por eco, cache de nodos) es el cuello de botella de I/O del pipeline: `recon-all` de FreeSurfer y `antsRegistration` abren y cierran miles de ficheros pequeños, de modo que **importa más la latencia de acceso aleatorio que el ancho de banda secuencial**. En el Tier 0 (1 run por sesión, PsiloData §6) el *workdir* por sujeto se mantiene en ~30–50 GB, lo que hace viable alojarlo en C: (NVMe) a pesar de ser el disco del sistema. La disposición óptima para maximizar velocidad es entonces: Docker Desktop y el *workdir* de fMRIPrep en **C: (NVMe, 241 GB libres)**; el *swapfile* de WSL 2 en **E:** (§2.1.3); y tanto el BIDS de entrada (post-NORDIC, `derivatives/ds006072/prep/nordic/data/<tier>`) como la salida (`derivatives/ds006072/prep/fmriprep/data/<tier>`) en **D: (HDD)** — las lecturas del input son mayoritariamente secuenciales y las escrituras de salida son chunky al final de cada sujeto, así que el HDD basta para ambas. Si en tiers posteriores se escala a varios runs activos por sujeto y el *workdir* se aproxima a 100 GB, basta con repuntar `-v` hacia otra unidad con más holgura. Si se omite el *bind mount*, fMRIPrep escribe en `/tmp/work` dentro del contenedor, consumiendo el VHDX de WSL 2 hasta llenar C: y provocar fallos silenciosos.

Crear el directorio host antes de la primera ejecución:

```
mkdir C:\DockerWork\fmriprep
```

Ejecutar fMRIPrep

```
docker run -ti --rm `
-v "D:\ProfessionalProyectos\PsyConn\derivatives\ds006072\prep\nordic\data\t0:/data:ro" `
-v "D:\ProfessionalProyectos\PsyConn\derivatives\ds006072\prep\fmripred\data\t0:/out" `
-v "C:\DockerWork\fmripred:/work" `
-v "D:\ProfessionalProyectos\PsyConn\license.txt:/opt/freesurfer/license.txt:ro" `
nipreps/fmripred:25.2.5 `
/data /out participant `
--work-dir /work `
--participant-label P1 `
--output-spaces MNI152NLin2009cAsym `
--me-output-echos `
--me-t2s-fit-method loglin `
--fs-license-file /opt/freesurfer/license.txt `
--nprocs 3 --omp-nthreads 2 --mem-mb 20000 `
--stop-on-first-crash
```

Eliminar el directorio host entre ejecuciones de distintos sujetos:

```
Remove-Item -Recurse -Force C:\DockerWork\fmripred\*
```

2.2.1 Explicación de los argumentos

- `-v "D:\...\prep\nordic\data\t0:/data:ro"`: monta el BIDS filtrado por tier —el output denoised de NORDIC, `derivatives/ds006072/prep/nordic/data/<tier>` (Psilo-Data §6)— en modo lectura. fMRIPrep lee cada BOLD/T1w entero por nodo (acceso secuencial), así que el HDD basta y libera la NVMe para el *workdir*.
- `-v "D:\...\prep\fmripred\data\t0:/out"`: monta el directorio de salida donde fMRIPrep escribe los derivados preprocesados y los reportes HTML de calidad. Se aloja bajo `derivatives/ds006072/prep/fmripred/data/<tier>`, en paralelo a `prep/nordic/data/` y separado de los derivados downstream (`tedana/`, `connectivity_*/`). Las escrituras son chunky y se producen al final de cada sujeto, por lo que el HDD es aceptable.
- `-v "C:\DockerWork\fmripred:/work"`: monta el *workdir* temporal en C: (NVMe, 241 GB libres). Permite el *caching* de *nipype* entre reintentos: si una ejecución falla, los nodos ya completados se marcan como cacheados y no se repiten en la siguiente pasada (ahorro de horas en `recon-all` y `antsRegistration`). La NVMe acelera notablemente las fases dominadas por I/O aleatorio de ficheros pequeños (FreeSurfer, ANTs); tras cada sujeto conviene limpiar el contenido del *workdir* para evitar que crezca linealmente.

- `-v "...\\license.txt:/opt/freesurfer/license.txt:ro"`: pasa la licencia de FreeSurfer al contenedor (requerida para la reconstrucción de superficies corticales).
- `nipreps/fmriprep:25.2.5`: imagen Docker de fMRIPrep, versión estable 25.2.5.
- `/data /out participant`: argumentos posicionales BIDS-Apps — directorio de entrada, directorio de salida y nivel de análisis (`participant`).
- `--work-dir /work`: ruta dentro del contenedor donde *nipype* escribe los archivos intermedios; se corresponde con el *bind mount* `C:\\DockerWork\\fmriprep`.
- `--participant-label P1`: identificador del sujeto a procesar (sin el prefijo `sub-`). Se procesa **un sujeto por ejecución** para acotar el tamaño del *workdir* y facilitar la inspección de los reportes antes de continuar con el siguiente.
- `--output-spaces MNI152Nlin2009cAsym`: espacios estándar de salida. `MNI152Nlin2009cAsym` es el template MNI asimétrico de 2009c, utilizado para el análisis de conectividad con atlas Schaefer.
- `--fs-license-file /opt/freesurfer/license.txt`: ruta explícita a la licencia de FreeSurfer dentro del contenedor. Aunque fMRIPrep detecta la licencia automáticamente si se monta en la ruta canónica, especificarla de forma explícita evita ambigüedades.
- `--nprocs 3`: número máximo de *workers* concurrentes de *nipype*. En un i5-10400F (6 cores / 12 threads) con WSL limitado a 10 processors, $3\text{ workers} \times 2\text{ hilos OMP} = 6$ hilos simultáneos; deja margen para el resto del sistema y, crucialmente, **limita a 3 el número de nodos T2SMap que pueden ejecutarse en paralelo**. El default de fMRIPrep (igual al número de CPUs) provoca fallos *Out-Of-Memory* en adquisiciones multi-echo: con 5 ecos y *curvefit*, cada T2SMap alcanza picos de ~8–10 GB; más de 2–3 nodos en paralelo exceden los 24 GB de WSL y el *OOM killer* mata los procesos.
- `--omp-nthreads 2`: número máximo de hilos OpenMP por proceso individual (ANTs, FreeSurfer). Reducido desde 4 para evitar sobre-suscripción de CPU.
- `--mem-mb 20000`: memoria máxima que el *scheduler* de *nipype* puede consumir (20 GB), dejando 4 GB de colchón dentro de los 24 GB asignados a WSL. Es el valor declarativo que *nipype* usa para decidir cuándo agendar nodos memory-hungry (T2SMap, *antsRegistration*); un valor más bajo serializa las tareas pesadas, un valor más alto las solapa y arriesga OOM.
- `--stop-on-first-crash`: detiene el pipeline al primer fallo en lugar de continuar y fallar silenciosamente en nodos dependientes. En datasets multi-echo permite detectar rápido un problema de memoria en T2SMap (que ocurre en los primeros minutos del procesamiento funcional) en lugar de esperar horas hasta que fallen todos los BOLD runs.

- `--me-output-echos`: guarda cada eco preprocesado por separado (`*_echo-N_part-mag_desc-preproc_bold.nii.gz`, en espacio BOLD nativo), además de la serie *optimally combined* (`*_space-MNI152NLin2009cAsym_desc-preproc_bold.nii.gz`). Estos ecos individuales alimentan TEDANA en §3.
- `--me-t2s-fit-method loglin`: método de estimación de los parámetros T_2^* y S_0 del modelo de decaimiento monoexponencial $S(TE) = S_0 \cdot e^{-TE/T_2^*}$. El algoritmo `loglin` ajusta una regresión lineal sobre el logaritmo de las intensidades ($\log S(TE) = \log S_0 - TE/T_2^*$) mediante una solución cerrada de mínimos cuadrados; el alternativo `curvefit` (default de fMRIPrep) ajusta directamente el modelo exponencial por mínimos cuadrados no-lineales (algoritmo Levenberg-Marquardt de `scipy.optimize.curve_fit`) de forma iterativa.

Justificación de la elección `loglin` para la estimación de T_2^* en fMRIPrep, adoptada como parte de un *split* estratégico coordinado con la estimación no-lineal ejecutada en TEDANA (véase §3):

1. **Uso downstream del T_2^* map de fMRIPrep: exclusivamente para la combinación óptima.** El único destino del mapa T_2^* calculado por fMRIPrep es ponderar los ecos en la serie *optimally combined* (`desc-preproc_bold.nii.gz`) mediante los pesos $w_k \propto TE_k \cdot e^{-TE_k/T_2^*}$ (Posse et al., 1999; Poser et al., 2006; Kundu et al., 2012). Esta combinación es **robusta frente a errores moderados en T_2^*** : los pesos varían suavemente con T_2^* y cualquier sesgo vóxel a vóxel queda atenuado tras la suma ponderada sobre los cinco ecos. En tejido cortical con SNR moderado a alto (Siemens Prisma 3T, TEs balanceados 14.2–113.12 ms) los mapas obtenidos por `loglin` y `curvefit` son **numéricamente indistinguibles** para este propósito (DuPre et al., 2021). Sobre parcelas Schaefer de ~1000 vóxeles con correlaciones de Pearson entre series temporales (Schaefer et al., 2018), las microvariaciones voxelwise quedan por debajo del ruido térmico y del jitter BOLD intrínseco.
2. **Split estratégico con TEDANA:** la precisión no-lineal del ajuste de T_2^* es **crítica** no en la combinación óptima, sino en la clasificación / de componentes ICA, paso ejecutado por TEDANA en §3. TEDANA **no reutiliza** el T_2^* map de fMRIPrep: lo recalcula internamente a partir de los ecos preprocesados (DuPre et al., 2021). Se adopta por tanto un *split* deliberado — `loglin` en fMRIPrep (rápido, suficiente para combinación óptima) y `curvefit` en TEDANA (no-lineal, donde la bondad del ajuste de T_2^* gobierna directamente las estadísticas pseudo-F (TE-dependencia) y (TE-independencia) que clasifican cada componente ICA como BOLD o non-BOLD; Kundu et al., 2012, 2013). Este patrón de delegación del cómputo no-lineal a la etapa donde domina el resultado es consistente con pipelines multi-eco Precision Functional Mapping (Lynch et al., 2020) y con la divergencia recomendada por la documentación de TEDANA para estudios en los que el *denoising* ICA es el eje del pipeline (DuPre et al., 2021).

3. **Coste computacional significativamente menor:** `curvefit` requiere aproximadamente **3–5× más memoria y tiempo** que `loglin` por nodo T2SMap (pico ~8–10 GB vs ~3–4 GB con 5 ecos y una adquisición *resting-state* de la duración del dataset). En configuraciones con restricciones de memoria (WSL 2 con 24 GB asignados y 32 GB físicos), `curvefit` en fMRIPrep provoca fallos recurrentes del *OOM killer* del kernel cuando varios nodos T2SMap se ejecutan en paralelo — comportamiento observado empíricamente en este entorno en ejecuciones previas. El paso a `loglin` en fMRIPrep elimina este riesgo y concentra el cómputo no-lineal en una única etapa (TEDANA), donde sí aporta valor metodológico.
 4. **Validez estadística del modelo log-lineal para la combinación óptima:** aunque la transformación logarítmica introduce heteroscedasticidad (los ecos con TE largo tienen SNR menor y por tanto varianza logarítmica mayor), esto solo sesga ligeramente la estimación puntual de T_2^* en vóxeles con S_0 bajo — precisamente los vóxeles que la combinación óptima pondera menos en la señal final, atenuando el impacto del sesgo (Kundu et al., 2012; DuPre et al., 2021).
- **Nota sobre corregistro multi-echo:** en datos multi-echo fMRIPrep utiliza la serie *optimally combined* (salida de `init_bold_t2s_wf()`) para el corregistro EPI→T1w vía `bbregister`, aprovechando el mayor SNR de la combinación T2*-ponderada respecto a cualquier eco individual (Posse et al., 1999; Kundu et al., 2012).

2.2.2 2.2.2 Reanudación tras fallo

El *workdir* C:\DockerWork\fmriprep y el directorio de salida D:\ProfessionalProyectos\PsyConn\derivative **no deben borrarse entre ejecuciones** del mismo sujeto. *nipype* detecta automáticamente los nodos ya completados (por hash de entradas y parámetros) y los reusa; fMRIPrep a su vez reutiliza los derivados anatómicos existentes (T1w, FreeSurfer) sin rehacer `recon-all`. Una vez completado con éxito el procesamiento de un sujeto, el *workdir* puede borrarse manualmente para liberar espacio antes de pasar al siguiente: `Remove-Item -Recurse -Force C:\DockerWork\fmriprep\*`.

2.2.3 2.2.3 Outputs esperados

fMRIPrep conforma sus salidas a la especificación **BIDS Derivatives** (Esteban et al., 2019, 2020) y produce tres familias de resultados: (i) reportes visuales de QA en HTML, (ii) derivados preprocesados (anatómicos y funcionales, incluyendo las transformaciones espaciales), y (iii) series temporales de *confounds* para el denoising downstream. Es importante subrayar que fMRIPrep **no aplica denoising por sí mismo** —ni GSR, ni regresión de movimiento, ni suavizado espacial— con la única excepción del filtrado temporal de alto paso aplicado antes del cómputo de los regresores CompCor (§2.2 y §3). Esta agnosticidad es deliberada: deja

en manos del investigador la elección del modelo de ruido, que en este pipeline se delega a TEDANA (§3) más una regresión residual 24P con *scrubbing*.

Con `--output-layout bids` (default desde v21.0), la estructura de `D:\ProfessionalProjects\PsyConn\derivatives` es:

```
derivatives/ds006072/prep/fmrip/derivatives/
dataset_description.json      # metadatos BIDS de los derivados
.bidsignore                  # exclusiones para el BIDS validator
logs/                        # boilerplate de citación (CC0)
sourcedata/
  freesurfer/                # subjects dir de FreeSurfer (recon-all)
    fsaverage{,5,6}/
    sub-P1_ses-1/            # FreeSurfer nombra el subject por la sesión anatómica
      mri/ surf/ label/ stats/
sub-P1.html                  # reporte visual de QA
sub-P1/
  anat/                      # derivados estructurales
  ses-6/func/                # derivados funcionales (sesión MTP)
  ses-11/func/               # derivados funcionales (sesión PSIL)
```

2.2.3.1 Reportes HTML (sub-<label>.html)

Un HTML autocontenido por sujeto, con reportlets SVG embebidos generados por `niworkflows`. Agrupa, en secciones navegables, la inspección visual de todas las etapas críticas (Esteban et al., 2019):

- **Procesamiento anatómico:** overlay de la máscara cerebral sobre el T1w INU-correcto, segmentación de tejidos (GM/WM/CSF mediante FAST de FSL) y normalización T1w→MNI152NLin2009cAsym visualizada como mosaico animado.
- **Superficies FreeSurfer:** contornos *pial*, *white* y *midthickness* superpuestos al T1w.
- **Corrección de distorsión por susceptibilidad (SDC):** comparación pre/post cuando existe fieldmap o syn-sdc.
- **Corregistro EPI→T1w:** *flicker* entre la BOLD de referencia y el T1w, ejecutado por `bbregister` (*boundary-based registration*) sobre la serie *optimally combined* en datos multi-echo (Posse et al., 1999; Kundu et al., 2012).
- **Carpetplot y confounds** (Power, 2017; Provins et al., 2022): panel superior con GS, CSF, WM, DVARS y FD; panel inferior con heatmap vóxel×tiempo segmentado por compartimento (cortical GM, deep GM, WM+CSF, cerebelo, *crown* o borde cerebral). Permite detectar artefactos globales y contaminación por ruido fisiológico/movimiento de un vistazo.

- **Varianza explicada por CompCor:** cuatro curvas acumuladas (aCompCor-CSF, aCompCor-WM, aCompCor-combined, tCompCor) con líneas de referencia al 50 / 70 / 90 % — por defecto solo se retienen los componentes que acumulan el top 50 %.
- **Matriz de correlación entre regresores** y correlación con la señal global — útil para diagnosticar *partial volume effects* y redundancia entre confounds.
- **Boilerplate de citación:** texto de métodos listo para reproducir verbatim en publicaciones (dominio público, licencia CC0).

El HTML es el **filtro de calidad primario** antes de proceder a TEDANA: cualquier sujeto con correregistro fallido, normalización degradada o FD mediana excesiva debe marcarse como candidato a descarte antes de invertir horas de cómputo multi-echo en §3.

2.2.3.2 Derivados anatómicos (sub-P1/anat/)

Todos los volúmenes en formato NIfTI-1 comprimido (.nii.gz):

Archivo	Contenido
sub-P1_desc-preproc_T1w.nii.gz	T1w corregido por inhomogeneidad de intensidad (N4BiasFieldCorrection de ANTs) en espacio nativo.
sub-P1_space-MNI152NLin2009cAsym_desc-preproc_T1w.nii.gz	T1w normalizado al template MNI asimétrico 2009c.
sub-P1_desc-brain_mask.nii.gz	Máscara cerebral binaria (uint8) derivada del skull-stripping.
sub-P1_dseg.nii.gz	Segmentación discreta de tejidos (1=CSF, 2=GM, 3=WM).
sub-P1_label-{CSF,GM,WM}_probseg.nii.gz	Mapas probabilísticos [0, 1] por tejido.

Transformaciones espaciales (HDF5 para warps compuestos, texto ITK para afines):

Archivo	Contenido
sub-P1_from-T1w_to-MNI152NLin2009cAsym_mode-image_xfm.h5	Warp compuesto T1w→MNI (affine + SyN no-lineal).
sub-P1_from-MNI152NLin2009cAsym_to-T1w_mode-image_xfm.h5	Transformación inversa MNI→T1w.
sub-P1_from-{fsnative,T1w}_to-{T1w,fsnative}_mode-image_xfm.txt	Affines entre el <i>conformed space</i> de FreeSurfer y el T1w nativo.

Si FreeSurfer está activo, se generan además las superficies corticales en GIFTI (*_hemi-[LR]_{white,midthickness,pial}.surf.gii), las esferas de registro a fsaverage y fsLR (con MSM-sulc si --msm), la máscara cortical (*_hemi-[LR]_desc-cortex_mask.label.gii) y las métricas morfométricas (thickness, curv, sulc) tanto en GIFTI nativo como en CIFTI-2 space-fsLR_den-32k (fMRIPrep invierte el signo de curv y sulc y enmascara la medial wall para consistencia con el HCP). En este pipeline, dado que la extracción de conectividad se ejecuta en volumen con el atlas Schaefer (§4) y no se solicita --cifti-output, las superficies y los CIFTI quedan como subproducto no utilizado aguas abajo — se conservan porque no añaden tiempo de cómputo adicional (son un subproducto directo de recon-all) y porque habilitan eventuales análisis complementarios sin reejecución.

2.2.3.3 Derivados de FreeSurfer (sourcedata/freesurfer/)

fMRIPrep aloja el *subjects directory* completo de FreeSurfer en sourcedata/ (la convención BIDS para insumos externos):

```
sourcedata/freesurfer/sub-P1/
  mri/          # volúmenes (orig, brainmask, aparc+aseg, wmparc)
  surf/         # superficies lh./rh. (white, pial, inflated, sphere)
  label/        # parcelaciones (lh.aparc.annot)
  stats/        # estadísticas morfométricas por ROI
```

Se copian también fsaverage{,5,6}/ cuando algún output se resampla a esas mallas. En la raíz del derivatives se dejan desc-aparc_dseg.tsv y desc-aparcaseg_dseg.tsv, que son los *lookup tables* (índice → nombre de ROI) — útiles si en el futuro se decide parcelar con Desikan-Killiany o Destrieux en lugar de Schaefer.

2.2.3.4 Derivados funcionales (sub-P1/ses-<N>/func/)

Todos los archivos llevan los *specifiers* BIDS obligatorios task-BOLDREST<1|2>_run-<1|2>. Formato NIfTI-1 comprimido para los volúmenes, ITK texto para las transformaciones rígidas.

Imágenes de referencia y máscaras.

Archivo	Contenido
*_desc-hmc_boldref.nii.gz	Referencia para la corrección de movimiento (promedio robusto tras alineamiento rígido).
*_desc-coreg_boldref.nii.gz	Boldref corregida al T1w (post bbgregister).
*_space-MNI152NLin2009cAsym_boldref.nii.gz	Boldref resamplada al template estándar.

Archivo	Contenido
<code>*_space-MNI152Nlin2009cAsym_desc-brain_mask.nii.gz</code>	Máscara cerebral BOLD en MNI.

Serie BOLD preprocesada — *optimally combined*.

El archivo `*_space-MNI152Nlin2009cAsym_desc-preproc_bold.nii.gz` es la serie 4D *optimally combined* generada por el workflow multi-echo `init_bold_t2s_wf()`. Cada vóxel es la suma ponderada por T_2^* de los cinco ecos (Posse et al., 1999; Kundu et al., 2012):

$$S_{\text{OC}}(t) = \sum_{k=1}^5 w_k \cdot S_k(t), \quad w_k \propto TE_k \cdot e^{-TE_k/T_2^*}$$

La serie ha pasado por $\text{STC} \rightarrow \text{HMC} \rightarrow \text{SDC} \rightarrow \text{corregistro EPI} \rightarrow \text{T1w} \rightarrow \text{resampling MNI}$ **concatenando todas las transformaciones en una única interpolación**, lo que minimiza el suavizado espacial acumulado por interpolaciones sucesivas. Las unidades son de intensidad BOLD arbitraria (BIDS no estandariza esta magnitud).

Mapa T_2^* .

El archivo `*_space-MNI152Nlin2009cAsym_T2starmap.nii.gz` contiene el mapa estático de T_2^* —fMRIPrep lo escribe **en segundos**— estimado por `tedana.t2smap` con `--me-t2s-fit-method loglin` (ver justificación en §2.2.1): regresión cerrada de mínimos cuadrados sobre $\log S(TE) = \log S_0 - TE/T_2^*$. Los valores típicos en GM cortical a 3T (30–50 ms) aparecen por tanto como 0.030–0.050 en el fichero. En este pipeline **su único uso downstream es la propia combinación óptima**: TEDANA lo recalcula internamente con `curvefit` para la clasificación / de componentes ICA (§3), por lo que el mapa exportado por fMRIPrep no se propaga al análisis de conectividad.

Ecos individuales preprocesados (con `--me-output-echos`).

Los cinco archivos `*_echo-{1,2,3,4,5}_part-mag_desc-preproc_bold.nii.gz` contienen cada eco ($TE = 14.20, 38.93, 63.66, 88.39, 113.12$ ms) corregido por STC, HMC y SDC, **en espacio BOLD nativo y no combinados**. A diferencia de la serie *optimally combined* —que sí se remuestrea a MNI152Nlin2009cAsym—, `--me-output-echos` emite los ecos individuales solo en el espacio nativo de la adquisición: no se remuestrean a los `--output-spaces`, porque el remuestreo a MNI se aplica una sola vez tras el denoising, dentro del flujo de TEDANA, que opera en espacio nativo (§3.9). Son el **insumo obligatorio de TEDANA**: la clasificación TE-dependiente (§3) requiere los ecos separados porque el ajuste (TE-dependencia) y (TE-independencia) se realiza vóxel a vóxel sobre los cinco puntos ($TE_k, S_k(t)$) para cada componente ICA (Kundu et al., 2012, 2013; DuPre et al., 2021). Sin la bandera `--me-output-echos` fMRIPrep descartaría los ecos individuales tras la combinación óptima y el pipeline multi-echo sería irrealizable.

Transformaciones funcionales.

Archivo	Contenido
<code>*_from-orig_to_boldref_mode-image_desc-hmc_xfm.txt</code>	Transformaciones rígidas (6 DOF) por volumen — MCFLIRT de FSL.
<code>*_from-boldref_to-T1w_mode-image_desc-coreg_xfm.txt</code>	Corregistro BOLD→T1w (bbregister, 6 DOF).
<code>*_from-boldref_to-{TOPUP,auto000XX}_mode-image_xfm.txt</code>	Registro al fieldmap, si aplica.

Estas transformaciones, combinadas con los warps T1w→MNI del directorio `anat/`, permiten resampling *ad hoc* de derivados futuros (p. ej. nuevos atlas) sin reejecutar fMRIPrep.

Outputs no generados en este pipeline.

Por concisión, se enumeran los derivados que fMRIPrep puede producir pero que **no se solicitan en ds006072**:

- Superficies BOLD (`*_hemi-[LR]_space-fsaverage5_bold.func.gii`): requerirían añadir `fsaverage5` a `--output-spaces`.
- Grayordinates CIFTI-2 (`*_bold.dtseries.nii`): requerirían `--cifti-output 91k`.
- Segmentaciones resampleadas al grid BOLD (`*_space-T1w_desc-{aparcaseg,aseg}_dseg.nii.gz`): no se generan porque la parcelación (Schaefer) se aplica directamente en MNI.

La decisión de limitar `--output-spaces` a `MNI152NLin2009cAsym` responde al diseño del análisis de conectividad (§4): atlas Schaefer en MNI, correlaciones de Pearson sobre parcelas volumétricas, sin análisis basado en superficie.

2.2.3.5 Confounds (`*_desc-confounds_timeseries.tsv + .json`)

Tabla TSV con cabecera; una fila por volumen funcional, una columna por regresor. El sidecar JSON documenta el método de cálculo y metadatos por columna (masks, varianza explicada, flag `Retained`). Los archivos de ds006072 contienen típicamente más de 100 columnas — **la selección de regresores para el modelo de denoising es responsabilidad del usuario** y debe hacerse antes de cualquier regresión (Parkes et al., 2018; Ciric et al., 2017). Incluir la tabla completa en una matriz de diseño es un error metodológico grave (colinealidad, pérdida de grados de libertad, doble denoising).

Parámetros de movimiento y expansiones (24P).

- **6P base**: `trans_x`, `trans_y`, `trans_z` (mm), `rot_x`, `rot_y`, `rot_z` (rad) — parámetros rígidos de HMC referidos a la imagen boldref.

- **Expansión Friston24 / Satterthwaite** (Friston et al., 1996; Satterthwaite et al., 2013): cada parámetro base se acompaña de `*_derivative1` (diferencia temporal), `*_power2` (término cuadrático) y `*_derivative1_power2` (cuadrado de la derivada). El total es $6 + 6 + 6 + 6 = 24$ regresores.

El modelo **24P** es el **regresor de movimiento base** del pipeline primario post-TEDANA (TEDANA+aCompCor; §4.3 y Section 3.13): captura el movimiento residual no eliminado por ME-ICA sin imponer la centralización forzada a media cero que introduciría la GSR (Murphy & Fox, 2017). El pipeline primario lo complementa con 10 componentes aCompCor (5 PC WM + 5 PC CSF; Behzadi 2007) para regresar el ruido fisiológico espacialmente difuso que la ICA espacial no separa por construcción (DuPre et al., 2021).

Señales tisulares medias y 36P con GSR.

- `csf`: media dentro de máscara CSF anatómicamente derivada y erosionada.
- `white_matter`: media dentro de máscara WM erosionada.
- `global_signal`: media dentro de la máscara cerebral.

Con sus expansiones `_derivative1`, `_power2` y `_derivative1_power2`, las tres señales tisulares suman 12 columnas adicionales. El modelo **36P = 24P + (csf, WM, GS) × 4 expansiones** (Satterthwaite et al., 2013) se reserva en este estudio para **triangulación metodológica con ds003059** — aplicarlo post-TEDANA constituiría un doble denoising perjudicial, pues TEDANA ya elimina los componentes globales no-BOLD por el criterio físico TE-dependiente (§3; Kundu et al., 2013).

Detección de outliers.

- `framewise_displacement` (FD): cuantificación bulk de movimiento frame-to-frame (Power et al., 2012), con rotaciones convertidas a mm asumiendo radio cerebral de 50 mm.
- `rmsd`: movimiento relativo por RMS (Jenkinson et al., 2002).
- `dvars`: derivada temporal de la RMS de intensidad sobre vóxeles (Power et al., 2012).
- `std_dvars`: DVARS estandarizado (adimensional, comparable entre runs).
- `non_steady_state_outlier_XX`: una columna por volumen inicial no estacionario (valor 1 en ese volumen, 0 en el resto). Se identifican automáticamente al inicio de la serie y se descartan del cómputo de cosines y CompCor.
- `motion_outlier_XX`: *spike regressors* generados con los umbrales por defecto `--fd-spike-threshold 0.5 mm` y `--dvars-spike-threshold 1.5`. Una columna por frame marcado, usada para *scrubbing* por regresión (equivalente a *censoring* sin pérdida de grados de libertad si se modela explícitamente; Power et al., 2012).

Regresores DCT (drift de baja frecuencia).

Las columnas `cosine_XX` son regresores de coseno discreto que implementan un filtro de paso alto equivalente a ~ 128 s de periodo. El número de columnas depende de la duración *efectiva*

de la serie tras descartar los non-steady-states. **Precaución:** si se aplica un filtro temporal externo, no incluir los `cosine_XX`; si se usan regresores CompCor, **sí es obligatorio incluirlos** porque fMRIPrep filtra antes de calcular CompCor y omitir los cosines reintroduce la varianza de baja frecuencia.

CompCor.

fMRIPrep implementa las variantes anatómica (aCompCor, Behzadi et al., 2007) y temporal (tCompCor) del método:

- `a_comp_cor_XX`: PCA sobre las series temporales dentro de tres máscaras anatómicas separadas (`Mask: CSF`, `Mask: WM`, `Mask: combined` — unión de CSF+WM erosionadas; Muschelli et al., 2014).
- `t_comp_cor_XX`: PCA sobre los vóxeles con mayor varianza temporal (top 2 %).

Por defecto se retienen los componentes que acumulan el top 50 % de varianza en su ROI; los descartados quedan registrados en el JSON con prefijo `dropped_XX` y `Retained: false`. El sidecar por componente tiene la forma:

```
{
  "a_comp_cor_00": {
    "CumulativeVarianceExplained": 0.108,
    "Mask": "combined",
    "Method": "aCompCor",
    "Retained": true,
    "SingularValue": 25.83,
    "VarianceExplained": 0.108
  }
}
```

En este pipeline, aCompCor se usa como **control data-driven sin GSR** (§4.3): típicamente los 5 primeros componentes de `Mask: combined` (implementación Behzadi) o los primeros N que acumulan 50 % en WM y CSF (implementación Muschelli). **Crítico:** consultar siempre el JSON para identificar el `Mask` de cada componente — mezclar `Mask: CSF`, `Mask: WM` y `Mask: combined` en la misma regresión introduce redundancia.

Regresores del *crown* cerebral.

fMRIPrep reutiliza el código de aCompCor sobre una máscara de vóxeles del borde cerebral (*crown*), generando 24 componentes PCA (Patriat et al., 2017; Provins et al., 2022). Capturan artefactos de borde (movimiento de cuero cabelludo, susceptibilidad dinámica) sin requerir máscaras tisulares, y son una alternativa útil cuando la segmentación WM/CSF es subóptima.

2.2.3.6 Consideraciones operacionales

1. **Desplazamiento por slice-timing:** STC referencia al slice medio por defecto, lo que desplaza los onsets de volumen en $+0.5 \cdot TR$. Irrelevante para resting-state (sin eventos), pero crítico en análisis GLM con regresores por evento — no aplica a este estudio, que es íntegramente resting.
2. **Los ecos individuales, no la serie OC, son el insumo de TEDANA.** TEDANA recomputa internamente su propia combinación óptima y su propio T_2^* map con `curvefit` a partir de los cinco archivos `*_echo-{1..5}_*_desc-preproc_bold.nii.gz` (§3).
3. **El mapa T_2^* de fMRIPrep es descartable tras la combinación óptima:** no alimenta TEDANA ni el análisis de conectividad.
4. **Filtrado obligatorio del TSV de confounds** antes de cualquier regresión: seleccionar explícitamente las 24 columnas del modelo 24P + las columnas `motion_outlier_*` para *scrubbing*. Automatizar esta selección reduce el riesgo de incluir columnas redundantes o contradictorias con TEDANA.

2.2.4 Interpretación de los resultados de fMRIPrep

Antes de pasar a §3 (TEDANA) cada sujeto debe superar una **inspección dual de QA**: (i) cuantitativa sobre los TSV de confounds y (ii) visual sobre los reportlets SVG embebidos en `sub-<label>.html`. Los criterios de admisibilidad aplicados en este pipeline son:

Criterio	Umbral	Fuente
FD mediana	$< 0.3 \text{ mm}$	Parkes et al. (2018)
% frames FD $> 0.5 \text{ mm}$	$< 20 \%$	Power et al. (2012)
sDVARs mediana	< 1.5	Power et al. (2012)
Corregistro EPI→T1w	sin desplazamientos visibles $> 2 \text{ mm}$	inspección visual
Histograma T_2^*	unimodal, pico 30–60 ms	Posse et al. (1999)
Máscara cerebral	sin agujeros ni extensiones extracraniales	inspección visual

Los runs que no superen algún criterio se marcan para revisión antes del análisis de grupo (§5). Esta sección se actualiza con una subsección por sujeto a medida que completa el preprocesamiento.

2.3 QC cuantitativo del bulk run fMRIPrep

Una vez completado fMRIPrep sobre todo el Tier 0 (PsiloData §6) en `D:/.../prep/fmriprep/data/t0/`, el siguiente chunk reproduce —sobre *cada* run preprocesado— el panel de criterios de admisibilidad descrito en Section 2.2.4, pero en forma **cuantitativa, reproducible y agregable a**

CSV. La motivación operacional es la misma que la del QC del bulk NORDIC (Section 1.9): inspeccionar 12 reportlets HTML uno a uno se vuelve impracticable conforme se añaden tiers, los umbrales (FD, DVARS, T2*, máscara, coregistro) deben quedar registrados en código en lugar de en el ojo del operador, y el verdict por run debe ser comparable entre el Tier 0 y los tiers posteriores.

Diferencias de diseño respecto al QC del bulk NORDIC:

1. **Sin pareja pre/post:** NORDIC se evalúa por contraste entre el run pre-NORDIC (pristino en `F:\`) y el post-NORDIC (en `prep/nordic/data/`). fMRIPrep no — el QC se evalúa sobre los outputs únicamente, contra umbrales calibrados de la literatura (Power et al. 2012; Parkes et al. 2018; Esteban et al. 2019; Posse et al. 1999).
2. **Fuente unificada de datos:** todas las métricas se derivan del directorio `prep/fmriprep/t0/sub-X/ses-Y/func/`. No hay paths cruzados entre discos ni inputs auxiliares; las series temporales nativas (5 ecos $\times$ `desc-preproc_bold.nii.gz`), el `T2starmap.nii.gz`, el `desc-brain_mask.nii.gz` y el `desc-confounds_timeseries.tsv` constituyen el conjunto cerrado de entrada.
3. **Idempotencia por filesystem:** igual que NORDIC, si los dos PNG del run ya existen en `qc/fmriprep/sub-X/ses-Y/` el run se salta sin re-leer NIfTIs. Permite añadir tiers posteriores y relanzar el chunk sin recomputar el Tier 0.
4. **Resolución multi-echo:** las métricas de tSNR y T2* se reportan **por eco** (5 ecos) y agregadas, para detectar fallos selectivos de un TE (por ejemplo, eco 5 con SNR insuficiente por dropout en zonas inferotemporales) que el OC enmascararía.
5. **Decoupling del verdict respecto al SVG:** los reportlets `desc-coreg_bold.svg`, `desc-sdc_bold.svg`, `desc-t2starhist_bold.svg`, `desc-rois_bold.svg` y `desc-carpetplot_bold.svg` siguen siendo la verdad visual y deben revisarse en Section 2.2.4 para los runs marcados como `REVISAR` o `FAIL` por este chunk. Las métricas automáticas son una *primera criba* que reproduce los criterios cuantitativos del Section 2.2.4; no sustituyen a la inspección de los SVG.

2.3.1 Métricas de QC

1. **Integridad estructural de outputs:** por cada run del Tier 0 deben existir los 5 ecos preproc nativos (`echo-{1..5}_part-mag_desc-preproc_bold.nii.gz`), el `T2starmap.nii.gz` y el `desc-preproc_bold.nii.gz` ya en `space-MNI152NLin2009cAsym` (la combinación óptima que fMRIPrep delega a TEDANA-T2smap interno, Section 2.2.4), las dos brain masks (nativa + MNI), el `desc-confounds_timeseries.tsv` y los transforms HMC/coreg/fmap. La ausencia de cualquiera dispara `FAIL` sin computar el resto de métricas (no hay sobre qué evaluar).
2. **Movimiento (FD, Power 2012; Parkes 2018):** mediana de `framewise_displacement`, máximo, y porcentaje de frames con `FD > 0.5 mm`. La mediana FD es el descriptor del run; el % de frames spike marca la *fracción de datos perdidos por scrubbing* si se aplicase censoring a 0.5 mm. La fracción de no-steady-state-outliers (NSS) se reporta aparte

porque ya está fuera de la serie analizable (los primeros 4–6 volúmenes que fMRIPrep marca por defecto).

3. **DVARS (Power 2012)**: mediana de `std_dvars` y % de frames con `std_dvars > 1.5`. DVARS captura saltos de intensidad globales que FD no ve (e.g., respiración profunda, spin-history sin desplazamiento neto); FD y DVARS son **complementarios**, no redundantes — un run con DVARS alto y FD bajo es sospechoso de artefacto fisiológico no captado por mcflirt.
4. **Outliers totales**: n° de columnas `motion_outlier_*` en el TSV (frames marcados por fMRIPrep con la regla por defecto `FD > 0.5 mm OR std_dvars > 1.5`) y % respecto al total de frames. Un run con `> 20 %` de frames flaggeados pierde el poder estadístico necesario para FC robusta (Parkes 2018, Table 1).
5. **CompCor — componentes retenidos**: n° de componentes guardados (top-50 % varianza explicada) por cada decomposición — aCompCor CSF, aCompCor WM, aCompCor combined, tCompCor — leído de la columna `Retained=True` del sidecar JSON. Más de **30 componentes retenidos en alguna ROI** indica que esa ROI está saturada de ruido (típicamente CSF en runs con artefactos respiratorios) y que el modelo de denoising por aCompCor competirá por varianza con la señal BOLD.
6. **Mapa T2* — plausibilidad fisiológica**: histograma del `T2starmap.nii.gz` (en MNI, vóxeles de la máscara cerebral). Debe ser **unimodal con moda en 30–60 ms** (Posse et al. 1999 — valor de referencia para córtex a 3T; valores más bajos en zonas con susceptibilidad como orbitofrontal y temporal inferior son esperables). Métricas derivadas: moda, mediana, % de vóxeles con $T2^* \in [25, 70]$ ms (fisiológicamente plausibles) y % de vóxeles con $T2^*$ fallido o saturado ($T2^* < 5$ ms o $T2^* > 150$ ms; señales de fit pobre por bajo SNR o saturación). Bi-modalidad o moda fuera de $[30, 60]$ problema multi-eco (eco 5 saturado, registro inter-eco mal estimado o fit `curvefit` divergiendo).
7. **Brain mask — integridad anatómica**: volumen total en ml y número de componentes conectados 3D del mask (debe ser 1). Componentes `> 1` máscara fracturada (típicamente artefacto en cerebelo o desconexión por ringing). La cobertura BOLD-vs-T1w no se cuantifica aquí (las máscaras BOLD-MNI y T1w-MNI están en grids distintos); se delega al reportlet `desc-sdc_bold.svg` (§2.3.2).
8. **tSNR por eco (foreground, máscara cerebral)**: mediana del tSNR vóxel-a-vóxel ($\bar{S}/\sigma_S$ temporal) de cada uno de los 5 ecos nativos `desc-preproc_bold`. **Expectativa multi-eco a 3T**: el tSNR crudo sigue la amplitud de señal $S_0 e^{-TE/T_2^*}$ y por tanto **decrece monótonamente con TE** — eco-1 es el de mayor tSNR, eco-5 el de menor. Lo que crece con TE hasta $TE \approx T_2^*$ es la sensibilidad/CNR BOLD ($\propto TE \cdot e^{-TE/T_2^*}$), no el tSNR crudo (Posse 1999; Kundu 2012). Una violación de la monotonía del perfil — un eco con menos tSNR que otro de TE mayor — señala un problema de registro inter-eco o de movimiento residual no captado por mcflirt; el QC vigila ese perfil, no un ranking concreto.
9. **Coregistro EPI→T1w — delegado al QC visual** (§2.3.2): el QC automatizado de coregistro EPI T1w via métrica intensity-based (NCC, NMI Studholme, `|Pearson r|/corratio`) **no es viable** sobre los outputs de fMRIPrep 25.x en este pipeline. Tres iteraciones documentadas: (i) NCC en MNI cayó en banda 0.42–0.61 por la inversión

GM/WM cross-modal (esperado para EPI T1w incluso con coregistro perfecto, ver Greve & Fischl 2009); (ii) NMI Studholme en MNI con 64 bins $\rightarrow$ 1.02–1.05 y en T1w-nativo con 32 bins + clipping $\rightarrow$ 1.00–1.01, ambos esencialmente en el baseline de independencia; (iii) $|r|$ en MNI con intersección de máscaras $\rightarrow$ 0.01–0.12, también baseline. Diagnóstico raíz: la affine del `desc-coreg_boldref.nii.gz` sigue apuntando a coordenadas BOLD-native con el xfm `bbregister` almacenado aparte en `from-boldref_to-T1w_mode-image_desc-coreg_xfm.txt` (NiPreps no “hornea” el xfm en la cabecera del output), por lo que `nilearn.resample_to_img` no puede alinear los dos volúmenes sin aplicar ese xfm explícitamente (Pearson $r = +0.07$ post-resample en T1w-native lo confirma). La calidad del coregistro se verifica entonces por **inspección visual** del reportlet `desc-coreg_bold.svg` en `sub-<label>.html` (§2.3.2 ítem 1, descrito al pie de Section 2.2.4) y por el hecho de que fMRIPrep aborta el workflow si `bbregister` diverge demasiado (degradación a FSL FLIRT): si fMRIPrep terminó con éxito y el SVG muestra contornos del córtex T1w alineados con el BOLDref (sin desplazamientos > 2 mm en orbitofrontal o temporal inferior), el coregistro es admisible. La automatización vía métrica cuantitativa requeriría parsear el cost final de `bbregister` desde los logs FreeSurfer en `sourcedata/freesurfer/sub-X/scripts/`, o aplicar el `_xfm.txt` manualmente con `nitransforms` antes del cálculo — fuera del alcance del Tier 0.

10. **Acoplamiento confound–señal global (proxy del carpetplot):** correlación de Pearson entre `framewise_displacement` y `global_signal` a lo largo del run. Valores > 0.5 implican que la señal global está dominada por movimiento (firma del banding horizontal en el carpetplot) y que cualquier pipeline sin GSR re-introducirá el ruido motor por contaminación de la señal media — relevante para la elección de pipeline de denoising en §4.3 (Power 2014; Parkes 2018).

Métrica	Rango esperado / umbral	Si sale fuera
Integridad de outputs	todos los archivos presentes	falta cualquiera FAIL (re-ejecutar fMRIPrep para el run)
<code>fd_median</code>	< 0.3 mm (Parkes 2018)	> 0.3 run de alto movimiento; > 0.5 candidato a exclusión
<code>fd_pct_over_0p5</code>	< 20 % (Power 2012)	20–35 % REVISAR ; > 35 % run perdido tras scrubbing
<code>std_dvars_median</code>	< 1.5 (Power 2012)	> 1.5 artefacto fisiológico/scanner; combinar con <code>fd_median</code> para diagnóstico
<code>std_dvars_pct_over_1p5</code>	< 15 %	> 15 % banding sostenido en carpetplot
<code>n_motion_outliers_pct</code>	< 20 %	> 20 % run no apto para FC (Parkes 2018)

Métrica	Rango esperado / umbral	Si sale fuera
<code>n_aCompCor_retained_combined</code>	1–20	> 30 ROI saturada (revisar SDC y registro CSF)
<code>t2star_mode_ms</code>	30–60 ms (córtex 3T; Posse 1999)	< 30 o > 60 fit T2* sospechoso; revisar <code>desc-t2starhist_bold.svg</code>
<code>t2star_pct_plausible</code> ([25,70] ms)	> 80 %	< 80 % fit fallido en gran parte del cerebro
<code>mask_volume_ml</code>	1100–1700 ml (adulto)	fuera máscara mal estimada (revisar <code>desc-rois_bold.svg</code>)
<code>mask_n_components</code>	1	> 1 máscara fracturada
<code>tsnr_echo</code> (mediana foreground)	perfil monótono decreciente eco-1→eco-5	no monótono registro inter-eco o movimiento residual
Coregistro EPI→T1w	delegado al QC visual del <code>desc-coreg_bold.svg</code> (§2.3.2)	desplazamiento > 2 mm visible FAIL manual
<code>fd_gs_corr</code>	abs < 0.30	> 0.50 señal global dominada por movimiento (relevante para GSR vs no-GSR)

Referencias. Power et al. 2012 (FD/DVARS thresholds; Neuroimage); Parkes et al. 2018 (admisibilidad de runs en FC; Neuroimage); Esteban et al. 2019 (fMRIPrep paper; Nat Methods); Posse et al. 1999 (T2* a 3T); Kundu et al. 2012 (multi-echo tSNR-vs-TE); Power 2014 (carpetplot y motion–global signal coupling).

2.3.2 2.3.2 Comprobaciones adicionales recomendadas (no automatizadas)

1. Reportlets SVG del propio fMRIPrep, abiertos desde `sub-<label>.html`:

- `desc-sdc_bold.svg` — overlay pre/post SDC; los bordes del córtex frontal y temporal inferior deben *moverse* hacia su posición anatómica.
- `desc-coreg_bold.svg` — contornos del T1w sobre el BOLDref; sin desplazamientos visibles > 2 mm en surcos principales.
- `desc-rois_bold.svg` — máscaras de tCompCor (azul) y aCompCor (magenta) sobre el BOLDref; deben quedar contenidas dentro del cráneo y no incluir señal cortical en el caso de aCompCor.
- `desc-t2starhist_bold.svg` — histograma de T2*; debe ser **unimodal** y centrado en 30–60 ms; bi-modalidad eco-5 saturado o fit `curvefit` divergiendo.

- **desc-carpetplot_bold.svg** — series globales (GS/CSF/WM/DVARS/FD) + carpet de las 5 regiones (corteza, gris profundo, cerebelo, blanca/CSF y *crown*; Provins et al. 2022). Bandas horizontales sincronizadas con picos de FD scrubbing necesario; banding sin correlato con FD artefacto fisiológico (respiración).
 - **desc-compcorvar_bold.svg** — curva de varianza acumulada por componente CompCor; debe alcanzar el 50 % en menos de ~10 componentes para aCompCor combined.
2. **Resumen desc-summary_bold.html** por run: lista los volúmenes excluidos por NSS, la TR efectiva, el método de SDC aplicado, y el método de combinación óptima de TEDANA-t2s. Verificar que los TEs reportados coinciden con los del JSON BIDS (regresiones por TE mal escrito en sidecar).
 3. **Reportlet anatómico** (**desc-reconall_T1w.svg**, **dseg.svg**, **space-MNI152NLin2009cAsym_T1w.svg**): la segmentación de FreeSurfer y la normalización a MNI son el escalón previo de todo el resto. Un error aquí se propaga a aCompCor (ROIs de CSF/WM mal trazadas), a la máscara BOLD-vs-T1w y a la combinación óptima.
 4. **Comparación inter-sujeto del T2* mediano en córtex**: si un sujeto presenta T2* mediano > 1.5 por encima/debajo del grupo, sospechar de adquisición fuera de protocolo (TEs distintos, voltaje de RF) o de un fit sistemáticamente sesgado.
 5. **Inspección del non_steady_state_outlier_* count**: si fMRIPrep marca más de 6 NSS (dummy scans esperados) en un run, comprobar el JSON BIDS (¿se importaron correctamente los dummies?) y el log (¿hay una transición de intensidad inicial real o es un artefacto del dataset?).

```
# =====
# QC del bulk run fMRIPrep - panel diagnóstico cuantitativo sobre cada run de
# `prep/fmriprep/data/{t0,t1,texc}/`, con la misma filosofía que @sec-nordic-qc-bulk:
#   - métricas derivadas de los outputs (no del SVG)
#   - umbrales calibrados de literatura (§2.3.1)
#   - idempotencia por filesystem (PNG presentes skip)
#   - tabla unificada `metrics.csv` con verdict por run
#
# Entrada:
#   - <tanda>/sub-X/ses-Y/func/*echo-{1..5}*desc-preproc_bold.nii.gz
#   - <tanda>/sub-X/ses-Y/func/*space-MNI*_T2starmap.nii.gz
#   - <tanda>/sub-X/ses-Y/func/*space-MNI*_boldref.nii.gz
#   - <tanda>/sub-X/ses-Y/func/*space-MNI*_desc-brain_mask.nii.gz
#   - <tanda>/sub-X/ses-1/anat/*desc-brain_mask.nii.gz    (T1w mask)
#   - <tanda>/sub-X/ses-1/anat/*space-MNI*desc-preproc_T1w.nii.gz
#   - <tanda>/sub-X/ses-Y/func/*desc-confounds_timeseries.tsv
#   - <tanda>/sub-X/ses-Y/func/*desc-confounds_timeseries.json
#
# Salida:
#   - QC_DIR/sub-X/ses-Y/<run>_summary.png    (FD, DVARS, T2*-hist, tSNR por eco)
```

```

# - QC_DIR/sub-X/ses-Y/<run>_carpet.png      (GS/CSF/WM + carpet por tejido)
# - QC_DIR/metrics.csv
# =====
import os, glob, json, re
from collections import defaultdict
import numpy as np
import pandas as pd
import nibabel as nib
import matplotlib.pyplot as plt
from scipy import ndimage

# fMRIPrep - output organizado en tres tandas bajo prep/fmriprep/data/.
# El QC recorre las tres; cada run registra su tanda para resolver func/anat.
FMRIPREP_DIRS = [
    "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/data/t0",
    "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/data/t1",
    "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/data/texc",
]
QC_DIR      = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/qc"
TR          = 1.761 # ds006072 (igual que @sec-nordic-qc-bulk)
os.makedirs(QC_DIR, exist_ok=True)

# Umbrales - calibrados de Power 2012, Parkes 2018, Posse 1999, Esteban 2019.
TH = dict(
    fd_median      = 0.30,
    fd_pct_over_0p5 = 20.0,
    dvars_median   = 1.5,
    dvars_pct_over  = 15.0,
    motion_out_pct  = 20.0,
    acompcor_max    = 30,
    t2star_mode_lo  = 30.0,
    t2star_mode_hi  = 60.0,
    t2star_plaus_pct = 80.0,
    mask_vol_lo_ml  = 1100.0,
    mask_vol_hi_ml  = 1700.0,
    fd_gs_corr_warn = 0.30,
    fd_gs_corr_fail = 0.50,
)

# -----
# Discovery - un registro por (subj, ses, task, run) con todos los paths
# -----

```

```

RUN_RE = re.compile(
    r"(sub-P\d+)_ (ses-[^_]+)_task-([^\_]+)_dir-([^\_]+)_ (run-\d+)_echo-(\d+)"
    r"_part-mag_desc-preproc_bold\.nii\.gz$"
)

def discover_runs():
    """Agrupa por (subj,ses,task,run) los 5 ecos preproc + outputs MNI + TSV.
    Recorre las tres tandas de fMRIPrep (t0, t1, texc); cada run registra la
    tanda que lo contiene para resolver func/anat dentro de ella."""
    runs = defaultdict(lambda: dict(echoes={}, missing=[], fmriprep_dir=None))
    claimed_by = {} # (subj,ses,task,run) → tanda que lo reclamó primero
    for fp_dir in FM RIPREP_DIRS:
        pattern = os.path.join(fp_dir, "sub-*", "ses-*", "func",
                                "*echo-*_part-mag_desc-preproc_bold.nii.gz")
        for p in sorted(glob.glob(pattern)):
            m = RUN_RE.search(os.path.basename(p))
            if not m: continue
            subj, ses, task, direction, run, echo = m.groups()
            run_key = (subj, ses, task, run)
            # dedup entre tandas: la primera (orden t0→t1→texc) reclama el run;
            # los duplicados de tandas posteriores (típicamente texc) se descartan.
            if claimed_by.setdefault(run_key, fp_dir) != fp_dir:
                continue
            runs[run_key]["echoes"][int(echo)] = p
            runs[run_key]["fmriprep_dir"] = fp_dir

    for key, info in runs.items():
        subj, ses, task, run = key
        fp_dir = info["fmriprep_dir"]
        func = os.path.join(fp_dir, subj, ses, "func")
        stub = f"{subj}_{ses}_task-{task}_dir-*_{run}_part-mag"
        # Resuelve por glob para no hard-codear `dir-AP`/`dir-PA`
        def first(g):
            hits = glob.glob(os.path.join(func, g))
            return hits[0] if hits else None
        info["boldref_mni"] = first(f"{stub}_space-MNI152Nlin2009cAsym_boldref.nii.gz")
        info["mask_mni"] = first(f"{stub}_space-MNI152Nlin2009cAsym_desc-brain_mask.nii.gz")
        info["preproc_mni"] = first(f"{stub}_space-MNI152Nlin2009cAsym_desc-preproc_bold.nii.gz")
        info["mask_native"] = first(f"{stub}_desc-brain_mask.nii.gz")
        info["confounds_tsv"] = first(f"{stub}_desc-confounds_timeseries.tsv")
        info["confounds_json"] = first(f"{stub}_desc-confounds_timeseries.json")
        info["t2star_mni"] = first(f"{stub}_desc-t2star_mni.nii.gz")

```



```

f"_space-MNI152NLin2009cAsym_T2starmap.nii.gz")
# Lista de echoes esperados (1..5) y los faltantes
info["missing"] = [e for e in (1, 2, 3, 4, 5) if e not in info["echoes"]]
return runs

# -----
# Métricas - devuelve un dict plano por run
# -----
def integrity_check(info):
    needed = ["boldref_mni", "mask_mni", "preproc_mni", "mask_native",
              "confounds_tsv", "confounds_json", "t2star_mni"]
    missing_files = [k for k in needed if not info.get(k) or not os.path.exists(info[k])]
    missing_echoes = list(info["missing"])
    ok = (not missing_files) and (not missing_echoes)
    return ok, missing_files, missing_echoes

def motion_metrics(tsv_path, json_path):
    df = pd.read_csv(tsv_path, sep="\t")
    fd = df["framewise_displacement"].astype(float).values
    dv = df["std_dvars"].astype(float).values
    fd_clean = fd[np.isfinite(fd)]
    dv_clean = dv[np.isfinite(dv)]
    motion_cols = [c for c in df.columns if c.startswith("motion_outlier")]
    nss_cols = [c for c in df.columns if c.startswith("non_steady_state_outlier")]
    n_frames = len(df)
    gs = df["global_signal"].astype(float).values
    fd_for_corr = np.where(np.isfinite(fd), fd, np.nanmedian(fd_clean))
    fd_gs_corr = float(np.corrcoef(fd_for_corr, gs)[0, 1])
    # CompCor retained per ROI
    meta = json.load(open(json_path))
    retained = {"CSF": 0, "WM": 0, "combined": 0, "tCompCor": 0}
    for col, m in meta.items():
        if not isinstance(m, dict) or not m.get("Retained"): continue
        if m.get("Method") == "tCompCor":
            retained["tCompCor"] += 1
        elif m.get("Method") == "aCompCor":
            retained[m.get("Mask", "combined")] += 1
    return dict(
        n_frames = int(n_frames),
        fd_median = float(np.nanmedian(fd_clean)) if fd_clean.size else np.nan,
        fd_max = float(np.nanmax(fd_clean)) if fd_clean.size else np.nan,
        fd_pct_over_0p5 = 100.0 * float((fd_clean > 0.5).mean()) if fd_clean.size else

```

```

        dvars_median          = float(np.nanmedian(dv_clean)) if dv_clean.size else np.nan,
        dvars_max             = float(np.nanmax(dv_clean))    if dv_clean.size else np.nan,
        dvars_pct_over_1p5    = 100.0 * float((dv_clean > 1.5).mean()) if dv_clean.size else
        n_motion_outliers     = len(motion_cols),
        n_motion_outliers_pct = 100.0 * len(motion_cols) / max(n_frames, 1),
        n_nss_outliers        = len(nss_cols),
        fd_gs_corr            = fd_gs_corr,
        n_acompcor_csf        = retained["CSF"],
        n_acompcor_wm         = retained["WM"],
        n_acompcor_combined   = retained["combined"],
        n_tcompcor            = retained["tCompCor"],
        _fd_series            = fd, _dvars_series = dv, _gs_series = gs,
    )

def t2star_metrics(t2star_path, mask_path):
    # fMRIPrep escribe el T2starmap en SEGUNDOS; las métricas y umbrales de
    # §2.3.1 están en ms (Posse 1999) → convertir a ms al cargar.
    t2s = nib.load(t2star_path).get_fdata(dtype=np.float32) * 1000.0
    mk = nib.load(mask_path).get_fdata(dtype=np.float32) > 0.5
    vals = t2s[mk & np.isfinite(t2s) & (t2s > 0)]
    if vals.size == 0:
        return dict(t2star_median=np.nan, t2star_mode_ms=np.nan,
                    t2star_pct_plausible=np.nan, t2star_pct_failed=np.nan,
                    _t2s_vals=np.array([]))
    hist, edges = np.histogram(vals, bins=200, range=(0, 200))
    mode = 0.5 * (edges[np.argmax(hist)] + edges[np.argmax(hist) + 1])
    plausible = float(((vals >= 25.0) & (vals <= 70.0)).mean())
    failed = float(((vals < 5.0) | (vals > 150.0)).mean())
    return dict(
        t2star_median          = float(np.median(vals)),
        t2star_mode_ms         = float(mode),
        t2star_pct_plausible   = 100.0 * plausible,
        t2star_pct_failed      = 100.0 * failed,
        _t2s_vals              = vals.astype(np.float32),
    )

def mask_metrics(mask_path):
    img = nib.load(mask_path)
    mk = img.get_fdata(dtype=np.float32) > 0.5
    vx_ml = float(np.prod(img.header.get_zooms()[:3])) / 1000.0
    vol_ml = float(mk.sum() * vx_ml)
    _, n_cc = ndimage.label(mk)

```

```

return dict(
    mask_volume_ml      = vol_ml,
    mask_n_components   = int(n_cc),
)

def tsnr_per_echo(echoes, mask_native_path):
    mk = nib.load(mask_native_path).get_fdata(dtype=np.float32) > 0.5
    out = {}
    for e in sorted(echoes):
        img = nib.load(echoes[e]).get_fdata(dtype=np.float32)
        mu = img.mean(-1); sd = img.std(-1)
        tsnr = np.zeros_like(mu); np.divide(mu, sd, out=tsnr, where=(sd > 0))
        out[e] = float(np.median(tsnr[mk])) if mk.any() else np.nan
    return out

def verdict(r):
    """Semáforo PASS / REVISAR / FAIL - mismas categorías que @sec-nordic-qc-bulk."""
    if not r["integrity_ok"]:
        return " FAIL (outputs incompletos)"
    fails, warns = [], []
    if r["fd_median"] > TH["fd_median"]: warns.append("FD>0.3")
    if r["fd_pct_over_0p5"] > TH["fd_pct_over_0p5"]: warns.append("FD spikes>20%")
    if r["fd_pct_over_0p5"] > 35.0: fails.append("FD spikes>35%")
    if r["dvars_median"] > TH["dvars_median"]: warns.append("DVARs>1.5")
    if r["dvars_pct_over_1p5"] > TH["dvars_pct_over"]: warns.append("DVARs spikes>15%")
    if r["n_motion_outliers_pct"] > TH["motion_out_pct"]: warns.append("motion_outliers>20%")
    if r["n_motion_outliers_pct"] > 50.0: fails.append("motion_outliers>50%")
    if r["n_acompcor_combined"] > TH["acompcor_max"]: warns.append("aCompCor combined>30")
    if not (TH["t2star_mode_lo"] <= r["t2star_mode_ms"] <= TH["t2star_mode_hi"]):
        warns.append(f"T2*mode={r['t2star_mode_ms']:.0f}")
    if r["t2star_pct_plausible"] < TH["t2star_plaus_pct"]: warns.append("T2*plaus<80%")
    if not (TH["mask_vol_lo_ml"] <= r["mask_volume_ml"] <= TH["mask_vol_hi_ml"]):
        warns.append(f"mask={r['mask_volume_ml']:.0f}ml")
    if r["mask_n_components"] > 1: warns.append(f"mask_cc={r['mask_n_...")
    if abs(r["fd_gs_corr"]) > TH["fd_gs_corr_fail"]: warns.append("FD~GS>0.5")
    # Perfil de tSNR por eco - el tSNR crudo sigue la amplitud S0·exp(-TE/T2*)
    # y debe decrecer monótonamente con TE; una violación de la monotonía
    # señala un problema de registro inter-eco o de movimiento (§2.3.1 métrica 8).
    tsnr_profile = [r[f"tsnr_echo_{e}"] for e in (1, 2, 3, 4, 5)
                    if f"tsnr_echo_{e}" in r]
    if any(a < b for a, b in zip(tsnr_profile, tsnr_profile[1:])):
        warns.append("tSNR perfil no monótono")

```

```

    if fails: return f" FAIL ({', '.join(fails)})"
    if warns: return f" REVISAR ({', '.join(warns)})"
    return " PASS"

# -----
# Plots - un summary multi-panel + un carpet derivado del confounds TSV
# -----
def plot_run_summary(key, r, out_dir):
    subj, ses, task, run = key
    fig, ax = plt.subplots(2, 2, figsize=(14, 8))
    # (a) FD
    fd = r["_fd_series"]; t = np.arange(len(fd)) * TR
    ax[0, 0].plot(t, fd, color="#1565c0", linewidth=0.8)
    ax[0, 0].axhline(0.5, color="#c62828", linestyle="--", linewidth=1,
                    label="0.5 mm (Power 2012)")
    ax[0, 0].axhline(0.3, color="#888", linestyle=":", linewidth=1,
                    label="0.3 mm (Parkes 2018)")
    ax[0, 0].set_xlabel("t (s)"); ax[0, 0].set_ylabel("FD (mm)")
    ax[0, 0].set_title(f"FD - median={r['fd_median']:.3f} "
                      f"%>0.5={r['fd_pct_over_0p5']:.1f}%")
    ax[0, 0].legend(fontsize=8)
    # (b) std_dvars
    dv = r["_dvars_series"]
    ax[0, 1].plot(t, dv, color="#2e7d32", linewidth=0.8)
    ax[0, 1].axhline(1.5, color="#c62828", linestyle="--", linewidth=1,
                    label="1.5 (Power 2012)")
    ax[0, 1].set_xlabel("t (s)"); ax[0, 1].set_ylabel("std_dvars")
    ax[0, 1].set_title(f"std_dvars - median={r['dvars_median']:.2f} "
                      f"%>1.5={r['dvars_pct_over_1p5']:.1f}%")
    ax[0, 1].legend(fontsize=8)
    # (c) histograma T2*
    vals = r["_t2s_vals"]
    if vals.size:
        ax[1, 0].hist(vals, bins=100, range=(0, 150), color="#6a1b9a", alpha=0.85)
        ax[1, 0].axvspan(30, 60, color="#c5e1a5", alpha=0.4,
                        label="rango fisiológico (Posse 1999)")
        ax[1, 0].axvline(r["t2star_mode_ms"], color="#c62828", linestyle="--",
                        linewidth=1, label=f"moda={r['t2star_mode_ms']:.1f} ms")
    ax[1, 0].set_xlabel("T2* (ms)"); ax[1, 0].set_ylabel("nº de vóxeles")
    ax[1, 0].set_title(f"T2* (en máscara cerebral, MNI) "
                      f"plausibles={r['t2star_pct_plausible']:.1f}%")
    ax[1, 0].legend(fontsize=8)

```

```

# (d) tSNR por eco
echoes = sorted([int(c.split("_")[-1]) for c in r if c.startswith("tsnr_echo_")])
tsnrs = [r[f"tsnr_echo_{e}"] for e in echoes]
ax[1, 1].bar(echoes, tsnrs, color="#ef6c00")
ax[1, 1].set_xticks(echoes); ax[1, 1].set_xlabel("echo")
ax[1, 1].set_ylabel("tSNR mediano (foreground)")
ax[1, 1].set_title("tSNR por eco - eco-2/3 deberían liderar (Kundu 2012)")
fig.suptitle(f"QC fMRIPrep - {subj} {ses} {task} {run} "
             f"verdict: {r['verdict']}", y=1.001, fontsize=12)
fig.tight_layout()
out_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_summary.png")
fig.savefig(out_png, dpi=120, bbox_inches="tight")
plt.close(fig)
return out_png

def plot_run_carpet(key, r, out_dir):
    """Carpetplot proxy - GS/CSF/WM/DVARs/FD sobre el tiempo + correlación FD-GS."""
    subj, ses, task, run = key
    fig, ax = plt.subplots(2, 1, figsize=(14, 5),
                           gridspec_kw=dict(height_ratios=[3, 1]))
    fd = r["_fd_series"]; gs = r["_gs_series"]; dv = r["_dvars_series"]
    t = np.arange(len(fd)) * TR
    # Normaliza a [0,1] para superponer
    def z01(x):
        x = np.array(x, dtype=float)
        x = x - np.nanmin(x); rng = np.nanmax(x) - np.nanmin(x) + 1e-9
        return x / rng
    ax[0].plot(t, z01(fd), label="FD", color="#1565c0", linewidth=0.7)
    ax[0].plot(t, z01(dv), label="std_dvars", color="#2e7d32", linewidth=0.7)
    ax[0].plot(t, z01(gs), label="global_sig", color="#6a1b9a", linewidth=0.7)
    ax[0].set_xlabel("t (s)"); ax[0].set_ylabel("normalizado [0,1]")
    ax[0].set_title(f"Confounds globales - corr(FD,GS)={r['fd_gs_corr']:+.2f}")
    ax[0].legend(fontsize=8, ncol=3)
    ax[1].axis("off")
    txt = (f"FD median={r['fd_median']:.3f} mm | %>0.5={r['fd_pct_over_0p5']:.1f}\n"
           f"std_dvars median={r['dvars_median']:.2f} | %>1.5={r['dvars_pct_over_1p5']:.1f}\n"
           f"motion_outliers={r['n_motion_outliers']} ({r['n_motion_outliers_pct']:.1f}%) \n"
           f"NSS outliers={r['n_nss_outliers']}\n"
           f"aCompCor retained: CSF={r['n_acompcor_csf']} WM={r['n_acompcor_wm']} "
           f"comb={r['n_acompcor_combined']} | tCompCor={r['n_tcompcor']}\n"
           f"T2* mode={r['t2star_mode_ms']:.1f} ms, median={r['t2star_median']:.1f} ms, "
           f"plausibles={r['t2star_pct_plausible']:.1f}%\n")

```

```

        f"mask={r['mask_volume_ml']:.0f} ml, cc={r['mask_n_components']}\n"
        f"verdict: {r['verdict']}")
    ax[1].text(0.0, 0.95, txt, va="top", ha="left", family="monospace",
              fontsize=9)
    fig.tight_layout()
    out_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_carpet.png")
    fig.savefig(out_png, dpi=120, bbox_inches="tight")
    plt.close(fig)
    return out_png

# -----
# Loop principal
# -----

runs = discover_runs()
print(f"[discover] {len(runs)} runs en fmripred/data (t0+t1+texc)")
rows, skipped, failed_runs = [], [], []

for key in sorted(runs):
    subj, ses, task, run = key
    info = runs[key]
    out_dir = os.path.join(QC_DIR, subj, ses); os.makedirs(out_dir, exist_ok=True)
    summary_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_summary.png")
    carpet_png = os.path.join(out_dir, f"{subj}_{ses}_{task}_{run}_carpet.png")

    if os.path.exists(summary_png) and os.path.exists(carpet_png):
        skipped.append(key); continue

    print(f"[qc] {subj} {ses} {task} {run}")
    ok, miss_files, miss_echoes = integrity_check(info)
    r = dict(subj=subj, ses=ses, task=task, run=run,
            integrity_ok=ok,
            missing_files=";".join(miss_files),
            missing_echoes=";".join(map(str, miss_echoes)))
    if not ok:
        r["verdict"] = f" FAIL (faltan: {r['missing_files'] or r['missing_echoes']})"
        rows.append(r); failed_runs.append(key); continue

    try:
        r.update(motion_metrics(info["confounds_tsv"], info["confounds_json"]))
        r.update(t2star_metrics(info["t2star_mni"], info["mask_mni"]))
        r.update(mask_metrics(info["mask_mni"]))
        tsnr = tsnr_per_echo(info["echoes"], info["mask_native"])

```

```

        for e, v in tsnr.items(): r[f"tsnr_echo_{e}"] = v
        r["verdict"] = verdict(r)
        plot_run_summary(key, r, out_dir)
        plot_run_carpet(key, r, out_dir)
        rows.append({k: v for k, v in r.items() if not k.startswith("_")})
    except Exception as e:
        print(f" [!] error: {type(e).__name__}: {e}")
        r["verdict"] = f" FAIL (excepción: {type(e).__name__})"
        rows.append({k: v for k, v in r.items() if not k.startswith("_")})
        failed_runs.append(key)

# -----
# CSV unificado + resumen
# -----
if rows:
    df = pd.DataFrame(rows)
    csv = os.path.join(QC_DIR, "metrics.csv")
    if os.path.exists(csv):
        df_prev = pd.read_csv(csv)
        df = pd.concat([df_prev[~df_prev.set_index(
            ["subj", "ses", "task", "run"]).index.isin(
                df.set_index(["subj", "ses", "task", "run"]).index)], df],
            ignore_index=True, sort=False)
    df = df.drop_duplicates(subset=["subj", "ses", "task", "run"], keep="last")
    df.to_csv(csv, index=False)
    print(f"[csv] {len(df)} runs en {csv}")
    print(df.groupby("verdict").size())

print(f"\n[summary] procesados={len(rows)}  skipped(idempotente)={len(skipped)}  "
      f"failed={len(failed_runs)}")

```

i Cuándo ejecutar y cómo interpretar el verdict

Este chunk se ejecuta **después** del bulk run de fMRIPrep sobre **t0/** (§2.2) y **antes** de TEDANA (§3). El verdict por run sigue las mismas tres categorías que Section 1.9:

- **PASS** — todas las métricas dentro de umbrales. El run pasa al pipeline TEDANA + extracción de FC sin más revisión.
- **REVISAR** — al menos una métrica fuera de su rango *warn* pero ninguna en rango *fail*. **Acción obligatoria:** abrir `sub-<label>.html` y revisar los reportlets SVG señalados en §2.3.2 para el run en cuestión, anotando en §2.3.3 si el hallazgo es admisible o conlleva exclusión.

- **FAIL** — falla estructural (faltan outputs) o $> 35\%$ de frames spike. **Acción:** el run no entra en §3; se reanuda fMRIPrep (§2.2.2) o se marca para exclusión definitiva en §4 con justificación documentada. La calidad del coregistro EPI→T1w se valida por inspección visual del `desc-coreg_bold.svg` (§2.3.2) y puede dar lugar a un **FAIL** manual si el SVG muestra desplazamientos > 2 mm.

i Coregistro EPI→T1w: QC cuantitativo pendiente vía nitransforms

La cuantificación del coregistro vía métricas intensity-based (NCC, NMI Studholme, $|r|$) no es viable directamente sobre los outputs de fMRIPrep 25.x porque el `desc-coreg_boldref` queda en espacio BOLD-native con el `_xfm.txt` almacenado aparte (no aplicado in-place). La validación se delega por ahora al reportlet visual `desc-coreg_bold.svg`. La automatización sería posible vía `nitransforms` aplicando el xfm explícitamente antes del cálculo de la métrica; queda anotada como mejora futura del chunk pero fuera del alcance del Tier 0 actual (cf. R1.1 «Limitaciones y caveats»).

3 3 Denoising multi-echo con TEDANA

3.1 3.1 Fundamento: separación BOLD vs. non-BOLD basada en TE-dependence

TEDANA (*TE-Dependent ANalysis*) es una librería Python para el denoising de datos fMRI multi-echo que explota un principio físico fundamental: la señal BOLD genuina es dependiente de T_2^* , mientras que los artefactos no-BOLD (movimiento, señales fisiológicas, artefactos de hardware) no lo son (Kundu et al., 2012, 2013; DuPre et al., 2021).

En una adquisición multi-echo, cada volumen se adquiere con múltiples tiempos de eco (TE). La señal BOLD en cada vóxel sigue un modelo de decaimiento monoexponencial:

$$S(TE) = S_0 \cdot e^{-TE/T_2^*}$$

donde S_0 es la intensidad a $TE = 0$ (proporcional a la densidad de protones y efectos de flujo) y T_2^* es el tiempo de relajación transversal efectivo, que varía con el nivel de oxigenación sanguínea (efecto BOLD). Los cambios neurales modulan T_2^* , produciendo fluctuaciones que escalan linealmente con TE (κ -dependencia). En cambio, los artefactos no-BOLD (movimiento, ruido fisiológico) modulan S_0 sin afectar T_2^* , produciendo fluctuaciones que son independientes de TE (ρ -dependencia).

TEDANA implementa los siguientes pasos (DuPre et al., 2021):

1. **Combinación óptima ponderada por T_2^* :** genera una serie temporal única con SNR maximizada, ponderando cada eco según su contribución a la señal BOLD.

2. **Descomposición PCA:** reduce la dimensionalidad de los datos combinados para estimar el número óptimo de componentes ICA.
3. **Descomposición ICA:** separa la señal combinada en componentes espacialmente independientes.
4. **Clasificación TE-dependiente:** cada componente ICA se clasifica como:
 - **Aceptado** (κ alto, ρ bajo): señal BOLD genuina, dependiente de T2*
 - **Rechazado** (κ bajo, ρ alto): artefacto no-BOLD, dependiente de S0
 - **Ignorado:** componentes ambiguos o de baja varianza
5. **Denoising:** los componentes rechazados se eliminan de los datos mediante regresión parcial (*non-aggressive*), preservando la varianza compartida con componentes aceptados.

3.2 3.2 Ventaja sobre métodos single-echo

A diferencia de las estrategias de regresión de confounds evaluadas para datos single-echo (24P, 36P, aCompCor; Ciric et al., 2017; Parkes et al., 2018), TEDANA:

- **Usa un criterio físico**, no estadístico: la dependencia de TE es una propiedad intrínseca de la señal BOLD, no un proxy indirecto como los parámetros de movimiento.
- **Preserva señal neural global:** la regresión de señal global (GSR) impone matemáticamente una distribución centrada en cero (Murphy & Fox, 2017), eliminando potencialmente fluctuaciones neurales de gran escala. TEDANA solo elimina componentes que *no* escalan con TE, preservando la señal BOLD global.
- **No requiere entrenamiento:** a diferencia de ICA-AROMA (Pruim et al., 2015), cuyos clasificadores fueron entrenados en datos single-echo en espacio MNI152NLin6Asym, TEDANA opera directamente sobre la física de la adquisición.
- **Elimina artefactos inaccesibles a otros métodos:** artefactos de hardware, susceptibilidad magnética dinámica y señales vasculares no capturadas por parámetros de movimiento o señales tisulares (Kundu et al., 2013).

Ciric et al. (2017) reconocieron explícitamente esta ventaja: “*improvements in image acquisition (including multi-echo techniques) may [...] change the methodological landscape of connectivity research*”.

3.3 3.3 Justificación de la estrategia de denoising multi-echo

La elección de TEDANA como estrategia primaria de denoising para ds006072 se justifica por múltiples líneas de evidencia convergentes:

1. **Criterio físico vs. proxy estadísticos.** Las estrategias evaluadas por Ciric et al. (2017) y Parkes et al. (2018) para datos single-echo (24P, 36P, aCompCor, ICA-AROMA) operan mediante regresión de señales que son *proxies* del ruido: parámetros de movimiento

(estimados de la realineación), señales tisulares (promediadas de máscaras anatómicas), o componentes ICA clasificados por heurísticas espaciotemporales. TEDANA, en cambio, separa señal BOLD de artefactos basándose en la **ecuación de decaimiento monoexponencial** $S(TE) = S_0 \cdot e^{-TE/T_2^*}$: los cambios neurales modulan T_2^* (métrica κ , TE-dependiente), mientras que los artefactos de movimiento, fisiológicos y de hardware modulan S_0 (métrica ρ , TE-independiente). Este criterio es intrínseco a la física de la adquisición y no requiere ningún modelo paramétrico del ruido (Kundu et al., 2012, 2013). Kundu et al. (2012) demostraron que los componentes ICA funcionales (redes neuronales) presentan valores κ altos y ρ bajos, mientras que los artefactos (movimiento, pulsatilidad) presentan el patrón inverso, con una separación altamente reproducible entre sujetos.

2. **Superioridad demostrada** Ciric et al. (2017) reconocieron explícitamente que “*improvements in image acquisition (including multi-echo techniques) may [...] change the methodological landscape of connectivity research*”.
3. **Preservación de señal neural global.** La GSR — el factor más eficaz para reducir QC-FC según Ciric et al. (2017) — impone matemáticamente que la distribución de correlaciones funcionales se centre en cero, generando anticorrelaciones que pueden ser artefactuales (Murphy & Fox, 2017). TEDANA elimina componentes globales no-BOLD (los que modulan S_0) sin afectar la señal BOLD global (que modula T_2^*). Esto permite observar la distribución *natural* de correlaciones funcionales sin el sesgo de centrado en cero, una ventaja crítica para el estudio de desincronización bajo psilocibina (Siegel et al., 2024), donde los cambios en la conectividad global son el hallazgo principal. Es notable que Siegel et al. (2024) tampoco aplicaron GSR en su pipeline original, lo que es consistente con este razonamiento.
4. **Eliminación de artefactos inaccesibles a métodos paramétricos.** Los artefactos de susceptibilidad magnética dinámica, las señales vasculares no correlacionadas con parámetros de movimiento, y los artefactos de hardware que afectan S_0 sin correlacionar con WM/CSF no son capturados por 24P, 36P ni aCompCor. TEDANA los identifica y elimina porque *cualquier* fluctuación TE-independiente es clasificada como non-BOLD, independientemente de su origen (Kundu et al., 2013).
5. **Ventajas sobre el pipeline original de Siegel et al. (2024).** Siegel et al. (2024, *Nature*) emplearon un pipeline *in-house* basado en NORDIC + combinación óptima + regresores tisulares + scrubbing (FD > 0.3 mm), **sin** descomposición ICA basada en TE-dependencia. Su pipeline utiliza la combinación óptima de ecos para maximizar SNR ($\sim 2\text{-}3\times$ vs. single-echo; Posse et al., 1999), pero no explota la información de TE-dependencia para separar componentes BOLD de non-BOLD a nivel de ICA. TEDANA va un paso más allá al aplicar ME-ICA sobre los datos multi-echo, lo que permite una clasificación física de cada componente como señal o artefacto, y una eliminación más completa de artefactos no accesibles mediante regresión de confounds convencional (Kundu et al., 2013). Aunque el presente pipeline difiere del original, esta divergencia es informativa: si los resultados de conectividad funcional convergen entre nuestro pipeline (TEDANA + regresión residual) y los hallazgos publicados de Siegel et al. (NORDIC +

OC + regresores tisulares), ello reforzaría la robustez de los efectos reportados frente a variaciones en la estrategia de denoising.

3.4 3.4 Impacto de TEDANA sobre las correlaciones de conectividad funcional

TEDANA modifica la estructura de las matrices de correlación de forma cualitativamente distinta a los métodos single-echo:

- **Correlaciones espurias de corta distancia (se reducen).** Power et al. (2012) demostraron que el movimiento infla correlaciones entre regiones cercanas porque un desplazamiento de cabeza afecta vóxeles vecinos de forma similar, produciendo señales artificialmente covariantes. TEDANA elimina estos componentes porque el artefacto de movimiento modula S_0 , no T_2^* — los componentes de movimiento tienen κ bajo y ρ alto, y son rechazados. Kundu et al. (2013) confirmaron que ME-ICA elimina la dependencia de distancia del artefacto de movimiento que persiste tras la regresión convencional de parámetros de movimiento.
- **Correlaciones genuinas de larga distancia (se preservan mejor).** El movimiento atenúa correlaciones entre regiones distantes (Power et al., 2012; Ciric et al., 2017). Al eliminar la varianza de movimiento por un criterio físico, TEDANA desenmascara correlaciones neurales de larga distancia que estaban suprimidas. Kundu et al. (2012) demostraron que la conectividad subcortical-cortical (hipocampo → corteza sensorial/pre-motora; tronco encefálico → ganglios basales) solo se hizo visible tras ME-ICA, siendo indetectable con denoising convencional — un hallazgo especialmente relevante para el estudio de la conectividad hipocampo–DMN bajo psilocibina.
- **Anti-correlaciones (más fiables que con GSR).** A diferencia de GSR, que impone matemáticamente distribución centrada en cero (Murphy & Fox, 2017), TEDANA no tiene este sesgo. Las anti-correlaciones observadas post-TEDANA tienen mayor probabilidad de reflejar inhibición recíproca genuina entre redes (e.g., DMN task-positive), análogamente a lo demostrado por Chai et al. (2012) con aCompCor.
- **Distribución de correlaciones.** Post-TEDANA, la distribución permanece sesgada hacia valores positivos (biológicamente esperado) con menor varianza espuria, a diferencia de post-GSR donde la distribución se centra forzosamente en cero.

3.5 3.5 Posición en el pipeline

TEDANA se aplica **después** de fMRIPrep y **antes** de la regresión residual de confounds:

fMRIPrep (`--me-output-echos`) → TEDANA → regresión residual (24P + scrubbing) → conectividad

Esta posición es la recomendada por la documentación de tedana: “*tedana now assumes that you’re working with data which has been previously preprocessed*” (DuPre et al., 2021). fMRIPrep realiza la corrección de movimiento, *slice-timing*, distorsión de susceptibilidad y normalización espacial sobre cada eco por separado; TEDANA recibe estos ecos preprocesados y realiza la separación BOLD/non-BOLD.

3.6 3.6 Consideración sobre GSR post-TEDANA

No se aplica GSR después de TEDANA. TEDANA ya elimina los componentes globales no-BOLD (movimiento, fisiológico) mediante la clasificación TE-dependiente. Aplicar GSR adicionalmente eliminaría señal neural global genuina que TEDANA preservó intencionalmente, constituyendo un doble denoising perjudicial (Kundu et al., 2013; DuPre et al., 2021). Por esta razón, la regresión residual post-TEDANA utiliza el modelo 24P (sin GSR) como pipeline primario, y aCompCor como control data-driven sin GSR. El modelo 36P (con GSR) se mantiene solo para triangulación metodológica con los datos de ds003059.

3.7 3.7 Ejecución de TEDANA

TEDANA se ejecuta sobre los ecos individuales preprocesados por fMRIPrep **en espacio BOLDref nativo** (post STC/HMC/SDC, sin resamplero a MNI). Ésta es la salida canónica de la bandera `--me-output-echos`: fMRIPrep no resampla los ecos individuales al espacio estándar porque el resamplero compuesto solo se realiza sobre la serie *optimally combined* (para minimizar interpolaciones redundantes y tamaño de disco). Los archivos correspondientes llevan el descriptor `part-mag` y **carecen** del descriptor `space-*` en el nombre:

```
sub-<subj>_ses-<ses>_task-<task>_dir-<PE>_run-<r>_echo-<k>_part-mag_desc-  
preproc_bold.nii.gz  
sub-<subj>_ses-<ses>_task-<task>_dir-<PE>_run-<r>_part-mag_desc-brain_mask.nii.gz
```

La salida de TEDANA queda igualmente en BOLDref nativo y debe **resamplearse a MNI152NLin2009cAsym a posteriori** (§3.11) componiendo las transformaciones `from-boldref_to-T1w` (.txt, affine 6 DOF de bbrregister) y `from-T1w_to-MNI152NLin2009cAsym` (.h5, affine + SyN no-lineal de ANTs) ya generadas por fMRIPrep.

3.8 3.8 Heterogeneidad de phase encoding y número de runs en ds006072

El dataset ds006072 combina dos direcciones de *phase encoding* (PE) y un número variable de runs por tarea según el sujeto. Esto obliga a que los patrones de búsqueda de archivos sean **dinámicos**, no estáticos:

Sujeto	PE (dir-)	Runs por tarea
sub-P1	AP	2 (run-1, run-2)
sub-P2	AP	1 (run-1)
sub-P3	AP	2 (run-1, run-2)
sub-P4	PA	1 (run-1)
sub-P5	PA	2 (run-1, run-2)
sub-P6	PA	1 (run-1)
sub-P7	PA	1 (run-1)

Phase encoding (AP vs. PA). La dirección de phase encoding determina el eje a lo largo del cual se acumulan las distorsiones geométricas por susceptibilidad magnética en EPI (Jezzard & Balaban, 1995). *Anterior→Posterior* (AP) comprime señal en corteza orbitofrontal y polos temporales hacia atrás; *Posterior→Anterior* (PA) lo hace en sentido opuesto. Tras la corrección de distorsión (SDC) aplicada por fMRIPrep con el fieldmap *PEpolar* — que sólo requiere un par de volúmenes con PE opuesto para estimar el campo (Andersson et al., 2003) — ambas adquisiciones quedan geoméricamente equivalentes en MNI. **No hay consecuencia analítica post-SDC**; la heterogeneidad PE de ds006072 sólo afecta al nombrado BIDS de los archivos (entity `dir-AP` vs. `dir-PA`).

Número de runs. Con `_run-N` siempre presente en los nombres (incluso para un único run), un glob sobre `run-2` simplemente devuelve lista vacía cuando falta, y los chunks hacen `continue` sin error. No se requiere lógica adicional para este caso.

Estrategia de robustez implementada en los chunks. Para evitar hardcodear `dir-AP` (lo que omitiría silenciosamente a sub-P4–P7), cada iteración (`subj`, `ses`, `task`, `run`) hace un *probe* inicial con wildcard `dir-*` sobre un archivo canónico del run (eco 1 para §3.7 y §4; `xfm boldref→T1w` para §3.11). Si el probe devuelve 0 coincidencias → el run no existe → se omite. Si devuelve 1 → se extrae la etiqueta (AP o PA) y se usa para construir el resto de nombres BIDS de ese run. Así un mismo chunk procesa correctamente las 7 combinaciones de PE × #runs sin intervención manual.

3.9 3.9 TEDANA en espacio nativo

El siguiente script procesa cada sujeto, sesión y run, aplicando TEDANA sobre los cinco ecos en BOLDref nativo:

```
#=====
# §3.7 - Denoising multi-echo con TEDANA
# Aplicado sobre los ecos preprocesados por fMRIPrep (--me-output-echos)
# Pipeline: fMRIPrep → TEDANA → regresión residual → conectividad
#
```

```

# Referencia principal: DuPre et al. (2021), JOSS 6(66):3669
# Fundamento: Kundu et al. (2012, NeuroImage; 2013, PNAS)
#=====

import os
import glob
import subprocess
import json

# -----
# Configuración
# -----

FMRIPREP_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/data/t0"
TEDANA_DIR   = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/tedana/tree/minima

# TEDANA opera en BOLDref nativo (los ecos no están resampleados a MNI;
# el resampleo a MNI se realiza post-TEDANA en §3.11)

# Tiempos de eco en segundos (BIDS convention; DuPre et al., 2021)
# Fuente: sidecars JSON de ds006072 (originales en ms: 14.20, 38.93,
# 63.66, 88.39, 113.12). TEDANA deprecó el input en ms; usar segundos.
ECHO_TIMES = [0.01420, 0.03893, 0.06366, 0.08839, 0.11312]

# Sujetos del protocolo principal (N=6; sub-P2 excluido en el pre-flight, PsiloData §3)
SUBJECTS = ["sub-P1", "sub-P3", "sub-P4",
            "sub-P5", "sub-P6", "sub-P7"]

# Mapeo sujeto → sesiones de droga (de session_data.csv)
DRUG_SESSIONS = {
    "sub-P1": [6, 11], # MTP, PSIL
    "sub-P3": [8, 12], # MTP, PSIL
    "sub-P4": [6, 10], # MTP, PSIL
    "sub-P5": [6, 10], # PSIL, MTP
    "sub-P6": [5, 9],  # MTP, PSIL
    "sub-P7": [7, 11], # PSIL, MTP
}

TASKS = ["BOLDREST1", "BOLDREST2", "BOLDREST3", "BOLDREST4"]
RUNS  = [1, 2] # Cada tarea BOLDREST contiene run-1 y run-2

os.makedirs(TEDANA_DIR, exist_ok=True)

# -----

```

```

# Procesar cada sujeto/sesión/tarea/run
# -----
for subj in SUBJECTS:
    for ses_num in DRUG_SESSIONS[subj]:
        ses = f"ses-{ses_num}"
        func_dir = os.path.join(FMRIPREP_DIR, subj, ses, "func")

        if not os.path.isdir(func_dir):
            print(f" Omitido: {subj} {ses} (directorio no encontrado)")
            continue

        for task in TASKS:
            for run_num in RUNS:
                run_label = f"run-{run_num}"

                # Detectar phase encoding (AP o PA) del run con un probe
                # sobre el eco 1. Si no existe → este run no existe para
                # este sujeto (ej. sujetos con un solo run): se omite.
                pe_probe = glob.glob(os.path.join(
                    func_dir,
                    f"{subj}_{ses}_task-{task}_dir-*_{run_label}"
                    f"_echo-1_part-mag_desc-preproc_bold.nii.gz"
                ))
                if not pe_probe:
                    # Caso normal para sub-P2, P4, P6, P7 con run-2 ausente
                    continue
                pe_dir = pe_probe[0].split("dir-")[1].split("_")[0]

                # Buscar los 5 ecos preprocesados (BOLDref nativo, sin space-*)
                echo_files = []
                for echo_idx in range(1, 6):
                    pattern = os.path.join(
                        func_dir,
                        f"{subj}_{ses}_task-{task}_dir-{pe_dir}_{run_label}"
                        f"_echo-{echo_idx}_part-mag"
                        f"_desc-preproc_bold.nii.gz"
                    )
                    matches = glob.glob(pattern)
                    if matches:
                        echo_files.append(matches[0])

                if len(echo_files) != 5:

```

```

        print(f" Omitido: {subj} {ses} {task} {run_label} "
              f"(encontrados {len(echo_files)}/5 ecos)")
        continue

# Buscar la máscara cerebral en BOLDref nativo (sin space-*)
mask_pattern = os.path.join(
    func_dir,
    f"{subj}_{ses}_task-{task}_dir-{pe_dir}_{run_label}"
    f"_part-mag_desc-brain_mask.nii.gz"
)
mask_files = glob.glob(mask_pattern)
if not mask_files:
    print(f" Omitido: {subj} {ses} {task} {run_label} "
          f"(máscara no encontrada)")
    continue

# Directorio de salida para este run
out_dir = os.path.join(
    TEDANA_DIR, subj, ses, task, run_label
)
os.makedirs(out_dir, exist_ok=True)

# Verificar si ya se procesó
dn_file = os.path.join(out_dir, "desc-denoised_bold.nii.gz")
if os.path.exists(dn_file):
    print(f" [OK] {subj} {ses} {task} {run_label} "
          f"(ya procesado)")
    continue

print(f"\n Ejecutando TEDANA: {subj} {ses} {task} {run_label}")
print(f" Ecos: {[os.path.basename(f) for f in echo_files]}")

# Construir comando tedana
# --fittype curvefit: regresión no lineal para T2*/S0
# --tedpca kic: selección de componentes PCA por criterio de
# información para datos multi-echo (Kundu et al., 2013).
# --tree minimal: árbol de decisión de tedana 25.x, simplificado,
# transparente y conservador frente al falso negativo (DuPre
# et al., 2021). Adoptado para todo el dataset (cf. §3.10).
# --mask: máscara cerebral de fMRIPrep
cmd = [
    "tedana",

```



```

        "-d"] + echo_files + [
        "-e"] + [str(te) for te in ECHO_TIMES] + [
        "--out-dir", out_dir,
        "--mask", mask_files[0],
        "--fittype", "curvefit",
        "--tedpca", "kic",
        "--tree", "minimal",
        "--convention", "bids",
        "--prefix", "",
        "--overwrite",
    ]

    print(f"    Comando: tedana -d [5 ecos] -e {ECHO_TIMES}")
    try:
        result = subprocess.run(
            cmd, capture_output=True, text=True, timeout=None
        )
        if result.returncode == 0:
            print(f"        TEDANA completado: {subj} {ses} "
                  f"{task} {run_label}")
        else:
            print(f"        Error TEDANA ({subj} {ses} "
                  f"{task} {run_label}):")
            print(result.stderr[-500:])
            if result.stderr else "Sin stderr")
    except subprocess.TimeoutExpired:
        print(f"        Timeout TEDANA ({subj} {ses} "
              f"{task} {run_label})")
    except FileNotFoundError:
        print("        tedana no encontrado. "
              "Instalar: pip install tedana")
        break

print("\n§3.7 completa (TEDANA multi-echo denoising en BOLDref nativo).")

```

3.10 Justificación de las opciones de TEDANA

Las opciones específicas elegidas para la ejecución de TEDANA (§3.7) se justifican como sigue:

- **--fittype curvefit**: regresión no lineal (Levenberg-Marquardt) del modelo de decaimiento $S(TE) = S_0 \cdot e^{-TE/T_2^*}$ para estimar mapas de T_2^* y S_0 voxel a voxel. La

alternativa **loglin** (regresión log-lineal, de solución cerrada) es más rápida pero menos precisa en vóxeles con baja SNR o con valores de T_2^* extremos, comunes en regiones de susceptibilidad (corteza orbitofrontal, lóbulo temporal medial) — regiones críticas para la DMN y para los efectos de la psilocibina sobre la conectividad hipocampo–corteza (Siegel et al., 2024). **La precisión de T_2^* es particularmente crítica aquí** porque las estadísticas pseudo- F (TE-dependencia, marcador BOLD) y (TE-independencia, marcador non-BOLD) que gobiernan la clasificación ICA de cada componente dependen directamente de la bondad del ajuste del modelo exponencial (Kundu et al., 2012, 2013). Un T_2^* sesgado por **loglin** en vóxeles OFC/MTL se traduce en atenuado y por tanto en componentes BOLD genuinos reclasificados como ruido (falsos negativos) — error especialmente costoso en un estudio centrado en la conectividad hipocampo-DMN bajo psilocibina (Siegel et al., 2024). Esta decisión es **complementaria con loglin en fM-RIPrep** (véase §2.2.1): el cómputo no-lineal se delega a la única etapa del pipeline donde la estimación de T_2^* domina el resultado final, patrón consistente con pipelines multi-echo Precision Functional Mapping (Lynch et al., 2020) y con las recomendaciones de la documentación de TEDANA para estudios centrados en la separación BOLD/non-BOLD basada en TE-dependencia (DuPre et al., 2021).

- **--tedpca kic**: criterio de información de Kundu (*Kundu Information Criterion*) para la selección del número de componentes PCA previo a ICA. A diferencia de **mdl** (Minimum Description Length) o **aic** (Akaike Information Criterion), **kic** fue diseñado específicamente para datos multi-echo fMRI y tiende a retener un número de componentes que preserva la dimensionalidad intrínseca de la señal BOLD sin sobreajustar a ruido térmico (Kundu et al., 2013). Para $N = 7$ sujetos con runs de ~15 min (513 volúmenes), **kic** proporciona un balance conservador entre retener señal y eliminar ruido.
- **--tree minimal**: árbol de decisión para la clasificación de componentes ICA. Se adopta **minimal** (introducido en tedana 25.x y documentado en DuPre et al., 2021): un árbol simplificado y transparente, con cada nodo aceptar/rechazar trazable y reproducible sobre el **desc-ICA_decision_tree.json** que TEDANA escribe por run. Su comportamiento es **conservador frente al falso negativo** — no descarta como ruido componentes BOLD genuinos —, una política especialmente apropiada en un estudio centrado en la desincronización de la conectividad bajo psilocibina (Siegel et al., 2024), donde los componentes BOLD globales son precisamente el objeto de interés. El ruido que un esquema más agresivo capturaría queda cubierto por las otras etapas del pipeline: NORDIC ya eliminó el ruido térmico (Section 1) y la regresión residual 24P + aCompCor + *scrubbing* posterior absorbe el movimiento y el ruido fisiológico residual (§4.3 y §5.2).
- **Denoising non-aggressive (default)**: TEDANA aplica regresión parcial para eliminar los componentes rechazados, preservando la varianza compartida entre componentes de señal y de ruido. La alternativa agresiva eliminaría *toda* la varianza de los componentes rechazados, incluyendo la porción correlacionada con señal neural — un riesgo especialmente relevante en análisis de conectividad funcional, donde la varianza compartida inter-componente puede reflejar acoplamiento funcional genuino.

3.11 3.11 Resampleo post-TEDANA a MNI152NLin2009cAsym

La serie `desc-denoised_bold.nii.gz` queda en BOLDref nativo (grid $\sim 110 \times 110 \times 72$, 2 mm iso). Para que §4 la consuma junto con el atlas Schaefer (ya en MNI) es necesario aplicar la composición de transformaciones generadas por fMRIPrep. Se usa `antsApplyTransforms` (interpolación Lanczos para preservar la resolución espectral) a través del contenedor Docker de fMRIPrep, que ya incluye ANTs 2.5.1 — evitando instalaciones adicionales:

```
#####
# §3.11 - Resampleo post-TEDANA: BOLDref nativo → MNI152NLin2009cAsym
# Compone: (boldref → T1w, bbrregister 6 DOF) (T1w → MNI, ANTs SyN)
# Interpolación: LanczosWindowedSinc (preserva espectro temporal BOLD)
#####

import os
import glob
import subprocess

FMRIPREP_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/fmriprep/data/t0"
TEDANA_DIR   = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/tedana/tree/minimal"
FMRIPREP_IMG = "nipreps/fmriprep:25.2.5"

# N=6; sub-P2 excluido en el pre-flight (PsiloData §3)
SUBJECTS = ["sub-P1", "sub-P3", "sub-P4",
            "sub-P5", "sub-P6", "sub-P7"]
DRUG_SESSIONS = {
    "sub-P1": [6, 11], "sub-P3": [8, 12],
    "sub-P4": [6, 10], "sub-P5": [6, 10], "sub-P6": [5, 9],
    "sub-P7": [7, 11],
}
TASKS = ["BOLDREST1", "BOLDREST2", "BOLDREST3", "BOLDREST4"]
RUNS = [1, 2]

def _find_one(pattern):
    m = glob.glob(pattern)
    return m[0] if m else None

for subj in SUBJECTS:
    # Transform T1w → MNI (anatómico, compartido por todas las sesiones)
    anat_pattern = os.path.join(
        FMRIPREP_DIR, subj, "ses-*", "anat",
        f"{subj}_ses-*_from-T1w_to-MNI152NLin2009cAsym_mode-image_xfm.h5"
    )
```

```

xfm_t1w_to_mni = _find_one(anat_pattern)
if xfm_t1w_to_mni is None:
    print(f" [skip] {subj}: no T1w->MNI xfm")
    continue

for ses_num in DRUG_SESSIONS[subj]:
    ses = f"ses-{ses_num}"
    func_dir = os.path.join(FMRIPREP_DIR, subj, ses, "func")
    if not os.path.isdir(func_dir):
        continue

    for task in TASKS:
        for run_num in RUNS:
            rlab = f"run-{run_num}"

            # Detectar phase encoding (AP o PA) del run. Usamos el xfm
            # boldref->T1w como probe - existe 1:1 con cada run válido.
            pe_probe = _find_one(os.path.join(
                func_dir,
                f"{subj}_{ses}_task-{task}_dir-*_{rlab}"
                f"_from-boldref_to-T1w_mode-image_desc-coreg_xfm.txt"
            ))
            if pe_probe is None:
                # Run no existe (sujetos con un único run): se omite
                continue
            pe_dir = os.path.basename(pe_probe).split("dir-")[1].split("_")[0]
            base = f"{subj}_{ses}_task-{task}_dir-{pe_dir}_{rlab}"

            in_nii = os.path.join(TEDANA_DIR, subj, ses, task, rlab,
                                   "desc-denoised_bold.nii.gz")
            if not os.path.exists(in_nii):
                print(f" [skip] {base}: sin output TEDANA")
                continue

            # Referencia de grid: boldref ya resampleado a MNI por fMRIPrep
            ref = _find_one(os.path.join(
                func_dir,
                f"{base}_part-mag_space-MNI152NLin2009cAsym_boldref.nii.gz"
            ))
            # Transform BOLDref -> T1w (específico por run)
            xfm_bold_to_t1w = _find_one(os.path.join(
                func_dir,

```

```

        f"{base}_from-boldref_to-T1w_mode-image_desc-coreg_xfm.txt"
    ))
    if not ref or not xfm_bold_to_t1w:
        print(f" [skip] {base}: faltan referencias o xfms")
        continue

    out_nii = in_nii.replace(
        "desc-denoised_bold.nii.gz",
        "space-MNI152NLin2009cAsym_desc-denoised_bold.nii.gz"
    )
    if os.path.exists(out_nii):
        print(f" [OK] {base}: ya resampleado")
        continue

# Traducción de rutas Windows a POSIX dentro del contenedor
def dockerize(path):
    return path.replace("\\", "/")

cmd = [
    "docker", "run", "--rm",
    "-v", f"{dockerize(FMRIPREP_DIR)}:/fmriprep:ro",
    "-v", f"{dockerize(TEDANA_DIR)}:/tedana",
    "--entrypoint", "antsApplyTransforms",
    FMRIPREP_IMG,
    "-d", "3", "-e", "3",
    "-i", dockerize(in_nii).replace(
        dockerize(TEDANA_DIR), "/tedana"),
    "-r", dockerize(ref).replace(
        dockerize(FMRIPREP_DIR), "/fmriprep"),
    "-o", dockerize(out_nii).replace(
        dockerize(TEDANA_DIR), "/tedana"),
    "-t", dockerize(xfm_t1w_to_mni).replace(
        dockerize(FMRIPREP_DIR), "/fmriprep"),
    "-t", dockerize(xfm_bold_to_t1w).replace(
        dockerize(FMRIPREP_DIR), "/fmriprep"),
    "-n", "LanczosWindowedSinc",

    "--float",
]
print(f" Resampleando {base} -> MNI")
r = subprocess.run(cmd, capture_output=True, text=True)
if r.returncode != 0:

```

```
print(f"      [error] {r.stderr[-400:]}")

print("\nFase 4.3.2 completa (denoised TEDANA en MNI152Nlin2009cAsym).")
```

Nota sobre orden de transformaciones. `antsApplyTransforms` aplica la lista de `-t` en **orden inverso** (stack): el último `-t` se aplica primero. Por tanto, para ir de BOLDref $\rightarrow$ MNI, se pasa primero `T1w  $\rightarrow$  MNI` y después `boldref  $\rightarrow$  T1w`, y ANTs ejecuta `boldref  $\rightarrow$  T1w  $\rightarrow$  T1w  $\rightarrow$  MNI`. La interpolación `LanczosWindowedSinc` es la recomendada por fMRIPrep para series temporales BOLD (preserva el espectro de frecuencias sin *ringing* apreciable; el default `Linear` suaviza en exceso).

3.12 3.12 Outputs de TEDANA

Para cada run procesado, TEDANA genera en el directorio de salida (todos los NIfTI en **BOLDref nativo**, tras §3.7):

Archivo	Descripción
<code>desc-denoised_bold.nii.gz</code>	Serie BOLD denoised en BOLDref nativo — resampleada a MNI en §3.9
<code>desc-optcom_bold.nii.gz</code>	Combinación óptima ponderada por T2* sin denoising (el “pre” interno para el QC)
<code>desc-tedana_metrics.tsv</code>	Métricas por componente ICA (κ , ρ , varianza, clasificación)
<code>desc-ICA_mixing.tsv</code>	Matriz de mezcla ICA
<code>desc-tedana_registry.json</code>	Registro de clasificación de componentes
<code>figures/</code>	Reportes HTML con visualizaciones de componentes

Tras §3.11 se añade además `space-MNI152Nlin2009cAsym_desc-denoised_bold.nii.gz` — **el input real para §4** (extracción de conectividad). La serie nativa se conserva para auditoría y para reprocesamiento con otras transformaciones si se desea.

3.13 3.13 QC cuantitativo del bulk run TEDANA

Una vez completado TEDANA sobre todo el Tier 0 (Section 3) en `D:/.../prep/tedana/tree/minimal/`, el siguiente chunk reproduce —sobre *cada* run denoised— el panel diagnóstico cuantitativo que cierra la cadena `NORDIC  $\rightarrow$  fMRIPrep  $\rightarrow$  TEDANA`. La motivación operacional es la misma que la de Section 1.9 y Section 2.3: inspeccionar 12 reportes HTML de TEDANA uno a uno y abrir 256 PNG de componentes por run es impracticable conforme se añaden tiers, los

umbrales sobre κ/ρ , varianza explicada, fit de T_2^* , máscara adaptativa y residuales tienen que quedar registrados en código en lugar de en el ojo del operador, y el verdict por run debe ser comparable entre Tier 0 y tiers posteriores.

Diferencias de diseño respecto al QC del bulk fMRIPrep:

1. **Sin pareja pre/post entre discos:** TEDANA se evalúa por contraste entre `desc-optcom_bold.nii.gz` (combinación óptima ponderada por T_2^* , sin denoising — el “pre” interno) y `desc-denoised_bold.nii.gz` (la misma serie tras proyectar los componentes rechazados por la clasificación TE-dependiente — el “post”). Ambos archivos coexisten dentro del mismo directorio de run, lo que simplifica el descubrimiento y la idempotencia (DuPre et al., 2021).
2. **Métricas ICA-céntricas en lugar de movimiento:** la evaluación no descansa en FD/DVARS (esas pertenecen a Section 2.3, evaluadas sobre los confounds de fMRIPrep) sino en los descriptores propios de la descomposición TEDANA — fracción de componentes aceptados, varianza explicada por aceptados vs rechazados, separación bimodal de κ/ρ respecto a sus elbows, y consistencia del fit monoexponencial (mapa de T_2^* , mapa de RMSE, máscara de fit failures). Estas métricas son las únicas que capturan el riesgo específico de TEDANA: *over-rejection* (descarta componentes BOLD genuinos) o *under-rejection* (deja componentes de ruido).
3. **Idempotencia por filesystem:** igual que NORDIC y fMRIPrep, si los dos PNG del run ya existen en `qc/tedana/sub-X/ses-Y/<task>/<run>/` el run se salta sin re-leer NIFTIs.
4. **Espacio nativo (BOLDref):** TEDANA escribe en BOLDref nativo (§3.7), por lo que el QC se computa en ese espacio. El paso a MNI (§3.11) se valida aparte como integridad de output: si falta el `space-MNI152NLin2009cAsym_desc-denoised_bold.nii.gz` el run no entra en §4.
5. **Decoupling del verdict respecto al `tedana_report.html`:** el reporte HTML interactivo (RICA, scree plots de κ/ρ , mapas de componentes) sigue siendo la verdad visual y debe revisarse para los runs marcados como `REVISAR` o `FAIL` por este chunk. Las métricas automáticas son una *primera criba* que reproduce los criterios cuantitativos sobre los outputs canónicos.

3.13.1 3.13.1 Métricas de QC

1. **Integridad estructural de outputs:** por cada run del Tier 0 deben existir los archivos canónicos del registro TEDANA — `desc-denoised_bold.nii.gz`, `desc-optcom_bold.nii.gz`, `T2starmap.nii.gz`, `S0map.nii.gz`, `desc-adaptiveGoodSignal_mask.nii.gz`, `desc-decayFitFailures_mask.nii.gz`, `desc-rmse_statmap.nii.gz`, `desc-tedana_metrics.tsv` + `.json`, `desc-ICACrossComponent_metrics.json` y `desc-ICA_decision_tree.json`. Adicionalmente debe existir el output resampleado a MNI tras §3.11 (`space-MNI152NLin2009cAsym_desc-denoised_bold.nii.gz`). La ausencia de cualquiera dispara `FAIL` sin computar el resto de métricas. Sobre la pareja `optcom/denoised`

se verifica `same_shape4d` y `same_affine` (TEDANA no debería alterar geometría ni número de frames).

2. **Descomposición ICA — número y proporción de componentes:** `n_components` (total tras `--tedpca kic`), `n_accepted`, `n_rejected`, y `pct_accepted = n_accepted / n_components`. Para `--tedpca kic` con runs de 510 frames de ds006072 (post-trim NORDIC), `n_components` típico es 25–80 según ruido del run, y `pct_accepted` típico es 10–40 % (Kundu et al., 2013; DuPre et al., 2021). **Fuera de rango:** `pct_accepted < 5 %` *over-rejection* (riesgo de pérdida de señal BOLD genuina); `pct_accepted > 60 %` *under-rejection* (componentes de ruido retenidos, denoising débil).
3. **Varianza explicada — accepted vs rejected:** `varex_accepted_total = Σ var. exp. de componentes accepted`, `varex_rejected_total`, y el ratio `total_var_exp_rejected_components_on_accepted` que TEDANA escribe directamente en `desc-ICACrossComponent_metrics.json`. Esta última métrica captura *cuánta varianza de los rechazados está correlacionada con la de los aceptados* tras la regresión non-aggressive — valores altos (> 30 %) indican que el denoising parcial dejó varianza de ruido en la serie denoised por compartimiento de subespacio. **Rango esperado para `pct_varex_accepted` post-NORDIC:** 10–70 % del total. La banda original de Kundu et al. 2013 (30–70 % en datos *raw*) se desplaza hacia abajo porque NORDIC (Section 1) ya eliminó el ruido térmico — la varianza absoluta del optcom es menor y los componentes BOLD explican una fracción más reducida del total. Fuera de [5, 90] % patológico.
4. **Separación κ/ρ — eficacia de la TE-dependence:** los κ/ρ elbows de Kundu (`kappa_elbow_kundu`, `rho_elbow_kundu`) escritos en `desc-ICACrossComponent_metrics.json` y, sobre `desc-tedana_metrics.tsv`, la mediana de κ y ρ por grupo (accepted/rejected). El ratio de separación `kappa_sep_ratio = median(_accepted) / median(_rejected)` debe ser > 2.5 (los aceptados son TE-dependientes). El ratio `rho_sep_ratio = median(_rejected) / median(_accepted)` idealmente sería > 1.5 , pero en la práctica muchos componentes rechazados son fisiológicos (similar a BOLD) y se descartan por *otros* criterios del árbol de decisión (varianza explicada, umbrales de /), no por un alto; por eso se exige solamente > 1.2 (warn si no, fail si < 0.8 , que sería una inversión genuina de la separación). **Solapamiento con elbow:** fracción de componentes accepted con $\kappa < \text{kappa_elbow}$ — el árbol de decisión puede admitir componentes bajo el elbow si pasan otros criterios, por lo que el umbral se fija en $< 25\%$ (warn) y $< 50\%$ (fail). Por encima del 50 % el clasificador está aceptando casi al azar respecto al criterio canónico de TE-dependencia (Kundu et al., 2012). Estas métricas son la **firma cuantitativa de la separación BOLD/non-BOLD** que justifica TEDANA frente a aCompCor/24P (Section 3).
5. **Mapa T_2^* — plausibilidad fisiológica del fit monoexponencial:** histograma de `T2starmap.nii.gz` dentro de la máscara adaptativa (vóxeles con ≥ 1 eco bueno). **Nota de unidades:** TEDANA escribe el mapa en **segundos** por convención BIDS (DuPre et al., 2021); el chunk autodetecta esa convención (mediana del foreground < 1.5 segundos)

y multiplica por 1000 antes del histograma y los umbrales en ms. Métricas derivadas: moda, mediana, % de vóxeles con $T_2^* \in [25, 70]$ ms (fisiológicamente plausibles a 3 T; Posse et al., 1999), y % de vóxeles con T_2^* fallido o saturado (< 5 ms o > 150 ms). Bi-modalidad o moda fuera de $[30, 60]$ ms — fit `curvefit` divergiendo o eco 5 saturado — error que se propaga directamente a κ y por tanto a la clasificación de componentes (§3.10).

6. **Mapa S0 y RMSE del fit:** mediana y rango dinámico de `S0map.nii.gz` dentro de la máscara adaptativa; mediana de `desc-rmse_statmap.nii.gz` (residual root-mean-square del ajuste exponencial vóxel-a-vóxel). RMSE alto en regiones plausibles — fit pobre, riesgo de κ/ρ ruidosos. El número de vóxeles en `desc-decayFitFailures_mask.nii.gz` (fit failures explícitos donde TEDANA cae al log-linear) debe ser $< 2\%$ de la máscara; valores altos señalan SNR insuficiente en una fracción del cerebro.
7. **Máscara adaptativa — perfil de cobertura multi-echo:** `desc-adaptiveGoodSignal_mask.nii.gz` toma valores $0-N_{\text{echos}}$ (0–5 en ds006072), donde el valor de cada vóxel es el número de ecos con “señal buena” según el método de dropout (DuPre et al., 2021). Métricas: `pct_full` = fracción de vóxeles intra-cerebro con valor = N_{echos} (incluido en optcom + ICA + denoising), `pct_lt3` = fracción de vóxeles con valor < 3 (excluidos del paso de clasificación ICA, aunque conservados en optcom). **Expectativa:** `pct_full` $> 70\%$ y `pct_lt3` $< 10\%$ para una adquisición sin dropout extremo. Valores bajos de `pct_full` indican adquisiciones afectadas por susceptibilidad u offset de cabeza pronunciado — frecuente en OFC y polos temporales con TEs largos en ds006072.
8. **Eficacia del denoising — tSNR optcom vs denoised:** mediana del tSNR ($\bar{S}/\sigma_S$ temporal) dentro de la máscara adaptativa ≥ 3 ecos, calculada sobre `desc-optcom_bold.nii.gz` (pre) y `desc-denoised_bold.nii.gz` (post). **Expectativa post-NORDIC:** `tsnr_gain` $\in [1.2, 3.5]$. Las bandas originales de Kundu 2012/2013 (1.5–2 $\times$ sobre datos *raw*) no se aplican directamente aquí porque NORDIC (Section 1) ya removi6 el ruido térmico antes de fMRIPrep — la varianza residual del optcom está dominada por componentes estructurados (movimiento, fisiológico, hardware) que son precisamente la *diana* de la clasificación TE-dependiente. Por tanto la reducción relativa de σ_S temporal puede ser sustancialmente mayor que la reportada en datos *raw* sin denoising previo. **Indicadores de over-rejection:** `tsnr_gain` > 5.0 (claro fail) combinado con `pct_acc_below_kelbow` alto o `psd_bold_ratio` < 0.85 — la ganancia se sostiene por pérdida de señal BOLD genuina. Ganancia < 0.95 — denoising sin efecto o regresión non-aggressive insuficiente. La magnitud por sí sola **no es diagn6stica**; debe combinarse con la PSD de banda BOLD (§punto 10) y con `pct_acc_below_kelbow` (§punto 4) para distinguir denoising legítimo de over-rejection.
9. **Conservación de la señal:** `mean_drift` = $|\text{mean}(\text{denoised}) - \text{mean}(\text{optcom})| / \text{mean}(\text{optcom})$ en la máscara adaptativa ≥ 1 . Debe ser $< 2\%$ (TEDANA aplica regresión parcial sobre los componentes rechazados, no debe alterar la media). Drift $> 5\%$ — bug grave en denoising o componentes globales mal clasificados.

10. **Espectro de potencia — preservación BOLD vs supresión de ruido:** PSD del foreground de optcom y denoised. La banda BOLD (0.01–0.10 Hz) debe preservarse (ratio post/pre ≈ 1); la banda alta de ruido (> 0.15 Hz) debe atenuarse en denoised si TEDANA capturó componentes fisiológicos o de hardware (Power et al., 2018). Atenuación uniforme en todas las frecuencias *over-denoising* (el árbol está rechazando señal); ningún cambio espectral entre optcom y denoised TEDANA no actuó (típicamente `n_accepted = n_components`, ningún componente clasificado como rechazado).
11. **Carpet plot proxy — global signal antes/después:** serie temporal del global signal en la máscara adaptativa ≥ 3 para optcom y denoised. Desviación típica del GS post debe ser $\leq$ pre (TEDANA elimina componentes globales no-BOLD; Kundu et al., 2013). El carpet completo (5 tejidos $\times T$ frames) está disponible como SVG nativo de TEDANA en `figures/carpet_optcom.svg`, `figures/carpet_denoised.svg`, `figures/carpet_accepted.svg`, `figures/carpet_rejected.svg` (Provins et al., 2022) y debe consultarse manualmente para runs marcados REVISAR.

Métrica	Rango esperado / umbral	Si sale fuera
Integridad de outputs (10 archivos + MNI denoised)	todos presentes	falta cualquiera FAIL (re-ejecutar TEDANA o §3.11 para el run)
<code>n_accepted</code>	3–60	< 3 FAIL over-rejection; > 60 REVISAR
<code>pct_accepted</code>	10–40 %	5–10 % o 40–60 % REVISAR; < 5 o > 60 FAIL
<code>pct_varex_accepted</code> (post-NORDIC)	10–70 %	< 5 o > 90 FAIL; 5–10 % o 70–90 % REVISAR
<code>varex_rej_on_acc</code> (cross-component)	< 30 %	30–50 REVISAR; > 50 FAIL
<code>kappa_sep_ratio</code> (acc/rej)	> 2.5	1.5–2.5 REVISAR; < 1.5 FAIL (TE-dependence rota)
<code>rho_sep_ratio</code> (rej/acc)	> 1.2	0.8–1.2 REVISAR; < 0.8 FAIL (inversión genuina)
<code>pct_acc_below_kappa_elbow</code>	< 25 %	25–50 REVISAR; > 50 FAIL
<code>t2star_mode_ms</code> (auto-convert s $\rightarrow$ ms)	30–60 ms (córtex 3 T; Posse 1999)	< 25 o > 80 FAIL; otros REVISAR
<code>t2star_pct_plausible</code> ([25,70] ms)	> 75 %	60–75 REVISAR; < 60 FAIL
<code>pct_decayfit_failures</code>	< 2 % de la máscara	2–5 REVISAR; > 5 FAIL
<code>rmse_median</code>	rango compatible entre runs (z-score por sujeto)	outlier > 2 REVISAR

Métrica	Rango esperado / umbral	Si sale fuera	
pct_adaptmask_full (= N_echos)	> 70 %	50–70 FAIL	REVISAR; < 50
pct_adaptmask_lt3 (< 3 ecos)	< 10 %	10–20 FAIL	REVISAR; > 20
tsnr_gain_denoised_optcom (post-NORDIC)	1.2–3.5	0.95–1.2 o 3.5–5.0 FAIL	REVISAR; < 0.95 o > 5.0
mean_drift_optcom_denoised	< 2 %	2–5	REVISAR; > 5 FAIL
PSD ratio en banda BOLD (0.01–0.1 Hz)	0.85–1.15	fuera	REVISAR (over-denoising si < 0.85)
gs_std_ratio (denoised/optcom)	≤ 1	> 1.05	REVISAR (denoising no removi3 GS no-BOLD)

Referencias. Kundu et al. 2012, 2013 (TE-dependence, κ/ρ , decision tree); DuPre et al. 2021 (TEDANA, JOSS; outputs can3nicos y m3tricas); Posse et al. 1999 (T_2^* cortical a 3 T); Power et al. 2018 (ME-ICA y motion); Provins et al. 2022 (carpet plot 5-tejidos).

3.13.2 3.13.2 Comprobaciones adicionales recomendadas (no automatizadas)

1. Reporte HTML interactivo `tedana_report.html`:

- **Summary view:** scatter κ vs ρ (cada componente coloreado por classification). Los aceptados deben caer en la regi3n κ alto / ρ bajo; rechazados en la regi3n opuesta. Bandas diagonales (componentes con $\kappa \approx \rho$) son ambiguas y deben revisarse uno a uno.
- **Kappa scree plot y Rho scree plot:** el codo de cada distribuci3n debe coincidir aproximadamente con el clasificador. Codos suaves pocos componentes claramente BOLD; codos pronunciados separaci3n neta (el caso favorable).
- **Individual component view:** para cada componente sospechoso, inspeccionar mapa espacial, time series, y power spectrum. Componentes aceptados con mapa de aspecto de movimiento (rim cortical) o frecuencias respiratorias (~0.3 Hz) deben reclasificarse manualmente con RICA (Section 3).

2. Carpet plots SVG nativos en `figures/`: `carpet_optcom.svg` (pre) y `carpet_denoised.svg` (post). Las bandas horizontales sincronizadas (movimiento) deben atenuarse claramente en denoised; si persisten revisar `carpet_rejected.svg` para ver si TEDANA captur3 el componente de movimiento o lo dej3 pasar.

3. Componentes individuales `figures/comp_XXX.png`: para los runs REVISAR, ojea los primeros 10 componentes ordenados por varianza explicada — los m3s informativos para diagn3stico (Kundu et al., 2012).

4. **Reclasificación manual con RICA** (`open_rica_report.py` está pre-cocinado en el directorio de cada run): herramienta interactiva del proyecto tedana para auditar visualmente componentes ambiguos. Si tras la auditoría un run se reclasifica, re-ejecutar `ica_reclassify` con la tabla manual (Section 3) y volver a lanzar este chunk; la idempotencia detecta el cambio si el resumen PNG existe — borrar el `summary.png` del run reclasificado para forzar el QC.
5. **Comparación inter-sujeto del fingerprint TEDANA**: agregada en `metrics.csv`, comparar `n_accepted`, `pct_varex_accepted`, `kappa_sep_ratio` entre sesiones MTP y PSIL del mismo sujeto. La psilocibina **debería** producir cambios cuantificables en la dimensionalidad de componentes BOLD (Tagliazucchi et al., 2014; Carhart-Harris et al., 2014), por lo que diferencias intra-sujeto son esperables, pero asimetrías sistemáticas entre sujetos (e.g., un sujeto con `n_accepted` 5× menor que el resto) apuntan a un problema técnico, no biológico.
6. **Decision tree status table** (`desc-ICA_status_table.tsv`): si un nodo del árbol clasifica > 50 % de los componentes en un solo paso, es señal de que el árbol `minimal` está actuando sobre datos fuera de su rango de calibración; el run pasa a **REVISAR** y se inspecciona manualmente vía `tedana_report.html`.

```
# =====
# QC del bulk run TEDANA - panel diagnóstico cuantitativo sobre cada run de
# `prep/tedana/tree/minimal/`, con la misma filosofía que @sec-nordic-qc-bulk y
# @sec-fmriprep-qc:
#   - métricas derivadas de los outputs (no del HTML)
#   - umbrales calibrados de literatura (§3.13.1)
#   - idempotencia por filesystem (PNG presentes skip)
#   - tabla unificada `metrics.csv` con verdict por run
#
# Entrada (por run):
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-denoised_bold.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-optcom_bold.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/T2starmap.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/S0map.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-adaptiveGoodSignal_mask.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-decayFitFailures_mask.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-rmse_statmap.nii.gz
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-tedana_metrics.tsv
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/desc-ICACrossComponent_metrics.json
#   - TEDANA_DIR/sub-X/ses-Y/<task>/<run>/space-MNI152Nlin2009cAsym_desc-denoised_bold.nii.gz
#
# Salida:
#   - QC_DIR/sub-X/ses-Y/<task>/<run>/summary.png    (/ scatter + T2* + tSNR + adaptive mask)
#   - QC_DIR/sub-X/ses-Y/<task>/<run>/spectra.png    (PSD + GS optcom vs denoised)
#   - QC_DIR/metrics.csv
```

```

# =====
import os, glob, json
from collections import defaultdict
import numpy as np
import pandas as pd
import nibabel as nib
import matplotlib.pyplot as plt
from scipy import ndimage, signal

TEDANA_DIR = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/tedana/tree/minimal"
QC_DIR      = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/tedana/qc/minimal"
TR          = 1.761    # ds006072 (igual que @sec-nordic-qc-bulk, @sec-fmriprip-qc)
N_ECHOS     = 5        # ds006072 multi-echo BOLD
os.makedirs(QC_DIR, exist_ok=True)

# Umbrales - calibrados de Kundu 2012/2013, DuPre 2021, Posse 1999, Power 2018.
# Ajustados para pipeline post-NORDIC (@sec-nordic): NORDIC ya removi  el ruido
# t rmico, as  que (i) la fracci n de varianza atribuida a componentes BOLD es
# t picamente menor que en datos raw (lo que sobra es BOLD + estructurado), y
# (ii) la ganancia tSNR de TEDANA sobre optcom puede ser sustancialmente > 1.6
# porque optcom todav a contiene la varianza fisiol gica/movimiento estructurada
# que TEDANA proyecta. Los rangos *raw* de Kundu 2012 (varex_acc 30-70 %,
# tsnr_gain 1.5-2) se relajan en consecuencia.
TH = dict(
    n_acc_lo      = 3,
    n_acc_hi      = 60,
    pct_acc_lo     = 10.0,
    pct_acc_hi     = 40.0,
    pct_acc_fail_lo = 5.0,
    pct_acc_fail_hi = 60.0,
    # Post-NORDIC: con thermal noise ya removido, BOLD explica menos varianza
    # absoluta del optcom que en raw data. Bandas m s generosas.
    pct_varex_acc_lo  = 10.0,
    pct_varex_acc_hi  = 70.0,
    pct_varex_fail_lo = 5.0,
    pct_varex_fail_hi = 90.0,
    varex_rej_on_acc  = 30.0,
    varex_rej_on_acc_f = 50.0,
    kappa_sep_ratio_lo = 2.5,
    kappa_sep_ratio_f  = 1.5,
    # -separaci n: muchos rechazados son fisiol gicos ( similar a BOLD) y se
    # descartan por varianza/topolog a, no por ; umbral menos estricto.

```

```

rho_sep_ratio_lo    = 1.2,
rho_sep_ratio_f     = 0.8,
# El árbol de decisión puede aceptar componentes con < elbow si pasan
# otros criterios (varianza, umbrales / ). Tolerar 25 % accepted bajo
# elbow antes de marcar warn, 50 % antes de fail.
pct_acc_below_elbow = 25.0,
pct_acc_below_f     = 50.0,
t2star_mode_lo      = 30.0,
t2star_mode_hi      = 60.0,
t2star_mode_fail_lo = 25.0,
t2star_mode_fail_hi = 80.0,
t2star_plaus_pct     = 75.0,
t2star_plaus_fail    = 60.0,
decayfit_fail_pct    = 2.0,
decayfit_fail_f      = 5.0,
pct_full_lo          = 70.0,
pct_full_fail        = 50.0,
pct_lt3_hi           = 10.0,
pct_lt3_fail         = 20.0,
# Post-NORDIC: la varianza estructurada que TEDANA elimina representa la
# mayor parte de la varianza residual del optcom; ganancias 1.2-3.5×
# son fisiológicas. >5× sí indica over-rejection.
tsnr_gain_lo         = 1.2,
tsnr_gain_hi         = 3.5,
tsnr_gain_fail_lo    = 0.95,
tsnr_gain_fail_hi    = 5.0,
mean_drift            = 0.02,
mean_drift_fail       = 0.05,
psd_bold_lo          = 0.85,
psd_bold_hi          = 1.15,
gs_std_ratio_hi       = 1.05,
)

# -----
# Discovery - cada run vive en TEDANA_DIR/sub-X/ses-Y/<task>/<run>/
# -----

def discover_runs():
    runs = {}
    for run_dir in sorted(glob.glob(
        os.path.join(TEDANA_DIR, "sub-*", "ses-*", "*", "run-*"))):
        parts = run_dir.replace("\\", "/").split("/")
        run, task, ses, subj = parts[-1], parts[-2], parts[-3], parts[-4]

```

```

        runs[(subj, ses, task, run)] = run_dir
    return runs

NEEDED = [
    "desc-denoised_bold.nii.gz",
    "desc-optcom_bold.nii.gz",
    "T2starmap.nii.gz",
    "S0map.nii.gz",
    "desc-adaptiveGoodSignal_mask.nii.gz",
    "desc-decayFitFailures_mask.nii.gz",
    "desc-rmse_statmap.nii.gz",
    "desc-tedana_metrics.tsv",
    "desc-ICACrossComponent_metrics.json",
    "desc-ICA_decision_tree.json",
    "space-MNI152NLin2009cAsym_desc-denoised_bold.nii.gz",
]

def integrity_check(run_dir):
    miss = [f for f in NEEDED if not os.path.exists(os.path.join(run_dir, f))]
    return (not miss), miss

# -----
# Métricas - devuelve un dict plano por run
# -----

def ica_metrics(run_dir):
    """Componentes ICA: counts, varex, separación / ."""
    df = pd.read_csv(os.path.join(run_dir, "desc-tedana_metrics.tsv"), sep="\t")
    cc = json.load(open(os.path.join(run_dir, "desc-ICACrossComponent_metrics.json")))
    cls = df["classification"].astype(str).str.lower()
    acc = df[cls.str.startswith("acc")]
    rej = df[cls.str.startswith("rej")]
    n_total, n_acc, n_rej = len(df), len(acc), len(rej)
    varex_total = float(df["variance explained"].sum()) + 1e-9
    varex_acc = float(acc["variance explained"].sum())
    varex_rej = float(rej["variance explained"].sum())
    # Separación / (acc vs rej)
    med_k_acc = float(acc["kappa"].median()) if n_acc else np.nan
    med_k_rej = float(rej["kappa"].median()) if n_rej else np.nan
    med_r_acc = float(acc["rho"].median()) if n_acc else np.nan
    med_r_rej = float(rej["rho"].median()) if n_rej else np.nan
    kappa_sep = (med_k_acc / med_k_rej) if (n_acc and n_rej and med_k_rej > 0) else np.nan
    rho_sep = (med_r_rej / med_r_acc) if (n_acc and n_rej and med_r_acc > 0) else np.nan

```

```

kappa_elbow = float(cc.get("kappa_elbow_kundu", np.nan))
rho_elbow   = float(cc.get("rho_elbow_kundu",   np.nan))
pct_acc_below_kelbow = (
    100.0 * float((acc["kappa"] < kappa_elbow).mean()) if n_acc and np.isfinite(kappa_elbow)
)
pct_acc_above_relbow = (
    100.0 * float((acc["rho"] > rho_elbow).mean()) if n_acc and np.isfinite(rho_elbow)
)
return dict(
    n_components      = n_total,
    n_accepted        = n_acc,
    n_rejected        = n_rej,
    pct_accepted       = 100.0 * n_acc / max(n_total, 1),
    pct_varex_accepted = 100.0 * varex_acc / varex_total,
    pct_varex_rejected = 100.0 * varex_rej / varex_total,
    varex_rej_on_acc   = float(cc.get(
        "total_var_exp_rejected_components_on_accepted", np.nan)),
    kappa_elbow        = kappa_elbow,
    rho_elbow          = rho_elbow,
    median_kappa_accepted = med_k_acc,
    median_kappa_rejected = med_k_rej,
    median_rho_accepted  = med_r_acc,
    median_rho_rejected  = med_r_rej,
    kappa_sep_ratio     = kappa_sep,
    rho_sep_ratio       = rho_sep,
    pct_acc_below_kelbow = pct_acc_below_kelbow,
    pct_acc_above_relbow = pct_acc_above_relbow,
    _df_metrics         = df,
)

def t2star_metrics(run_dir):
    """Histograma T2*/S0 en máscara adaptativa (>=1 eco).

    TEDANA escribe `T2starmap.nii.gz` en **segundos** (BIDS convention;
    DuPre 2021). Detectamos unidades por la mediana del foreground: si
    < 1.5 → segundos → multiplicamos por 1000 para los thresholds en ms.
    """
    t2s = nib.load(os.path.join(run_dir, "T2starmap.nii.gz")).get_fdata(dtype=np.float32)
    s0  = nib.load(os.path.join(run_dir, "S0map.nii.gz")).get_fdata(dtype=np.float32)
    mk  = nib.load(os.path.join(run_dir,
        "desc-adaptiveGoodSignal_mask.nii.gz")).get_fdata(dtype=np.float32) >= 1
    fail = nib.load(os.path.join(run_dir,

```



```

        "desc-decayFitFailures_mask.nii.gz")).get_fdata(dtype=np.float32) > 0.5
rmse = nib.load(os.path.join(run_dir,
        "desc-rmse_statmap.nii.gz")).get_fdata(dtype=np.float32)
vals = t2s[mk & np.isfinite(t2s) & (t2s > 0)]
t2s_units = "ms"
if vals.size and np.median(vals) < 1.5:
    # T2* en segundos → convertir a ms para histograma y umbrales
    vals = vals * 1000.0
    t2s_units = "s→ms"
if vals.size == 0:
    out = dict(t2star_median=np.nan, t2star_mode_ms=np.nan,
              t2star_pct_plausible=np.nan, t2star_pct_failed=np.nan,
              t2star_units=t2s_units)
else:
    hist, edges = np.histogram(vals, bins=200, range=(0, 200))
    mode = 0.5 * (edges[np.argmax(hist)] + edges[np.argmax(hist) + 1])
    out = dict(
        t2star_median      = float(np.median(vals)),
        t2star_mode_ms     = float(mode),
        t2star_pct_plausible = 100.0 * float(((vals >= 25.0) & (vals <= 70.0)).mean()),
        t2star_pct_failed   = 100.0 * float(((vals < 5.0) | (vals > 150.0)).mean()),
        t2star_units       = t2s_units,
    )
s0_vals = s0[mk & np.isfinite(s0) & (s0 > 0)]
rmse_vals = rmse[mk & np.isfinite(rmse) & (rmse > 0)]
out.update(
    s0_median      = float(np.median(s0_vals)) if s0_vals.size else np.nan,
    s0_iqr         = float(np.percentile(s0_vals, 75) - np.percentile(s0_vals, 25))
                    if s0_vals.size else np.nan,
    rmse_median     = float(np.median(rmse_vals)) if rmse_vals.size else np.nan,
    pct_decayfit_failures = 100.0 * float(fail.sum()) / max(float(mk.sum()), 1.0),
    _t2s_vals       = vals.astype(np.float32),
)
return out

def adaptive_mask_metrics(run_dir):
    """Perfil 0..N_ECHOS de cobertura multi-echo."""
    am = nib.load(os.path.join(run_dir,
        "desc-adaptiveGoodSignal_mask.nii.gz")).get_fdata().astype(int)
    intra = am > 0
    n_intra = max(int(intra.sum()), 1)
    counts = np.bincount(am[intra].ravel(), minlength=N_ECHOS + 1)

```

```

return dict(
    adaptmask_n_intra    = n_intra,
    pct_adaptmask_full   = 100.0 * float(counts[N_ECHOS]) / n_intra,
    pct_adaptmask_lt3    = 100.0 * float(counts[1:3].sum()) / n_intra,
    adaptmask_hist       = counts.tolist(),
)

def denoising_metrics(run_dir):
    """tSNR/drift/PSD/GS sobre la pareja optcom denoised."""
    optcom_img = nib.load(os.path.join(run_dir, "desc-optcom_bold.nii.gz"))
    denoised_img = nib.load(os.path.join(run_dir, "desc-denoised_bold.nii.gz"))
    am = nib.load(os.path.join(run_dir,
        "desc-adaptiveGoodSignal_mask.nii.gz")).get_fdata().astype(int)
    same_shape4d = optcom_img.shape == denoised_img.shape
    same_affine = np.allclose(optcom_img.affine, denoised_img.affine, atol=1e-5)
    if not same_shape4d:
        return dict(same_shape4d=False, same_affine=same_affine, denoising_ok=False)

    optcom = optcom_img.get_fdata(dtype=np.float32)
    denoised = denoised_img.get_fdata(dtype=np.float32)
    fg = am >= 3 # mask de clasificación ICA - misma usada por TEDANA internamente
    fg1 = am >= 1 # mask de optcom/denoised (más liberal)
    T = optcom.shape[-1]

    mu_o = optcom.mean(-1); sd_o = optcom.std(-1)
    mu_d = denoised.mean(-1); sd_d = denoised.std(-1)
    tsnr_o = np.zeros_like(mu_o); np.divide(mu_o, sd_o, out=tsnr_o, where=(sd_o > 0))
    tsnr_d = np.zeros_like(mu_d); np.divide(mu_d, sd_d, out=tsnr_d, where=(sd_d > 0))

    mean_drift = abs(mu_d[fg1].mean() - mu_o[fg1].mean()) / (abs(mu_o[fg1].mean()) + 1e-9)
    tsnr_o_med = float(np.median(tsnr_o[fg]))
    tsnr_d_med = float(np.median(tsnr_d[fg]))
    tsnr_gain = tsnr_d_med / (tsnr_o_med + 1e-9)

    # Global signal mean over fg (vectorizado por slabs para memoria)
    gs_o = np.zeros(T, dtype=np.float64); gs_d = np.zeros(T, dtype=np.float64)
    n_vox = max(int(fg.sum()), 1)
    for z in range(optcom.shape[2]):
        fz = fg[:, :, z]
        if not fz.any(): continue
        gs_o += optcom[:, :, z, :][fz].sum(0)
        gs_d += denoised[:, :, z, :][fz].sum(0)

```

```

gs_o /= n_vox; gs_d /= n_vox
gs_std_ratio = float(np.std(gs_d) / (np.std(gs_o) + 1e-9))

# PSD del GS
nperseg = min(256, T)
f_psd, P_o = signal.welch(gs_o, fs=1/TR, nperseg=nperseg)
_, P_d = signal.welch(gs_d, fs=1/TR, nperseg=nperseg)
bold_band = (f_psd >= 0.01) & (f_psd <= 0.10)
high_band = (f_psd >= 0.15)
psd_bold_ratio = float(P_d[bold_band].sum() / (P_o[bold_band].sum() + 1e-12))
psd_high_ratio = float(P_d[high_band].sum() / (P_o[high_band].sum() + 1e-12))

return dict(
    same_shape4d          = True,
    same_affine            = same_affine,
    denoising_ok          = True,
    n_frames               = int(T),
    tsnr_optcom_med        = tsnr_o_med,
    tsnr_denoised_med      = tsnr_d_med,
    tsnr_gain_denoised_opt = float(tsnr_gain),
    mean_drift              = float(mean_drift),
    gs_std_ratio            = gs_std_ratio,
    psd_bold_ratio          = psd_bold_ratio,
    psd_high_ratio          = psd_high_ratio,
    _gs_o = gs_o, _gs_d = gs_d, _psd_f = f_psd, _psd_o = P_o, _psd_d = P_d,
)

# -----
# Verdict - semáforo  PASS /  REVISAR /  FAIL
# -----
def verdict(r):
    if not r.get("integrity_ok", False):
        return " FAIL (outputs incompletos)"
    fails, warns = [], []
    # Estructura optcom  denoised
    if not r.get("same_shape4d", True): fails.append("shape mismatch")
    if not r.get("same_affine", True): fails.append("affine mismatch")
    # Componentes
    if r["n_accepted"] < TH["n_acc_lo"]: fails.append(f"n_acc={r['n_accepted']}")
    if r["pct_accepted"] < TH["pct_acc_fail_lo"]: fails.append(f"pct_acc={r['pct_accepted']}")
    elif r["pct_accepted"] > TH["pct_acc_fail_hi"]: fails.append(f"pct_acc={r['pct_accepted']}")
    elif not (TH["pct_acc_lo"] <= r["pct_accepted"] <= TH["pct_acc_hi"]):

```

```

        warns.append(f"pct_acc={r['pct_accepted']:.1f}")
# Varianza
if r["pct_varex_accepted"] < TH["pct_varex_fail_lo"] or \
    r["pct_varex_accepted"] > TH["pct_varex_fail_hi"]:
    fails.append(f"varex_acc={r['pct_varex_accepted']:.1f}")
elif not (TH["pct_varex_acc_lo"] <= r["pct_varex_accepted"] <= TH["pct_varex_acc_hi"]):
    warns.append(f"varex_acc={r['pct_varex_accepted']:.1f}")
if np.isfinite(r["varex_rej_on_acc"]):
    if r["varex_rej_on_acc"] > TH["varex_rej_on_acc_f"]:
        fails.append(f"varex_rej_on_acc={r['varex_rej_on_acc']:.1f}")
    elif r["varex_rej_on_acc"] > TH["varex_rej_on_acc"]:
        warns.append(f"varex_rej_on_acc={r['varex_rej_on_acc']:.1f}")
# Separación /
if np.isfinite(r["kappa_sep_ratio"]):
    if r["kappa_sep_ratio"] < TH["kappa_sep_ratio_f"]:
        fails.append(f"sep={r['kappa_sep_ratio']:.2f}")
    elif r["kappa_sep_ratio"] < TH["kappa_sep_ratio_lo"]:
        warns.append(f"sep={r['kappa_sep_ratio']:.2f}")
if np.isfinite(r["rho_sep_ratio"]):
    if r["rho_sep_ratio"] < TH["rho_sep_ratio_f"]:
        fails.append(f"sep={r['rho_sep_ratio']:.2f}")
    elif r["rho_sep_ratio"] < TH["rho_sep_ratio_lo"]:
        warns.append(f"sep={r['rho_sep_ratio']:.2f}")
if np.isfinite(r["pct_acc_below_kelbow"]):
    if r["pct_acc_below_kelbow"] > TH["pct_acc_below_f"]:
        fails.append(f"acc< elbow={r['pct_acc_below_kelbow']:.0f}%")
    elif r["pct_acc_below_kelbow"] > TH["pct_acc_below_elbow"]:
        warns.append(f"acc< elbow={r['pct_acc_below_kelbow']:.0f}%")
# T2*
if r["t2star_mode_ms"] < TH["t2star_mode_fail_lo"] or \
    r["t2star_mode_ms"] > TH["t2star_mode_fail_hi"]:
    fails.append(f"T2*mode={r['t2star_mode_ms']:.0f}")
elif not (TH["t2star_mode_lo"] <= r["t2star_mode_ms"] <= TH["t2star_mode_hi"]):
    warns.append(f"T2*mode={r['t2star_mode_ms']:.0f}")
if r["t2star_pct_plausible"] < TH["t2star_plaus_fail"]:
    fails.append(f"T2*plaus<{TH['t2star_plaus_fail']:.0f}%")
elif r["t2star_pct_plausible"] < TH["t2star_plaus_pct"]:
    warns.append(f"T2*plaus={r['t2star_pct_plausible']:.0f}%")
# Decay fit failures
if r["pct_decayfit_failures"] > TH["decayfit_fail_f"]:
    fails.append(f"fitfail={r['pct_decayfit_failures']:.1f}%")
elif r["pct_decayfit_failures"] > TH["decayfit_fail_pct"]:

```

```

        warns.append(f"fitfail={r['pct_decayfit_failures']:.1f}%")
# Adaptive mask
if r["pct_adaptmask_full"] < TH["pct_full_fail"]:
    fails.append(f"adaptfull<{TH['pct_full_fail']:.0f}%")
elif r["pct_adaptmask_full"] < TH["pct_full_lo"]:
    warns.append(f"adaptfull={r['pct_adaptmask_full']:.0f}%")
if r["pct_adaptmask_lt3"] > TH["pct_lt3_fail"]:
    fails.append(f"adapt<3={r['pct_adaptmask_lt3']:.0f}%")
elif r["pct_adaptmask_lt3"] > TH["pct_lt3_hi"]:
    warns.append(f"adapt<3={r['pct_adaptmask_lt3']:.0f}%")
# Denoising
g = r["tsnr_gain_denoised_opt"]
if g < TH["tsnr_gain_fail_lo"] or g > TH["tsnr_gain_fail_hi"]:
    fails.append(f"tsnr_gain={g:.2f}")
elif not (TH["tsnr_gain_lo"] <= g <= TH["tsnr_gain_hi"]):
    warns.append(f"tsnr_gain={g:.2f}")
if r["mean_drift"] > TH["mean_drift_fail"]:
    fails.append(f"drift={r['mean_drift']:.3f}")
elif r["mean_drift"] > TH["mean_drift"]:
    warns.append(f"drift={r['mean_drift']:.3f}")
if not (TH["psd_bold_lo"] <= r["psd_bold_ratio"] <= TH["psd_bold_hi"]):
    warns.append(f"psd_BOLD={r['psd_bold_ratio']:.2f}")
if r["gs_std_ratio"] > TH["gs_std_ratio_hi"]:
    warns.append(f"gs_std={r['gs_std_ratio']:.2f}")
if fails: return f" FAIL ({', '.join(fails)})"
if warns: return f" REVISAR ({', '.join(warns)})"
return " PASS"

# -----
# Plots - un summary multi-panel + un PSD/GS auxiliar
# -----
def plot_summary(key, r, out_dir):
    subj, ses, task, run = key
    df = r["_df_metrics"]
    cls = df["classification"].astype(str).str.lower()
    acc = df[cls.str.startswith("acc")]
    rej = df[cls.str.startswith("rej")]

    fig, ax = plt.subplots(2, 2, figsize=(14, 9))
    # (a) vs scatter - la firma visual de TEDANA
    ax[0, 0].scatter(rej["kappa"], rej["rho"], s=18, c="#666", label=f"rejected n={len(rej)}")
    ax[0, 0].scatter(acc["kappa"], acc["rho"], s=22, c="#1565c0", label=f"accepted n={len(acc)}")

```

```

if np.isfinite(r["kappa_elbow"]):
    ax[0, 0].axvline(r["kappa_elbow"], color="#1565c0", linestyle="--", linewidth=1,
                    label=f" elbow={r['kappa_elbow']:.1f}")
if np.isfinite(r["rho_elbow"]):
    ax[0, 0].axhline(r["rho_elbow"], color="#c62828", linestyle="--", linewidth=1,
                    label=f" elbow={r['rho_elbow']:.1f}")
ax[0, 0].set_xlabel(" (TE-dependence)")
ax[0, 0].set_ylabel(" (TE-independence)")
ax[0, 0].set_title(f" vs - pct_acc={r['pct_accepted']:.1f}% "
                  f"varex_acc={r['pct_varex_accepted']:.1f}%")
ax[0, 0].legend(fontsize=8, loc="upper right")
# (b) histograma T2*
vals = r["_t2s_vals"]
if vals.size:
    ax[0, 1].hist(vals, bins=100, range=(0, 150), color="#6a1b9a", alpha=0.85)
    ax[0, 1].axvspan(30, 60, color="#c5e1a5", alpha=0.4,
                    label="fisiológico 3 T (Posse 1999)")
    ax[0, 1].axvline(r["t2star_mode_ms"], color="#c62828", linestyle="--",
                    linewidth=1, label=f"moda={r['t2star_mode_ms']:.1f} ms")
ax[0, 1].set_xlabel("T2* (ms)"); ax[0, 1].set_ylabel("nº de vóxeles")
ax[0, 1].set_title(f"T2* (mask adaptativa 1) "
                  f"plausibles={r['t2star_pct_plausible']:.1f}% "
                  f"fitfail={r['pct_decayfit_failures']:.1f}%")
ax[0, 1].legend(fontsize=8)
# (c) histograma adaptive mask (0..N_ECHOS)
hist = r["adaptmask_hist"]
ax[1, 0].bar(range(len(hist)), hist, color="#ef6c00")
ax[1, 0].set_xticks(range(len(hist)))
ax[1, 0].set_xlabel("nº de ecos con señal buena")
ax[1, 0].set_ylabel("nº de vóxeles")
ax[1, 0].set_title(f"Adaptive mask full={r['pct_adaptmask_full']:.1f}% "
                  f"<3 ecos={r['pct_adaptmask_lt3']:.1f}%")
# (d) tSNR bars optcom vs denoised
ax[1, 1].bar(["optcom", "denoised"],
             [r["tsnr_optcom_med"], r["tsnr_denoised_med"]],
             color=["#888", "#1565c0"])
ax[1, 1].set_ylabel("tSNR mediano (mask 3 ecos)")
ax[1, 1].set_title(f"tSNR optcom-denoised ganancia ×{r['tsnr_gain_denoised_opt']:.2f} "
                  f"drift={r['mean_drift']*100:.2f}%")
fig.suptitle(f"QC TEDANA - {subj} {ses} {task} {run} verdict: {r['verdict']}",
            y=1.001, fontsize=12)
fig.tight_layout()

```

```

out_png = os.path.join(out_dir, "summary.png")
fig.savefig(out_png, dpi=120, bbox_inches="tight")
plt.close(fig)
return out_png

def plot_spectra(key, r, out_dir):
    subj, ses, task, run = key
    fig, ax = plt.subplots(1, 2, figsize=(14, 4))
    t = np.arange(r["n_frames"]) * TR
    ax[0].plot(t, r["_gs_o"], color="#666", linewidth=0.7, label="optcom")
    ax[0].plot(t, r["_gs_d"], color="#1565c0", linewidth=0.7, label="denoised")
    ax[0].set_xlabel("t (s)"); ax[0].set_ylabel("global signal (mask 3)")
    ax[0].set_title(f"GS optcom vs denoised - std ratio={r['gs_std_ratio']:.2f}")
    ax[0].legend(fontsize=8)
    ax[1].semilogy(r["_psd_f"], r["_psd_o"], color="#666", label="optcom")
    ax[1].semilogy(r["_psd_f"], r["_psd_d"], color="#1565c0", label="denoised")
    ax[1].axvspan(0.01, 0.10, color="#c5e1a5", alpha=0.3, label="BOLD band")
    ax[1].axvline(0.15, color="#c62828", linestyle=":", linewidth=1,
                  label="ruido > 0.15 Hz")
    ax[1].set_xlabel("Hz"); ax[1].set_ylabel("power (log)")
    ax[1].set_title(f"PSD - BOLD ratio={r['psd_bold_ratio']:.2f} "
                    f"high ratio={r['psd_high_ratio']:.2f}")
    ax[1].legend(fontsize=8)
    fig.suptitle(f"{subj} {ses} {task} {run}", y=1.001, fontsize=11)
    fig.tight_layout()
    out_png = os.path.join(out_dir, "spectra.png")
    fig.savefig(out_png, dpi=120, bbox_inches="tight")
    plt.close(fig)
    return out_png

# -----
# Loop principal
# -----

runs = discover_runs()
print(f"[discover] {len(runs)} runs en {TEDANA_DIR}")
rows, skipped, failed_runs = [], [], []

for key in sorted(runs):
    subj, ses, task, run = key
    run_dir = runs[key]
    out_dir = os.path.join(QC_DIR, subj, ses, task, run); os.makedirs(out_dir, exist_ok=True)
    summary_png = os.path.join(out_dir, "summary.png")

```

```

spectra_png = os.path.join(out_dir, "spectra.png")
if os.path.exists(summary_png) and os.path.exists(spectra_png):
    skipped.append(key); continue

print(f"[qc] {subj} {ses} {task} {run}")
ok, miss = integrity_check(run_dir)
r = dict(subj=subj, ses=ses, task=task, run=run,
         integrity_ok=ok, missing_files=";".join(miss))
if not ok:
    r["verdict"] = f" FAIL (faltan: {r['missing_files']})"
    rows.append(r); failed_runs.append(key); continue

try:
    r.update(ica_metrics(run_dir))
    r.update(t2star_metrics(run_dir))
    r.update(adaptive_mask_metrics(run_dir))
    r.update(denoising_metrics(run_dir))
    r["verdict"] = verdict(r)
    plot_summary(key, r, out_dir)
    plot_spectra(key, r, out_dir)
    rows.append({k: v for k, v in r.items() if not k.startswith("_")})
except Exception as e:
    print(f" [!] error: {type(e).__name__}: {e}")
    r["verdict"] = f" FAIL (excepción: {type(e).__name__})"
    rows.append({k: v for k, v in r.items() if not k.startswith("_")})
    failed_runs.append(key)

# -----
# CSV unificado + resumen
# -----
if rows:
    df = pd.DataFrame(rows)
    csv = os.path.join(QC_DIR, "metrics.csv")
    if os.path.exists(csv):
        df_prev = pd.read_csv(csv)
        df = pd.concat([df_prev[~df_prev.set_index(
            ["subj", "ses", "task", "run"]).index.isin(
                df.set_index(["subj", "ses", "task", "run"]).index]], df],
            ignore_index=True, sort=False)
    df = df.drop_duplicates(subset=["subj", "ses", "task", "run"], keep="last")
    df.to_csv(csv, index=False)
    print(f"[csv] {len(df)} runs en {csv}")

```



```

show_cols = [c for c in [
    "subj", "ses", "task", "run", "n_components", "n_accepted",
    "pct_accepted", "pct_varex_accepted", "varex_rej_on_acc",
    "kappa_sep_ratio", "rho_sep_ratio", "t2star_mode_ms",
    "t2star_pct_plausible", "pct_decayfit_failures",
    "pct_adaptmask_full", "pct_adaptmask_lt3",
    "tsnr_gain_denoised_opt", "mean_drift", "gs_std_ratio",
    "psd_bold_ratio", "verdict"] if c in df.columns]
with pd.option_context("display.float_format", "{:.3f}".format,
    "display.max_columns", None,
    "display.width", 240):
    print("\n" + df[show_cols].to_string(index=False))
    print("\n" + df.groupby("verdict").size().to_string())

print(f"\n[summary] procesados={len(rows)}  skipped(idempotente)={len(skipped)}  "
    f"failed={len(failed_runs)}")

```

i Cuándo ejecutar y cómo interpretar el verdict

Este chunk se ejecuta **después** del bulk run de TEDANA sobre $t_0/$ (§3.7) y del resamplio a MNI (§3.11), y **antes** de la extracción de conectividad (Section 4). El verdict por run sigue las mismas tres categorías que Section 1.9 y Section 2.3:

- **PASS** — todas las métricas dentro de umbrales. El `space-MNI152Nlin2009cAsym_desc-denoised_bold.nii.gz` del run entra en §4 (extracción de FC) sin más revisión.
- **REVISAR** — al menos una métrica fuera de su rango *warn* pero ninguna en *fail*. **Acción obligatoria:** abrir `tedana_report.html` del run y revisar los puntos señalados en §3.13.2 (scatter κ/ρ , scree plots, mapas de componentes ambiguos, carpet pre/post). Si el hallazgo es admisible se documenta en §3.13.3; si no, se reclasifica con RICA (Section 3) y se re-ejecuta este chunk borrando el `summary.png` correspondiente para forzar el QC.
- **FAIL** — falla estructural (faltan outputs, optcom/denoised desemparejados), descomposición patológica ($n_accepted < 3$, $varex_acc$ fuera de $[15, 90]$ %), separación κ/ρ rota, T_2^* implausible, fit failures > 5 %, máscara adaptativa colapsada, o denoising sin efecto/over-denoising. **Acción:** el run no entra en §4 y se documenta como exclusión definitiva en PsiloData §4.

La tabla `metrics.csv` es la entrada canónica al análisis de covariables de §4 (efectos de denoising y dimensionalidad ICA sobre la FC) y al benchmark QC-FC contra los pipelines single-echo de referencia (Parkes et al., 2018). Mantener el chunk idempotente garantiza que la tabla pueda extenderse en tiers posteriores sin recomputar el Tier 0.

4 4 Extracción de conectividad funcional para ds006072 (Python)

La extracción de conectividad funcional para el dataset ds006072 (psilocibina) sigue la misma lógica que la rama LSD (LSDPrep §5, ds003059), pero con diferencias sustanciales en la estrategia de denoising derivadas de la naturaleza multi-echo de los datos. Mientras que para ds003059 (single-echo, derivados) el pipeline primario es 36P con GSR, para ds006072 (multi-echo, datos crudos) el denoising primario es **TEDANA** — un método que explota la dependencia del contraste BOLD con el tiempo de eco (TE) para separar señal neural de artefactos no-BOLD mediante un criterio **físico**, no estadístico (Kundu et al., 2012, 2013; DuPre et al., 2021).

Es importante señalar que la estrategia de denoising aquí adoptada **difiere deliberadamente** del pipeline original de Siegel et al. (2024), quienes emplearon un pipeline *in-house* consistente en: (1) reducción de ruido térmico con NORDIC PCA (Moeller et al., 2021; Vizioli et al., 2021), (2) combinación óptima de ecos por suma ponderada (*optimally combined*; Posse et al., 1999), (3) regresores tisulares volumétricos (WM, ventrículos, CSF extra-axial; Behzadi et al., 2007), (4) filtrado bandpass, y (5) scrubbing con umbral FD > 0.3 mm. Crucialmente, Siegel et al. **no** aplicaron descomposición ICA basada en TE-dependencia (ni TEDANA ni ME-ICA), sino que se limitaron a la combinación óptima de ecos seguida de regresión de confounds convencional. La elección de TEDANA en el presente trabajo se justifica por las ventajas teóricas y empíricas de la clasificación TE-dependiente sobre los métodos de regresión convencionales, como se detalla a continuación.

4.1 4.1 Por qué NO aplicar GSR después de TEDANA (en el pipeline primario)

La exclusión de GSR del pipeline primario merece justificación explícita, dado que Ciric et al. (2017) identificaron GSR como el factor más eficaz para reducir QC-FC en datos single-echo:

1. **Doble eliminación de señal global.** TEDANA ya identifica y elimina los componentes ICA cuyas fluctuaciones son TE-independientes (modulan S_0 , no T_2^*). Estos componentes incluyen artefactos de movimiento global, señales cardíacas y respiratorias, y fluctuaciones vasculares no neurales — precisamente las fuentes que GSR captura en datos single-echo (Power et al., 2014, 2016). Aplicar GSR adicionalmente eliminaría fluctuaciones neurales globales genuinas (dependientes de T_2^*) que TEDANA preservó intencionalmente (Kundu et al., 2012, 2013).
2. **Sesgo matemático de centrado en cero.** Murphy y Fox (2017) demostraron que GSR impone que la media de todas las correlaciones sea cero, lo que puede generar anticorrelaciones artefactuales y alterar métricas de grafos (modularidad, eficiencia global, clustering). En el contexto de la psilocibina, donde Siegel et al. (2024) reportaron una desincronización masiva que afecta la conectividad global, GSR podría sesgar artificialmente la magnitud del efecto.

3. **Los benchmarks de Ciric et al. (2017) no aplican directamente a datos post-TEDANA.** Las evaluaciones de Ciric et al. (2017) y Parkes et al. (2018) se realizaron sobre datos single-echo donde *toda* la carga de denoising recae en la regresión de confounds. En datos post-TEDANA, la mayoría del ruido no-BOLD ya fue eliminado por la clasificación TE-dependiente, dejando una señal más limpia donde GSR aportaría beneficio marginal frente a un coste alto (eliminación de señal neural global).

No obstante, dado que GSR fue el regresor más eficaz en los benchmarks single-echo de Ciric et al. (2017) y Parkes et al. (2018), incluimos el pipeline OC+36P+GSR como **análisis de sensibilidad explícito**: si los efectos de psilocibina sobre la conectividad global desaparecen únicamente bajo GSR, es atribuible al sesgo matemático de centrado en cero (Murphy & Fox, 2017) más que al ruido global. Si persisten en los siete pipelines, el efecto es robusto frente a las dos extremas filosofías de denoising (sin GSR vs con GSR + 36 confounds).

4.2 4.2 Atlas de parcelación: corteza + subcortex (Schaefer 200 + Tian S2)

La elección del atlas constriñe directamente qué hipótesis pueden testearse. Los efectos cardinales de los psicodélicos serotoninérgicos no son exclusivamente corticales: la conectividad **tálamo-cortical** se ha identificado como el correlato más replicado del estado LSD/psilocibina (Müller et al., 2017, 2018; Tagliazucchi et al., 2016; Preller et al., 2018), y el desacoplamiento **parahipocampo-retroespleneal** correlaciona con disolución del ego con $r = 0.82$ (Carhart-Harris et al., 2016, *PNAS*). El atlas Schaefer 2018 (Schaefer et al., *Cereb Cortex*) es el estándar moderno para parcelación cortical funcional volumétrica — supera en homogeneidad funcional y conectiva a Gordon, Glasser, Shen y Craddock (Schaefer et al., 2018, Fig. 3) — pero **es exclusivamente cortical**: sus autores recomiendan explícitamente combinarlo con un atlas subcortical separado, dada la diferencia de SNR y la aplicabilidad limitada de gradientes corticales a estructuras profundas (Schaefer et al., 2018, p. 3108).

Para el subcortex se adopta **Tian et al. (2020, *Nat Neurosci*) en su resolución S2** (32 ROIs: 16 por hemisferio cubriendo tálamo, estriado dorsal y ventral, palidio, hipocampo y amígdala). Tian S2 es el atlas subcortical de gradientes funcionales más usado en la literatura psicodélica reciente: Singleton et al. (2022, *Nat Commun*) lo emparejan con Schaefer 200 (atlas combinado de 232 ROIs) para su análisis de *network control theory* aplicado a LSD/psilocibina; Luppi et al. (2021, *NeuroImage*) usan Lausanne 463 — que también incluye subcortex — para el análisis de integración-segregación dinámica bajo LSD. Esta combinación captura exactamente las regiones subcorticales que la bibliografía de Müller, Carhart-Harris y Preller identifica como críticas para el efecto serotoninérgico.

Compatibilidad técnica. Ambos atlas son volumétricos en espacio MNI y se consumen con `NiftiLabelsMasker` de `nilearn` sin modificación. Schaefer se distribuye nativamente en `MNI152NLin2009cAsym` (mismo espacio que la salida de `fMRIPrep`, §2); Tian se distribuye en múltiples espacios — se descarga la variante `Tian_Subcortex_S2_3T_2009cAsym.nii.gz` del repositorio oficial, ya en `MNI152NLin2009cAsym`, evitando la conversión `MNI152NLin6Asym`

→ 2009cAsym. La única operación es un resampleo a la grilla 2 mm de Schaefer mediante interpolación de vecino más cercano (`nilearn.image.resample_to_img, interpolation='nearest'`), que preserva la integridad de las etiquetas enteras y se aplica solo si las grillas difieren.

Convención de IDs (atlas combinado de 232 ROIs). Las etiquetas se concatenan con offset: Schaefer ocupa los IDs 1–200, Tian ocupa los IDs 201–232. En cualquier vóxel donde Schaefer asigne corteza (ID 1–200) y Tian asigne subcortex (ID 1–32), Schaefer prevalece — la regla resuelve la ambigüedad por tipo de tejido sin necesidad de máscaras adicionales y respeta que Schaefer fue diseñado sobre superficie cortical reproyectada a MNI. La matriz de conectividad final tiene dimensiones 232×232 , organizada en bloques cortex–cortex (200×200), cortex–subcortex (200×32) y subcortex–subcortex (32×32). Los bloques mixtos son los teóricamente más informativos: la submatriz cortex–subcortex contiene los términos tálamo–DMN, estriado–visual, hipocampo–DMN, etc., que constituyen las hipótesis principales del estudio.

Caveat de SNR. Las regiones subcorticales —especialmente el estriado ventral y la cabeza del caudado— sufren caídas de SNR BOLD del orden del 30–50 % respecto a corteza en adquisiciones a 3 T. NORDIC (Section 1) atenúa pero no elimina esta diferencia. En §5.1 se reporta SNR temporal y QC-FC región a región, y en PsyConn §2 se contemplan análisis de sensibilidad excluyendo regiones con SNR por debajo de un umbral preespecificado. La triangulación de §4.3 (TEDANA / OC+aCompCor / OC+36P+GSR) permite además detectar artefactos subcorticales específicos: TEDANA elimina varianza non-BOLD con eficacia particular en estructuras profundas (Kundu et al., 2012, demostrando recuperación de conectividad hipocampo–corteza invisible con denoising convencional), por lo que efectos PSIL > MTP que persistan en TEDANA y aCompCor pero desaparezcan en 36P+GSR son atribuibles al sesgo de centrado en cero de GSR sobre subcortex (Murphy & Fox, 2017), no a artefacto.

```
#=====
# §4 - Extracción de conectividad funcional (ds006072)
# §4.2 - Construcción del atlas combinado Schaefer 200 + Tian S2 (232 ROIs)
#=====
# Descarga Tian S2 desde el repositorio oficial (yetianmed/subcortex en GitHub),
# lo resamplea a la grilla de Schaefer (MNI152NLin2009cAsym, 2 mm) y genera
# el atlas combinado de 232 ROIs. Idempotente: solo descarga/construye si los
# archivos no existen.
#
# Salidas:
#   derivatives/atlas/Tian_Subcortex_S2_3T.nii.gz          (descarga directa)
#   derivatives/atlas/Tian_Subcortex_S2_3T_label.txt      (descarga directa)
#   derivatives/atlas/Schaefer200_Tian32_MNI2009c.nii.gz  (atlas combinado)
#   derivatives/atlas/atlas_labels_232.csv                (id, label)
#=====

import os
```

```

import urllib.request
import numpy as np
import pandas as pd
from nilearn import datasets, image as nimg

ATLAS_DIR = "D:/ProfessionalProyectos/PsyConn/derivatives/atlas"
os.makedirs(ATLAS_DIR, exist_ok=True)

COMBINED_ATLAS_PATH = os.path.join(
    ATLAS_DIR, "Schaefer200_Tian32_MNI2009c.nii.gz")
COMBINED_LABELS_PATH = os.path.join(ATLAS_DIR, "atlas_labels_232.csv")

# -----
# 1. Descarga Tian S2 (NIIfTI + labels) si no están presentes
# -----
TIAN_BASE = ("https://raw.githubusercontent.com/yetianmed/subcortex/"
             "master/Group-Parcellation/3T/Subcortex-Only")
# Versión nativa en MNI152Nlin2009cAsym - mismo espacio que Schaefer y fMRIPrep,
# evita errores de registro entre MNI152Nlin6Asym (FSL) y 2009cAsym.
TIAN_NII_URL = f"{TIAN_BASE}/Tian_Subcortex_S2_3T_2009cAsym.nii.gz"
TIAN_LBL_URL = f"{TIAN_BASE}/Tian_Subcortex_S2_3T_label.txt"

tian_nii = os.path.join(ATLAS_DIR, "Tian_Subcortex_S2_3T_2009cAsym.nii.gz")
tian_lbl = os.path.join(ATLAS_DIR, "Tian_Subcortex_S2_3T_label.txt")

for url, dst in [(TIAN_NII_URL, tian_nii), (TIAN_LBL_URL, tian_lbl)]:
    if not os.path.exists(dst):
        print(f"Descargando {os.path.basename(dst)} ...")
        try:
            urllib.request.urlretrieve(url, dst)
        except Exception as e:
            raise RuntimeError(
                f"Fallo al descargar {url}: {e}\n"
                "Descarga manual desde https://github.com/yetianmed/subcortex "
                f"y coloca el archivo en {ATLAS_DIR}"
            )
print(f" Tian S2 NIIfTI: {tian_nii}")

# -----
# 2. Carga Schaefer 200 / Yeo-7 / 2 mm (espacio MNI152Nlin2009cAsym)
# -----
schaefer = datasets.fetch_atlas_schaefer_2018(

```

```

    n_rois=200, yeo_networks=7, resolution_mm=2)
schaefer_img = nimg.load_img(schaefer.maps)
print(f"  Schaefer 200 cargado: shape={schaefer_img.shape}")

# -----
# 3. Alinea Tian S2 a la grilla de Schaefer
#   Ambos están en MNI152NLin2009cAsym; resample solo armoniza voxel-grid
#   (Tian se distribuye típicamente a 1 mm; Schaefer aquí está a 2 mm).
#   Nearest-neighbor para preservar etiquetas enteras.
# -----
tian_img = nimg.load_img(tian_nii)
print(f"  Tian S2 nativo: shape={tian_img.shape}")

if tian_img.shape != schaefer_img.shape:
    tian_resamp = nimg.resample_to_img(
        tian_img, schaefer_img,
        interpolation="nearest",
        force_resample=True,
        copy_header=True,
    )
    print(f"  Tian S2 resampleado a grilla Schaefer: shape={tian_resamp.shape}")
else:
    tian_resamp = tian_img
    print("  Tian S2 ya en grilla de Schaefer - no requiere resample.")

# -----
# 4. Combinación: Schaefer 1..200 + Tian (1..32 → 201..232)
#   Donde ambos asignen, Schaefer prevalece (subcortex no debe pisar cortex).
# -----
sch_data = np.asarray(schaefer_img.dataobj).astype(np.int32)
tian_data = np.asarray(tian_resamp.dataobj).astype(np.int32)

combined = sch_data.copy()
mask = (tian_data > 0) & (combined == 0)
combined[mask] = tian_data[mask] + 200

n_subcort_voxels = int(mask.sum())
n_overlap = int(((tian_data > 0) & (sch_data > 0)).sum())
print(f"  Vóxeles asignados a Tian: {n_subcort_voxels}")
print(f"  Vóxeles Tian descartados por solape con Schaefer: {n_overlap}")

combined_img = nimg.new_img_like(schaefer_img, combined)

```

```

combined_img.to_filename(COMBINED_ATLAS_PATH)
print(f" Atlas combinado guardado: {COMBINED_ATLAS_PATH}")

# -----
# 5. Etiquetas combinadas (Background + 200 Schaefer + 32 Tian)
# -----
sch_labels = [
    lab.decode("utf-8") if isinstance(lab, bytes) else str(lab)
    for lab in schaefer.labels
]
# Algunas versiones de nilearn devuelven "Background" como primera entrada;
# se elimina aquí para evitar duplicado al prependerlo más abajo.
if sch_labels and sch_labels[0].lower() == "background":
    sch_labels = sch_labels[1:]
assert len(sch_labels) == 200, (
    f"Schaefer 200 debe tener 200 etiquetas tras strip de Background, "
    f"encontré {len(sch_labels)}"
)

with open(tian_lbl, "r", encoding="utf-8") as f:
    tian_labels = [ln.strip() for ln in f if ln.strip()]

if len(tian_labels) != 32:
    print(f" Tian label file tiene {len(tian_labels)} líneas (esperado 32). "
          "Generando etiquetas genéricas.")
    tian_labels = [f"S2_{i:02d}" for i in range(1, 33)]

# Prefijo identificable para distinguir cortex vs subcortex en análisis posterior
tian_labels = [f"Subcortex_{lbl}" for lbl in tian_labels]

all_labels = ["Background"] + sch_labels + tian_labels
assert len(all_labels) == 233, f"Esperaba 233 etiquetas, encontré {len(all_labels)}"

pd.DataFrame({
    "id": list(range(0, 233)),
    "label": all_labels,
}).to_csv(COMBINED_LABELS_PATH, index=False)
print(f" Etiquetas combinadas: {COMBINED_LABELS_PATH}")
print(f" Cortex (Schaefer): IDs 1..200")
print(f" Subcortex (Tian): IDs 201..232")

# -----

```

```

# 6. Coordenadas centroidales (MNI mm) - necesarias para análisis de
#     dependencia de distancia QC-FC en §5.1 y para visualizaciones de
#     conectomas en PsyConn §1.
# -----
from nilearn import plotting

print(" Calculando coordenadas centroidales (find_parcellation_cut_coords)...")
coords = plotting.find_parcellation_cut_coords(combined_img)
assert coords.shape == (232, 3), f"Esperaba (232,3), encontré {coords.shape}"

coords_df = pd.DataFrame({
    "id":      list(range(1, 233)),
    "label":   all_labels[1:],    # excluir Background
    "x":       coords[:, 0],
    "y":       coords[:, 1],
    "z":       coords[:, 2],
})
coords_path = os.path.join(ATLAS_DIR, "Schaefer200_Tian32_coords.csv")
coords_df.to_csv(coords_path, index=False)
print(f" Coordenadas guardadas: {coords_path}")

```

4.3 4.3 Diseño de triangulación para ds006072

Para garantizar la robustez de los resultados se ejecutan **siete pipelines en paralelo** sobre los datos ya pre-denoised por NORDIC (Section 1) y preprocesados por fMRIPrep (§2). La lógica de triangulación está diseñada para aislar las contribuciones independientes de (i) la reducción de ruido térmico (NORDIC, común a todos), (ii) el denoising ICA TE-dependiente (TEDANA vs OC), (iii) la regresión de movimiento *adicional* sobre la salida de TEDANA (TEDANA vs TEDANA+24P), (iv) la regresión de **ruido fisiológico difuso** sobre la salida de TEDANA mediante aCompCor (TEDANA vs TEDANA+aCompCor — capturando varianza respiratoria que la ICA espacial no puede separar por construcción; DuPre et al., 2021), (v) la estrategia de regresión de confounds sobre OC (Siegel-like vs 24P vs aCompCor vs 36P+GSR), y (vi) el efecto específico de GSR (incluido solo en el último pipeline):

Pipeline	Datos de entrada	Regresión residual	GSR	Bandpass	Censurado FD > 0.3	Propósito
TEDANA (sensibilidad ME-ICA-only)	TEDANA denoised	— (solo scrubbing)	No	No	Sí	Posición Kundu (2012, 2013, 2017): tras NORDIC + ME-ICA no haría falta nuisance externo. Contrastada empíricamente en este dataset (cf. Section 3.13)
TEDANA+24P (sensibilidad motor)	TEDANA denoised	24P (Friston, 1996)	No	No	Sí	Patrón “Denoising Data with Components” recomendado por la documentación oficial de tedana (DuPre et al., 2021) — testea si la regresión de motion <i>adicional</i> añade valor sobre ME-ICA

Pipeline	Datos de entrada	Regresión residual	GSR	Bandpass	Censurado FD > 0.3	Propósito
TEDANA + aICOM- pCor (primario)	TEDANA denoised	24P + 5 PC WM + 5 PC CSF	No	No	Sí	Recomen- dación explícita de la docu- mentación de tedana para ruido espacial- mente difuso (DuPre et al., 2021); selec- cionado tras §5.2 como pipeline primario por mejor QC-FC entre los pipelines ME-ICA (Behzadi et al., 2007; Power et al., 2018)
OC+Siegel- like (repli- cación)	Combi- nación óptima	12 motion + WM + CSF	No	0.009–0.08 Hz	Sí	Repli- cación fiel del pipeline de Siegel et al. (2024)

Pipeline	Datos de entrada	Regresión residual	GSR	Bandpass	Censurado FD > 0.3	Propósito
OC+24P (contra-partida single-echo)	Combinación óptima	24P (Friston, 1996)	No	No	Sí	Contra-partida OC de TEDANA+24P — regresores idénticos, distinto input; aísla el aporte de ME-ICA bajo regresión de movimiento sola
OC+aCompCor (top performer Ciric)	Combinación óptima	24P + 5 PC WM + 5 PC CSF	No	No	Sí	Mejor pipeline single-echo sin GSR (Behzadi et al., 2007; Ciric et al., 2017)
OC+36P+GSR (sensibilidad GSR)	Combinación óptima	36P (24 motion + 3 tissue × {x, x ² , dx, dx ² } = 24 + 12)	Sí	No	Sí	Análisis de sensibilidad para el sesgo de GSR (Murphy & Fox, 2017; Ciric et al., 2017)

Pipeline primario: TEDANA + aCompCor + scrubbing. El diseño de triangulación se construyó originalmente sobre la hipótesis Kundu (2012, 2013, 2017) de que tras ME-ICA no haría falta nuisance externo: Kundu et al. (2012) demostraron que tras ME-ICA “*no other filtering was applied*” y que las trazas DVARS del output BOLD (high- κ) eran planas sin regre-

si3n de movimiento ni filtrado bandpass; Kundu et al. (2013) confirmaron un incremento de $4 \times$ *entSNR*ylaeliminacindeladependenciadedistanciadelartefactodemovimiento;ylarevisindeKunduetal.(2017). ICA"nonecesitarapasosdeacondicionamientocomofiltradobandpassnisuavizadoespacial".Elrbolpre-registrado fijaba TEDANA como primario condicional a que los pipelines ME-ICA convergieran ($r = 0.85$) sobre el `group_diff` PSIL-MTP. La ejecuci3n del QC-FC (§5.1) y de la triangulaci3n inter-pipeline (§5.2) sobre este dataset **no sostiene esa convergencia**: $r(\text{TEDANA}, \text{TEDANA} + \text{aCompCor}) = 0.68$ y $r(\text{TEDANA}, \text{TEDANA} + 24\text{P}) = 0.69$ caen por debajo del umbral pre-registrado, y el benchmark 1 QC-FC muestra contaminaci3n de movimiento residual sustancial en TEDANA puro y TEDANA+24P sobre PSIL ($r_{\text{FD}} = 0.87$ y 0.76 respectivamente; $\text{pct_sig_QCFC} \geq 15\%$ frente al $< 5\%$ esperado bajo no contaminaci3n seg3n Ciric 2017). Tras la cl3usula expl3cita del 3rbol de decisi3n (“si TEDANA+aCompCor diverge sustancialmente de TEDANA $\rightarrow$ TEDANA+aCompCor pasa a candidato a primario”), **se selecciona TEDANA+aCompCor como pipeline primario**: reduce r_{FD} a 0.50 y pct_sig_QCFC a 5.4% , ocupa el centro del cl3ster denso de la matriz 7×7 ($r \geq 0.84$ con TEDANA+24P, OC+24P y OC+aCompCor), preserva la ventaja estructural del dataset multi-eco (clasificaci3n TE-dependiente), y materializa la recomendaci3n expl3cita de la documentaci3n de tedana para ruido espacialmente difuso (DuPre et al., 2021). El pipeline regresa los 24 par3metros de movimiento expandidos de Friston (1996) m3s los 5 primeros componentes principales de WM y los 5 primeros de CSF (Behzadi et al., 2007), sin filtrado bandpass — manteniendo la l3gica Kundu de evitar el difuminado $\text{bandpass} \times \text{spike}$ (Hallquist et al., 2013). El censurado $\text{FD} > 0.3$ mm se conserva como red de seguridad ante artefactos respiratorios pseudo-movimiento que sobreviven ME-ICA (Power et al., 2018; Fair et al., 2020); el umbral 0.3 mm coincide con Siegel et al. (2024) y es m3s conservador que el $\text{FD} > 0.5$ mm com3nmente usado en datos de adultos sanos (Power et al., 2014). El veredicto completo, las tablas de QC-FC y el dendrograma est3n en Section 3.13. **TEDANA solo + scrubbing** pasa a ser ahora *control de sensibilidad ME-ICA-only* (control 1, contrastando emp3ricamente la posici3n Kundu); el resto de la l3gica de triangulaci3n (controles 1–6) sigue siendo v3lida con s3lo la redesignaci3n del primario.

Control 1: TEDANA+24P (sensibilidad a regresi3n de motion adicional). Aplica regresi3n de los 24 par3metros de movimiento expandidos de Friston et al. (1996) — 6 r3gidos + 6 derivadas + 6 cuadrados + 6 derivadas² — sobre la salida denoised de TEDANA, antes de la extracci3n de conectividad. Es el patr3n expl3citamente descrito por la documentaci3n oficial de tedana en la secci3n “Denoising Data with Components” (DuPre et al., 2021), donde se muestra la concatenaci3n `np.hstack((rejected_components, confounds))` con `confounds` siendo los 6P + derivadas/cuadrados de fMRIPrep como ejemplo can3nico de buena pr3ctica. La inclusi3n de este pipeline cumple dos funciones complementarias: (i) testear emp3ricamente si el residual de motion no capturado por la clasificaci3n TE-dependiente a3ade varianza espuria detectable a la FC — la posici3n de Kundu et al. (2012, 2017) es que no, pero Power et al. (2018) sugieren que los artefactos respiratorios *pseudo-movimiento* s3 pueden sobrevivir ME-ICA y persistir en la salida denoised; (ii) servir de puente entre la interpretaci3n estricta de Kundu (TEDANA primario, sin regresi3n) y los controles single-eco basados en OC (que necesariamente requieren regresi3n externa porque no han pasado por ME-ICA), permitiendo *atribuir* las diferencias

inter-pipeline al denoising ICA en sí mismo en lugar de a la presencia/ausencia de regresores de movimiento. Si TEDANA y TEDANA+24P producen FC indistinguibles, la posición de Kundu queda corroborada empíricamente *sobre este dataset*; si difieren sustancialmente, es señal de que los 24P sí capturan varianza residual que ME-ICA dejó pasar.

Control 2: TEDANA+aCompCor (sensibilidad a ruido fisiológico difuso). Aplica 24P + top-10 componentes principales de aCompCor (Behzadi et al., 2007 — máscara erosionada de WM+CSF combinada, primeros 5 PC de WM y 5 de CSF) *sobre* la salida denoised de TEDANA. Esta combinación es la recomendación explícita de la documentación de tedana para ruido espacialmente difuso: “*Due to the constraints of spatial ICA, TEDICA is able to identify and remove spatially localized noise components, but it cannot identify components that are spread out throughout the whole brain. [...] Methods which have been employed in the past include [...] anatomical CompCor, Go Decomposition (GODEC), and robust PCA. Currently, tedana implements GSR and MIR.*” (DuPre et al., 2021). El motivo es estructural: la ICA espacial separa fuentes con mapas estadísticamente independientes en el dominio espacial, lo que funciona bien para artefactos focalizados (movimiento, artefactos de bobina) pero **no para señales globales o casi-globales** como las pulsaciones respiratorias profundas (Power et al., 2018) y el componente cardíaco lento — esas modulan amplios territorios cerebrales de forma sincronizada y por construcción no se descomponen en componentes ICA independientes. aCompCor las captura porque opera sobre la varianza temporal de regiones anatómicas conocidas (WM y CSF erosionadas), donde la señal BOLD verdadera es despreciable y casi toda la varianza es fisiológica. Comparar TEDANA+aCompCor vs TEDANA cuantifica exactamente cuánto ruido fisiológico difuso sobrevive a ME-ICA en *este dataset*; si la diferencia es nula, la posición Kundu queda corroborada; si es sustancial, **TEDANA+aCompCor pasa a candidato a pipeline primario**. La elección de aCompCor sobre GSR para este propósito es clave: aCompCor preserva la señal global neural genuina (que GSR elimina por construcción; Murphy & Fox, 2017) mientras captura ruido fisiológico anatómicamente localizado.

Control 3: OC+Siegel-like (replicación fiel). Replica el pipeline de Siegel et al. (2024) sobre los datos OC de fMRIPrep: 12 parámetros de movimiento (6 rígidos + 6 derivadas), señales medias de WM y CSF (regresores tisulares; Behzadi et al., 2007), scrubbing FD > 0.3 mm, y bandpass 0.009–0.08 Hz aplicado en el masker de Nilearn. Las diferencias residuales respecto al pipeline original son únicamente: (a) fMRIPrep en lugar del pipeline *in-house* de Siegel; (b) atlas Schaefer-200/Yeo-7 en lugar del FreeSurfer/Glasser empleado por Siegel. NORDIC y el umbral de scrubbing FD > 0.3 mm son idénticos. El bandpass se conserva exclusivamente en este pipeline para preservar la fidelidad metodológica al estudio de referencia, aunque se reconoce que aplicar bandpass simultáneamente con regresores spike en Nilearn introduce sesgo de difuminado del censurado (Hallquist et al., 2013); para mitigarlo, el censurado se aplica como **descarte de volúmenes post-masker.transform()** en lugar de como spike regressors, garantizando que el filtro no opera sobre frames marcados.

Control 4: OC+24P (contrapartida single-echo de TEDANA+24P). Aplica los mismos 24 parámetros de movimiento expandidos (Friston et al., 1996) que el Control 1, pero sobre los datos de combinación óptima de fMRIPrep en lugar de la salida denoised de

TEDANA. Es a TEDANA+24P lo que OC+aCompCor es a TEDANA+aCompCor: un par de pipelines con **regresores idénticos y distinto input**, de modo que cualquier diferencia entre ambos es atribuible exclusivamente al denoising TE-dependiente de ME-ICA. La comparación TEDANA+aCompCor OC+aCompCor cuantifica el aporte de ME-ICA cuando además se regresa ruido fisiológico difuso; la comparación TEDANA+24P OC+24P lo cuantifica bajo regresión de movimiento sola, es decir, en el régimen de denoising externo más parsimonioso. Si TEDANA+24P y OC+24P producen FC indistinguibles, ME-ICA no añade nada sobre la simple regresión 24P sobre este dataset; si divergen, ME-ICA elimina varianza non-BOLD —plausiblemente pseudo-movimiento respiratorio (Power et al., 2018)— que los 24P no capturan.

Control 5: OC+aCompCor (top performer Ciric). Aproxima al mejor pipeline sin GSR del benchmark de Ciric et al. (2017): 24 parámetros de movimiento expandidos (Friston et al., 1996) más los primeros 5 componentes principales de las señales de WM y los primeros 5 de las señales de CSF (aCompCor, Behzadi et al., 2007). aCompCor extrae varianza fisiológica sin requerir GSR ni invertir el signo de las correlaciones; es la alternativa estándar a GSR cuando se quiere preservar la dirección biológica de las anti-correlaciones (Chai et al., 2012). Permite evaluar el rendimiento de TEDANA frente al mejor competidor single-echo sin sesgo de centrado en cero. **Nota sobre la simetría con TEDANA+aCompCor:** la única diferencia entre OC+aCompCor y TEDANA+aCompCor es el dato de entrada (OC vs TEDANA denoised); los regresores son idénticos (24P + 10 PC aCompCor). Esto permite atribuir limpiamente cualquier diferencia entre los dos a la presencia/ausencia del denoising TE-dependiente de ME-ICA — es la comparación más informativa dentro del diseño de triangulación para *cuantificar el aporte neto de TEDANA* sobre un pipeline single-echo bien configurado.

Control 6: OC+36P+GSR (análisis de sensibilidad). Implementa la familia 36P+GSR identificada como mejor en QC-FC global por Ciric et al. (2017) y Parkes et al. (2018) sobre datos single-echo: 24 parámetros de movimiento expandidos (Friston, 1996) + 3 señales tisulares ($\overline{S_{\text{global}}}, \overline{S_{\text{WM}}}, \overline{S_{\text{CSF}}}$) $\times \{x, x^2, dx, dx^2\}$ aplicado a cada una = 24 + 12 = 36 columnas (Satterthwaite et al., 2013; Ciric et al., 2017). **Incluye GSR explícitamente.** Este pipeline no es nuestra recomendación principal por las razones detalladas arriba (doble eliminación, sesgo de centrado en cero), pero su inclusión cumple una función crítica de transparencia: si el efecto de psilocibina desaparece **solo** bajo este pipeline mientras persiste en los otros seis, es atribuible al sesgo matemático de GSR, no a un confound real (Murphy & Fox, 2017). Si persiste en los siete, es robusto frente a la dicotomía con-GSR / sin-GSR.

Lógica de triangulación:

- **Efecto en los 7 pipelines** → Resultado robusto: la señal persiste con/sin denoising ICA, con/sin GSR, con/sin bandpass, con/sin regresión externa de motion, y con/sin regresión externa de ruido fisiológico. Es la evidencia más sólida posible dentro de este TFM.
- **Efecto en TEDANA + TEDANA+24P + TEDANA+aCompCor + Siegel-like + OC+24P + OC+aCompCor, NO en 36P+GSR** → Posible sesgo de centrado en

cero por GSR (Murphy & Fox, 2017); apoya la interpretación biológica del efecto.

- **TEDANA TEDANA+24P TEDANA+aCompCor (los tres pipelines ME-ICA convergen) pero distintos de los pipelines OC** → Confirma empíricamente la posición de Kundu (2012, 2017): la regresión externa adicional sobre ME-ICA no aporta información (ni motion ni fisiológico) — los confounds externos sólo importan cuando no se ha aplicado denoising ICA. El efecto observado es atribuible a la separación BOLD/non-BOLD por TE-dependencia.
- **TEDANA TEDANA+24P (divergen sustancialmente)** → ME-ICA dejó pasar varianza residual de motion en sub-P1/P3/P4 (sospechosos en §3.13) o en runs con artefactos respiratorios pseudo-movimiento (Power et al., 2018). En ese caso, **TEDANA+24P pasa a ser candidato a pipeline primario**.
- **TEDANA TEDANA+aCompCor (divergen sustancialmente)** → ME-ICA dejó pasar **ruido fisiológico difuso** (respiratorio, cardíaco lento) que aCompCor sí captura por su anclaje anatómico en WM/CSF. En ese caso, **TEDANA+aCompCor pasa a ser candidato a pipeline primario** — y el patrón específico (afectación de DMN o redes vasculares) determinaría si el efecto biológico de psilocibina queda intacto o se atenúa.
- **TEDANA+24P TEDANA+aCompCor (convergen entre sí pero distintos de TEDANA solo)** → Tanto motion residual como fisiología difusa contribuyen, pero la regresión adicional satura — los dos pipelines convergen porque están limpiando ruido superpuesto. En este caso el pipeline primario debería ser el más conservador de los dos (TEDANA+aCompCor cubre más fuentes de ruido).
- **Efecto solo en pipelines ME-ICA (cualquiera de los tres) y no en OC** → Revelado por el denoising TE-dependiente: ME-ICA eliminó artefactos non-BOLD que enmascaraban la señal en los pipelines OC. Verificar con reportes / de TEDANA.
- **Efecto solo en OC y no en pipelines ME-ICA** → Posible artefacto non-BOLD que TEDANA eliminó correctamente; o señal BOLD eliminada como falso negativo por TEDANA (verificar componentes accepted/rejected del run en `tedana_report.html`).
- **OC+aCompCor TEDANA+aCompCor** → Caso especialmente informativo: si los dos pipelines con regresores idénticos (24P + 10 PC aCompCor) producen FC indistinguibles cambiando solo el input (OC vs TEDANA denoised), entonces el aporte neto de ME-ICA en este dataset es **marginal** y aCompCor por sí solo ya hace el trabajo. Si difieren sustancialmente, ME-ICA elimina varianza non-BOLD que aCompCor no captura — refuerza la elección del pipeline primario basado en TEDANA.
- **OC+24P TEDANA+24P** → Versión del caso anterior bajo regresión de movimiento sola: si los dos pipelines con 24P idénticos convergen al cambiar solo el input, ME-ICA no aporta sobre la regresión 24P en este dataset; si divergen, ME-ICA capta varianza de movimiento residual (pseudo-movimiento respiratorio; Power et al., 2018) que los 24P no eliminan. Junto con la comparación aCompCor, acota el aporte de ME-ICA en los dos regímenes de regresión externa.

4.4 4.4 Filtrado temporal y censurado: decisiones revisadas

Bandpass: eliminado de seis de los siete pipelines. En la versión inicial de este pipeline se aplicó bandpass 0.01–0.08 Hz a varios pipelines vía `NiftiLabelsMasker(low_pass=, high_pass=)` de Nilearn. Esta decisión se ha revisado a la luz de Kundu et al. (2012, 2013, 2017): tras ME-ICA + scrubbing, el filtrado bandpass es **redundante** porque (a) los componentes de baja frecuencia físicamente plausibles ya están preservados por la clasificación TE-dependiente, y (b) los componentes de alta frecuencia rechazados ya fueron retirados de la serie. Kundu et al. (2012) reportan literalmente que “*no other filtering was applied*” tras ME-ICA. Más críticamente, aplicar bandpass + regresores spike simultáneamente en un solo paso (modelo de Nilearn) **difumina** los regresores spike sobre frames adyacentes (Hallquist et al., 2013; Power et al., 2014), reduciendo la eficacia del censurado. Por tanto:

- **TEDANA, TEDANA+24P, TEDANA+aCompCor, OC+24P, OC+aCompCor, OC+36P+GSR: sin bandpass.** Sin filtro paso-bajo ni paso-alto en el masker (`low_pass=None, high_pass=None`).
- **OC+Siegel-like: bandpass 0.009–0.08 Hz preservado.** El estudio de Siegel et al. (2024) aplica explícitamente este bandpass como parte de su pipeline; preservarlo aquí garantiza fidelidad metodológica al estudio de referencia. Para mitigar el problema Hallquist 2013, el censurado se aplica como descarte de volúmenes `post-masker.transform()`, no como spike regressors dentro del masker.

Censurado: spike regressors → descarte post-transform. En la versión inicial el censurado se implementaba como columnas spike añadidas al matrix de confounds que pasaba al masker. Esta estrategia es estándar en la literatura (Power et al., 2014; Ciric et al., 2017) pero interactúa mal con bandpass: el filtro temporal “ve” los regresores spike como sub-segundas Dirac y los difumina sobre los frames vecinos, contaminando frames limpios y atenuando el censurado real. La solución empleada aquí es (i) dejar que el masker filtre/regress sobre los confounds **no-spike** (bandpass solo en Siegel-like, regresores tisulares/movimiento/aCompCor según pipeline), y (ii) descartar los frames con $FD > 0.3$ mm directamente del array de series temporales devuelto por `masker.transform()` antes del cómputo de la matriz de correlación. Esto preserva la lógica del scrubbing sin comprometer al filtrado.

Umbral de censurado: $FD > 0.3$ mm. Reducido desde el $FD > 0.5$ mm utilizado inicialmente para igualar el criterio de Siegel et al. (2024). Power et al. (2014) muestran que $FD > 0.5$ mm es adecuado para datos pediátricos y de adultos con alto movimiento, pero conservador para adquisiciones limpias de adultos sanos como ds006072; $FD > 0.3$ mm es el estándar para Precision Functional Mapping (Lynch et al., 2020) y para los estudios de psicodélicos del laboratorio de Siegel/Dosenbach. Se mantiene el umbral global de exclusión de runs si > 15 % de los frames son censurados (Ciric et al., 2017; Parkes et al., 2018).

El TR de ds006072 es **1.761 s**, lo que debe especificarse en los maskers (cuando bandpass está activo, en Siegel-like) y en cualquier cálculo de espectros.

4.5 4.5 Series temporales y matrices de conectividad

```
#=====
# §4 - Extracción de conectividad funcional (ds006072)
# §4.5 - Series temporales y matrices de conectividad (7 pipelines)
#=====
# Siete pipelines de denoising ejecutados en paralelo sobre datos pre-NORDIC
# (@sec-nordic) + fMRIPrep (§2):
#
# 1. TEDANA (sensibilidad ME-ICA-only) - TEDANA denoised +
# scrubbing FD>0.3, sin bandpass, sin regresión residual.
# Posición Kundu (2012, 2013, 2017; Dowdle et al., 2023) ejecutada
# como hipótesis contrastable; fallida en este dataset según
# §5.2 (cf. @sec-tedana-qc).
#
# 2. TEDANA+24P (sensibilidad motor) - TEDANA denoised + 24P
# (Friston, 1996) + scrubbing FD>0.3, sin bandpass.
# Patrón "Denoising Data with Components" recomendado por la doc
# oficial de tedana (DuPre et al., 2021); testea si la regresión
# de motion adicional sobre la salida de ME-ICA aporta valor.
#
# 3. TEDANA+aCompCor (PRIMARIO) - TEDANA denoised +
# 24P + 10 PC aCompCor + scrubbing FD>0.3, sin bandpass.
# Recomendación explícita de la doc de tedana (DuPre et al., 2021)
# para ruido espacialmente difuso (respiratorio/cardíaco) que la
# ICA espacial no captura. Regresores idénticos a OC+aCompCor; la
# única diferencia es el input (TEDANA denoised vs OC) → aísla el
# aporte neto de ME-ICA sobre un pipeline single-echo bien configurado.
# Seleccionado como primario tras §5.2 (mejor QC-FC entre los
# pipelines ME-ICA; cf. @sec-tedana-qc).
#
# 4. OC+Siegel-like (replicación) - OC + 12 motion + WM + CSF
# + scrubbing FD>0.3 + bandpass 0.009-0.08 Hz
# (replica fielmente Siegel et al., 2024)
#
# 5. OC+24P (contrapartida 24P) - OC + 24P (Friston, 1996)
# + scrubbing FD>0.3, sin bandpass.
# Mismos regresores que TEDANA+24P; la única diferencia es el input
# (OC vs TEDANA denoised) → aísla el aporte de ME-ICA bajo regresión
# 24P, igual que OC+aCompCor lo aísla bajo aCompCor.
#
# 6. OC+aCompCor (top sin GSR) - OC + 24P + 10 PC aCompCor
```

```

# + scrubbing FD>0.3, sin bandpass
# (Behzadi et al., 2007; Ciric et al., 2017 best non-GSR)
#
# 7. OC+36P+GSR (sensibilidad GSR) - OC + 36P + GSR
# + scrubbing FD>0.3, sin bandpass
# (Ciric et al., 2017 best overall, demuestra efecto Murphy & Fox 2017)
#
# Decisiones revisadas vs versión inicial:
# - Bandpass eliminado de TEDANA/aCompCor/36P+GSR (Kundu 2017 redundante;
# Hallquist 2013: filter+spike difumina censurado).
# Conservado solo en Siegel-like para fidelidad al estudio original.
# - FD threshold bajado a 0.3 mm (matchea Siegel 2024; Power 2014).
# - Censurado: descarte de volúmenes POST-masker.transform() en lugar de
# spike regressors (evita el difuminado bandpass*spike en Siegel-like).
#=====

import os
import re
import sys
import glob
import json
import functools
import traceback
import numpy as np
import pandas as pd

print = functools.partial(print, flush=True)

os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

from nilearn.maskers import NiftiLabelsMasker
from nilearn import datasets

# =====
# 1. Rutas
# =====
# fMRIPrep se ejecutó en DOS tandas (t0, t1): cada una contiene la mitad de
# las tareas BOLDREST de cada sesión (p.ej. ses-7 → BOLDREST2 en t0,
# BOLDREST1 en t1). Se listan ambas y cada run se resuelve, en el loop
# principal (§5), a la tanda que realmente lo contiene.

```

```

FMRIPREP_DIRS = [
    "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/fmriprep/data/t0",
    "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/fmriprep/data/t1",
]
TEDANA_DIR      = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/tedana/tree/minima
STATS_DIR       = "D:/ProfessionalProjects/PsyConn/results/ds006072/stats"
SPACE           = "MNI152NLin2009cAsym"

# Directorios de salida - uno por pipeline
OUT_TEDANA      = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/TEDANA"
OUT_TEDANA_24P  = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/TEDANA_24P"
OUT_TEDANA_ACOMP COR = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/TEDANA_aComp
OUT_OC_SIEGEL   = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/OC_siegel"
OUT_OC_24P      = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/OC_24P"
OUT_OC_ACOMP COR = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/OC_aCompCor
OUT_OC_GSR      = "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/conn/OC_36P_GSR"

# =====
# 2. Sujetos, sesiones y runs
# =====
# N=6; sub-P2 excluido en el pre-flight (PsiloData §3)
SUBJECTS = ["sub-P1", "sub-P3", "sub-P4",
            "sub-P5", "sub-P6", "sub-P7"]

# Mapeo sujeto → sesiones de droga y tipo (de session_data.csv y README)
# Orden: P1=MTP/PSIL, P3=MTP/PSIL, P4=MTP/PSIL,
#         P5=PSIL/MTP, P6=MTP/PSIL, P7=PSIL/MTP
DRUG_SESSIONS = {
    "sub-P1": {"PSIL": 11, "MTP": 6},
    "sub-P3": {"PSIL": 12, "MTP": 8},
    "sub-P4": {"PSIL": 10, "MTP": 6},
    "sub-P5": {"PSIL": 6, "MTP": 10},
    "sub-P6": {"PSIL": 9, "MTP": 5},
    "sub-P7": {"PSIL": 7, "MTP": 11},
}

TASKS = ["BOLDREST1", "BOLDREST2", "BOLDREST3", "BOLDREST4"]

# TR de ds006072 (Siegel et al., 2024): 1.761 s
TR = 1.761

# Umbral de scrubbing - alineado con Siegel et al. (2024) y PFM (Lynch 2020)

```

```

FD_THRESHOLD = 0.3

# Censurado por % de frames scrubbed: criterio en dos niveles (Parkes 2018
# como warning, Birn 2013 como hard cut). Permite distinguir runs con
# outliers aislados (warning, conservados con flag) de runs con motion
# contamination sistemática (excluidos). Coherente con LSDPrep §5.
PCT_CENSOR_WARN = 15 # flag para revisión QA, run conservado (Parkes 2018)
PCT_CENSOR_HARD = 25 # exclusión dura: para BOLDREST de ~15 min (513 vols ×
# 1.761 s) deja >11 min retenidos - muy por encima
# del mínimo Birn 2013 (5 min para conectividad estable)

# =====
# 3. Definición de columnas de confounds por pipeline
# =====
# Helper: 12 motion (6 rigid + 6 derivatives) - base para Siegel-like
MOTION_12P = [
    "trans_x", "trans_y", "trans_z",
    "rot_x", "rot_y", "rot_z",
    "trans_x_derivative1", "trans_y_derivative1", "trans_z_derivative1",
    "rot_x_derivative1", "rot_y_derivative1", "rot_z_derivative1",
]

# Helper: 24P (Friston et al., 1996) - base para aCompCor y 36P
MOTION_24P = MOTION_12P + [
    "trans_x_power2", "trans_y_power2", "trans_z_power2",
    "rot_x_power2", "rot_y_power2", "rot_z_power2",
    "trans_x_derivative1_power2", "trans_y_derivative1_power2",
    "trans_z_derivative1_power2",
    "rot_x_derivative1_power2", "rot_y_derivative1_power2",
    "rot_z_derivative1_power2",
]

# .....
# Pipeline 2 - TEDANA+24P (sensibilidad a regresión de motion adicional)
# .....
# Sobre la salida denoised de TEDANA, regresa 24P (Friston, 1996) sin
# bandpass. Patrón "Denoising Data with Components" recomendado por la
# documentación oficial de tedana (DuPre et al., 2021), donde se muestra
# np.hstack((rejected_components, confounds)) con confounds = motion 24P.
# En este pipeline los rejected_components ya fueron proyectados durante
# la ejecución de tedana (non-aggressive default) - aquí solo añadimos
# los 24P sobre la serie ya denoised.

```

```

# Si TEDANA TEDANA+24P → confirma posición Kundu (2012, 2017): regresión
# motion adicional redundante. Si TEDANA TEDANA+24P → ME-ICA dejó pasar
# varianza de motion residual (e.g. respiratorio pseudo-mov; Power 2018)
# y TEDANA+24P pasa a primario.
# El mismo conjunto de columnas lo reutiliza OC+24P (pipeline 5): la
# diferencia entre ambos es solo el input (TEDANA denoised vs OC).
# .....
CONFOUND_COLS_24P = MOTION_24P # solo motion; compartido por pipelines 2 y 5

# .....
# Pipeline 3 - TEDANA+aCompCor (sensibilidad a ruido fisiológico difuso)
# .....
# Sobre la salida denoised de TEDANA, regresa 24P + top-10 PCs aCompCor
# (Behzadi 2007). Recomendación EXPLÍCITA de la doc de tedana (DuPre 2021):
# "Methods which have been employed [post-TEDANA] include anatomical
# CompCor [...]. Currently, tedana implements GSR and MIR."
#
# Motivo estructural: la ICA espacial de TEDANA separa fuentes con mapas
# estadísticamente independientes en el dominio ESPACIAL - funciona bien
# para artefactos focalizados (movimiento, bobina) pero NO para señales
# globales/casi-globales (respiración profunda, cardíaco lento) que
# modulan amplios territorios cerebrales de forma sincronizada y por
# construcción no se descomponen en componentes ICA independientes.
# aCompCor las captura porque opera sobre la varianza temporal de regiones
# anatómicas conocidas (WM/CSF erosionadas) donde la señal BOLD genuina
# es despreciable y casi toda la varianza es fisiológica.
#
# Regresores IDÉNTICOS a OC+aCompCor (pipeline 6) → la única diferencia
# entre pipeline 3 y pipeline 6 es el input (TEDANA denoised vs OC) → la
# comparación 3 vs 6 cuantifica limpiamente el aporte neto de ME-ICA.
# .....
# Las columnas concretas a_comp_cor_* se filtran dinámicamente en el loop
# principal (no aquí) porque dependen del TSV de fMRIPrep por run.
# .....

# .....
# Pipeline 4 - OC+Siegel-like (replicación fiel)
# .....
# 12 motion + WM + CSF (medias). Bandpass 0.009-0.08 Hz se aplica en el
# masker, scrubbing FD>0.3 se aplica POST-transform.
# .....
CONFOUND_COLS_SIEGEL = MOTION_12P + ["white_matter", "csf"]

```

```

# .....
# Pipeline 5 - OC+24P (contrapartida single-echo de TEDANA+24P)
# .....
# OC + 24P (Friston, 1996), sin bandpass. Reutiliza CONFOUND_COLS_24P:
# regresores idénticos al pipeline 2; la única diferencia es el input
# (OC vs TEDANA denoised). La comparación pipeline 2 vs pipeline 5 aísla
# el aporte de ME-ICA bajo regresión 24P, igual que 3 vs 6 lo hace para
# aCompCor.
# .....

# .....
# Pipeline 6 - OC+aCompCor (Behzadi 2007; Ciric 2017 best non-GSR)
# .....
# 24P + top-10 PCs aCompCor de máscara WM+CSF combinada (ROI anatómica
# erosionada generada por fMRIPrep). Las columnas se llaman
# a_comp_cor_00 .. a_comp_cor_NN, ordenadas por varianza explicada.
# Tomamos las 10 primeras (~aprox 5 WM + 5 CSF en práctica), siguiendo
# la recomendación de Behzadi 2007 / Ciric 2017.
# IMPORTANTE: comparte el mismo conjunto de regresores que TEDANA+aCompCor
# (pipeline 3); la diferencia es solo el input fMRI.
# .....
N_ACOMP_COR_PCS = 10 # top-K componentes de aCompCor combinado

# .....
# Pipeline 7 - OC+36P+GSR (sensibilidad)
# .....
# 24P motion + 3 señales tisulares (WM, CSF, GS) × 4 expansiones
# (signal, derivative, power2, derivative_power2) = 24 + 3×4 = 36
# (Satterthwaite et al., 2013; Ciric et al., 2017).
# Con global_signal => GSR explícito (Murphy & Fox, 2017).
# .....
CONFOUND_COLS_36P_GSR = MOTION_24P + [
    "white_matter", "white_matter_derivative1",
    "white_matter_power2", "white_matter_derivative1_power2",
    "csf", "csf_derivative1",
    "csf_power2", "csf_derivative1_power2",
    "global_signal", "global_signal_derivative1",
    "global_signal_power2", "global_signal_derivative1_power2",
]

# =====
# 4. Atlas combinado Schaefer 200 + Tian S2 (232 ROIs)

```

```

# =====
# Atlas generado en §4.2 (cortex + subcortex, MNI152NLin2009cAsym, 2 mm).
# Captura las 7 redes Yeo en cortex (Schaefer) y los hubs subcorticales
# (tálamo, estriado, hipocampo, amígdala, palidio) críticos para el efecto
# serotoninérgico de psilocibina (Müller 2018; Carhart-Harris 2016;
# Singleton 2022). Si se desea simetría inter-dataset con LSDPrep §5,
# re-ejecutar esa sección sustituyendo su atlas Schaefer 200 por este atlas
# combinado.
# =====

ATLAS_PATH = ("D:/ProfessionalProyectos/PsyConn/derivatives/atlas/"
              "Schaefer200_Tian32_MNI2009c.nii.gz")

if not os.path.exists(ATLAS_PATH):
    raise FileNotFoundError(
        f"Atlas combinado no encontrado en {ATLAS_PATH}. "
        "Ejecutar §4.2 antes que §4.5."
    )

print(f"Cargando atlas combinado Schaefer 200 + Tian S2 (232 ROIs)...")
print(f" {ATLAS_PATH}")

# Shape esperada de cada matriz de conectividad. Se usa en extract_and_save
# para idempotencia: si un connectome ya existe con esta shape se omite el
# reprocesado; si existe con shape distinta (e.g., 200×200 de una corrida
# previa con Schaefer-only) se sobrescribe.
EXPECTED_SHAPE = (232, 232)

# =====
# 5. Maskers - uno SIN bandpass (TEDANA, aCompCor, 36P+GSR), uno CON bandpass
# (Siegel-like, 0.009-0.08 Hz). TR = 1.761 s (ds006072).
# =====

masker_nofilt = NiftiLabelsMasker(
    labels_img=ATLAS_PATH,
    standardize=True,
    detrend=True,
    low_pass=None, high_pass=None,      # Kundu 2017: bandpass redundante post-ME-ICA
    t_r=TR,
    resampling_target="data",
    verbose=0,
)

masker_siegel = NiftiLabelsMasker(

```

```

labels_img=ATLAS_PATH,
standardize=True,
detrend=True,
low_pass=0.08, high_pass=0.009,    # Siegel et al. (2024) bandpass canónico
t_r=TR,
resampling_target="data",
verbose=0,
)

# Buscar imagen de referencia para fit()
_ref = None
for subj in SUBJECTS:
    for drug, ses_num in DRUG_SESSIONS[subj].items():
        ses = f"ses-{ses_num}"
        candidates = [
            os.path.join(TEDANA_DIR, subj, ses, "BOLDREST1", "run-1",
                          "space-MNI152Nlin2009cAsym_desc-denoised_bold.nii.gz"),
        ]
        for fp_dir in FMRIPREP_DIRS:
            candidates.append(os.path.join(
                fp_dir, subj, ses, "func",
                f"{subj}_{ses}_task-BOLDREST1_dir-AP_run-1"
                f"_part-mag_space-{SPACE}_desc-preproc_bold.nii.gz"))
        for candidate in candidates:
            if os.path.exists(candidate):
                _ref = candidate
                break
        if _ref:
            break
    if _ref:
        break

if _ref is None:
    raise FileNotFoundError(
        "No se encontró ningún archivo BOLD para fit(). "
        "Ejecutar primero fMRIPrep ($2) y TEDANA ($3).")

print(f" Referencia para fit(): {os.path.basename(_ref)}")
masker_nofilt.fit(_ref)
masker_siegel.fit(_ref)
print(f" Maskers ajustados (TR = {TR} s; siegel: bandpass 0.009-0.08 Hz)")

```



```

# =====
# 6. Funciones auxiliares
# =====
def censor_postprocess(timeseries, fd_array, threshold):
    """Descarta frames con FD > threshold del array de series temporales.
    Aplicado POST-masker.transform() para evitar la difuminación
    bandpass × spike (Hallquist et al., 2013).
    Devuelve (timeseries_censored, n_dropped, pct_dropped).
    """
    fd_clean = np.nan_to_num(fd_array, nan=0.0)
    keep = fd_clean <= threshold
    n_drop = int((~keep).sum())
    pct = n_drop / len(fd_clean) * 100
    return timeseries[keep, :], n_drop, pct

def to_zcorr(timeseries):
    """Pearson → z-Fisher con clip y diagonal a 0."""
    corr = np.corrcoef(timeseries.T)
    corr = np.nan_to_num(corr, nan=0.0)
    corr = np.clip(corr, -0.999999, 0.999999)
    z = np.arctanh(corr)
    np.fill_diagonal(z, 0)
    return z

def extract_and_save(masker, func_file, confounds_array, fd_array,
                    out_dir, prefix, label):
    """Extrae series temporales con el masker, descarta frames FD>0.3
    (post-transform), calcula z-Fisher y guarda outputs estándar.

    Idempotencia: si el connectome ya existe con la shape esperada
    (EXPECTED_SHAPE), se omite el reprocesado. Si existe con shape distinta
    (e.g., 200×200 de una corrida previa con Schaefer-only) o no se puede
    leer, se reprocesa y sobrescribe.
    """
    out_npy = os.path.join(out_dir, f"{prefix}_connectome.npy")
    if os.path.exists(out_npy):
        try:
            existing_shape = np.load(out_npy, mmap_mode="r").shape
        except Exception:
            existing_shape = None
        if existing_shape == EXPECTED_SHAPE:
            print(f" {label}: ya existe ({prefix}_connectome.npy, ")

```

```

        f"shape={existing_shape})), skip")
    return True
print(f" {label}: existe con shape={existing_shape} "
      f" {EXPECTED_SHAPE} - reprocesando y sobrescribiendo")

try:
    ts = masker.transform(func_file, confounds=confounds_array)
    ts_cens, n_drop, pct = censor_postprocess(ts, fd_array, FD_THRESHOLD)
    print(f" {label}: ts {ts.shape} → tras censurado {ts_cens.shape} "
          f"({n_drop} frames descartados, {pct:.1f}%)")

    # Si tras censurar quedan <50 frames, el run no es analizable
    if ts_cens.shape[0] < 50:
        print(f" {label}: <50 frames tras censurado, omitido")
        return False

    np.savetxt(os.path.join(out_dir, f"{prefix}_timeseries.csv"),
               ts_cens, delimiter=",")

    z = to_zcorr(ts_cens)
    np.savetxt(os.path.join(out_dir, f"{prefix}_zcorr.csv"),
               z, delimiter=",")
    np.save(os.path.join(out_dir, f"{prefix}_connectome.npy"), z)
    print(f" {label} guardado: {prefix}_connectome.npy")
    return True

except Exception:
    print(f" ERROR {label}:")
    traceback.print_exc()
    return False

# =====
# 7. Loop principal: 7 pipelines en paralelo
# =====
RUNS = [1, 2] # Cada tarea BOLDREST contiene run-1 y run-2

for subj in SUBJECTS:
    for drug, ses_num in DRUG_SESSIONS[subj].items():
        ses = f"ses-{ses_num}"
        cond_label = drug.lower() # "psil" o "mtp"

        for task in TASKS:

```

```

for run_num in RUNS:
    run_label = f"run-{run_num}"
    out_prefix = f"{subj}_{ses}_{task}_{run_label}"

    # Localizar el run: se prueba el TSV de confounds en cada
    # tanda de fMRIPrep (t0/t1). La primera coincidencia fija
    # func_dir para el resto de archivos del run. Si no aparece
    # en ninguna → el run no existe para este sujeto.
    conf_probe = []
    for fp_dir in FM RIPREP_DIRS:
        conf_probe = glob.glob(os.path.join(
            fp_dir, subj, ses, "func",
            f"{subj}_{ses}_{task}_{task}_dir-*_{run_label}"
            f"_part-mag_desc-confounds_timeseries.tsv"
        ))
        if conf_probe:
            break
    if not conf_probe:
        continue
    func_dir = os.path.dirname(conf_probe[0])
    # Phase encoding (AP/PA) extraído del nombre del TSV
    pe_dir = os.path.basename(
        conf_probe[0]).split("dir-")[1].split("_")[0]

    # ---- Archivos de entrada ----
    # TEDANA denoised en MNI (pipeline 1)
    tedana_file = os.path.join(
        TEDANA_DIR, subj, ses, task, run_label,
        "space-MNI152Nlin2009cAsym_desc-denoised_bold.nii.gz"
    )
    # Combinación óptima de fMRIPrep (pipelines 2-4)
    oc_file = os.path.join(
        func_dir,
        f"{subj}_{ses}_{task}_{task}_dir-{pe_dir}_{run_label}"
        f"_part-mag_space-{SPACE}_desc-preproc_bold.nii.gz"
    )
    conf_file = conf_probe[0]

    has_tedana = os.path.exists(tedana_file)
    has_oc = os.path.exists(oc_file)
    if not (has_tedana or has_oc):
        print(f" Omitido: {subj} {ses} {task} {run_label} ")

```

```

        f"(ni TEDANA ni OC encontrados)")
    continue

# ---- Leer confounds ----
conf_df = pd.read_csv(conf_file, sep="\t")
fd = conf_df["framewise_displacement"].values

# ---- Pre-check de censurado: criterio en dos niveles ----
# > PCT_CENSOR_HARD → exclusión (Birn 2013: <5 min de datos)
# > PCT_CENSOR_WARN → flag QA, run conservado (Parkes 2018)
fd_clean = np.nan_to_num(fd, nan=0.0)
pct_above = (fd_clean > FD_THRESHOLD).mean() * 100
n_total = len(fd_clean)
n_keep = (fd_clean <= FD_THRESHOLD).sum()
min_kept = n_keep * TR / 60.0
if pct_above > PCT_CENSOR_HARD:
    print(f"    {subj} {ses} {task} {run_label}: "
          f"{pct_above:.1f}% frames > FD {FD_THRESHOLD} mm "
          f"> {PCT_CENSOR_HARD}% - run EXCLUIDO "
          f"({min_kept:.1f} min restantes < 5 min, Birn 2013)")
    continue
if pct_above > PCT_CENSOR_WARN:
    print(f"    {subj} {ses} {task} {run_label}: "
          f"{pct_above:.1f}% frames > FD {FD_THRESHOLD} mm "
          f"> {PCT_CENSOR_WARN}% - FLAG QA, run conservado "
          f"({min_kept:.1f} min restantes 5 min)")

# ---- Construir matrices de confounds por pipeline ----
def cols_to_array(col_list):
    avail = [c for c in col_list if c in conf_df.columns]
    missing = set(col_list) - set(avail)
    if missing:
        print(f"    Confounds ausentes: {sorted(missing)}")
    return conf_df[avail].fillna(0).replace(
        [np.inf, -np.inf], 0).to_numpy()

# Pipelines 2 y 5: 24P (TEDANA+24P y OC+24P comparten regresor)
conf_24p = cols_to_array(CONFOUND_COLS_24P)

# Pipeline 4: Siegel-like (12 motion + WM + CSF)
conf_siegel = cols_to_array(CONFOUND_COLS_SIEGEL)

```

```

# Pipelines 3 y 6: aCompCor (24P + top-K aCompCor PCs)
# MISMOS regresores para TEDANA+aCompCor y OC+aCompCor; lo
# que cambia entre pipelines es el input (TEDANA denoised
# vs OC), no las columnas regresadas.
# aCompCor: 5 PC de WM + 5 PC de CSF (Behzadi 2007). fMRIPrep
# numera `a_comp_cor_NN` mezclando las máscaras CSF/WM/combined;
# sin filtrar por `Mask` no se garantiza el balance 5+5 ni se
# evita la redundancia con la máscara combinada (§2.2.3 lo
# advierte explícitamente). Se filtra por el sidecar JSON.
conf_json_path = conf_file.replace(".tsv", ".json")
acompcor_cols = []
try:
    with open(conf_json_path) as _f:
        _conf_meta = json.load(_f)
    def _acompcor_top(mask_name, k):
        cands = [
            (c, m.get("VarianceExplained", 0.0))
            for c, m in _conf_meta.items()
            if isinstance(m, dict)
            and m.get("Method") == "aCompCor"
            and m.get("Mask") == mask_name
            and m.get("Retained", False)
            and c in conf_df.columns
        ]
        cands.sort(key=lambda x: -x[1])
        return [c for c, _ in cands[:k]]
    acompcor_cols = _acompcor_top("WM", 5) \
        + _acompcor_top("CSF", 5)
except (OSError, json.JSONDecodeError, KeyError) as e:
    print(f"    No se pudo leer aCompCor por máscara del JSON "
          f"({e}); fallback a top-{N_ACOMPCOR_PCS} por índice.")
    acompcor_cols = sorted([
        c for c in conf_df.columns
        if re.fullmatch(r"a_comp_cor_\d+", c)
    ][:N_ACOMPCOR_PCS])
if len(acompcor_cols) < N_ACOMPCOR_PCS:
    print(f"    Solo {len(acompcor_cols)} componentes aCompCor "
          f"disponibles (esperados {N_ACOMPCOR_PCS}): "
          f"5 WM + 5 CSF)")
conf_acompcor = cols_to_array(MOTION_24P + acompcor_cols)

# Pipeline 7: 36P + GSR

```

```

conf_gsr = cols_to_array(CONFOUND_COLS_36P_GSR)

print(f"\nProcesando {subj} | {ses} ({drug}) "
      f"| {task} {run_label} | dir-{pe_dir}")

# =====
# Pipeline 1 - TEDANA (sensibilidad ME-ICA-only)
# =====
# Hipótesis Kundu (2017): tras ME-ICA no haría falta nuisance
# externo. Contrastada empíricamente en §5.2 - fallida en
# este dataset (cf. @sec-tedana-qc). Se conserva
# como rama de sensibilidad que muestra qué pasa al confiar
# sólo en la clasificación TE-dependiente.
if has_tedana:
    out_dir = os.path.join(OUT_TEDANA, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, tedana_file,
        confounds_array=None,      # Kundu 2017: sin regresión
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="TEDANA")

# =====
# Pipeline 2 - TEDANA + 24P (sensibilidad a motion adicional)
# =====
# Patrón "Denoising Data with Components" recomendado por la
# documentación oficial de tedana (DuPre 2021): regresión 24P
# sobre la serie ya denoised por ME-ICA. Testea si los 24P
# capturan varianza residual de motion que ME-ICA dejó pasar.
if has_tedana:
    out_dir = os.path.join(OUT_TEDANA_24P, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, tedana_file,
        confounds_array=conf_24p,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="TEDANA+24P")

# =====
# Pipeline 3 - TEDANA + aCompCor (PRIMARIO)

```

```

# =====
# Pipeline primario del análisis (seleccionado en §5.2;
# cf. @sec-tedana-qc). Recomendación EXPLÍCITA
# de la doc oficial de tedana (DuPre 2021) para ruido
# espacialmente difuso (respiratorio / cardíaco lento) que
# la ICA espacial NO puede capturar por construcción -
# modulan amplios territorios cerebrales de forma
# sincronizada y no se descomponen en componentes ICA
# independientes.
# Regresores idénticos a OC+aCompCor (24P + top-10 PCs
# aCompCor) → la comparación pipeline 3 vs pipeline 6
# cuantifica limpiamente el aporte neto de ME-ICA.
if has_tedana:
    out_dir = os.path.join(OUT_TEDANA_ACOMP COR, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, tedana_file,
        confounds_array=conf_acompcor,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="TEDANA+aCompCor")

# =====
# Pipeline 4 - OC + Siegel-like (replicación)
# =====
if has_oc:
    out_dir = os.path.join(OUT_OC_SIEGEL, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_siegel, oc_file,
        confounds_array=conf_siegel,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="OC+Siegel")

# =====
# Pipeline 5 - OC + 24P (contrapartida single-echo de
# TEDANA+24P)
# =====
# Mismos regresores 24P que el pipeline 2; la única
# diferencia es el input (OC vs TEDANA denoised) → la
# comparación pipeline 2 vs pipeline 5 aísla el aporte de

```

```

# ME-ICA bajo regresión 24P.
if has_oc:
    out_dir = os.path.join(OUT_OC_24P, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, oc_file,
        confounds_array=conf_24p,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="OC+24P")

# =====
# Pipeline 6 - OC + aCompCor
# =====
if has_oc:
    out_dir = os.path.join(OUT_OC_ACOMP COR, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, oc_file,
        confounds_array=conf_acompcor,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="OC+aCompCor")

# =====
# Pipeline 7 - OC + 36P + GSR (sensibilidad GSR)
# =====
if has_oc:
    out_dir = os.path.join(OUT_OC_GSR, cond_label, subj)
    os.makedirs(out_dir, exist_ok=True)
    extract_and_save(
        masker_nofilt, oc_file,
        confounds_array=conf_gsr,
        fd_array=fd,
        out_dir=out_dir,
        prefix=out_prefix, label="OC+36P+GSR")

print("\n$4.5 completa "
      "(TEDANA, TEDANA+24P, TEDANA+aCompCor, "
      "OC+Siegel, OC+24P, OC+aCompCor, OC+36P+GSR).")

```


4.6 4.6 Promedios de conectividad por sujeto y grupo

Una vez extraídas las matrices de conectividad por run, este paso consolida una matriz de conectividad **por condición y sujeto** (PSIL y MTP) y los correspondientes promedios grupales y diferencias intra-sujeto PSIL – MTP, análogamente a la consolidación equivalente del pipeline LSD (LSDPrep §5). El diseño cruzado (*crossover*) de ds006072 permite la comparación intra-sujeto controlando la variabilidad inter-individual.

Estado actual del dataset (cf. Section 1, §2). Aunque el protocolo nominal de ds006072 contempla 2 tareas REST $\times$ 2 runs = 4 adquisiciones por sesión (~60 min totales), tras múltiples iteraciones de NORDIC + fMRIPrep + TEDANA varios runs han resultado inservibles para el análisis por motivos heterogéneos (denoising fallido, frames truncados, sujetos excluidos en el paper original P2, etc.). Los runs supervivientes se reorganizaron en **dos subconjuntos disjuntos**, procesados en las tandas de fMRIPrep t0 y t1, cada uno con un run por sesión y por tarea BOLDREST. Ambas tandas están ya preprocesadas y §4.5 las consolida en un único directorio `conn/<pipeline>/<condición>/<sujeto>/`, de modo que cada sujeto y condición dispone de **2 runs BOLDREST**.

Promediado entre runs. El chunk promedia los runs disponibles por condición y sujeto (mejora la SNR de la estimación de conectividad; Birn et al., 2013) y genera los **archivos de salida estándar** (`*_mean_connectome.npy`, `*_diff_connectome.npy`, `group_mean_*.npy`, `all_*.connectomes.npy`), de modo que los chunks downstream que los consumen (§5 y PsyConn §2-§4) funcionan sin cambios. Los promedios se generan para los siete pipelines de la triangulación (TEDANA, TEDANA+24P, TEDANA+aCompCor, OC+Siegel, OC+24P, OC+aCompCor, OC+36P+GSR).

```
#####
# §4 - Extracción de conectividad funcional (ds006072)
# §4.6 - Consolidación de conectividad por sujeto y grupo
#####
# Consolida una matriz de conectividad por condición (PSIL, MTP) y sujeto a
# partir de los runs extraídos en §4.5, y calcula promedios grupales y
# diferencias intra-sujeto PSIL - MTP.
#
# Estado actual del dataset (cf. @sec-nordic, §2): tras la limpieza por NORDIC +
# fMRIPrep + TEDANA varios runs nominales resultaron inservibles (denoising
# fallido, frames truncados, exclusiones del paper original P2, etc.). Los
# runs supervivientes se reorganizaron en dos subconjuntos disjuntos -las
# tandas de fMRIPrep t0 y t1, ambas ya preprocesadas-; §4.5 las consolida
# en un único directorio conn/<pipeline>/<cond>/<subj>/, de modo que cada
# sujeto y condición dispone de 2 runs BOLDREST.
#
# El chunk promedia los runs disponibles por condición y sujeto (mejora la
```

```

# SNR de la estimación de conectividad; Birn et al., 2013) y genera los
# archivos de salida (mean_connectome, diff_connectome, group_*,
# all_*_connectomes) que consumen sin cambios los chunks downstream
# (§5 y PsyConn §2-§4).
#
# Se procesan los siete pipelines de la triangulación (§4.3):
#   TEDANA, TEDANA+24P, TEDANA+aCompCor, OC+Siegel, OC+24P, OC+aCompCor,
#   OC+36P+GSR.
#=====

import os
import glob
import numpy as np
import pandas as pd
from nilearn import datasets

STATS_DIR = "D:/ProfessionalProyects/PsyConn/results/ds006072/stats"
os.makedirs(STATS_DIR, exist_ok=True)

# Etiquetas del atlas combinado Schaefer 200 + Tian S2 (generado en §4.2)
# Estructura: 1 Background + 200 Schaefer (Yeo-7) + 32 Tian S2 = 233 entradas
ATLAS_LABELS_CSV = ("D:/ProfessionalProyects/PsyConn/derivatives/atlas/"
                    "atlas_labels_232.csv")

if not os.path.exists(ATLAS_LABELS_CSV):
    raise FileNotFoundError(
        f"Etiquetas combinadas no encontradas en {ATLAS_LABELS_CSV}. "
        "Ejecutar §4.2 antes que §4.6."
    )

labels_df = pd.read_csv(ATLAS_LABELS_CSV)
# Persistir en formato compatible con downstream (una columna 'label',
# Background incluido como primera fila)
labels_df[["label"]].to_csv(
    os.path.join(STATS_DIR, "atlas_labels_ds006072.csv"), index=False
)
print(f"Atlas combinado: {len(labels_df)} etiquetas guardadas "
      f"(1 Background + 200 Schaefer + 32 Tian)")

# -----
# Definición de pipelines
# -----

```

```

PIPELINES = [
    ("TEDANA",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/TEDANA",
     "Primario: NORDIC + TEDANA ICA denoising + scrubbing FD>0.3 "
     "(Kundu et al., 2013; Moeller et al., 2021; Siegel et al., 2024)"),
    ("TEDANA+24P",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/TEDANA_24P",
     "Sensibilidad motor: TEDANA denoised + 24P (Friston, 1996) + scrubbing "
     "FD>0.3 - patrón Denoising Data with Components (DuPre et al., 2021)"),
    ("TEDANA+aCompCor",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/TEDANA_aCompCor",
     "Sensibilidad fisiológica: TEDANA denoised + 24P + 10 PC aCompCor + "
     "scrubbing FD>0.3 - recomendación oficial tedana para ruido difuso "
     "(Behzadi et al., 2007; DuPre et al., 2021)"),
    ("OC+Siegel",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/OC_siegel",
     "Replicación Siegel: OC + 12P motion + WM + CSF + bandpass 0.009-0.08 Hz "
     "+ scrubbing FD>0.3 (Siegel et al., 2024)"),
    ("OC+24P",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/OC_24P",
     "Contrapartida single-echo de TEDANA+24P: OC + 24P (Friston, 1996) + "
     "scrubbing FD>0.3 - aísla el aporte de ME-ICA bajo regresión 24P"),
    ("OC+aCompCor",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/OC_aCompCor",
     "Mejor non-GSR: OC + 24P + 5 PC WM + 5 PC CSF + scrubbing FD>0.3 "
     "(Behzadi et al., 2007; Ciric et al., 2017)"),
    ("OC+36P+GSR",
     "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/OC_36P_GSR",
     "Sensibilidad GSR: OC + 36P (24 motion + 3 tissue × 4 expansiones = 24+12) + "
     "scrubbing FD>0.3 (Ciric et al., 2017; Murphy & Fox, 2017)"),
]

CONDITIONS = ["psil", "mtp"]

def save_csv(mat, out_path):
    pd.DataFrame(mat).to_csv(out_path)

# -----
# Procesar cada pipeline
# -----
for pipe_name, conn_dir, pipe_desc in PIPELINES:
    psil_dir = os.path.join(conn_dir, "psil")

```

```

mtp_dir = os.path.join(conn_dir, "mtp")

if not os.path.isdir(psil_dir):
    print(f"\n Directorio {pipe_name} no encontrado ({psil_dir}) "
          f"- ejecutar primero $4.5")
    continue

print(f"\n{'='*60}")
print(f"Promedios {pipe_name} - {pipe_desc}")
print(f"{'='*60}")

subjects = sorted(
    d for d in os.listdir(psil_dir)
    if os.path.isdir(os.path.join(psil_dir, d)) and d != "group"
)
print(f"Sujetos detectados: {subjects}")

group_psil, group_mtp, group_diff = [], [], []

for subj in subjects:
    # Buscar connectomes por condición. Nominalmente hasta 4 runs
    # (BOLDREST1/2 × run-1/2); en la práctica varía por sujeto tras
    # la limpieza NORDIC+fMRIPrep+TEDANA (ver comentario inicial).
    # Se excluyen las propias salidas de este chunk (mean_connectome).
    psil_files = sorted(
        f for f in glob.glob(os.path.join(
            psil_dir, subj, f"{subj}*_connectome.npy"))
        if "mean_connectome" not in os.path.basename(f)
        and "diff_connectome" not in os.path.basename(f)
    )
    mtp_files = sorted(
        f for f in glob.glob(os.path.join(
            mtp_dir, subj, f"{subj}*_connectome.npy"))
        if "mean_connectome" not in os.path.basename(f)
        and "diff_connectome" not in os.path.basename(f)
    )

    if not psil_files or not mtp_files:
        print(f" Omitido {subj} ({pipe_name}): "
              f"PSIL={len(psil_files)}, MTP={len(mtp_files)}")
        continue

```

```

# Carga + promedio entre runs (mejora la SNR de la estimación de
# conectividad; Birn et al., 2013). Se guarda con nombre
# 'mean_connectome' por compatibilidad con los chunks downstream.
psil_mats = [np.load(f) for f in psil_files]
mtp_mats  = [np.load(f) for f in mtp_files]

n_psil = len(psil_mats)
n_mtp  = len(mtp_mats)

mean_psil = np.mean(psil_mats, axis=0)
mean_mtp  = np.mean(mtp_mats,  axis=0)
diff      = mean_psil - mean_mtp

psil_tag = f"promedio de {n_psil} runs"
mtp_tag  = f"promedio de {n_mtp} runs"

subj_psil_dir = os.path.join(psil_dir, subj)
subj_mtp_dir  = os.path.join(mtp_dir,  subj)

np.save(os.path.join(subj_psil_dir, f"{subj}_mean_connectome.npy"),
        mean_psil)
np.save(os.path.join(subj_mtp_dir,  f"{subj}_mean_connectome.npy"),
        mean_mtp)
np.save(os.path.join(subj_psil_dir, f"{subj}_diff_connectome.npy"),
        diff)

save_csv(mean_psil, os.path.join(subj_psil_dir,
                                f"{subj}_mean_psil.csv"))
save_csv(mean_mtp,  os.path.join(subj_mtp_dir,
                                f"{subj}_mean_mtp.csv"))
save_csv(diff,      os.path.join(subj_psil_dir,
                                f"{subj}_diff.csv"))

print(f" {subj}: PSIL={mean_psil.shape} ({psil_tag}), "
      f"MTP={mean_mtp.shape} ({mtp_tag})")

group_psil.append(mean_psil)
group_mtp.append(mean_mtp)
group_diff.append(diff)

# ---- Promedios grupales ----
if group_psil:

```

```

n = len(group_psil)
group_mean_psil = np.mean(group_psil, axis=0)
group_mean_mtp = np.mean(group_mtp, axis=0)
group_mean_diff = np.mean(group_diff, axis=0)

group_out = os.path.join(conn_dir, "group")
os.makedirs(group_out, exist_ok=True)

np.save(os.path.join(group_out, "group_mean_psil.npy"), group_mean_psil)
np.save(os.path.join(group_out, "group_mean_mtp.npy"), group_mean_mtp)
np.save(os.path.join(group_out, "group_diff.npy"), group_mean_diff)

save_csv(group_mean_psil, os.path.join(group_out, "group_mean_psil.csv"))
save_csv(group_mean_mtp, os.path.join(group_out, "group_mean_mtp.csv"))
save_csv(group_mean_diff, os.path.join(group_out, "group_diff.csv"))

# Guardar también las matrices individuales apiladas para análisis
# estadístico posterior (tests pareados PSIL vs MTP)
np.save(os.path.join(group_out, "all_psil_connectomes.npy"),
        np.array(group_psil))
np.save(os.path.join(group_out, "all_mtp_connectomes.npy"),
        np.array(group_mtp))
np.save(os.path.join(group_out, "all_diff_connectomes.npy"),
        np.array(group_diff))

print(f"\n Promedios grupales {pipe_name}: "
      f"n={n} sujetos, shape={group_mean_psil.shape}")
print(f" Matrices individuales apiladas: "
      f"({n}, {group_mean_psil.shape[0]}, {group_mean_psil.shape[1]})")
else:
    print(f"\nSin resultados grupales {pipe_name}.")

print("\n$4.6 completa "
      "(promedios TEDANA, TEDANA+24P, TEDANA+aCompCor, "
      "OC+Siegel, OC+24P, OC+aCompCor, OC+36P+GSR).")

```

5 5 Control de calidad

El movimiento de cabeza durante la adquisición de fMRI constituye la principal fuente de artefacto en estudios de conectividad funcional en reposo (rs-fcMRI). Power et al. (2012)

demonstraron que desplazamientos incluso submilimétricos generan correlaciones espurias pero sistemáticas: las conexiones de corto alcance se inflan artificialmente mientras que las de largo alcance se atenúan, un patrón que persiste tras la realineación volumétrica y la regresión estándar de parámetros de movimiento. Este hallazgo motivó la propuesta de dos índices de calidad frame-a-frame —*Framewise Displacement* (FD) y DVARS— y la técnica de *scrubbing* (censura temporal de volúmenes contaminados).

Evaluaciones sistemáticas posteriores han confirmado la heterogeneidad en la eficacia de las estrategias de denoising. Ciric et al. (2017, *NeuroImage*) compararon 14 pipelines en 393 jóvenes y encontraron que los modelos de 36 parámetros (36P: 24 parámetros de movimiento expandidos + 8 señales tisulares + 4 señal global) con censura temporal fueron los más eficaces en reducir las correlaciones QC-FC (calidad–conectividad), con menos del 7% de aristas mostrando relación significativa con el movimiento. Notablemente, los seis pipelines con mejor rendimiento incluyeron regresión de señal global (GSR), que resultó ser la estrategia individual más eficaz para minimizar el acoplamiento movimiento–conectividad. Sin embargo, la GSR exacerbó la dependencia de distancia del artefacto residual, mientras que las técnicas de censura redujeron ambos (Ciric et al., 2017).

Parkes et al. (2018, *NeuroImage*) extendieron esta evaluación a 19 pipelines en cuatro datasets independientes con cinco benchmarks: correlaciones QC-FC, dependencia de distancia QC-FC, contrastes entre sujetos de alto y bajo movimiento (HLM), pérdida de grados de libertad temporales (tDOF-loss) y fiabilidad test-retest (TRT). Sus resultados confirmaron que: (1) la regresión simple de parámetros de movimiento (6HMP, 24HMP) es insuficiente; (2) aCompCor solo es viable en datos de bajo movimiento; (3) ICA-AROMA ofrece buen rendimiento con baja pérdida de datos; (4) GSR mejoró prácticamente todos los pipelines en la mayoría de benchmarks; y (5) las diferencias grupales en conectividad son altamente dependientes de la estrategia de preprocesamiento.

Respecto a la GSR, Murphy y Fox (2017, *NeuroImage*) alcanzaron un consenso: la GSR no revela la naturaleza “verdadera” del cerebro, sino que diferentes estrategias de procesamiento producen perspectivas complementarias. La GSR impone matemáticamente que las correlaciones promedio sean cero, generando anti-correlaciones que pueden ser artefactuales. Sin embargo, la GSR ofrece ventajas documentadas: mayor correspondencia con la anatomía por DTI, mejor delineación de núcleos subcorticales, mayor especificidad de correlaciones positivas y reducción de señales de movimiento, cardíacas y respiratorias. Crucialmente, la GSR altera métricas de teoría de grafos —reduce las estimaciones de heredabilidad del coeficiente de clustering, la modularidad y la eficiencia global (Schwarz y McGonigle, 2010; Sinclair et al., 2015)—, lo que debe considerarse al interpretar los análisis de esta sección y de PsyConn §2.

Todos los datos fueron preprocesados con fMRIPrep (Esteban et al., 2019, *Nature Methods*), un pipeline robusto y agnóstico al análisis que se adapta automáticamente al dataset de entrada mediante el estándar BIDS (Gorgolewski et al., 2016). fMRIPrep integra herramientas de FSL, ANTs, FreeSurfer y AFNI, produciendo datos preprocesados con mayor precisión espacial y menor suavizado no controlado que pipelines alternativos como FEAT (Esteban et al., 2019). Críticamente, fMRIPrep genera automáticamente las matrices de confounds (FD,

DVARS, parámetros de movimiento, señales tisulares, componentes aCompCor) que alimentan las estrategias de denoising evaluadas en este control de calidad.

El presente control de calidad se aplica a **ds006072 (psilocibina)** procesado con **NORDIC + fMRIPrep + TEDANA** (Section 1, §2, §3) y a los **siete pipelines de denoising paralelos** definidos en §4.3 (TEDANA, TEDANA+24P, TEDANA+aCompCor, OC+Siegel-like, OC+24P, OC+aCompCor, OC+36P+GSR). Reportar las métricas QA en los siete pipelines permite verificar la lógica de triangulación de §4: si una métrica QA mejora consistentemente bajo los tres pipelines ME-ICA (TEDANA, TEDANA+24P, TEDANA+aCompCor) y bajo OC+aCompCor, pero empeora bajo 36P+GSR, el efecto refleja un beneficio real del denoising, no un sesgo introducido por GSR; si se observa el patrón inverso, conviene revisar la fidelidad de la extracción TEDANA. Tres comparaciones específicas son especialmente diagnósticas: **(a)** TEDANA vs TEDANA+24P resuelve si los 24 parámetros de motion aportan valor *adicional* sobre la salida ME-ICA (Kundu 2017 predice que no; Power 2018 sugiere que sí en runs con respiración profunda); **(b)** TEDANA vs TEDANA+aCompCor resuelve si el ruido fisiológico difuso (respiratorio/cardíaco) sobrevive ME-ICA y requiere aCompCor para limpiarse (DuPre 2021 lo recomienda explícitamente); **(c)** TEDANA+aCompCor vs OC+aCompCor — con regresores idénticos pero input distinto — **cuantifica directamente el aporte neto de ME-ICA** sobre un pipeline single-echo bien configurado.

- **§5.1** — Correlaciones FD/DVARS – RSFC a nivel global por pipeline (benchmark 1 de Power 2012), reportada para los 7 pipelines. Los benchmarks 2 (% aristas con QC-FC significativa) y 3 (dependencia de distancia) se mantienen en el chunk como anexo descriptivo: fueron diseñados para $N > 30$ y carecen de poder con $N=6$.
- **§5.2** — Triangulación inter-pipeline + tDOF + convergencia de efectos PSIL–MTP (similitud 7×7 de **group_diff**, pérdida de grados de libertad temporales por pipeline, test pareado de una muestra y Cohen’s d_z por arista; Cumming 2014), con un árbol de decisión pre-registrado para la selección del pipeline primario. Es el QA verdaderamente informativo con $N=6$.

Caveat de tamaño muestral. ds006072 es un estudio de Precision Functional Mapping (PFM); tras la exclusión de sub-P2 en el pre-flight (PsiloData §3) el subconjunto preprocesado cuenta con **N=6 sujetos por condición** (sub-P1, P3, P4, P5, P6, P7) y **k=2 runs por (sujeto, condición)** —BOLDREST1 run-1 y run-2, integrados desde las dos tandas **t0/t1** de fMRIPrep—. Las métricas QC-FC clásicas (Power 2012; Ciric 2017) fueron diseñadas para $N>30$; con $N=6$ se reportan como sanity check y para detectar diferencias gruesas entre pipelines, no como inferencia confirmatoria. La triangulación inter-pipeline de §5.2 — y, aguas abajo, el análisis de sensibilidad de PsyConn §7 (LOO + bootstrap + permutación intra-sujeto + convergencia inter-pipeline) — sustituye a la corrección por atenuación basada en ICC test-retest como vía de robustez estadística defendible para un diseño PFM con $N=6$ (decisión registrada en [reviews/PsiloPrep_§7-qc_ronda3_estadistico.md](#)).

5.1 5.1 Framewise Displacement, DVARS y QC-FC

Evaluación de la contaminación por movimiento según los tres benchmarks estándar de la literatura (Power et al., 2012; Ciric et al., 2017; Parkes et al., 2018): (1) correlación FD/DVARS–RSFC media, (2) proporción de aristas con correlación QC-FC significativa, y (3) dependencia de distancia de las correlaciones QC-FC.

⚠ Benchmarks 2 y 3: rol descriptivo con N=6

Los benchmarks 2 y 3 (% aristas con QC-FC significativa y dependencia de distancia) fueron diseñados para muestras con $N > 30$ (Power 2012; Ciric 2017; Parkes 2018). Con $N=6$ sus p-valores cross-subject son ruidosos: se reportan como **figuras descriptivas para comparar pipelines entre sí**, no como inferencia confirmatoria sobre la contaminación por movimiento. El benchmark 1 (correlación FD/DVARS – mean RSFC) sí opera sobre runs y conserva poder con $N=6$. La métrica diagnóstica primaria del QA con $N=6$ es la triangulación inter-pipeline de §5.2; la robustez estadística se cierra aguas abajo en PsyConn §7 (LOO + bootstrap + permutación + convergencia inter-pipeline).

```
#####
# §5.1 - FD, DVARS y QC-FC (Quality Control - Functional Connectivity)
#####
# Aplicado a ds006072 (psilocibina, PFM design) procesado con NORDIC +
# fMRIPrep + TEDANA (@sec-nordic, $2, $3). Itera sobre los 7 pipelines de §4.3:
#   1. TEDANA          - sensibilidad ME-ICA-only (posición Kundu 2017,
#                       - contrastada y falsada empíricamente en §5.2)
#   2. TEDANA+24P      - sensibilidad motor (DuPre 2021, doc oficial tedana)
#   3. TEDANA+aCompCor - PRIMARIO post-§5.2 (DuPre 2021; @sec-tedana-qc)
#   4. OC+Siegel-like  - replicación de Siegel 2024
#   5. OC+24P          - contrapartida OC de TEDANA+24P
#   6. OC+aCompCor     - best non-GSR (Ciric 2017); comparación canónica
#                       - con primario (regresores idénticos, input distinto)
#   7. OC+36P+GSR     - sensibilidad GSR (Ciric 2017; Murphy & Fox 2017)
#
# Implementa los tres benchmarks estándar:
#   1. Correlación FD/DVARS - RSFC media (Power 2012)
#   2. Proporción de aristas con QC-FC significativa (Ciric 2017)
#   3. Dependencia de distancia de las QC-FC (Parkes 2018)
#
# CAVEAT N pequeña: tras la exclusión de sub-P2 en el pre-flight (PsiloData
# $3), N=6 sujetos por condición y k=2 runs intra-sujeto (BOLDREST1 run-1
# + run-2). Las métricas QC-FC cross-subject (benchmarks 2 y 3) tienen
# poder muy limitado con este N (los benchmarks se diseñaron para N>30).
```

```

# Se reportan para comparar pipelines (lógica de triangulación), no como
# inferencia confirmatoria. La triangulación inter-pipeline de §5.2 es el
# QA verdaderamente informativo en diseño PFM; la robustez estadística se
# resuelve aguas abajo en PsyConn §7 (LOO + bootstrap + permutación +
# convergencia inter-pipeline). Umbral mínimo N3 sujetos por condición
# (mínimo matemático para cor.test); por debajo el benchmark se omite con
# aviso.
#
# Confounds: leídos de los TSV de fMRIPrep (independientes del pipeline de
# denoising - son los mismos para los 7). Los connectomas se leen de los
# directorios conn/<pipeline>/ generados en §4.5.
# Umbral FD = 0.3 mm (Siegel 2024; Lynch 2020), no 0.5 mm.
#=====

library(tidyverse)
library(patchwork)

# =====
# 0. Configuración: ds006072 + 7 pipelines
# =====
DERIV_DIR      <- "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep"
# fMRIPrep se ejecutó en dos tandas bajo fmriprep/data/: t0 cubre el primer
# run de BOLDREST1; t1 cubre los runs adicionales (BOLDREST1 run-2 y, cuando
# existe, BOLDREST2). Los runs son disjuntos entre tandas → la unión cubre
# la sesión completa, sin duplicados, y se combinan en el descubrimiento (§1).
FMRIPREP_DIRS <- file.path(DERIV_DIR, "fmriprep", "data", c("t0", "t1"))
ATLAS_DIR     <- "D:/ProfessionalProyectos/PsyConn/derivatives/atlas"
RESULTS_DIR   <- "D:/ProfessionalProyectos/PsyConn/results/ds006072"
out_dir       <- file.path(REULTS_DIR, "stats/qa")
plot_dir      <- file.path(REULTS_DIR, "figures/qa")
dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

# Pipelines de §4.3 - se itera sobre los 7 para triangulación
PIPELINES <- tibble::tribble(
  ~name,          ~conn_dir,
  "TEDANA",       "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/TEDANA",
  "TEDANA+24P",   "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/TEDANA_24P",
  "TEDANA+aCompCor", "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/TEDANA_aCompCor",
  "OC+Siegel",    "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/OC_siegel",
  "OC+24P",       "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/OC_24P",
  "OC+aCompCor",  "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn/OC_aCompCor"
)

```

```

"OC+36P+GSR",      "D:/ProfessionalProyects/PsyConn/derivatives/ds006072/conn/OC_36P_GSR"
)

# Mapeo sujeto → sesiones de droga (de §4.5, basado en session_data.csv y
# README). sub-P2 excluido en pre-flight (PsiloData §3 - N=6).
DRUG_SESSIONS <- list(
  "sub-P1" = list(PSIL = 11, MTP = 6),
  "sub-P3" = list(PSIL = 12, MTP = 8),
  "sub-P4" = list(PSIL = 10, MTP = 6),
  "sub-P5" = list(PSIL = 6,  MTP = 10),
  "sub-P6" = list(PSIL = 9,  MTP = 5),
  "sub-P7" = list(PSIL = 7,  MTP = 11)
)
SUBJECTS <- names(DRUG_SESSIONS)

# Umbrales - alineados con §4.5 (Siegel 2024; Lynch 2020; Power 2014)
FD_SCRUB_THRESH <- 0.3
DVARs_SCRUB_THRESH <- 1.5
TR_SECONDS <- 1.761 # ds006072 - necesario para min_retained
# Censurado en dos niveles (coherente con §4.5 Python y LSDPrep §5)
PCT_CENSOR_WARN <- 15 # flag QA, run conservado (Parkes 2018)
PCT_CENSOR_HARD <- 25 # exclusión (Birn 2013: <5 min restantes)
FD_EXCLUSION_SUBJ <- 0.3 # mean FD por sujeto/sesión

# =====
# 1. Leer FD y DVARs desde los TSVs de fMRIPrep (idénticos para los 7 pipes)
# -----
# Descubrimiento dinámico: para cada (sujeto, droga) se enumeran TODOS los
# confound files presentes en func/ y se parsean task/run/dir del nombre.
# Esto es necesario porque tras la limpieza NORDIC+fMRIPrep+TEDANA el run
# superviviente varía por sujeto (p.ej. sub-P6 conserva BOLDREST4 mientras
# otros conservan BOLDREST1/2). Iterar sobre TASKS y RUNS hardcodedos
# perdería esos sujetos en silencio.
# =====
CONFOUND_FNAME_PATTERN <- paste0(
  "(sub-[^_]+)_(ses-[^_]+)_task-(BOLDREST\\d+)_dir-([A-Z]+)_run-(\\d+)",
  "_part-mag_desc-confounds_timeseries\\.tsv$"
)

discover_runs_for_subj_drug <- function(subj, drug) {
  ses_num <- DRUG_SESSIONS[[subj]][[drug]]
  ses <- sprintf("ses-%d", ses_num)

```

```

# Enumera los confound TSV de la sesión en AMBAS tandas (t0, t1) y los
# combina. Los runs son disjuntos entre tandas (p.ej. BOLDREST2 en t0,
# BOLDREST1 en t1) → la unión cubre la sesión completa, sin duplicados.
conf_files <- character(0)
for (fp_dir in FMRIPREP_DIRS) {
  func_dir <- file.path(fp_dir, subj, ses, "func")
  if (!dir.exists(func_dir)) next
  conf_files <- c(conf_files,
                  list.files(func_dir, pattern = CONFOUND_FNAME_PATTERN,
                             full.names = TRUE))
}
if (length(conf_files) == 0) return(NULL)

map_dfr(conf_files, function(conf_file) {
  fname <- basename(conf_file)
  m      <- regmatches(fname, regexec(CONFOUND_FNAME_PATTERN, fname))[[1]]
  task   <- m[4]
  run_num <- as.integer(m[6])

  df <- read_tsv(conf_file, show_col_types = FALSE,
                 na = c("", "NA", "n/a"))
  if (!"framewise_displacement" %in% names(df)) return(NULL)

  fd_vec    <- df$framewise_displacement
  dvars_vec <- if ("std_dvars" %in% names(df)) df$std_dvars else NA_real_

  n_total  <- sum(!is.na(fd_vec))
  n_scrub  <- sum(fd_vec > FD_SCRUB_THRESH, na.rm = TRUE)
  pct_scrub <- ifelse(n_total > 0, 100 * n_scrub / n_total, NA_real_)

  # Criterio en dos niveles (coherente con §4.5 Python):
  #   excluded = TRUE si pct > PCT_CENSOR_HARD (Birn 2013, <5 min)
  #   flag_warn = TRUE si PCT_CENSOR_WARN < pct < PCT_CENSOR_HARD
  #               (run conservado pero requiere inspección, Parkes 2018)
  min_retained <- ifelse(n_total > 0,
                        (n_total - n_scrub) * TR_SECONDS / 60.0,
                        NA_real_)

  tibble(
    subject = subj,
    condition = tolower(drug),
    session = ses,
    task = task,

```

```

    run          = run_num,
    mean_FD      = mean(fd_vec,    na.rm = TRUE),
    median_FD    = median(fd_vec,  na.rm = TRUE),
    max_FD       = max(fd_vec,     na.rm = TRUE),
    mean_DVARS   = mean(dvars_vec, na.rm = TRUE),
    n_volumes    = n_total,
    n_scrubbed   = n_scrub,
    pct_scrubbed = pct_scrub,
    min_retained = min_retained,
    flag_warn_qa = !is.na(pct_scrub) &
                    pct_scrub > PCT_CENSOR_WARN &
                    pct_scrub <= PCT_CENSOR_HARD,
    excluded     = !is.na(pct_scrub) & pct_scrub > PCT_CENSOR_HARD
  )
})
}

motion_df <- expand_grid(
  subject = SUBJECTS,
  drug    = c("PSIL", "MTP")
) %>%
  pmap_dfr(function(subject, drug) {
    discover_runs_for_subj_drug(subject, drug)
  })

write.csv(motion_df, file.path(out_dir, "FD_DVARS_by_run.csv"), row.names = FALSE)
cat(sprintf(
  "Movimiento leído: %d runs encontrados por descubrimiento dinámico\n",
  nrow(motion_df)
))
cat(sprintf(" Sujetos con datos: %d/%d | Sujetos × condición con datos: %d/%d\n",
            length(unique(motion_df$subject)), length(SUBJECTS),
            nrow(distinct(motion_df, subject, condition)),
            length(SUBJECTS) * 2L))
if (nrow(motion_df) > 0) {
  task_run_summary <- motion_df %>%
    count(task, run, name = "n_runs") %>%
    arrange(task, run)
  cat(" Tasks/runs detectados:\n")
  print(as.data.frame(task_run_summary), row.names = FALSE)
}

```

```

# Sujetos × condición flaggeados por movimiento excesivo.
# Nota: `flag_high_motion` se usa como ANOTACIÓN de QA, no como filtro. Con
# N=6 no se descarta a un sujeto por mean FD; el único filtro real es
# `!excluded` (a nivel de run, > PCT_CENSOR_HARD). La sensibilidad a sujetos
# flaggeados se evalúa aguas abajo en PsyConn §7.
subject_motion <- motion_df %>%
  group_by(subject, condition) %>%
  summarize(
    mean_FD_session      = mean(mean_FD, na.rm = TRUE),
    mean_DVARS_session   = mean(mean_DVARS, na.rm = TRUE),
    pct_scrubbed_avg      = mean(pct_scrubbed, na.rm = TRUE),
    n_runs_warn_qa        = sum(flag_warn_qa),      # entre WARN y HARD
    n_runs_excluded       = sum(excluded),          # > HARD
    .groups = "drop"
  ) %>%
  mutate(flag_high_motion = mean_FD_session > FD_EXCLUSION_SUBJ)

write.csv(subject_motion, file.path(out_dir, "subject_motion_summary.csv"),
  row.names = FALSE)
cat(sprintf("Sujetos × condición flaggeados (mean FD > %.2f mm): %d/%d\n",
  FD_EXCLUSION_SUBJ, sum(subject_motion$flag_high_motion),
  nrow(subject_motion)))

# =====
# 2. Función auxiliar: leer matriz zcorr.csv (formato np.savetxt, sin header)
# =====
read_zcorr <- function(f) {
  m <- as.matrix(read.csv(f, header = FALSE))
  storage.mode(m) <- "numeric"
  diag(m) <- 0
  m
}

mean_run_rsfc <- function(f) {
  m <- read_zcorr(f)
  mean(m[upper.tri(m)], na.rm = TRUE)
}

# =====
# 3. Procesar QA por pipeline - leer zcorr.csv y unir a motion_df
# =====
process_pipeline <- function(pipe_name, conn_dir) {

```

```

cat(sprintf("\n=== Pipeline: %s ===\n", pipe_name))

fd_rsfc_df <- motion_df %>%
  rowwise() %>%
  mutate(
    conn_file = {
      cond_dir <- file.path(conn_dir, condition, subject)
      f <- file.path(cond_dir,
                     sprintf("%s_%s_%s_run-%d_zcorr.csv",
                             subject, session, task, run))
      if (file.exists(f)) f else NA_character_
    }
  ) %>%
  ungroup() %>%
  mutate(mean_RSFC = sapply(conn_file, function(f) {
    if (is.na(f)) NA_real_ else mean_run_rsfc(f)
  })))

fd_rsfc_df$pipeline <- pipe_name
fd_rsfc_df_valid <- fd_rsfc_df %>%
  filter(!is.na(mean_RSFC), !excluded)
cat(sprintf("  Runs con zcorr válido y FD aceptable: %d/%d\n",
            nrow(fd_rsfc_df_valid), nrow(fd_rsfc_df)))
fd_rsfc_df_valid
}

all_pipelines_df <- map2_dfr(PIPELINES$name, PIPELINES$conn_dir,
                           process_pipeline)
write.csv(all_pipelines_df,
          file.path(out_dir, "FD_DVARS_RSFC_by_run_all_pipelines.csv"),
          row.names = FALSE)

# =====
# 4. Correlaciones FD-RSFC y DVARS-RSFC por pipeline (benchmark 1)
# =====

corr_global <- all_pipelines_df %>%
  group_by(pipeline, condition) %>%
  summarize(
    r_FD_spearman = cor(mean_FD, mean_RSFC, method = "spearman"),
    r_FD_pearson = cor(mean_FD, mean_RSFC, method = "pearson"),
    r_DVARS_spearman = cor(mean_DVARS, mean_RSFC, method = "spearman",
                           use = "complete.obs"),

```

```

    r_DVARs_pearson = cor(mean_DVARs, mean_RSFC, method = "pearson",
                          use = "complete.obs"),
    n_runs          = n(),
    .groups = "drop"
)

write.csv(corr_global,
          file.path(out_dir, "FD_DVARs_RSFC_correlation_global.csv"),
          row.names = FALSE)

cat("\n=== Correlaciones globales FD/DVARs - RSFC por pipeline ===\n")
print(as.data.frame(corr_global), row.names = FALSE)

# =====
# 5. QC-FC por arista y dependencia de distancia (benchmarks 2 y 3)
#   CAVEAT: con ~5-6 sujetos por condición las correlaciones cross-subject
#   son ruidosas (muy por debajo del rango clásico Power/Ciric/Parkes,
#   N>30). Se reportan para comparar entre pipelines, no como inferencia.
#   Umbral mínimo N3 (mínimo matemático para cor.test); por debajo se
#   omite con aviso.
# =====
QCFC_MIN_SUBJECTS <- 3L
COORDS_FILE <- file.path(ATLAS_DIR, "Schaefer200_Tian32_coords.csv")
roi_coords <- if (file.exists(COORDS_FILE)) read.csv(COORDS_FILE) else NULL
if (is.null(roi_coords))
  warning("No se encontraron coordenadas (ejecutar §4.2); ",
          "distance-dependence omitida.")

compute_qcfc_pipeline <- function(pipe_name, conn_dir) {
  subj_means <- all_pipelines_df %>%
    filter(pipeline == pipe_name) %>%
    group_by(subject, condition) %>%
    summarize(mean_FD_subj = mean(mean_FD), .groups = "drop")

  qcfc_per_cond <- list()
  for (cond in c("psil", "mtp")) {
    cond_subjs <- subj_means %>% filter(condition == cond)
    if (nrow(cond_subjs) < QCFC_MIN_SUBJECTS) {
      cat(sprintf(" %s | %s: solo %d sujetos (<%d), omitido QC-FC\n",
                  pipe_name, cond, nrow(cond_subjs), QCFC_MIN_SUBJECTS))
      next
    }
  }
}

```



```

mats <- lapply(cond_subjs$subject, function(s) {
  cond_dir <- file.path(conn_dir, cond, s)
  files <- list.files(cond_dir, pattern = "_zcorr\\.csv$",
                      full.names = TRUE)
  if (length(files) == 0) return(NULL)
  mat_list <- lapply(files, read_zcorr)
  Reduce("+", mat_list) / length(mat_list)
})
valid <- !sapply(mats, is.null)
mats <- mats[valid]
fd_v <- cond_subjs$mean_FD_subj[valid]
if (length(mats) < QCFC_MIN_SUBJECTS) {
  cat(sprintf(" %s | %s: solo %d sujetos con zcorr, omitido QC-FC\n",
              pipe_name, cond, length(mats)))
  next
}

n_roi <- nrow(mats[[1]])
idx <- which(upper.tri(matrix(0, n_roi, n_roi)), arr.ind = TRUE)
n_edges <- nrow(idx)

# Edge values: matriz n_subj x n_edges
edge_vals <- t(sapply(mats, function(m) m[upper.tri(m)]))

qcfc <- apply(edge_vals, 2, function(e) cor(fd_v, e, use = "complete.obs"))
qcfc_p <- apply(edge_vals, 2, function(e) {
  tryCatch(cor.test(fd_v, e)$p.value, error = function(x) NA_real_)
})

distances <- NULL
rho_dist <- NA_real_
if (!is.null(roi_coords) && nrow(roi_coords) >= n_roi) {
  distances <- sqrt(
    (roi_coords$x[idx[, 1]] - roi_coords$x[idx[, 2]])^2 +
    (roi_coords$y[idx[, 1]] - roi_coords$y[idx[, 2]])^2 +
    (roi_coords$z[idx[, 1]] - roi_coords$z[idx[, 2]])^2
  )
  rho_dist <- cor(distances, qcfc, method = "spearman",
                  use = "complete.obs")
}

qcfc_per_cond[[cond]] <- list(

```

```

    pipeline      = pipe_name,
    condition     = cond,
    n_subjects    = length(mats),
    n_edges       = n_edges,
    pct_sig_p05   = 100 * mean(qcfc_p < 0.05, na.rm = TRUE),
    median_abs_qcfc = median(abs(qcfc), na.rm = TRUE),
    distance_dependence_rho = rho_dist,
    qcfc          = qcfc,
    qcfc_p        = qcfc_p,
    distances     = distances
  )
}
qcfc_per_cond
}

cat("\nCalculando QC-FC por arista para los 7 pipelines (puede tardar 1-3 min)...\n")
all_qcfc <- map(transpose(PIPELINES), function(p) {
  compute_qcfc_pipeline(p$name, p$conn_dir)
})
names(all_qcfc) <- PIPELINES$name

# Resumen tabular
qcfc_summary <- map_dfr(all_qcfc, function(res_pipe) {
  map_dfr(res_pipe, function(r) {
    if (is.null(r)) return(NULL)
    tibble(
      pipeline      = r$pipeline,
      condition     = r$condition,
      n_subjects    = r$n_subjects,
      n_edges       = r$n_edges,
      pct_sig_p05   = r$pct_sig_p05,
      median_abs_qcfc = r$median_abs_qcfc,
      distance_dependence_rho = r$distance_dependence_rho
    )
  })
})

write.csv(qcfc_summary, file.path(out_dir, "QCFC_summary_all_pipelines.csv"),
  row.names = FALSE)

cat("\n=== QC-FC y distance-dependence por pipeline ===\n")
cat("Caveat: ~5-6 sujetos por condición → poder cross-subject muy limitado.\n")

```

```

cat("Refs: Ciric 2017 (<5% sig = denoising eficaz);\n")
cat("      Parkes 2018 (|rho_dist| > 0.10 = dependencia residual).\n\n")
if (nrow(qcfc_summary) > 0) {
  print(as.data.frame(qcfc_summary), row.names = FALSE)
} else {
  cat("(QC-FC no calculado en ningún pipeline - N por condición ",
      sprintf("< %d.)\n", QCFC_MIN_SUBJECTS), sep = "")
}

# =====
# 6. Visualizaciones QA
# =====

PIPE_COLORS <- c(
  "TEDANA"           = "#E74C3C", # rojo
  "TEDANA+24P"       = "#F39C12", # naranja
  "TEDANA+aCompCor"  = "#D35400", # naranja oscuro (familia ME-ICA)
  "OC+Siegel"        = "#3498DB", # azul
  "OC+24P"           = "#16A085", # teal (contrapartida OC de TEDANA+24P)
  "OC+aCompCor"       = "#28B463", # verde
  "OC+36P+GSR"       = "#9B59B6"  # morado
)

# Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C (MTP).
# Coherente con la convención global de PsyConn (cond_colors); el par
# rojo/azul está reservado para POS/NEG en §5/§6.3 de PsyConn y su
# reutilización para condición es contraproducente.
COND_COLORS <- c("psil" = "#C2185B", "mtp" = "#00695C")

# Orden de pipelines para el layout asimétrico patchwork (4 filas: 1+2+2+2)
PIPE_LAYOUT <- c("TEDANA+aCompCor", "TEDANA+24P", "TEDANA",
  "OC+aCompCor", "OC+24P", "OC+Siegel", "OC+36P+GSR")

# 6a. FD vs RSFC por pipeline - layout 1 fila completa + 3x2 (patchwork)
make_fd_panel <- function(pipe_name) {
  is_primary <- pipe_name == "TEDANA+aCompCor"
  ggplot(all_pipelines_df %>% filter(pipeline == pipe_name),
    aes(x = mean_FD, y = mean_RSFC, color = condition)) +
    geom_point(alpha = 0.6, size = 1.5) +
    geom_smooth(method = "lm", se = TRUE, alpha = 0.15, linewidth = 0.7) +
    theme_minimal(base_size = 10) +
    scale_color_manual(values = COND_COLORS) +
    labs(title = if (is_primary) paste0(pipe_name, " [primario]") else pipe_name,
      x = "Mean FD (mm)", y = "Mean RSFC (z)") +

```

```

    theme(legend.position = if (is_primary) "right" else "none",
          plot.title = element_text(size = 10,
                                     face = if (is_primary) "bold" else "plain"))
  }
fd_panels <- setNames(lapply(PIPE_LAYOUT, make_fd_panel), PIPE_LAYOUT)
p_fd <- (fd_panels[["TEDANA+aCompCor"]] /
  (fd_panels[["TEDANA+24P"]] | fd_panels[["TEDANA"]]) /
  (fd_panels[["OC+aCompCor"]] | fd_panels[["OC+24P"]]) /
  (fd_panels[["OC+Siegel"]] | fd_panels[["OC+36P+GSR"]])) +
plot_layout(heights = c(2, 1, 1, 1)) +
plot_annotation(
  title = "[QA] FD vs mean RSFC por run, los 7 pipelines (ds006072)",
  subtitle = "Power 2012; Ciric 2017"
)
ggsave(file.path(plot_dir, "QA_FD_vs_RSFC.png"), p_fd,
       width = 10, height = 14, dpi = 300)

# 6b. DVARs vs RSFC por pipeline - layout 1 fila completa + 3x2 (patchwork)
make_dvars_panel <- function(pipe_name) {
  is_primary <- pipe_name == "TEDANA+aCompCor"
  df_pipe <- all_pipelines_df %>% filter(pipeline == pipe_name, !is.na(mean_DVARs))
  ggplot(df_pipe, aes(x = mean_DVARs, y = mean_RSFC, color = condition)) +
    geom_point(alpha = 0.6, size = 1.5) +
    geom_smooth(method = "lm", se = TRUE, alpha = 0.15, linewidth = 0.7) +
    theme_minimal(base_size = 10) +
    scale_color_manual(values = COND_COLORS) +
    labs(title = if (is_primary) paste0(pipe_name, " [primario]") else pipe_name,
         x = "Mean std_DVARs", y = "Mean RSFC (z)") +
    theme(legend.position = if (is_primary) "right" else "none",
          plot.title = element_text(size = 10,
                                     face = if (is_primary) "bold" else "plain"))
}
dvars_panels <- setNames(lapply(PIPE_LAYOUT, make_dvars_panel), PIPE_LAYOUT)
p_dvars <- (dvars_panels[["TEDANA+aCompCor"]] /
  (dvars_panels[["TEDANA+24P"]] | dvars_panels[["TEDANA"]]) /
  (dvars_panels[["OC+aCompCor"]] | dvars_panels[["OC+24P"]]) /
  (dvars_panels[["OC+Siegel"]] | dvars_panels[["OC+36P+GSR"]])) +
plot_layout(heights = c(2, 1, 1, 1)) +
plot_annotation(
  title = "[QA] DVARs vs mean RSFC por run, los 7 pipelines",
  subtitle = "DVARs estandarizado (Power 2012; Nichols 2013)"
)

```

```

ggsave(file.path(plot_dir, "QA_DVARS_vs_RSFC.png"), p_dvars,
        width = 10, height = 14, dpi = 300)

# 6c. Distribución de QC-FC por pipeline
# Si QC-FC se omitió en todos los pipelines (N<umbral), all_qcfc devuelve
# listas vacías → map_dfr produce un tibble de 0 columnas. Se blindan los
# accesos a 'qcfc' contra ese caso para no romper la ejecución del chunk.
has_any_qcfc <- any(map_lgl(all_qcfc, ~ length(.x) > 0))

if (has_any_qcfc) {
  qcfc_dist_df <- map_dfr(all_qcfc, function(res_pipe) {
    map_dfr(res_pipe, function(r) {
      if (is.null(r)) return(NULL)
      tibble(qcfc = r$qcfc, condition = r$condition, pipeline = r$pipeline)
    })
  }) %>% filter(!is.na(qcfc))

  if (nrow(qcfc_dist_df) > 0) {
    make_qcfc_hist_panel <- function(pipe_name) {
      is_primary <- pipe_name == "TEDANA+aCompCor"
      df_pipe <- qcfc_dist_df %>% filter(pipeline == pipe_name)
      ggplot(df_pipe, aes(x = qcfc, fill = condition)) +
        geom_histogram(bins = 80, alpha = 0.55, position = "identity") +
        geom_vline(xintercept = 0, linetype = "dashed", color = "grey40") +
        scale_y_continuous(labels = scales::label_number()) +
        theme_minimal(base_size = 10) +
        scale_fill_manual(values = COND_COLORS) +
        labs(title = if (is_primary) paste0(pipe_name, " [primario]") else pipe_name,
             x = "QC-FC (Pearson r)", y = "N aristas") +
        theme(legend.position = if (is_primary) "right" else "none",
              plot.title = element_text(size = 10,
                                         face = if (is_primary) "bold" else "plain"))
    }

    qcfc_hist_panels <- setNames(lapply(PIPE_LAYOUT, make_qcfc_hist_panel),
                                PIPE_LAYOUT)

    p_qcfc_hist <- (qcfc_hist_panels[["TEDANA+aCompCor"]] /
      (qcfc_hist_panels[["TEDANA+24P"]] | qcfc_hist_panels[["TEDANA"]])) /
      (qcfc_hist_panels[["OC+aCompCor"]] | qcfc_hist_panels[["OC+24P"]]) /
      (qcfc_hist_panels[["OC+Siegel"]] | qcfc_hist_panels[["OC+36P+GSR"]])) +
      plot_layout(heights = c(2, 1, 1, 1)) +
      plot_annotation(
        title = "[QA] Distribución de QC-FC por arista, los 7 pipelines",

```

```

      subtitle = "Ciric 2017 - referencia: <5% aristas significativas (p<.05)"
    )
    ggsave(file.path(plot_dir, "QA_QCFC_distribution.png"),
           p_qcfc_hist, width = 10, height = 14, dpi = 300)
  }
} else {
  cat("[QA] 6c - Distribución QC-FC omitida (sin pipelines con N suficiente).\n")
}

# 6d. Distance dependence por pipeline
has_dist_dep <- has_any_qcfc &&
  "distance_dependence_rho" %in% names(qcfc_summary) &&
  any(!is.na(qcfc_summary$distance_dependence_rho))

if (has_dist_dep) {
  dist_dep_df <- map_dfr(all_qcfc, function(res_pipe) {
    map_dfr(res_pipe, function(r) {
      if (is.null(r) || is.null(r$distances)) return(NULL)
      tibble(distance = r$distances, qcfc = r$qcfc,
              condition = r$condition, pipeline = r$pipeline)
    })
  }) %>% filter(!is.na(qcfc), !is.na(distance))

  if (nrow(dist_dep_df) > 0) {
    make_dist_dep_panel <- function(pipe_name) {
      is_primary <- pipe_name == "TEDANA+aCompCor"
      df_pipe <- dist_dep_df %>% filter(pipeline == pipe_name)
      ggplot(df_pipe, aes(x = distance, y = qcfc, color = condition)) +
        geom_point(alpha = 0.02, size = 0.3) +
        geom_smooth(method = "loess", se = FALSE, linewidth = 1) +
        geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
        theme_minimal(base_size = 10) +
        scale_color_manual(values = COND_COLORS) +
        labs(title = if (is_primary) paste0(pipe_name, " [primario]") else pipe_name,
             x = "Distancia inter-ROI (mm)", y = "QC-FC (Pearson r)") +
        theme(legend.position = if (is_primary) "right" else "none",
              plot.title = element_text(size = 10,
                                         face = if (is_primary) "bold" else "plain"))
    }
    dist_dep_panels <- setNames(lapply(PIPE_LAYOUT, make_dist_dep_panel),
                               PIPE_LAYOUT)
    p_dist <- (dist_dep_panels[["TEDANA+aCompCor"]]) /

```

```

      (dist_dep_panels[["TEDANA+24P"]] | dist_dep_panels[["TEDANA"]]) /
      (dist_dep_panels[["OC+aCompCor"]] | dist_dep_panels[["OC+24P"]]) /
      (dist_dep_panels[["OC+Siegel"]] | dist_dep_panels[["OC+36P+GSR"]])) +
    plot_layout(heights = c(2, 1, 1, 1)) +
    plot_annotation(
      title = "[QA] Dependencia de distancia QC-FC, los 7 pipelines",
      subtitle = "Power 2012; Ciric 2017; Parkes 2018 | cortex+subcortex (232 ROIs)"
    )
    ggsave(file.path(plot_dir, "QA_QCFC_distance.png"), p_dist,
           width = 10, height = 14, dpi = 300)
  }
} else {
  cat("[QA] 6d - Dependencia de distancia QC-FC omitida ",
      "(sin coordenadas ROI o sin pipelines con N suficiente).\n", sep = "")
}

# 6e. Movimiento por sujeto
p_motion <- subject_motion %>%
  ggplot(aes(x = reorder(subject, mean_FD_session),
             y = mean_FD_session, fill = condition)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6, alpha = 0.85) +
  geom_hline(yintercept = FD_EXCLUSION_SUBJ,
            linetype = "dashed", color = "red", linewidth = 0.5) +
  annotate("text", x = 1, y = FD_EXCLUSION_SUBJ + 0.02,
          label = sprintf("Umbral: %.2f mm", FD_EXCLUSION_SUBJ),
          hjust = 0, size = 3, color = "red") +
  theme_minimal(base_size = 11) +
  scale_fill_manual(values = COND_COLORS) +
  labs(title = "[QA] Mean FD por sujeto y condición (ds006072)",
       subtitle = "Umbral 0.3 mm = Siegel 2024 / PFM standard",
       x = "", y = "Mean FD (mm)") +
  coord_flip()
ggsave(file.path(plot_dir, "QA_motion_per_subject.png"), p_motion,
       width = 8, height = 6, dpi = 300)

cat("\nOK §5.1 (ds006072, 7 pipelines) completo - outputs en:\n")
cat(sprintf(" Stats: %s\n", out_dir))
cat(sprintf(" Figures: %s\n", plot_dir))

```

5.2 5.2 Triangulación inter-pipeline + tDOF + convergencia de efectos

El control de calidad de §5.1 (QC-FC) evalúa cada pipeline **en aislamiento**. Pero la pregunta operacional del TFM no es “¿cuál pipeline es el mejor en QC-FC?”, sino “**¿qué pipeline elijo como primario y los efectos PSIL vs MTP que reporto son robustos frente a esa elección?**”. Esta sección integra tres análisis complementarios diseñados específicamente para responder esa pregunta con $N=6$, donde las métricas QC-FC clásicas (Power 2012; Ciric 2017) tienen poder estadístico limitado:

1. **Similaridad inter-pipeline (clustermap 7×7)** — Correlación entre las matrices `group_diff` (PSIL – MTP) de los 7 pipelines, con clustering jerárquico. Responde: ¿los 7 pipelines producen FC convergente o radicalmente distinta? ¿Los 3 pipelines ME-ICA forman un cluster separado de los 4 pipelines OC? ¿TEDANA+aCompCor se agrupa con OC+aCompCor (regresores idénticos) o con TEDANA (input idéntico)? La estructura del dendrograma revela qué factor — *input* (ME-ICA vs OC) o *regresores externos* (motion/aCompCor/GSR) — domina la varianza inter-pipeline.
2. **Pérdida de grados de libertad temporales (tDOF) por pipeline** — Cálculo explícito de DOF efectivos = `n_frames` - `n_regressors` - `n_scrubbed` por run y pipeline. TEDANA solo gasta 0 regresores externos (Kundu 2017) frente a los 36 del pipeline 36P+GSR; con $N=6$ cada DOF cuenta. La tabla cuantifica el coste estadístico de cada elección como input al cálculo de Cohen’s d_z por arista de esta misma sección.
3. **Convergencia de efectos PSIL vs MTP** — Para cada arista, test pareado de una muestra sobre la matriz `all_diff_connectomes.npy` ($N \times 232 \times 232$; H : la diferencia PSIL – MTP es cero a nivel grupal) y tamaño del efecto Cohen’s d_z (Cohen 1988; cf. Cumming 2014 para paired designs). Comparación cruzada: dado un pipeline declarado *primario* (TEDANA+aCompCor tras la ejecución de §5.2; configurable vía `PRIMARY_PIPE`), ¿qué fracción de sus aristas significativas (a) mantiene el mismo signo en los otros 6 pipelines, y (b) alcanza también significancia? La tabla de convergencia es **la métrica que defiende los hallazgos ante el tribunal**: efectos que replican en 6/6 pipelines de sensitivity son sólidos; los que solo aparecen en el primario son sospechosos.

Caveat $N=6$ explícito. Con 6 sujetos, los p-valores de los tests pareados son ruidosos y no se aplica corrección por múltiples comparaciones a nivel de arista ($232 \times 231 / 2 = 26\,796$ tests). El umbral nominal $p < 0.05$ se usa aquí como *marcador descriptivo* de tendencia, no como inferencia confirmatoria — PsyConn §6 aplicará FDR/permutación sobre el pipeline primario una vez seleccionado. La métrica realmente diagnóstica de esta sección es la **fracción de aristas que coinciden en signo** entre pipelines (no requiere significancia, robusta a N pequeña).

```
#####  
# §5.2 - Triangulación inter-pipeline + tDOF + convergencia de efectos  
#####
```



```

# Tres análisis complementarios sobre los 7 pipelines de §4.3 para decidir
# pipeline primario con criterios pre-registrados:
#
# 1. Similitud inter-pipeline (clustermap 7×7 de group_diff).
# 2. tDOF por pipeline × run.
# 3. Convergencia de efectos PSIL-MTP (signo + significancia descriptiva).
#
# Output:
# stats/QA/triangulation_similarity.csv      (matriz 7×7 + clustering)
# stats/QA/triangulation_t dof.csv          (tDOF por run × pipeline)
# stats/QA/triangulation_convergence.csv    (replicación inter-pipeline)
# figures/QA/triangulation_clustermap.png
# figures/QA/triangulation_t dof.png
# figures/QA/triangulation_convergence.png
#
# CAVEAT N=6: los p-valores de los tests pareados son ruidosos; la métrica
# diagnóstica primaria es la concordancia de signo entre pipelines, no
# la significancia.
#=====

import os
import glob
import re
from pathlib import Path
from functools import partial
import numpy as np
import pandas as pd
from scipy import stats, spatial
from scipy.cluster.hierarchy import linkage, dendrogram, leaves_list
import matplotlib.pyplot as plt

print = partial(print, flush=True)

# =====
# 0. Configuración
# =====
CONN_BASE = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/conn"
# fMRIPrep se ejecutó en dos tandas bajo fmrip/derivatives/: t0 cubre el primer
# run de BOLDREST1; t1 cubre los runs adicionales (BOLDREST1 run-2 y, cuando
# existe, BOLDREST2). La sección 3 (tDOF) recorre ambas; los runs son
# disjuntos entre tandas, la unión los cubre sin duplicados.
FMRIPREP_DIRS = [

```

```

    "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/fmriprep/data/t0",
    "D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/fmriprep/data/t1",
]
RESULTS_DIR = "D:/ProfessionalProjects/PsyConn/results/ds006072"
QA_STATS = os.path.join(RESULTS_DIR, "stats", "qa")
QA_FIGS = os.path.join(RESULTS_DIR, "figures", "qa")
os.makedirs(QA_STATS, exist_ok=True)
os.makedirs(QA_FIGS, exist_ok=True)

# Pipelines (etiqueta visible, directorio en /conn/, alias, n_regresores externos)
# n_regressors = parámetros pasados a confounds_array en §4.5; los rejected
# components de TEDANA ya están proyectados y NO se cuentan aquí (Kundu 2017
# los considera parte del denoising interno).
PIPELINES = [
    ("TEDANA", "TEDANA", 0),
    ("TEDANA+24P", "TEDANA_24P", 24),
    ("TEDANA+aCompCor", "TEDANA_aCompCor", 34), # 24P + 10 PC
    ("OC+Siegel", "OC_siegel", 14), # 12P + WM + CSF
    ("OC+24P", "OC_24P", 24), # 24P
    ("OC+aCompCor", "OC_aCompCor", 34), # 24P + 10 PC
    ("OC+36P+GSR", "OC_36P_GSR", 36),
]
PIPE_COLORS = {
    "TEDANA": "#E74C3C",
    "TEDANA+24P": "#F39C12",
    "TEDANA+aCompCor": "#D35400",
    "OC+Siegel": "#3498DB",
    "OC+24P": "#16A085",
    "OC+aCompCor": "#28B463",
    "OC+36P+GSR": "#9B59B6",
}

# Pipeline primario tras ejecución del output (cf. @sec-tedana-qc):
# el árbol de decisión disparó la cláusula "TEDANA+aCompCor diverge
# sustancialmente de TEDANA (r=0.68 < 0.80) → TEDANA+aCompCor pasa a primario".
# El benchmark 1 QC-FC (§5.1) y la similaridad inter-pipeline (§5.2)
# confirmaron la decisión: TEDANA+aCompCor reduce r_FD a 0.50 (vs 0.87 TEDANA
# puro en PSIL), pct_sig_QCFC a 5.4% (vs 15.1%), y queda en el centro del
# clúster denso de la matriz 7x7 (r >= 0.84 con TEDANA+24P, OC+24P y OC+aCompCor).
PRIMARY_PIPE = "TEDANA+aCompCor"
TR = 1.761
FD_THRESHOLD = 0.3
SIG_ALPHA = 0.05 # umbral descriptivo, no inferencia confirmatoria

```

```

N_ROI          = 232          # Schaefer 200 + Tian S2 (§4.2)

# =====
# 1. Cargar group_diff.npy + all_diff_connectomes.npy de cada pipeline
# =====
def load_pipeline_diffs(dir_name):
    g = Path(CONN_BASE) / dir_name / "group" / "group_diff.npy"
    a = Path(CONN_BASE) / dir_name / "group" / "all_diff_connectomes.npy"
    if not (g.exists() and a.exists()):
        return None, None
    return np.load(g), np.load(a)

group_diffs = {}
all_diffs   = {}
for label, dir_name, _ in PIPELINES:
    gd, ad = load_pipeline_diffs(dir_name)
    if gd is None:
        print(f"    {label}: faltan group_diff.npy o all_diff_connectomes.npy")
        continue
    if gd.shape != (N_ROI, N_ROI):
        print(f"    {label}: shape {gd.shape}  esperada {(N_ROI, N_ROI)}; omitido")
        continue
    group_diffs[label] = gd
    all_diffs[label]   = ad
    print(f"    {label}: group_diff {gd.shape}, all_diffs {ad.shape}")

if len(group_diffs) < 2:
    raise RuntimeError(f"Solo {len(group_diffs)} pipelines disponibles; "
                       "ejecutar primero §4.5 y §4.6.")
PIPE_LABELS = list(group_diffs.keys())
N_PIPES = len(PIPE_LABELS)

# =====
# 2. Similaridad inter-pipeline (matriz 7×7 de Pearson sobre el triángulo
#    superior de group_diff) + clustering jerárquico
# =====
tri = np.triu_indices(N_ROI, k=1)
sim_mat = np.zeros((N_PIPES, N_PIPES))
for i, p1 in enumerate(PIPE_LABELS):
    e1 = group_diffs[p1][tri]
    for j, p2 in enumerate(PIPE_LABELS):
        e2 = group_diffs[p2][tri]

```

```

sim_mat[i, j] = np.corrcoef(e1, e2)[0, 1]

# Clustering: distancia = 1 - r (Pearson), linkage average
dist_mat = np.clip(1 - sim_mat, 0, 2)
np.fill_diagonal(dist_mat, 0)
sq_dist = spatial.distance.squareform(dist_mat, checks=False)
Z_link = linkage(sq_dist, method="average")
order = leaves_list(Z_link)

sim_df = pd.DataFrame(sim_mat, index=PIPE_LABELS, columns=PIPE_LABELS)
sim_df.to_csv(os.path.join(QA_STATS, "triangulation_similarity.csv"))
print("\n=== Similaridad inter-pipeline (Pearson r sobre group_diff) ===")
print(sim_df.round(3).to_string())

# -----
# Figura: dendrograma arriba + heatmap reordenado
# -----

fig = plt.figure(figsize=(10, 11))
# 2 cols: col 0 = dendrograma + heatmap (mismo ancho exacto), col 1 = colorbar
gs = fig.add_gridspec(2, 2, height_ratios=[1, 3],
                    width_ratios=[9, 1],
                    hspace=0.05, wspace=0.3)

# Dendrograma - col 0 únicamente, mismo ancho que el heatmap
ax_d = fig.add_subplot(gs[0, 0])
dendro = dendrogram(Z_link, labels=PIPE_LABELS, ax=ax_d,
                    color_threshold=0.4, above_threshold_color="#888")
ax_d.set_xticks([]) # las etiquetas van en el heatmap inferior
ax_d.set_ylabel("1 - Pearson r")
ax_d.set_title("Triangulación inter-pipeline - clustering jerárquico (group_diff PSIL-MTP)",
               pad=10, fontsize=11)
for spine in ("top", "right"):
    ax_d.spines[spine].set_visible(False)

# Heatmap reordenado - col 0
ax_h = fig.add_subplot(gs[1, 0])
ord_lbl = [PIPE_LABELS[i] for i in order]
ord_mat = sim_mat[np.ix_(order, order)]
im = ax_h.imshow(ord_mat, cmap="RdBu_r", vmin=-1, vmax=1, aspect="auto")
ax_h.set_xticks(range(N_PIPES)); ax_h.set_xticklabels(ord_lbl, rotation=30, ha="right")
ax_h.set_yticks(range(N_PIPES)); ax_h.set_yticklabels(ord_lbl)
for i in range(N_PIPES):

```

```

    for j in range(N_PIPES):
        v = ord_mat[i, j]
        ax_h.text(j, i, f"{v:.2f}", ha="center", va="center",
                  color="white" if abs(v) > 0.5 else "black", fontsize=10)

# Colorbar - col 1, eje propio: no roba ancho al heatmap ni al dendrograma
ax_c = fig.add_subplot(gs[1, 1])
fig.colorbar(im, cax=ax_c, label="Pearson r")

out_png = os.path.join(QA_FIGS, "triangulation_clustermap.png")
fig.savefig(out_png, dpi=200, bbox_inches="tight")
plt.close(fig)
print(f"\n Clustermap guardado: {out_png}")

# Resumen interpretativo automático
print("\n=== Interpretación automática del clustering ===")
me_ica = [p for p in PIPE_LABELS if p.startswith("TEDANA")]
oc      = [p for p in PIPE_LABELS if p.startswith("OC")]
if len(me_ica) >= 2 and len(oc) >= 2:
    me_pairs = [(i, j) for i, p1 in enumerate(PIPE_LABELS) if p1 in me_ica
                  for j, p2 in enumerate(PIPE_LABELS) if p2 in me_ica and i < j]
    oc_pairs = [(i, j) for i, p1 in enumerate(PIPE_LABELS) if p1 in oc
                  for j, p2 in enumerate(PIPE_LABELS) if p2 in oc and i < j]
    cross     = [(i, j) for i, p1 in enumerate(PIPE_LABELS) if p1 in me_ica
                  for j, p2 in enumerate(PIPE_LABELS) if p2 in oc]
    r_within_meica = np.mean([sim_mat[i, j] for i, j in me_pairs])
    r_within_oc     = np.mean([sim_mat[i, j] for i, j in oc_pairs])
    r_cross         = np.mean([sim_mat[i, j] for i, j in cross])
    print(f"  r medio dentro de cluster ME-ICA : {r_within_meica:.3f}")
    print(f"  r medio dentro de cluster OC      : {r_within_oc:.3f}")
    print(f"  r medio entre clusters (cross)      : {r_cross:.3f}")
    if min(r_within_meica, r_within_oc) - r_cross > 0.10:
        print("    → Cluster ME-ICA vs OC bien separado: el INPUT (TEDANA vs OC)")
        print("    es el factor dominante de varianza inter-pipeline.")
    else:
        print("    → Sin separación clara ME-ICA/OC: los REGRESORES externos")
        print("    dominan la varianza inter-pipeline (no el input fMRI).")

# Comparación canónica TEDANA+aCompCor (primario) vs OC+aCompCor
if "TEDANA+aCompCor" in PIPE_LABELS and "OC+aCompCor" in PIPE_LABELS:
    i = PIPE_LABELS.index("TEDANA+aCompCor")
    j = PIPE_LABELS.index("OC+aCompCor")

```

```

r_critical = sim_mat[i, j]
print(f"\n TEDANA+aCompCor (primario) OC+aCompCor (regresores idénticos, input distinto)")
print(f"    r = {r_critical:.3f}")
if r_critical > 0.90:
    print("    → ME-ICA aporta MARGINAL valor neto sobre aCompCor en este dataset.")
elif r_critical < 0.70:
    print("    → ME-ICA aporta valor neto SUSTANCIAL sobre single-echo con regresores idénticos")
else:
    print("    → ME-ICA aporta valor neto MODERADO; revisar análisis de sensibilidad en L1")

# =====
# 3. tDOF por pipeline × run
# =====
print("\n=== Cálculo de tDOF por run × pipeline ===")
# sub-P2 excluido en pre-flight (PsiloData §3 - N=6).
DRUG_SESSIONS = {
    "sub-P1": {"PSIL": 11, "MTP": 6},
    "sub-P3": {"PSIL": 12, "MTP": 8},
    "sub-P4": {"PSIL": 10, "MTP": 6},
    "sub-P5": {"PSIL": 6, "MTP": 10},
    "sub-P6": {"PSIL": 9, "MTP": 5},
    "sub-P7": {"PSIL": 7, "MTP": 11},
}
CONF_RE = re.compile(
    r"(sub-P\d+)_ (ses-[^_]+)_task-(BOLDREST\d+)_dir-[A-Z]+_run-(\d+)"
    r"_part-mag_desc-confounds_timeseries\.tsv$"
)

tdof_rows = []
for subj, sessions in DRUG_SESSIONS.items():
    for drug, ses_num in sessions.items():
        ses = f"ses-{ses_num}"
        # Confounds TSV de la sesión en ambas tandas de fMRIPrep (t0, t1):
        # los runs son disjuntos entre tandas → la unión los cubre todos.
        conf_paths = []
        for fp_dir in FMRIPREP_DIRS:
            func_dir = os.path.join(fp_dir, subj, ses, "func")
            if not os.path.isdir(func_dir):
                continue
            conf_paths.extend(glob.glob(os.path.join(
                func_dir, "*_desc-confounds_timeseries.tsv")))
        for conf_path in sorted(conf_paths):

```

```

m = CONF_RE.search(os.path.basename(conf_path))
if not m:
    continue
_, _, task, run = m.groups()
try:
    df = pd.read_csv(conf_path, sep="\t")
except Exception as e:
    print(f"    {os.path.basename(conf_path)}: error lectura ({e})")
    continue
fd = df.get("framewise_displacement", pd.Series(dtype=float))
fd_clean = fd.fillna(0).values
n_total = len(fd_clean)
n_scrubbed = int((fd_clean > FD_THRESHOLD).sum())
nss_cols = [c for c in df.columns if c.startswith("non_steady_state_outlier")]
n_nss = len(nss_cols)
n_analytical = n_total - n_nss
for label, _, n_reg in PIPELINES:
    tdof = n_analytical - n_reg - n_scrubbed
    tdof_rows.append(dict(
        subj=subj, condition=drug.lower(), ses=ses,
        task=task, run=int(run),
        pipeline=label,
        n_total_frames=n_total,
        n_nss=n_nss,
        n_analytical=n_analytical,
        n_regressors=n_reg,
        n_scrubbed=n_scrubbed,
        tdof_effective=tdof,
        pct_scrubbed=100.0 * n_scrubbed / max(n_total, 1),
    ))

tdof_df = pd.DataFrame(tdof_rows)
tdof_df.to_csv(os.path.join(QA_STATS, "triangulation_tdof.csv"), index=False)

# Resumen por pipeline
tdof_summary = (tdof_df.groupby("pipeline")
                .agg(tdof_median=("tdof_effective", "median"),
                    tdof_mean =("tdof_effective", "mean"),
                    tdof_min  =("tdof_effective", "min"),
                    tdof_max  =("tdof_effective", "max"),
                    n_runs    =("tdof_effective", "size"))
                .reindex([lbl for lbl, _, _ in PIPELINES])

```

```

        .reset_index())
print("\n=== tDOF efectivo por pipeline (frames retenidos tras regresión + scrubbing) ===")
print(tdof_summary.round(1).to_string(index=False))

# Figura tDOF
fig, ax = plt.subplots(figsize=(10, 5))
positions = np.arange(N_PIPES)
sub_tdof = [tdof_df[tdof_df["pipeline"] == p]["tdof_effective"].values
             for p in PIPE_LABELS]
bp = ax.boxplot(sub_tdof, positions=positions, widths=0.5,
                 patch_artist=True, showfliers=True)
for patch, lbl in zip(bp["boxes"], PIPE_LABELS):
    patch.set_facecolor(PIPE_COLORS.get(lbl, "#888"))
    patch.set_alpha(0.7)
ax.set_xticks(positions); ax.set_xticklabels(PIPE_LABELS, rotation=30, ha="right")
ax.set_ylabel("tDOF efectivos por run\n(frames - regresores - scrubbed)")
ax.set_title(f"Pérdida de DOF por pipeline (FD > {FD_THRESHOLD} mm scrubbed, ds006072)",
             fontsize=11)
ax.axhline(y=tdof_summary["tdof_median"].max(), color="grey", linestyle=":",
           linewidth=0.7, label=f"max mediana = {tdof_summary['tdof_median'].max():.0f}")
ax.legend(fontsize=9, loc="lower left")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
out_png = os.path.join(QA_FIGS, "triangulation_tdof.png")
plt.savefig(out_png, dpi=200, bbox_inches="tight")
plt.close(fig)
print(f"\n tDOF figura guardada: {out_png}")

# =====
# 4. Convergencia de efectos PSIL - MTP por arista
# =====
# Para cada pipeline: t-test pareado de una muestra (H0: diff_PSIL-MTP = 0)
# + Cohen's d_z (paired effect size; Cumming 2014).
# Cohen's d_z = mean(diff) / sd(diff) - apropiado para diseño pareado.
# =====
print("\n=== Test pareado PSIL - MTP por arista ===")
effects = {}
for p in PIPE_LABELS:
    arr = all_diffs[p]                                # shape (N, 232, 232)
    n_subj = arr.shape[0]
    edge_vals = arr[:, tri[0], tri[1]]                # shape (N, n_edges)
    mean_d = edge_vals.mean(axis=0)

```



```

sd_d    = edge_vals.std(axis=0, ddof=1)
d_z     = np.where(sd_d > 1e-9, mean_d / sd_d, 0.0)
t_val, p_val = stats.ttest_1samp(edge_vals, 0, axis=0)
effects[p] = dict(d=d_z, t=t_val, p=p_val, n=n_subj)
sig = int(np.sum(p_val < SIG_ALPHA))
print(f"  {p:<18s}: N={n_subj}  aristas p<{SIG_ALPHA} = {sig:>5d} "
      f"({100*sig/len(p_val):.1f}%)  |d_z| medio = {np.mean(np.abs(d_z)):.3f}")

# -----
# Tabla de convergencia con PRIMARY_PIPE como referencia
# -----
if PRIMARY_PIPE not in effects:
    print(f"\n Pipeline primario '{PRIMARY_PIPE}' no disponible; convergencia omitida.")
else:
    prim = effects[PRIMARY_PIPE]
    sig_idx = prim["p"] < SIG_ALPHA
    n_sig_prim = int(sig_idx.sum())
    print(f"\n=== Convergencia respecto a primario = {PRIMARY_PIPE} "
          f"({n_sig_prim} aristas con p<{SIG_ALPHA}) ===")

    conv_rows = []
    for p, eff in effects.items():
        # Concordancia de signo en TODAS las aristas (no solo las sig.)
        same_sign_all = (np.sign(eff["d"]) == np.sign(prim["d"])) & \
            (np.abs(prim["d"]) > 1e-9 & (np.abs(eff["d"]) > 1e-9))
        pct_signo_global = 100.0 * same_sign_all.sum() / len(prim["d"])

        # Sobre las aristas sig. del primario
        if n_sig_prim > 0:
            same_sign_sig = np.sign(eff["d"][sig_idx]) == np.sign(prim["d"][sig_idx])
            also_sig_sig = eff["p"][sig_idx] < SIG_ALPHA
            both = same_sign_sig & also_sig_sig
            pct_signo_sig = 100.0 * same_sign_sig.sum() / n_sig_prim
            pct_replic = 100.0 * both.sum() / n_sig_prim
        else:
            pct_signo_sig = np.nan
            pct_replic = np.nan

        # Correlación de d_z entre pipelines (todas las aristas)
        r_d = np.corrcoef(prim["d"], eff["d"])[0, 1]

        conv_rows.append(dict(

```

```

        pipeline                = p,
        r_d_vs_primary          = r_d,
        pct_signo_global        = pct_signo_global,
        pct_signo_sig_primary   = pct_signo_sig,
        pct_replic_sig_primary  = pct_replic,
        n_sig_pipeline          = int((eff["p"] < SIG_ALPHA).sum()),
    ))

conv_df = pd.DataFrame(conv_rows).set_index("pipeline")
conv_df = conv_df.reindex(PIPE_LABELS)
conv_df.to_csv(os.path.join(QA_STATS, "triangulation_convergence.csv"))
print(conv_df.round(2).to_string())

# Interpretación automática
print(f"\n Lectura:")
print(f"    - r_d_vs_primary alto + pct_signo_global 100% pipelines convergen,")
print(f"      la elección de primario es académica.")
print(f"    - pct_replic_sig_primary alto en los 5 sensitivity efectos sig. del")
print(f"      primario son robustos (publicables como hallazgo principal).")
print(f"    - pct_signo_sig_primary alto pero pct_replic bajo los efectos están")
print(f"      en la misma dirección pero pierden significancia (consistente con")
print(f"      poder limitado N=6, no con artefacto).")

# Figura: heatmap de convergencia - primario en fila 0, resto por r_d desc.
non_primary = [p for p in conv_df.index if p != PRIMARY_PIPE]
sorted_others = (conv_df.loc[non_primary, "r_d_vs_primary"]
                 .sort_values(ascending=False, na_position="last")
                 .index.tolist())
plot_order = [PRIMARY_PIPE] + sorted_others
conv_df_plot = conv_df.reindex(plot_order)

metrics_to_plot = ["r_d_vs_primary", "pct_signo_global", "pct_replic_sig_primary"]
metric_labels = ["r(d_z) vs primario",
                 "% signo coincidente\n(todas aristas)",
                 "% replic. sig.\nde primario"]
plot_labels = conv_df_plot.index.tolist()
n_rows = len(plot_labels)

plot_mat = conv_df_plot[metrics_to_plot].values.astype(float)
plot_norm = np.copy(plot_mat)
plot_norm[:, 0] = (plot_norm[:, 0] + 1) / 2 # r [-1,1] → [0,1]
plot_norm[:, 1] = plot_norm[:, 1] / 100 # % → [0,1]

```

```

plot_norm[:, 2] = plot_norm[:, 2] / 100

# 2 cols: heatmap (ancho) + colorbar (estrecha); evita que colorbar robe ancho
fig, (ax, ax_c) = plt.subplots(1, 2, figsize=(9, 5),
                                gridspec_kw={"width_ratios": [20, 1]})
im = ax.imshow(plot_norm, cmap="RdYlGn", vmin=0, vmax=1, aspect="auto")
ax.set_xticks(range(len(metrics_to_plot)))
ax.set_xticklabels(metric_labels, fontsize=10)
ax.set_yticks(range(n_rows))
ax.set_yticklabels(
    [f" {lbl} [primario]" if lbl == PRIMARY_PIPE else lbl
     for lbl in plot_labels],
    fontsize=10)
# Separador visual entre fila primaria y sensitivity pipelines
ax.axhline(0.5, color="black", linewidth=1.5, linestyle="--")
for i in range(n_rows):
    for j in range(len(metrics_to_plot)):
        v = plot_mat[i, j]
        txt = f"{v:.2f}" if j == 0 else f"{v:.0f}%"
        ax.text(j, i, txt, ha="center", va="center",
                fontsize=10, color="black")
ax.set_title(f"Convergencia de efectos PSIL - MTP por pipeline\n"
             f"(primario = {PRIMARY_PIPE}; ds006072; verde = convergencia alta)",
             fontsize=11)
fig.colorbar(im, cax=ax_c, label="Convergencia (normalizada)")
out_png = os.path.join(QA_FIGS, "triangulation_convergence.png")
fig.savefig(out_png, dpi=200, bbox_inches="tight")
plt.close(fig)
print(f"\n Convergencia figura guardada: {out_png}")

# =====
# 5. Resumen ejecutivo final (impreso) - guía para la decisión de primario
# =====
print("\n" + "=" * 70)
print("RESUMEN EJECUTIVO - guía para selección de pipeline primario")
print("=" * 70)
print(f"\n1. SIMILARIDAD: r medio inter-pipeline = "
      f"{np.mean(sim_mat[np.triu_indices(N_PIPES, k=1)):.3f}")
# Tres comparaciones canónicas (cf. §5 intro):
# (a) TEDANA TEDANA+24P - ¿24P externos aportan sobre ME-ICA?
# (b) TEDANA TEDANA+aCompCor - ¿ruido fisiológico difuso sobrevive ME-ICA?
# (c) TEDANA+aCompCor OC+aCompCor - aporte neto de ME-ICA (regresores idénticos)

```

```

def _report_pair(p1, p2, comment_low, comment_high, thr_low=0.80, thr_high=0.95):
    if p1 in PIPE_LABELS and p2 in PIPE_LABELS:
        i1, i2 = PIPE_LABELS.index(p1), PIPE_LABELS.index(p2)
        r = sim_mat[i1, i2]
        print(f"    {p1}    {p2}: r = {r:.3f}")
        if r > thr_high:
            print(f"        → {comment_high}")
        elif r < thr_low:
            print(f"        → {comment_low}")

_report_pair("TEDANA", "TEDANA+24P",
            "24P externos modifican la FC sobre ME-ICA (Power 2018: posible respiración pro",
            "24P no añade información sobre TEDANA (Kundu 2017 corroborado).")
_report_pair("TEDANA", "TEDANA+aCompCor",
            "aCompCor cambia FC; considerar TEDANA+aCompCor como primario (DuPre 2021).",
            "aCompCor no añade información sobre TEDANA (Kundu 2017 corroborado).")
_report_pair("TEDANA+aCompCor", "OC+aCompCor",
            "ME-ICA aporta valor neto SUSTANCIAL sobre single-echo con regresores idénticos",
            "ME-ICA aporta MARGINAL valor neto sobre aCompCor en este dataset.")

ted_mediana = tdof_summary.loc[tdof_summary['pipeline'] == 'TEDANA', 'tdof_median']
gsr_mediana = tdof_summary.loc[tdof_summary['pipeline'] == 'OC+36P+GSR', 'tdof_median']
if len(ted_mediana) and len(gsr_mediana):
    print(f"\n2. tDOF: TEDANA conserva {ted_mediana.values[0]:.0f} DOF/run "
          f"vs {gsr_mediana.values[0]:.0f} DOF/run en 36P+GSR.")

if PRIMARY_PIPE in effects and n_sig_prim > 0:
    medianas_replic = conv_df.drop(PRIMARY_PIPE)["pct_replic_sig_primary"].median()
    medianas_signo = conv_df.drop(PRIMARY_PIPE)["pct_signo_global"].median()
    print(f"\n3. CONVERGENCIA respecto a {PRIMARY_PIPE}:")
    print(f"    Mediana de % signo coincidente (todas aristas) = {medianas_signo:.1f}%")
    print(f"    Mediana de % replic. sig. (aristas sig. primario)= {medianas_replic:.1f}%")
    if medianas_signo > 70:
        print("        → Efectos robustos en signo; primario defensible.")
    if medianas_replic > 50:
        print("        → Efectos sig. del primario replican mayoritariamente.")

print("\nÁrbol de decisión pre-registrado (cf. §4.3):")
print("    - r entre pipelines ME-ICA    0.85 y pct_signo_global    70% en TEDANA vs el")
print("    resto → PRIMARIO = TEDANA (más parsimonioso, Kundu 2017).")
print("    - Si TEDANA+aCompCor difiere sustancialmente (r < 0.80) → reconsiderar")
print("    PRIMARIO = TEDANA+aCompCor (ruido fisiológico residual real).")

```

```
print(" - El veredicto final se documenta en @sec-tedana-qc.")

print(f"\nOK §5.2 (triangulación inter-pipeline, ds006072, 7 pipelines) completo")
print(f" Stats: {QA_STATS}")
print(f" Figures: {QA_FIGS}")
```

i Cuándo ejecutar y cómo interpretar el output

Este chunk se ejecuta **después** de §4.5 (extracción FC) y §4.6 (consolidación grupal), y **antes** de PsyConn §6/§7 (análisis estadístico) — necesita los `group_diff.npy` y `all_diff_connectomes.npy` de los 7 pipelines.

Outputs de inferencia (en orden de prioridad para la decisión de primario):

1. **triangulation_similarity.csv** + clustermap PNG: matriz 7×7 . Si los 7 pipelines convergen (r medio > 0.85), la elección de primario es académica y se justifica por parsimonia (TEDANA). Si divergen (cluster ME-ICA vs cluster OC visiblemente separados, o TEDANA+aCompCor cae lejos de TEDANA solo), hay que defender la elección con criterio.
2. **triangulation_convergence.csv** + heatmap PNG: tabla `pipeline × {r_d, % signo, % replic}`. La columna `pct_signo_global` (concordancia de signo en todas las aristas) es la métrica **menos sensible al N**: si los 6 pipelines de sensitivity replican el signo del primario en $> 80\%$ de las aristas, los hallazgos de PsyConn §6 son robustos.
3. **triangulation_tdof.csv** + boxplot PNG: tDOF efectivo por run $\times$ pipeline. Solo desempata en caso de convergencia: si dos pipelines producen FC indistinguible, gana el que conserva más DOF.

Acción tras el output:

- Si convergencia alta (r medio > 0.85 y mediana de signo $> 70\%$) mantener TEDANA primario, documentar la elección en Section 3.13.
- Si divergencia ME-ICA vs OC con cluster claro TEDANA primario sigue siendo defendible (ME-ICA es la innovación metodológica del TFM).
- Si TEDANA+aCompCor diverge de TEDANA y converge con OC+aCompCor considerar TEDANA+aCompCor como primario (ruido fisiológico residual real); justificar en Section 3.13.
- Si OC+36P+GSR es el único divergente confirma el sesgo de centrado en cero de Murphy & Fox (2017); excluirlo del análisis principal y reportarlo solo como sensitivity de GSR.

La métrica `pct_replic_sig_primary` (replicación de significancia) está incluida solo como referencia; con $N=6$ los p-valores son ruidosos y no debe usarse como criterio único.

La métrica diagnóstica primaria es **pct_signo_global** porque captura la dirección del efecto sin depender del umbral de significancia.